

O'REILLY®

Архитектуры данных

современные решения для любых задач



SPRINT
book

Джеймс Серра

Deciphering Data Architectures

*Choosing Between a Modern Data Warehouse, Data
Fabric, Data Lakehouse, and Data Mesh*

James Serra

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Архитектуры данных

*современные решения
для любых задач*

Джеймс Серра

SPRiNT
book

2026

ББК 32.973.233-018
УДК 004.62
С33

Серра Джеймс

С33 Архитектуры данных: современные решения для любых задач. — Астана: «Спринт Бук», 2026. — 288 с.: ил.

ISBN 978-601-12-5033-7

Ткань данных, озеро данных и сетка данных появились относительно недавно и стали рассматриваться как конкурентные альтернативы современному хранилищу данных. У новых архитектур действительно есть весомые преимущества, но, помимо этого, они окружены ореолом предубеждений и мифов. Книга содержит практический обзор архитектур данных, который поможет специалистам в области обработки данных понять преимущества и недостатки каждой из них.

Джеймс Серра, архитектор решений Big Data и хранилищ данных, работающий в Microsoft, рассматривает общие концепции архитектур данных, в том числе то, как хранилищам данных пришлось эволюционировать, чтобы работать с функциями озер данных. Вы узнаете, чем хороши озера-хранилища данных, а также как отличить раздуваемый вокруг сеток данных ажиотаж от их реальных возможностей. А самое главное, вы научитесь определять наиболее подходящую архитектуру данных для своих нужд.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.233-018
УДК 004.62

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

ISBN 978-1098150761 англ.

Authorized Russian translation of the English edition of Deciphering Data Architectures ISBN 9781098150761 © 2024 James Serra
This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.
© Перевод на русский язык ТОО «Спринт Бук», 2025
© Издание на русском языке, оформление ТОО «Спринт Бук», 2025

ISBN 978-601-12-5033-7

Краткое содержание

https://t.me/IT_Portal

Часть I. Основные понятия

Глава 1. Большие данные (Big Data)	28
Глава 2. Типы архитектур данных	38
Глава 3. Дизайн-сессия по разработке архитектуры	50

Часть II. Общие понятия архитектуры данных

Глава 4. Реляционное хранилище данных.....	70
Глава 5. Озеро данных	86
Глава 6. Решения для хранения данных и процессы работы с данными.....	103
Глава 7. Подходы к проектированию данных	121
Глава 8. Подходы к моделированию данных.....	134
Глава 9. Способы приема данных	157

Часть III. Архитектуры данных

Глава 10. Современное хранилище данных	168
Глава 11. Ткань данных	184
Глава 12. Озеро-хранилище данных	191
Глава 13. Основы сетки данных	202
Глава 14. Стоит ли внедрять сетку данных? Мифы, проблемы и перспективы	221

Часть IV. Люди, процессы и технологии

Глава 15. Люди и процессы	242
Глава 16. Технологии	258
Об авторе	281
Иллюстрация на обложке	282

Оглавление

Отзывы о книге «Архитектуры данных: современные решения для любых задач»	16
От издательства	21
О научном редакторе русского издания	21
Предисловие	22
Введение	23
Условные обозначения	24
Благодарности	25

Часть I

Основные понятия

Глава 1. Большие данные (Big Data)	28
Что такое большие данные и чем они могут быть вам полезны?	29
Зрелость данных	33
Уровень 1. Реактивный	33
Уровень 2. Информативный	34
Уровень 3. Прогностический	35
Уровень 4. Трансформационный	35
Бизнес-аналитика самообслуживания	35
Итоги	36
Глава 2. Типы архитектур данных	38
Эволюция архитектур данных	39
Реляционное хранилище данных (relational data warehouse)	42
Озеро данных (data lake)	43
Современное хранилище данных (modern data warehouse)	46
Ткань данных (data fabric)	46
Озеро-хранилище данных (data lakehouse)	47

Сетка данных (data mesh)	47
Итоги	49
Глава 3. Дизайн-сессия по разработке архитектуры.....	50
Что такое ADS?	50
Зачем проводить ADS?	50
Перед проведением ADS	51
Подготовка.....	52
Приглашение участников	55
Проведение ADS	56
Представление участников.....	56
Дискавери.....	57
Работа с маркерной доской.....	63
После проведения ADS.....	64
Советы по проведению ADS.....	65
Итоги	67

Часть II

Общие понятия архитектуры данных

Глава 4. Реляционное хранилище данных	70
Что такое реляционное хранилище данных?	70
Чем хранилище данных не является	73
Нисходящий подход к проектированию (сверху вниз)	74
Почему стоит использовать реляционное хранилище данных?.....	77
Недостатки использования реляционного хранилища данных.....	80
Наполнение хранилища данных	81
Как часто извлекать данные	82
Методы извлечения.....	82
Как определять, какие данные изменились с момента последнего извлечения	83
Слухи о смерти реляционных хранилищ данных сильно преувеличены.....	84
Итоги	85
Глава 5. Озеро данных	86
Что такое озеро данных?	86
Почему следует использовать озеро данных?	87

Восходящий подход к проектированию (снизу вверх).....	89
Лучшие практики проектирования озер данных.....	91
Несколько озер данных.....	97
Преимущества.....	97
Недостатки.....	101
Итоги	101
Глава 6. Решения для хранения данных и процессы работы с данными	103
Решения для хранения данных.....	104
Витрины данных	104
Хранилища оперативных данных	105
Концентраторы данных	107
Процессы работы с данными.....	110
Управление мастер-данными	110
Виртуализация и федерализация данных.....	111
Каталоги данных	117
Маркетплейсы данных	118
Итоги	120
Глава 7. Подходы к проектированию данных	121
Оперативная обработка транзакций (OLTP) и оперативная аналитическая обработка (OLAP).....	122
Оперативные и аналитические данные.....	124
Симметричная многопроцессорная обработка и массово-параллельная обработка.....	125
Лямбда-архитектура	126
Каппа-архитектура.....	129
Полиглотное хранение и полиглотные хранилища данных.....	131
Итоги	132
Глава 8. Подходы к моделированию данных.....	134
Реляционное моделирование	134
Ключи.....	134
Диаграммы «сущность — связь».....	135
Правила и формы нормализации	136
Отслеживание изменений.....	138

Размерное моделирование	138
Факты, измерения и ключи	138
Отслеживание изменений.....	139
Денормализация.....	141
Общая модель данных	141
Модель Data Vault.....	143
Методологии хранения данных Кимбалла и Инмона	145
Методология Инмона — подход «сверху вниз»	146
Методология Кимбалла — подход «снизу вверх»	147
Выбор методологии.....	149
Гибридные модели.....	150
Мифы о методологиях Инмона и Кимбалла	152
Итоги	155
Глава 9. Способы приема данных	157
ETL и ELT	157
Обратный ETL	160
Пакетная обработка и обработка в реальном времени.....	161
Плюсы и минусы пакетной обработки.....	162
Плюсы и минусы обработки в реальном времени.....	163
Управление данными (data governance).....	164
Итоги	165

Часть III

Архитектуры данных

Глава 10. Современное хранилище данных.....	168
Архитектура MDW.....	168
Плюсы и минусы архитектуры MDW	174
Комбинирование RDW и озера данных	175
Озеро данных	175
Реляционное хранилище данных	176
Промежуточные ступени на пути к MDW.....	177
Расширенное EDW (augmented EDW)	177
Временное озеро данных плюс EDW.....	178
«Все в одном».....	179

Сценарий использования: стратегический переход компании Wilson & Gunkerk на MDW	180
Проблема.....	181
Решение	181
Результат.....	181
Итоги	182
Глава 11. Ткань данных	184
Архитектура «ткань данных».....	185
Политики доступа к данным.....	185
Каталог метаданных	187
Управление мастер-данными (MDM).....	187
Виртуализация данных	187
Обработка в реальном времени	188
API	188
Сервисы	188
Продукты	188
Зачем переходить с MDW на архитектуру «ткань данных»?.....	189
Потенциальные недостатки.....	189
Итоги	190
Глава 12. Озеро-хранилище данных	191
Функции Delta Lake.....	192
Повышение производительности	194
Архитектура озера-хранилища данных.....	195
Что, если отказаться от реляционного хранилища данных?	197
Реляционный сервисный уровень	200
Итоги	201
Глава 13. Основы сетки данных.....	202
Децентрализованная архитектура данных	203
Ажиотаж вокруг сетки данных	204
Четыре принципа сетки данных по Дегани.....	205
Принцип № 1. Владение доменом.....	206
Принцип № 2. Данные как продукт	206
Принцип № 3. Самообслуживаемая инфраструктура данных как платформа.....	208
Принцип № 4. Федеративное вычислительное управление	209

«Настоящая» сетка данных	211
Домены данных	212
Логическая архитектура сетки данных	214
Различные топологии	215
Сетка данных в сравнении с тканью данных	216
Сценарии использования	218
Итоги	220

Глава 14. Стоит ли внедрять сетку данных? Мифы, проблемы

и перспективы.....221

Мифы.....	221
Миф: использование сетки данных — это универсальное средство, которое поможет решить все проблемы с данными	221
Миф: сетка данных заменит озеро данных и хранилище данных.....	222
Миф: все проекты хранилищ данных провальные, но сетка данных решит эту проблему	222
Миф: создание сетки данных означает децентрализацию абсолютно всего.....	222
Миф: с помощью виртуализации данных можно создать сетку данных.....	223
Проблемы	224
Философские и концептуальные вопросы	224
Объединение данных в децентрализованной среде	225
Другие вопросы децентрализации	226
Сложность.....	227
Дублирование	228
Возможность реализации.....	229
Люди	231
Препятствия на уровне доменов	232
Организационная оценка: стоит ли внедрять сетку данных?	234
Рекомендации по внедрению успешно работающей сетки данных.....	235
Перспективы сетки данных	237
Общая картина: представление об архитектурах данных и их применении	238
Итоги	239

Часть IV

Люди, процессы и технологии

Глава 15. Люди и процессы.....	242
Организация команды: роли и обязанности	243
Роли для работы с современным хранилищем данных, тканью данных или озером-хранилищем данных	243
Роли при реализации сетки данных	245
Почему проекты проваливаются: распространенные ошибки и способы их избежать	248
Ошибка: позволить руководителю думать, что BI — «это просто»	249
Ошибка: использование неподходящих технологий	249
Ошибка: собирать слишком много бизнес-требований.....	249
Ошибка: собирать слишком мало бизнес-требований.....	249
Ошибка: представление отчетов без их предварительной проверки	250
Ошибка: привлечение неопытной консалтинговой IT-компании.....	250
Ошибка: привлекать консалтинговую компанию, которая передает разработку на аутсорсинг офшорным разработчикам	250
Ошибка: передача прав на владение проектом консалтинговой компании	251
Ошибка: пренебрежение передачей информации обратно в организацию	251
Ошибка: сокращение бюджета в середине проекта	251
Ошибка: исходить из даты окончания проекта и строго следовать графику.....	251
Ошибка: создание структуры хранилища данных в соответствии с исходными данными, а не с потребностями бизнеса.....	252
Ошибка: представлять конечным пользователям решение с большим временем отклика или другими проблемами производительности	252
Ошибка: чрезмерное (или недостаточное) проектирование архитектуры данных (overdesigning/underdesigning)	253
Ошибка: плохая коммуникация между IT-специалистами и бизнес-доменами.....	253
Рекомендации по достижению успеха	254
Не экономьте на инвестициях	254
Привлекайте пользователей, покажите им результаты и заинтересуйте их.....	255

Повысьте ценность новых отчетов и дашбордов.....	255
Попросите конечных пользователей создать прототип	255
Найдите спонсора проекта	256
Составьте план проекта с целевым показателем эффективности работы сотрудников 80 %.....	256
Итоги	257
Глава 16. Технологии	258
Выбор платформы.....	258
Решения с открытым исходным кодом (OSS).....	258
Локальные решения.....	261
Облачные решения.....	263
Модели облачных сервисов.....	265
Ведущие облачные провайдеры	268
Мультиоблачные решения	269
Программные фреймворки	272
Hadoop.....	272
Databricks.....	276
Snowflake	278
Итоги	279
Об авторе	281
Иллюстрация на обложке.....	282

*Светлой памяти моих бабушек
и дедушек — Долли (Dolly), Билла (Bill),
Марты (Martha) и Берта (Bert)*

Отзывы о книге «Архитектуры данных: современные решения для любых задач»

В книге «Архитектуры данных: современные решения для любых задач» Джеймс Серра прекрасно описывает эволюцию ведущих архитектур данных, а также их преимущества и недостатки. Эту книгу должны обязательно прочитать как опытные, так и начинающие архитекторы данных.

*Билл Антон (Bill Anton), Data Geek,
Opifex Solutions*

В этой исчерпывающей и написанной очень доступным языком книге Джеймс аккумулировал более 30 лет знаний и опыта в области архитектуры данных. Тем, кто выполняет сложную практическую работу, а не просто восхваляет аналитику данных, она просто обязательна к прочтению.

*Д-р Барри Девлин (Dr. Barry Devlin),
основатель и руководитель компании
9sight Consulting*

Этот путеводитель должен стоять на книжной полке каждого архитектора данных. Благодаря четкому и содержательному описанию используемых и только разрабатываемых технологий читатель получит представление о том, как направить свою компанию на решение сложных задач формирующегося ландшафта данных. Это бесценный источник информации как для начинающих, так и для опытных архитекторов данных.

*Майк Фунг (Mike Fung), главный
архитектор облачных решений, Oracle*

Маркетинговая шумиха и болтовня лидеров мнений ИТ-отрасли породили массу заблуждений относительно паттернов архитектур данных. Благодаря своему богатому опыту и мастерству изложения Джеймс Серра пробивается сквозь этот информационный шум и вносит ясность как в давно устоявшиеся модели архитектур данных, так и в передовые методы, которые помогут в работе и специалистам-практикам, и техлидам в области данных. Держите эту книгу под рукой — вы будете часто в нее заглядывать.

*Сойер Найквист (Sawyer Nyquist), владелец,
автор статей и консультант, The Data Shop*

Мир архитектур данных сложен и полон шума. Эта книга предлагает свежий, практичный взгляд, сложившийся на основе десятилетий опыта. Независимо от того, новичок он или эксперт, каждый, кто интересуется данными, должен прочитать эту книгу!

*Питейн Стренгхольм (Piethein Strengholt),
автор книги Data Management at Scale*

Жемчужина обучающей литературы! В книге «Архитектуры данных: современные решения для любых задач» соблюден идеальный баланс между простотой и глубиной изложения, благодаря чему технологические специалисты любого уровня смогут разобраться в ключевых концепциях обработки данных и понять основные компромиссные решения, которые действительно важны при планировании работы с данными.

*Бен Рейес (Ben Reyes), соучредитель
и управляющий партнер,
ZetaMinusOne LLC*

Я рекомендую книгу «Архитектуры данных: современные решения для любых задач» как источник знаний, позволяющий понять и сориентироваться в доступных вариантах при разработке архитектур данных.

*Майк Шелтон (Mike Shelton), архитектор
облачных решений, Microsoft*

Управление данными — важная составляющая успеха любого бизнеса. В книге «Архитектуры данных: современные решения для любых задач» все модные словечки превращаются в простые и понятные концепции и практические решения, которые помогут найти правильную архитектуру для вашего датасета.

*Мэмм Ашер (Matt Usher),
директор Pure Storage*

Как консультант и лидер сообщества, я часто рекомендую людям блог Джеймса Серра, в котором можно узнать актуальную и подробную информацию о современных архитектурах данных. Эта книга — великолепная подборка, в которой Серра представил все разнообразие архитектур без привязки к поставщикам. Мне больше всего нравится часть III, где Джеймс обсуждает плюсы и минусы каждого архитектурного проекта. Я считаю, что эта книга принесет огромную пользу любой организации, которая планирует модернизировать работу с данными.

*Тео Лачев (Teo Lachev), консультант,
Prologika*

Блог Джеймса стал для меня основным источником, помогающим понять суть архитектурных концепций, техническую терминологию и жизнь архитектора решений или инженера данных. Способность Джеймса понятно и четко объяснять сложные технические концепции поистине поразительна. Эта книга — бесценная коллекция его наработок, исчерпывающий справочник по проектированию и осмыслению архитектур.

*Энни Сюй (Annie Xu), старший инженер
данных по работе с заказчиками, Google*

Суперспособностью Джеймса всегда было брать сложные темы и объяснять их простым языком. В этой книге он описывает все самое главное, помогая выбрать правильную архитектуру данных и избежать распространенных (и дорогостоящих!) ошибок.

*Род Колледж (Rod Colledge), старший
технический специалист (в области
данных и искусственного интеллекта),
Microsoft*

Эта книга представляет собой важную веху эволюции обработки данных в технологической отрасли на протяжении нескольких десятилетий, что для большинства равносильно всей карьере. Книга предлагает новому поколению специалистов по работе с данными отличные идеи, на что стоит обратить внимание при разработке будущих решений.

*Пол Эндрю (Paul Andrew), технический
директор Cloud Formations Consulting*

Эта книга — фантастическое руководство для архитекторов данных, наполненное опытом и полезными соображениями. Всесторонний разбор меняющихся тенденций и разнообразных подходов делает ее незаменимым источником информации для всех, кто хочет расширить свой кругозор в этой области.

*Саймон Уайтли (Simon Whiteley),
технический директор, Advancing Analytics
Limited*

Нет никого, чьим знаниям об архитектурах и процессах обработки данных я доверял бы больше, чем знаниям Джеймса Серра. Эта книга не только содержит полное и четкое описание ключевых архитектурных принципов, подходов и «подводных камней», но и затрагивает важнейшие человеческие, культурные и организационные вопросы, которые слишком часто губят проекты по работе с данными еще до их запуска. Эта книга должна стать учебником для студентов колледжей

и бизнес-профессионалов, работающих с данными впервые (а может, даже во второй и в третий раз)!

*Уэйн Экерсон (Wayne Eckerson),
президент Eckerson Group*

Книга «Архитектуры данных: современные решения для любых задач» — это незаменимое руководство для современных специалистов по работе с данными, не привязанное к поставщикам. В нем наглядно сравниваются прежние и современные архитектуры, подчеркиваются их ключевые преимущества и недостатки, а также рассматриваются все нюансы принятия решений при выборе подходящей архитектуры в условиях постоянно меняющегося ландшафта данных.

*Стейсия Варга (Stacia Varga),
автор и консультант по аналитике
данных, Data Inspirations*

Глубокая, практическая мудрость, самые свежие сценарии на современном рынке с учетом конкретного поставщика, новейшая терминология и опции, предлагаемые при продаже услуг. Джеймс использует свой многолетний опыт, чтобы представить универсальный, кросс-облачный, независимый, вертикальный подход от общей структуры до самых мелких деталей.

*Джордан Мартц (Jordan Martz), старший
инженер по продажам, Fivetran*

Озеро данных, озеро-хранилище данных, ткань данных, сетка данных... Нелегко отделить зерна от плевел. Знания и опыт Джеймса Серра — отличный источник информации для всех, кто занимается архитектурой данных.

*Дэйв Уэллс (Dave Wells), отраслевой
аналитик, eLearningcurve*

Слишком часто книги ограничиваются простым описанием решения, «как сделать что-нибудь», не раскрывая его предыстории и логики, — но эта книга не из таких. Получив полное представление о том, почему данные организованы именно так, а не иначе, вы узнаете, каким образом правильно реализовать это самое «как».

*Бак Вуди (Buck Woody), главный
специалист по data science, Microsoft*

Книга «Архитектуры данных: современные решения для любых задач» не только написана подробно и обстоятельно, но и представляет критический взгляд на то, что работает, и, что еще важнее, на то, что может работать не слишком хорошо.

Независимо от того, обсуждаются ли старые подходы к работе с данными или новые, такие как сетка данных, книга предлагает мудрые советы и уроки на основе опыта, что поможет любому практикующему специалисту по работе с данными быстрее освоиться на своем пути.

*Эрик Брода (Eric Broda), предприниматель,
консультант по данным, автор книги
Implementing Data Mesh.*

Ни одна известная мне книга не объясняет так подробно суть понятий озера, хранилища, сетки, ткани и озера-хранилища данных! Эта книга обязательна к прочтению для всех архитекторов и инженеров данных.

*Винсент Райнарди (Vincent Rainardi),
архитектор данных и автор*

От издательства

Мы выражаем огромную благодарность клубу рецензентов ИТ-литературы ReadIT Club за помощь в работе над русскоязычным изданием книги и вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@sprintbook.kz (издательство «SprintBook», компьютерная редакция).

Мы будем рады узнать ваше мнение!

О научном редакторе русского издания

Александр Петраки — главный специалист команды DataOps РСХБ-ИНТЕХ. Занимается проектированием архитектуры высоконагруженных приложений и реализацией DataOps процессов.

Предисловие

Никогда еще в истории современного технологически ориентированного бизнеса ландшафт данных не менялся так быстро. По мере ускорения темпов этих изменений экосистема данных все более усложняется — особенно цепочки создания ценности, связывающие поставщиков и клиентов с организациями. Кажется, что данные повсюду. Они стали одним из основных стратегических активов любой организации, лежащим в основе цифровой трансформации, автоматизации, искусственного интеллекта, инноваций и многих других процессов.

Растущий темп изменений повышает значение оптимизации архитектуры данных, чтобы она могла непрерывно обеспечивать адаптивность, совместимость и работоспособность всех систем компании. В этой книге Джеймс Серра (James Serra) предлагает понятный набор рабочих вариантов для архитектора данных, независимо от поставленной перед ним цели — создать отказоустойчивый надежный продукт или просто устранить технический долг.

Наверное, любой, кто работает с информацией, может рассказать историю встречи с инженерами данных и архитекторами, которая напоминала бы мультсериал «Дилберт» про жизнь офиса, где кажется, что все говорят на разных языках, а решения слишком сложные и непонятные. Давая определения понятиям, разбирая сложные вопросы, развеивая мифы и предлагая обходные пути, Джеймс делится с читателем полезными знаниями об архитектурах данных и придает уверенность в принятии обоснованных решений. Как руководитель, я доволен тем, насколько содержание книги помогает добиться слаженности в работе моих команд обработки данных, обеспечивая их общим понятным языком и справочной информацией.

Когда я познакомился с Джеймсом около 15 лет назад, он подумывал расширить свою специализацию, перейдя от простого администрирования баз данных к бизнес-аналитике и системной аналитике. Его ненасытное желание узнавать новое и делиться тем, чему он научился, чтобы приносить пользу общему делу, произвело на меня сильное впечатление. Таков он и сегодня. Бесчисленные публикации в блогах, презентации и выступления, в которых он делится своим богатым опытом, воплотились в этом обширном труде. Эта книга будет полезна всем нам, кто работает с данными, поскольку мы должны ориентироваться в неопределенности будущего.

Понимание основных принципов архитектуры данных позволит управлять ветром перемен по мере появления новых платформ данных, провайдеров технологий и инноваций. Джеймс создал фундаментальный источник информации для всех нас. Эта книга поможет увидеть общую картину и спроектировать понятное будущее, в котором ваши данные обеспечат вам необходимое конкурентное преимущество.

*Шон Макколл (Sean McCall), директор по
работе с данными, Oceaneering International
Хьюстон, декабрь 2023 года*

Введение

Я занимаюсь информационными технологиями почти 40 лет. Я работал в компаниях самых разных размеров, был консультантом и владельцем собственного бизнеса. Последние 9 лет я работаю в Microsoft в качестве архитектора данных, а хранилищами данных занимаюсь уже 15 лет. Я рассказывал о данных тысячи раз, выступая перед клиентами и группами специалистов.

За свою карьеру я видел, как появлялись и исчезали разные архитектуры данных. Я видел, как множество компаний спорили о том, какой подход лучше, а в итоге выбирали неверную архитектуру данных — совершая ошибку ценой в миллионы долларов и месяцы потерянного времени, которая оставляла их далеко позади конкурентов.

Добавок архитектуры данных сложны. Я на собственном опыте убедился, что большинство людей не понимают их концепций, если вообще что-то знают о них. Все жонглируют такими терминами, как сетка данных, хранилище данных и озеро-хранилище данных, но если вы спросите 10 человек, что такое сетка данных, то получите 11 разных ответов.

С чего же начать? Что кроется за этими умными словами — дань моде, в которой много хайпа, но мало сути, или же это жизнеспособные подходы? В теории эти концепции могут звучать прекрасно, но насколько они практичны? Каковы преимущества и недостатки каждой архитектуры?

Ни одна из архитектур, о которых пойдет речь в этой книге, не является «плохой» или «хорошей». Все они имеют право на существование, но только в определенных сценариях использования. И ни одна архитектура не является универсальной, поэтому моя книга не ставит цель убедить вас выбрать одну архитектуру в ущерб другим. Напротив, здесь вы найдете объективное обсуждение всех достоинств и недостатков каждой архитектуры. У всего есть плюсы и минусы, и важно понять, в чем они заключаются, а не просто выбрать архитектуру, которая больше раскручена. В каждой архитектуре есть что-то, что можно учесть в работе, даже если не использовать ее целиком. Например, понимание того, как работает сетка данных, приведет вас к концепции владения данными — понятию, которое может быть применимо к любой архитектуре.

Эта книга дает базовые знания об общих концепциях архитектуры данных. Концепций так много, что понять, какую из них выбрать и как ее реализовать, нелегко. Я хочу помочь вам получить общее представление обо всех этих понятиях и архитектурах, чтобы вы знали, какие варианты существуют и какой из них лучше всего подходит в вашей ситуации. Цель книги — научить вас рассуждать о концепциях и архитектурах данных обоснованно и выбирать те, которые подходят к создаваемому вами решению.

Стандартных определений концепций и архитектур данных не существует. Иначе эта книга была бы не нужна. Я надеюсь дать общие определения, которые помогут всем говорить на одном языке и облегчат обсуждение. Не питаю иллюзий, что мои определения станут общепринятыми, но я хотел бы дать нам всем отправную точку для дискуссий о том, как скорректировать эти определения.

Я написал эту книгу для всех, кому важно извлечение ценности из данных, будь то разработчик или администратор баз данных, архитектор данных, технический директор или директор по информационным технологиям, или даже человек, не имеющий отношения к ИТ-сфере. Вы можете быть как новичком, так и опытным специалистом. Единственные навыки, которые вам потребуются, — это общее представление о данных, с которыми вы работаете, и здоровое любопытство.

Для читателей, не имеющих опыта работы в этой сфере, я предлагаю обзор больших данных (глава 1) и архитектур данных (глава 2), а также базовых понятий из области данных (часть II). Если вы уже давно работаете с данными, но вам необходимо разобраться в новых архитектурах, вы можете найти много полезного в части III, где подробно рассматриваются конкретные архитектуры, а также кратко повторяются некоторые основы. Смело пропускайте разделы с материалом, который вы и так хорошо знаете. Также отмечу, что, несмотря на то, что основное внимание уделяется большим данным, рассматриваемые концепции и архитектуры применимы и к «малым» данным.

Материал книги абсолютно не привязан к поставщикам услуг. Вы сможете применять архитектуры и концепции, о которых здесь рассказывается, к любому облачному решению. Я являюсь сотрудником компании Microsoft, однако мнения, высказанные здесь, принадлежат только мне и не отражают точку зрения моего работодателя.

Я написал эту книгу, поскольку мое врожденное любопытство побуждает меня разбираться в вещах, а затем делиться информацией так, чтобы ее мог понять каждый. Эта книга — итог работы всей моей жизни. Надеюсь, что она будет вам полезна.

Условные обозначения

В этой книге используются следующие обозначения.

Курсив

Курсивом выделены новые термины, ссылки на сайты, адреса электронной почты, имена и расширения файлов.

Моноширинный шрифт

Используется для листингов программ, а также внутри абзацев для обозначения таких элементов, как имена переменных или функций, базы данных, типы данных, переменные среды, операторы и ключевые слова.



Этот рисунок указывает на совет или предложение.



Этот рисунок указывает на общее примечание.

Благодарности

Эта книга не была бы написана без неизменной поддержки и терпения моей жены Мэри, что было особенно важно в долгие ночи, проведенные за бумагой и означавшие, что придется пропустить карточную партию с семьей и друзьями. Ее присутствие внушало чувство комфорта и придавало мотивации.

Мой нелегкий путь был поддержан семьей: моими родителями Джимом и Лоррейн; сестрами Дениз, Мишель и Николь; и моими уже взрослыми детьми Лорен, Рэй-Энн и Джеймсом, которые стали для меня источником вдохновения, пусть даже не имели ни малейшего представления, о чем эта книга!

Сердечное спасибо моим наставникам и коллегам за годы работы в Microsoft. Такие люди, как Стив Басби (Steve Busby), Эдуардо Касснер (Eduardo Kassner) и Мартин Ли (Martin Lee), помогали мне профессионально расти, делясь со мной мудростью, которая нашла отражение на этих страницах.

Хочу выразить благодарность тем, кто помог мне критическим взглядом и конструктивной обратной связью, в частности Питейну Стренгхолту (Piethein Strengholt), Барри Девлину (Barry Devlin), Биллу Инмону (Bill Inmon) и Майку Шелтону (Mike Shelton). Ваши советы были бесценны.

Я особенно благодарен Шону Макколлу не только за то, что много лет назад он познакомил меня с миром хранилищ данных, но и за то, что остается моим верным другом и согласился написать предисловие к этой книге.

Наконец, спасибо всем замечательным сотрудникам издательства O'Reilly, благодаря которым эта книга появилась на свет: Саре Грей (Sarah Grey), чьи потрясающие редакторские правки и рекомендации сделали эту книгу намного лучше, чем если бы я работал над ней в одиночку; Аарону Блэку (Aaron Black) за то, что помог мне создать аннотацию к книге и получить ее одобрение; Поле Флеминг (Paula Fleming) за отличную литературную редактуру; Кэти Тозер (Katie Tozer) за руководство выпуском книги; Кристен Браун (Kristen Brown) за то, что все прошло гладко; и Сюзанне Хастон (Suzanne Huston) за замечательный маркетинг книги.

Я хотел бы выразить глубокую признательность вам, читателям, — благодаря вашему интересу и участию бесчисленные часы написания не просто стоили того, но и принесли глубокое удовлетворение.

Завершая эту главу своей жизни и с нетерпением вглядываясь в новые горизонты, я испытываю глубокое чувство благодарности за путь, который прошел с этой книгой, и за невероятных людей, которые стали частью этого пути.

Основные понятия

https://t.me/IT_Portal

В первой части этой книги рассмотрены базовые понятия архитектур данных. Глава 1 начинается с определения понятия больших данных (Big Data), в главе 2 представлен обзор типов архитектур данных и их эволюции. В главе 3 описывается процесс проведения дизайн-сессии по разработке архитектуры, которая поможет определить лучшую архитектуру данных для проекта.

Эта часть послужит хорошей отправной точкой для понимания ценности больших данных и истории архитектур, использующих эти данные. В следующих главах мы углубимся в детали.

ГЛАВА 1

Большие данные (Big Data)

В 2020-х годах число компаний, создающих архитектуры данных, резко возросло. Этот рост вряд ли замедлится в ближайшее время во многом потому, что данных стало доступно больше, чем когда-либо прежде: из социальных сетей, устройств интернета вещей (IoT), непрофессиональных приложений и стороннего ПО, и это лишь несколько источников. Согласно исследованию BCG (Boston Consulting Group) за 2023 год (<https://oreil.ly/hpOPt>), «объем генерируемых данных с 2018 по 2021 год увеличился примерно в два раза и составил около 84 зеттабайт, и ожидается, что темпы роста сохранятся». По оценкам исследователей, «в период с 2021 по 2024 год объем генерируемых данных будет увеличиваться с совокупным годовым приростом (CAGR, compound annual growth rate) в 21 % и достигнет 149 зеттабайт». Компании знают, что могут сэкономить миллионы долларов и увеличить свой доход, собирая эти данные и используя их для анализа прошлого и настоящего, а также для прогнозирования будущего, но для этого им нужен способ хранения всех этих данных.

Во всем деловом мире спешат как можно быстрее создавать архитектуры данных. Эти архитектуры должны быть готовы обрабатывать любые будущие данные — независимо от их размера, скорости обработки или типа — и поддерживать их точность. И тем из нас, кто имеет дело с архитектурами данных, необходимо четко понимать, как они работают и какие их разновидности существуют. Здесь и пригодится эта книга. Я не понаслышке знаю, к чему приводит неверное понимание сути архитектуры данных. Одна известная мне компания построила архитектуру данных стоимостью 100 миллионов долларов за два года, а потом обнаружила, что в ней использовалась неподходящая технология, она была слишком сложна и недостаточно гибка для обработки определенных типов данных. Пришлось все снести и начать все заново. Не дайте подобному случиться с вами!

Главное — донести нужную информацию до нужных людей в нужное время и в правильной форме. Для этого необходима архитектура данных, позволяющая получать, хранить, преобразовывать и моделировать данные (все это — обработка больших данных), чтобы их можно было правильно и легко использовать. Нужна архитектура, которая позволит любому конечному пользователю, не обладающему обширными техническими знаниями, анализировать данные и создавать отчеты и информационные панели (дашборды), а не рассчитывать, что специалисты в ИТ-отделе сделают это за них.

Глава 1 начинается с введения в теорию больших данных и описания некоторых ее основополагающих принципов. Затем мы обсудим, как компании используют свои данные, с акцентом на бизнес-аналитику и расширение их использования по мере развития архитектуры данных в компании.

Что такое большие данные и чем они могут быть вам полезны?

Несмотря на то что в термине «*большие данные*» (Big Data) фигурирует слово «*большие*», он описывает не только размер данных. Он подразумевает все данные, большие или малые, внутри компании и все данные за ее пределами, которые могут быть вам полезны. Данные могут быть представлены в любом формате, а их сбор может происходить с любой частотой. Поэтому лучший способ определить понятие «*большие данные*» — считать, что это *все* данные, независимо от их размера (объема), скорости обработки (скорости) или типа (разнообразия). В дополнение к этим критериям для описания данных можно использовать еще три: достоверность, изменчивость и ценность. Вместе они известны как «*шесть V*» больших данных (volume, velocity, variety, veracity, variability, value), как показано на рис. 1.1.



Рис. 1.1. Шесть V больших данных (источник: The Cloud Data Lake by Rukmani Gopalan [O'Reilly, 2023])

Рассмотрим каждый из этих критериев подробнее.

Объем (Volume)

Объем — это огромное количество сгенерированных и сохраненных данных. Он может составлять от терабайта до петабайта данных из самых разных источников, включая социальные сети, транзакции электронной коммерции, научные эксперименты, данные датчиков IoT-устройств и многие другие. Например, данные из системы ввода заказов могут составлять пару терабайт в день, а IoT-устройства — обрабатывать миллионы событий в минуту и генерировать сотни терабайт данных в день.

Разнообразие (Variety)

Под *разнообразием* понимается широкий спектр источников и форматов данных. Их можно разделить на *структурированные данные* (из реляционных баз данных), *полуструктурированные данные* (например, логи и документы в форматах CSV, XML и JSON), *неструктурированные данные* (например, электронные письма, документы и PDF-файлы) и *бинарные данные* (изображения, аудио, видео). Так, данные из системы ввода заказов будут структурированными, поскольку они поступают из реляционной базы данных, а данные с IoT-устройств, скорее всего, будут в формате JSON.

Скорость (Velocity)

Скорость — это скорость, с которой генерируются и обрабатываются данные. Сбор данных, производящийся редко, обычно называют *пакетной обработкой*; например, когда все заказы за день собираются и обрабатываются вечером. Однако данные могут собираться и достаточно часто или даже в режиме реального времени, особенно если они генерируются с высокой скоростью, как, например, данные из социальных сетей, с IoT-устройств и из мобильных приложений.

Достоверность (Veracity)

Достоверность — это точность и надежность данных. Большие данные поступают из самых разных источников. Использование ненадежных или неполных источников может отрицательно сказаться на качестве данных. Например, данные поступают с IoT-устройства — камеры наружного наблюдения, расположенной перед домом и направленной на подъездную дорожку, и она отправляет текстовое сообщение, когда обнаруживает человека. Не исключено, что подобное устройство сработает ложно из-за факторов окружающей среды, таких как погода, что приведет к ошибке в данных. Таким образом, при получении данных их необходимо проверять.

Изменчивость (Variability)

Под *изменчивостью* понимается согласованность (или несогласованность) данных с точки зрения их качества, содержания и смысла. Для обработки и анализа структурированных, полуструктурированных и неструктурированных форматов данных требуются разные инструменты и методики. Например, тип, частота и качество данных, поступающих с датчиков IoT-устройств, могут

сильно различаться. Датчики температуры и влажности могут генерировать данные через регулярные промежутки времени, в то время как датчики движения генерируют данные только при наличии движения.

Ценность (Value)

Важнейший V-фактор — *ценность* — связан с полезностью и актуальностью данных. Компании используют большие данные для получения информации и принятия решений, ведущих к созданию таких бизнес-ценностей, как повышение эффективности, снижение затрат или новые источники дохода. Например, анализируя данные о клиентах, организации могут лучше понять их поведение, предпочтения и потребности. Они могут использовать эту информацию для разработки таргетированных маркетинговых кампаний, повышения качества обслуживания клиентов и стимулирования продаж.

Собирая большие данные, компании получают информацию, которая помогает им принимать более обоснованные бизнес-решения. *Прогностический анализ* — это тип анализа данных, который предполагает использование статистических алгоритмов и машинного обучения для анализа исторических данных и составления прогнозов относительно будущих событий и тенденций. Это позволяет компаниям не просто реагировать, а действовать на опережение.

В последнее время многие компании называют данные «новой нефтью», поскольку в современной цифровой экономике они стали невероятно ценным ресурсом, подобно нефти в индустриальную эпоху. Данные похожи на нефть во многих отношениях:

- Это сырье, которое необходимо добыть, очистить и переработать, чтобы оно стало полезным. В случае с данными речь идет об их сборе, хранении и анализе для получения информации, которая может стать основой для принятия бизнес-решений.
- Они невероятно ценны. Компании, которые собирают и анализируют большие объемы данных, могут использовать их для улучшения своих продуктов и услуг, принятия более обоснованных бизнес-решений и получения конкурентных преимуществ.
- Данные можно использовать самыми разными способами. Например, обученные на данных алгоритмы машинного обучения в дальнейшем можно применять для автоматизации задач, выявления закономерностей и составления прогнозов.
- Это мощный ресурс, способный кардинально изменить общество. Широкое распространение нефти стимулировало развитие многих отраслей промышленности и новых технологий, а появление больших данных привело к прогрессу в таких областях, как искусственный интеллект, машинное обучение и предиктивная аналитика.
- Именно вследствие всех вышеперечисленных факторов данные могут стать основой могущества и влияния.

Большие данные можно использовать для создания отчетов и дашбордов, где они подскажут, в каких точках продажи отстают, и предпринять шаги «постфактум» для повышения продаж. Машинное обучение может предсказать, в каких точках продажи упадут в будущем, что позволит предпринять упреждающие шаги, чтобы предотвратить это падение. Это называется *бизнес-аналитикой* (business intelligence, BI): процесс сбора, анализа и использования данных, помогающий бизнесу принимать более обоснованные решения.

Согласно рис. 1.2, данные можно собирать из новых источников, например IoT-устройств, блогов и социальных сетей, а также из старых источников, таких как бизнес-приложения, приложения для планирования ресурсов предприятия (ERP, enterprise resource planning) и управления отношениями с клиентами (CRM, customer relationship management). Эти данные могут быть представлены в различных форматах: CSV, JSON, Parquet и др. Они могут поступать пакетами, например раз в час, или передаваться несколько раз в секунду (это называется *поточковой передачей в реальном времени*).

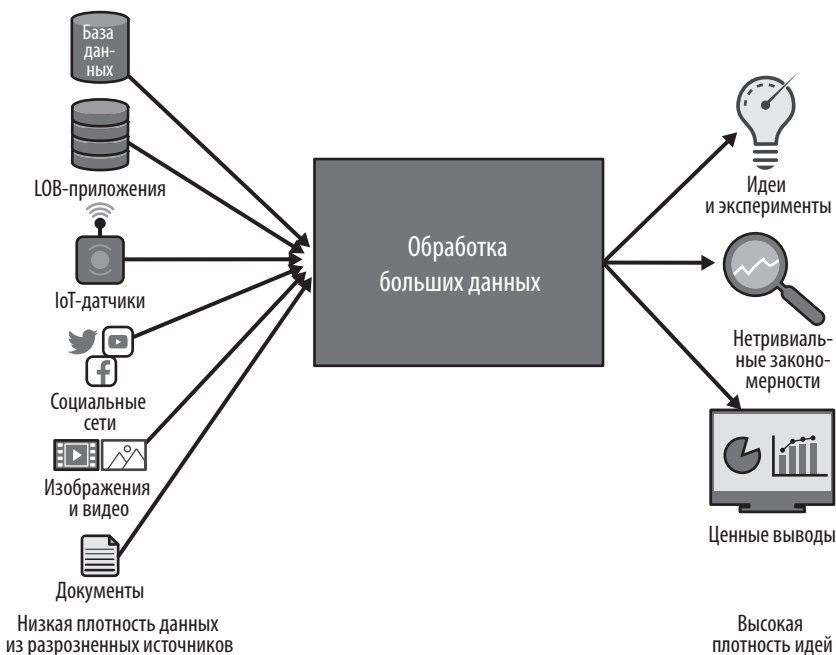


Рис. 1.2. Обработка больших данных (источник: The Cloud Data Lake by Rukmani Gopalan [O'Reilly, 2023])

Компаниям важно понимать, на каком уровне использования данных они находятся по сравнению с другими игроками. Существует концепция *зрелости данных* (data maturity), и в следующем разделе мы рассмотрим ее уровни, чтобы вы могли понять, на каком из них находится ваша компания.

Зрелость данных

Возможно, вы уже не раз слышали такой термин, как «*цифровая трансформация*», который означает, что компания внедряет цифровые технологии, чтобы коренным образом изменить процесс извлечения ценности из данных, а также собственные операции и способы предоставления ценности клиентам. Цифровая трансформация предполагает переход от традиционных, ручных или бумажных процессов к цифровым, с использованием всех возможностей ИТ-технологий для повышения эффективности, производительности и инновационности. Важной частью этой трансформации обычно является использование данных для оптимизации бизнеса, что может означать создание профиля клиента 360¹ (<https://oreil.ly/rSF6P>) для повышения качества обслуживания клиентов или применение машинного обучения для повышения скорости и точности работы производственных линий.

Цифровую трансформацию можно разделить на четыре этапа, которые называются уровнями *зрелости корпоративных данных* (рис. 1.3). Хотя термин широко используется в ИТ, у меня есть свой взгляд на подобное разделение. Эти этапы описывают уровень развития и степень сложности, которых достигла организация в управлении, использовании и извлечении ценности из своих данных. Эта модель — способ оценить возможности организации по управлению данными и ее готовность к использованию расширенной аналитики, искусственного интеллекта, а также других систем, работающих с Big Data. На каждом этапе эффективность использования данных при создании бизнес-ценности и принятии решений растет. Оставшаяся часть раздела содержит описание этих этапов.

Уровень 1. Реактивный

На первом уровне данные компании находятся в разных местах, скорее всего, в виде множества электронных таблиц Excel и/или локальных баз данных, в различных файловых системах, и все это пересылается по электронной почте по всей компании. Архитекторы данных называют это *витриной таблиц* (spreadmart) (сокращение от «spreadsheet data mart» — «витрина таблиц данных»): неформализованный, децентрализованный набор данных, часто встречающийся в организациях, где используются электронные таблицы для хранения, управления и анализа данных. Отдельные сотрудники или команды обычно создают и поддерживают витрины таблиц независимо от централизованной системы управления данными или официального хранилища данных. Витрины таблиц страдают несогласованностью данных, отсутствием управления, ограниченной масштабируемостью и неэффективностью (поскольку часто требуют дублирующих действий).

¹ Профиль клиента 360 — это база данных, которая включает всю историю взаимодействия клиента с брендом; данные собираются из разных источников и постоянно обновляются. — *Примеч. пер.*



Рис. 1.3. Уровни зрелости корпоративных данных

Уровень 2. Информативный

Компании достигают второго уровня зрелости, когда начинают централизовывать свои данные, что значительно упрощает проведение анализа и создание отчетов. Уровни 1 и 2 предназначены для создания *ретроспективных (исторических) статистических отчетов* или отслеживания трендов и закономерностей в прошлом, поэтому на рис. 1.3 они обозначены как «взгляд в прошлое». На этих этапах вы только реагируете на то, что уже произошло.

На уровне 2 решение, созданное для сбора данных, обычно не очень хорошо масштабируется. Как правило, объем и типы данных, которые оно может обрабатывать, ограничены, и оно может принимать данные относительно редко (к примеру, каждую ночь). Большинство компаний находятся на уровне 2, особенно если их инфраструктура все еще остается *локальной*¹.

¹ Под «локальной» понимается такая ИТ-инфраструктура организации — серверы, системы хранения данных и сетевое оборудование, — размещение и управление которой производится в собственных физических помещениях организации, обычно называемых дата-центрами. В этом их отличие от облачных сервисов, где ресурсы размещаются и управляются сторонними провайдерами, такими как Azure, Amazon Web Services (AWS) или Google Cloud Platform (GCP), в удаленных дата-центрах. О преимуществах перехода от локальной инфраструктуры к облачной я расскажу в главе 16, а пока примите к сведению, что переход с локальных серверов на облачные является важной частью цифровой трансформации большинства предприятий.

Уровень 3. Прогностический

На третьем уровне находятся компании, которые уже перешли на облачные технологии и создали систему, способную обрабатывать большие объемы данных, разные типы данных и данные, которые поступают часто (каждый час или в потоковом режиме). Они также улучшили процесс принятия решений, внедрив машинное обучение (расширенную аналитику), чтобы принимать решения в режиме реального времени. Например, когда пользователь заходит в книжный интернет-магазин, система может рекомендовать ему дополнительные книги на странице оформления заказа, основываясь на его предыдущих покупках.

Уровень 4. Трансформационный

Наконец, на четвертом уровне компания создает решение, способное обрабатывать любые данные, независимо от их объема, скорости поступления и типа. Новые данные можно легко и быстро использовать в работе, поскольку архитектура способна их обрабатывать и имеет инфраструктуру для их поддержки. Это решение позволяет конечным пользователям, не обладающим техническими знаниями, легко создавать отчеты и дашборды с помощью выбранных ими инструментов.

В этой книге мы сосредоточимся на уровнях 3 и 4. В частности, процесс, когда конечные пользователи сами создают отчеты, называется *бизнес-аналитика самообслуживания*, о ней речь пойдет в следующем разделе.

Бизнес-аналитика самообслуживания

В течение многих лет, если конечному пользователю в организации требовался отчет или дашборд, ему приходилось собирать все свои материалы (необходимые исходные данные, а также описание того, как должен выглядеть отчет или дашборд), заполнять форму запроса в ИТ-отдел и ждать. ИТ-отдел создавал отчет. Процесс включал извлечение данных, загрузку их в хранилище, построение модели данных и, наконец, формирование отчета или дашборда. Конечный пользователь просматривал его и либо утверждал, либо запрашивал изменения. Это часто приводило к длинной очереди запросов, так что в итоге ИТ-отдел становился серьезным «узким местом». Конечным пользователям требовались дни, недели или даже месяцы, чтобы начать извлекать пользу из данных. Сейчас этот процесс называют «традиционной бизнес-аналитикой», поскольку в последние годы появилось нечто лучшее — бизнес-аналитика самообслуживания (self-service BI).

Цель любого решения архитектуры данных, которое вы создаете, должна заключаться в том, чтобы каждый конечный пользователь, независимо от своих технических навыков, мог быстро и легко запрашивать данные и создавать отчеты и дашборды. Для выполнения этой работы не нужно привлекать ИТ-специалистов — пользователи должны справляться с этим самостоятельно.

Чтобы этого добиться, придется проделать большую предварительную работу: пообщаться со всеми конечными пользователями, чтобы выяснить, какие данные им нужны, а затем построить архитектуру данных с учетом их потребностей. Но результат будет стоить того, поскольку позволит экономить время на создание отчетов. Такой подход избавляет от очередей и переписки с ИТ-отделом, сотрудники которого, как правило, мало что понимают в конкретных данных. Вместо этого конечный пользователь, который лучше всех разбирается в данных, получает прямой доступ к ним, подготавливает их, создает модель данных, формирует отчеты и проверяет их правильность. Такой рабочий процесс гораздо более продуктивен.

Создание столь простого в применении решения по работе с данными приводит к появлению бизнес-аналитики самообслуживания. Сформировать отчет должно быть так же просто, как перетащить поле в документе. Конечные пользователи не должны разбираться, как объединять данные из разных таблиц, или беспокоиться о том, что отчет обрабатывается слишком медленно. Создавая систему обработки данных, всегда спрашивайте себя: *легко ли будет другим людям формировать собственные отчеты?*

Итоги

Прочитав эту главу, вы узнали, что такое Big Data и как они могут помочь вам и вашей организации принимать более эффективные бизнес-решения, особенно в сочетании с машинным обучением. Вы научились описывать большие данные с помощью шести V, а также узнали, что такое зрелость данных и как определять ее уровни. Наконец, вы поняли разницу между традиционной бизнес-аналитикой (BI) и бизнес-аналитикой самообслуживания: цель последней состоит в том, чтобы каждый мог использовать данные для быстрого и простого создания отчетов и получения аналитических выводов.

Теперь позвольте вкратце описать, что вас ждет в следующих главах. В главе 2 я расскажу о том, что такое архитектура данных, и представлю общий обзор изменений типов архитектур данных со временем. В главе 3 покажу, как провести дизайн-сессию по разработке архитектуры, помогающую определить, какая архитектура данных лучше всего подходит для ваших целей.

В части II «Общие понятия архитектуры данных» более подробно рассказывается о различных архитектурах. В главе 4 речь пойдет о том, что такое хранилище данных и чем оно не является, а также для чего его нужно использовать. Я остановлюсь на подходе «сверху вниз», отвечу на вопрос, пришел ли конец реляционным хранилищам данных, и расскажу о способах заполнения хранилища. В главе 5 описывается, что такое озеро данных и почему нужно его использовать. В ней также обсуждается подход «снизу вверх», а затем более подробно рассматривается проектирование озера данных и случаи, когда стоит использовать несколько озер данных.

Глава 6 посвящена общим концепциям архитектуры данных, связанным с хранением данных, включая витрины данных (data mart), хранилища оперативных

данных, управление мастер-данными и виртуализацию данных. В главе 7 рассматриваются общие понятия архитектуры данных, связанные с проектированием, в том числе оперативная обработка транзакций (OLTP, Online Transaction Processing) в сравнении с оперативной аналитической обработкой (OLAP, Online Analytical Processing), оперативные данные в сравнении с аналитическими, симметричная многопроцессорная архитектура (SMP, symmetric multiprocessing) в сравнении с массово-параллельной (MPP, massive parallel processing), а также лямбда-архитектуры, капша-архитектуры и полиглотное хранение (polyglot persistence). В главе 8 речь идет о моделировании данных, в том числе реляционном и размерном моделировании, дискуссии между Кимбаллом (Kimball) и Инмоном (Inmon), общей модели данных и моделям типа Data Vault (свод данных). А в главе 9 вы прочтете о приеме данных, в том числе об операциях ETL (extract, transform, load — извлечение, преобразование, загрузка) в сравнении с ELT (extract, load, transform — извлечение, загрузка, преобразование), обратном ELT, пакетной обработке в сравнении с обработкой в реальном времени, а также об управлении данными.

Часть III посвящена конкретным архитектурам данных. В главе 10 описано современное хранилище данных и пять этапов его создания. В главе 11 рассматриваются архитектурный паттерн «ткань данных» (Data Fabric) и варианты его использования. В главе 12 обсуждается архитектура озера-хранилища данных (data lakehouse), а также плюсы и минусы отказа от использования реляционных хранилищ данных.

Главы 13 и 14 посвящены архитектурам сеток данных (data mesh) — здесь есть о чем поговорить! Глава 13 фокусируется на децентрализованном подходе и четырех принципах построения сетки данных, а также описывает, что такое домены данных и продукты данных. Глава 14 посвящена проблемам и вызовам, связанным с построением сеток данных, и развенчивает некоторые распространенные мифы о них. Она поможет проверить, готовы ли вы к внедрению сетки данных. В заключение мы поговорим о том, каким может быть будущее сетки данных.

В главе 15 рассматриваются причины успеха и неудач различных проектов, а также описывается, как правильно организовать команду для создания архитектуры данных. Наконец, глава 16 посвящена обсуждению открытого исходного кода, преимуществ облачных технологий, основных облачных провайдеров, мультиоблачности и программных фреймворков.

Пришла пора в корне изменить ваши представления о данных. Готовы?

Типы архитектур данных

Очень важно уделить должное время разработке и созданию правильной архитектуры данных. Я убедился в этом на собственном горьком опыте в начале своей карьеры. Я был так увлечен созданием собственного решения, что легкомысленно отнесся к важным вопросам, связанным с проектированием архитектуры и с тем, какие продукты следует использовать. Через три месяца после начала проекта я понял, что архитектура не будет поддерживать некоторые из необходимых источников данных. По сути, нам пришлось перезапускать проект с нуля, придумывать другую архитектуру и выбирать другие продукты, потеряв на этом уйму времени и денег. Без правильного планирования конечные пользователи не смогут вовремя извлечь ценность из вашего решения, они будут злиться из-за сорванных сроков, а ваша компания рискует окончательно отстать от своих конкурентов.

При создании решения по работе с данными нужна хорошо продуманная схема. Именно в этот момент на помощь приходит архитектура данных. *Архитектура данных* определяет высокоуровневый архитектурный подход и концепцию, которой необходимо следовать, описывает набор необходимых технологий и поток данных, который будет использоваться для сбора Big Data в создаваемом вами решении. Выбор архитектуры данных может оказаться непростой задачей, поскольку универсальной архитектуры на все случаи жизни не существует. Нельзя просто пролистать книгу и найти список архитектур с соответствующими им продуктами. Нет простого дерева решений, которое приведет вас к созданию идеальной архитектуры. Используемый подход и технологии будут сильно отличаться от клиента к клиенту, от одного сценария к другому.

Основные типы высокоуровневых архитектурных подходов и концепций — это именно то, о чем повествует книга. Она содержит далеко не полный их список, но он даст вам общую картину того, что они собой представляют. Хотя разбить все архитектуры данных на типы в зависимости от их характеристик будет полезным, что я и сделаю в этой книге, это не то же самое, что просто выбрать нужный шаблон из множества заранее определенных «на все случаи жизни». Каждая архитектура данных уникальна и требует индивидуального подхода для удовлетворения конкретных бизнес-потребностей.

Под *архитектурой данных* понимаются проектирование и организация данных в информационной системе. Предопределенные архитектурные шаблоны могут показаться простым способом быстро настроить новую систему. Однако они часто

не учитывают конкретные требования и ограничения системы, к которой применяются, что может привести к проблемам с качеством данных, производительностью системы и обслуживанием. Кроме того, со временем потребности организации и системы данных могут измениться, что потребует обновления и корректировки архитектуры данных. Стандартизированный шаблон может оказаться недостаточно гибким, чтобы учесть эти изменения, и в итоге вы столкнетесь с неэффективностью и ограничениями системы.

В этой главе приведен краткий обзор основных типов, которые будут рассмотрены в книге: реляционное хранилище данных, озеро данных, современное хранилище данных, ткань данных, озеро-хранилище данных и сетка данных. Каждому типу будет посвящена отдельная глава с подробным описанием.

Эволюция архитектур данных

Реляционная база данных хранит данные в структурированном виде, а связи между элементами данных в ней определяются ключами. Данные обычно организованы в таблицы, каждая из которых состоит из строк и столбцов. Каждая строка представляет собой отдельный экземпляр данных, а каждый столбец — определенный атрибут данных.

Реляционные базы данных предназначены для работы со структурированными данными и предоставляют платформу для создания, изменения и запроса данных с помощью стандартизированного языка, известного как язык структурированных запросов (SQL, Structured Query Language). Реляционная модель была впервые предложена Эдгаром Ф. Коддом (Edgar F. Codd) в 1970 году¹, а с середины 70-х годов она стала доминировать в системах управления базами данных. Большинство операционных приложений нуждаются в постоянном хранении данных, и реляционная БД, как правило, является для них предпочтительным инструментом.

В реляционных БД, где согласованность и целостность данных имеют первостепенное значение, данные обычно упорядочиваются с помощью подхода, называемого «*схема при записи*» (schema-on-write). Под *схемой* понимается формальная структура, определяющая организацию таблиц, полей, типов данных и ограничений, а также отношения между ними. Она служит шаблоном хранения и управления данными и обеспечивает согласованность, целостность и эффективную организацию БД. Реляционные базы данных (и реляционные хранилища данных) требуют определенной предварительной работы, прежде чем можно будет размещать в них данные. Необходимо создать базу данных, ее таблицы, поля и схему, а затем написать код для передачи в нее данных. При использовании подхода «*схема при записи*» схема данных определяется и применяется при записи или загрузке данных

¹ “A Relational Model of Data for Large Shared Data Banks”, Communications of the ACM, 13, no. 6 (1970): 377–87 (<https://oreil.ly/vJwgQ>).

в базу. Данные должны соответствовать заранее определенной схеме, включая типы данных, ограничения и связи, прежде чем они поступят в хранилище.

Подход «схема при чтении» (schema-on-read) применяется при считывании или доступе к данным, а не при их записи. Данные могут поступать в систему хранения без соблюдения строгой схемы, а их структура определяется только при запросе или потреблении данных. Такой подход обеспечивает большую гибкость при хранении неструктурированных или полуструктурированных данных и широко используется в озерах данных, о которых речь пойдет далее в этой главе.

На высоком уровне архитектуры данных предоставляют платформу для организации и управления данными таким образом, чтобы они соответствовали потребностям организации. Это подразумевает определение методов сбора, хранения, обработки данных и доступа к ним, а также поддержание качества и обеспечение безопасности и конфиденциальности данных. Хотя архитектуры данных могут принимать множество различных форм, можно выделить в них некоторые общие элементы.

Хранение данных

Все архитектуры данных должны определять способ хранения данных, включая физический носитель (например, жесткие диски или облачные хранилища) и структуры данных, используемые для упорядочивания.

Обработка данных

Архитектуры данных должны определять методы обработки данных, включая любые преобразования или вычисления, которые выполняются с данными перед их хранением или анализом.

Доступ к данным

Архитектуры данных должны предоставлять механизмы доступа к данным, включая пользовательские и программные интерфейсы приложения (API), которые позволяют запрашивать и анализировать данные.

Безопасность и конфиденциальность данных

Архитектуры данных должны включать механизмы обеспечения безопасности и конфиденциальности данных, такие как контроль доступа, шифрование и маскирование данных.

Управление данными

Архитектуры данных должны предоставлять фреймворки для управления данными, включая стандарты качества, отслеживание происхождения и политики сохранения.

В целом главная задача архитектуры данных — обеспечить эффективное управление и использование активов данных в организации, содействуя достижению бизнес-целей и процессам принятия решений.

В качестве отправной точки при определении того, какая архитектура лучше всего подойдет для конкретного сценария, можно использовать таблицу 2.1. В ней представлено сравнение характеристик архитектур данных, о которых я рассказываю в книге.

Таблица 2.1. Сравнение архитектур данных

Характеристика	Реляционное хранилище данных	Озеро данных	Современное хранилище данных	Ткань данных	Озеро-хранилище данных	Сетка данных
Год внедрения	1984	2010	2011	2016	2020	2019
Централизованная/децентрализованная	Централизованная	Централизованная	Централизованная	Централизованная	Централизованная	Децентрализованная
Тип хранения	Реляционный	Объектный	Реляционный и объектный	Реляционный и объектный	Объектный	Зависит от домена
Тип схемы	Схема при записи	Схема при чтении	Схема при чтении и схема при записи	Схема при чтении и схема при записи	Схема при чтении	Зависит от домена
Безопасность данных	Высокая	От низкой до средней	От средней до высокой	Высокая	Средняя	Зависит от домена
Задержка в доступе к данным	Малая	Большая	От малой до большой	От малой до большой	От средней до большой	Зависит от домена
Время разработки и ввода в эксплуатацию	Среднее	Небольшое	Небольшое	Небольшое	Небольшое	Большое
Общая стоимость решения	Высокая	Низкая	Средняя	От средней до высокой	От низкой до средней	Высокая
Поддержка сценариев использования	Низкий уровень	От низкого до среднего уровня	Средний уровень	От среднего до высокого уровня	Высокий уровень	Высокий уровень
Сложность разработки	Низкая	Средняя	Средняя	Средняя	От средней до высокой	Высокая
Зрелость технологии	Высокая	Средняя	От средней до высокой	От средней до высокой	От средней до высокой	Низкая
Необходимый уровень квалификации сотрудников	Низкий	От низкого до среднего	Средний	От среднего до высокого	От среднего до высокого	Высокий

Реляционное хранилище данных (relational data warehouse)

Реляционные базы данных были основой хранения данных на протяжении нескольких десятилетий. Первым реляционным хранилищем данных, используемым в коммерческих целях, стала система Teradata, разработанная в Стэнфордском университете доктором Джеком Э. Шемером (Dr. Jack E. Shemer), который в 1979 году основал корпорацию Teradata. Банк Wells Fargo установил первую систему Teradata в 1983 году (<https://oreil.ly/BMbzc>) и использовал ее для анализа финансовых данных.

По мере того как организации стали генерировать все бóльшие объемы данных, обрабатывать и анализировать их без длительных задержек становилось все сложнее. Ограничения реляционных БД привели к разработке реляционных хранилищ данных¹, которые получили широкое распространение в конце 1980-х годов², примерно через 15 лет после появления реляционных баз данных. *Реляционное хранилище данных* (RDW, relational data warehouse) — это особый тип реляционной базы данных, предназначенный для хранилищ данных и приложений бизнес-аналитики, с оптимизированной производительностью запросов и поддержкой крупномасштабного анализа данных. Хотя и реляционные хранилища данных, и транзакционные БД используют реляционную модель для организации данных, реляционное хранилище обычно больше по масштабу и оптимизировано под аналитические запросы.

RDW имеют как вычислительный механизм, так и хранилище. Вычислительный механизм — это вычислительная мощность, используемая для обработки запросов к данным. Хранилище — это *реляционное хранилище*, в котором хранятся данные, структурированные в виде таблиц, строк и столбцов. Вычислительная мощность RDW может быть использована только для реляционного хранилища — эти концепты связаны в единое целое.

К числу наиболее важных функций RDW относятся поддержка транзакций (обеспечение надежной и последовательной обработки данных), аудиторский след (запись всех действий, выполняемых с данными в системе) и принудительное применение схемы (обеспечение организации и структурирования данных заданным образом).

В 1970-х и 1980-х годах организации использовали реляционные базы данных для оперативных приложений, таких как ввод заказов и управление запасами.

¹ Примечание по терминологии: применительно к архитектуре реляционного хранилища данных вы можете встретить название *традиционное хранилище данных*. В этой книге я чаще всего буду использовать термин *реляционное хранилище данных* (RDW), иногда сокращая его до просто *хранилища данных*. Все эти термины обозначают одно и то же понятие.

² Популярность RDW возросла во многом благодаря Барри Девлину (Barry Devlin) и Биллу Инмону (Bill Inmon), о которых я расскажу в главе 8.

Эти приложения называются системами *оперативной обработки транзакций* (OLTP, Online Transaction Processing). Системы OLTP могут создавать, читать, обновлять и удалять данные в базе данных или, другими словами, выполнять *CRUD-операции*¹. CRUD-операции составляют основу процессов обработки данных и управления ими в архитектурах данных. Они необходимы для проектирования и внедрения систем хранения данных и пользовательских интерфейсов, взаимодействующих с данными.

CRUD-операции требуют быстрого отклика приложения, чтобы конечных пользователей не раздражало долгое обновление данных. Можно обрабатывать запросы и создавать отчеты в реляционной БД, которая используется оперативным приложением, но это потребует большого количества ресурсов и может привести к конфликтам с другими CRUD-операциями, выполняемыми в это же время. Это может замедлить всю работу.

RDW были разработаны отчасти для решения этой проблемы. Данные из реляционной базы копируются в хранилище данных, и пользователи могут выполнять запросы и создавать отчеты не в реляционной базе данных, а в хранилище данных. Таким образом, они не нагружают систему, в которой размещена реляционная БД, и не замедляют работу приложения для конечных пользователей. RDW также централизуют данные из нескольких приложений, чтобы улучшить отчетность, как показано на рис. 2.1.

В главе 4 RDW будут рассмотрены более подробно.

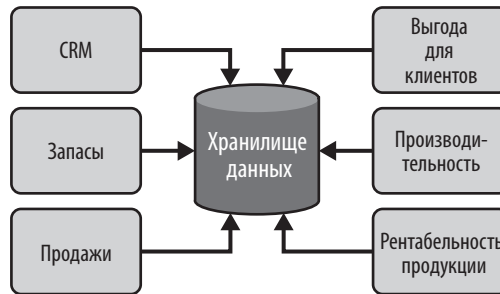


Рис. 2.1. Хранилище данных

Озеро данных (data lake)

Озеро данных — это более поздняя концепция, впервые появившаяся примерно в 2010 году. Можно представить себе озеро данных как усовершенствованную

¹ CRUD — это аббревиатура, обозначающая четыре основные операции управления данными: create, read, update, delete, то есть создание, чтение, обновление и удаление. — *Примеч. пер.*

файловую систему, не сильно отличающуюся от файловой системы на вашем ноутбуке. *Озеро данных* — это просто хранилище; в отличие от реляционного хранилища данных, с ним не связан никакой вычислительный механизм. К счастью, существует множество вычислительных движков, которые могут работать с озерами данных, поэтому вычислительные ресурсы для озера обычно обходятся дешевле, чем для реляционного хранилища данных. Еще одно отличие заключается в том, что если в RDW используется реляционное хранение, то в озерах данных *хранение объектное*, которое не требует структурирования данных в строки и столбцы.

Применение технологии озера данных началось с создания распределенной файловой системы Apache Hadoop (HDFS, Hadoop Distributed File System) — бесплатной платформы с открытым исходным кодом, которая была очень популярна в начале 2010-х годов¹. HDFS — это масштабируемая, отказоустойчивая распределенная система хранения данных, предназначенная для работы на недорогом стандартном оборудовании. Она является основным компонентом экосистемы Apache Hadoop, о которой я подробнее расскажу в главе 16. Поскольку важность облачных вычислений продолжала расти, озера данных стали создаваться в облаке на основе другого типа хранилища, и большинство озер данных сегодня существуют в облаке.

В отличие от реляционного хранилища данных, озеро данных работает по принципу «схема при чтении»; это означает, что для размещения в озере данные не требуют никакой предварительной обработки: можно просто скопировать туда файлы, как папки на ноутбуке. Данные в озере данных хранятся в необработанном (или сыром) формате, то есть могут попадать из системы-источника в озеро данных без преобразования в другой формат. Например, если данные экспортируются из реляционной базы данных в файл в необработанном формате CSV, их можно хранить в озере данных без изменений. Однако если их потребуется хранить в реляционном хранилище данных, их придется преобразовать, чтобы они соответствовали строкам и столбцам таблицы.

При копировании файла данных в озеро схема данных может не скопироваться или вообще находиться в другом файле. Поэтому необходимо создать или получить ее из конкретного файла — отсюда и термин «схема при чтении». Как показано на рис. 2.2, в озеро могут попадать данные из БД операционных приложений, с различных датчиков, из социальных сетей и т. д. Эти файлы могут содержать структурированные данные (например, из реляционных БД), полуструктурированные (например, файлы CSV, логи, XML или JSON-файлы) или неструктурированные (например, из электронных писем, документов и PDF-файлов). Они могут даже содержать бинарные данные (изображения, аудио- и видеофайлы).

¹ Под открытым исходным кодом (<https://opensource.org>) понимается программное обеспечение, исходный код которого бесплатно предоставляется всем желающим для использования, модификации и распространения.

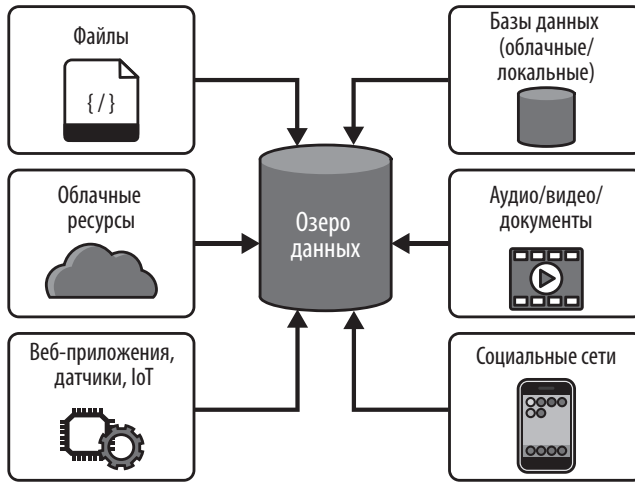


Рис. 2.2. Озеро данных

Озера данных изначально задумывались как решение всех проблем реляционных хранилищ данных, а именно высокой стоимости, ограниченной масштабируемости, низкой производительности, накладных расходов на подготовку данных и ограниченной поддержки сложных типов данных. Компании, продающие Hadoop и озера данных, такие как Cloudera, Hortonworks и MapR, рекламировали их как волшебные инструменты, с легкостью копирующие и очищающие данные, делая их доступными для конечных пользователей. Они утверждали, что озера данных могут полностью заменить реляционные хранилища в соответствии с подходом «одна технология на все случаи жизни». И многие решили сэкономить деньги, используя для всех своих технологий бесплатные инструменты с открытым исходным кодом.

Проблема заключалась в том, что формировать запросы к озеру данных на самом деле не так уж просто: для этого требуется довольно высокая квалификация. ИТ-специалисты говорили конечным пользователям: «Мы скопировали все нужные вам данные в это озеро данных. Просто откройте блокнот Jupyter и создавайте отчеты с этими файлами при помощи Hive и Python». Но это оказалось невозможным, поскольку большинство конечных пользователей и близко не обладали такими навыками. Компании на собственном горьком опыте убедились, что эти сложные, неудобные в использовании решения в итоге обходятся гораздо дороже из-за расходов на оборудование и поддержку, задержек в производстве и снижения производительности. Кроме того, озера данных не обладали рядом функций, которые так нравились пользователям в хранилищах данных, например функциями поддержки транзакций, принудительного применения схем и аудиторских следов. Это привело к тому, что два из трех ведущих поставщиков озер данных, Hortonworks и MapR, перестали существовать.

Но озера данных не исчезли. Просто их цель трансформировалась в другую, такую же полезную: промежуточное хранение и подготовка данных. В главе 5 мы подробно поговорим об этой архитектуре.

Современное хранилище данных (modern data warehouse)

Реляционные хранилища данных и озера данных сами по себе являются упрощенными архитектурами. В них используется только одна технология для централизации данных, а вспомогательных продуктов практически нет. Если вы используете больше технологий и продуктов для поддержки реляционных хранилищ данных или озер данных, они развиваются в архитектуры, рассматриваемые в этой и следующих главах. Озера данных не смогли заменить реляционные хранилища, однако они по-прежнему обладают некоторыми полезными функциями в плане промежуточного хранения и подготовки данных. Почему бы не воспользоваться преимуществами обоих вариантов? Примерно с 2011 года многие компании начали создавать архитектуры, в которых озера соседствуют с реляционными хранилищами, образуя то, что мы теперь называем *современным хранилищем данных* (MDW, modern data warehouse), показанным на рис. 2.3. Слово «современное» в этом определении означает, что в данной архитектуре используются новейшие технологии и подходы в области хранения данных.

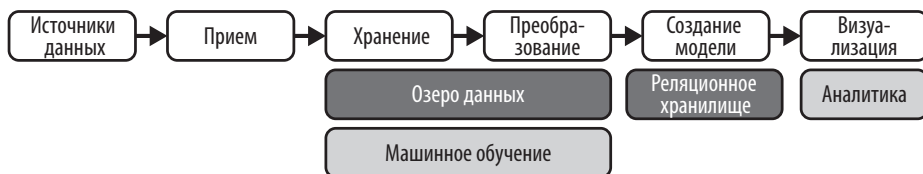


Рис. 2.3. Современное хранилище данных (MDW)

Это подход совмещает в себе все лучшее от обоих предшественников. Озеро данных служит для промежуточного хранения и подготовки данных, а специалисты data science используют его для построения моделей машинного обучения. Хранилище данных предназначено для обслуживания, обеспечения безопасности и приведения данных в соответствие с нормативными требованиями, а бизнес-пользователи осуществляют с его помощью запросы и составляют отчеты. В главе 10 мы более подробно рассмотрим современные архитектуры хранилищ данных.

Ткань данных (data fabric)

Архитектуры типа «ткань данных» начали появляться примерно с 2016 года. Эту архитектуру можно рассматривать как эволюцию архитектуры современного хранилища данных с добавлением технологий, позволяющих получать из

источников больше данных, лучше защищать их и делать их более доступными. Кроме того, были усовершенствованы способы получения данных, их преобразования, формирования запросов, поиска данных и доступа к ним (рис. 2.4). Благодаря всем этим дополнениям система превратилась в «ткань» — большой фреймворк, который способен принимать любые данные. Более подробно о нем будет рассказано в главе 11.

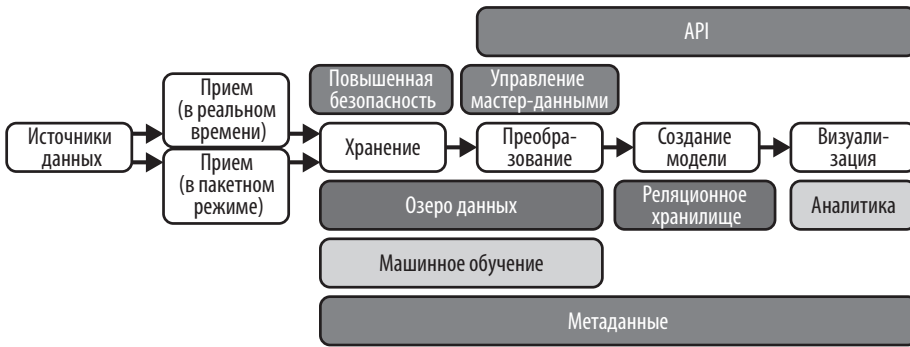


Рис. 2.4. Ткань данных

Озеро-хранилище данных (data lakehouse)

Термин «озеро-хранилище данных» — это гибрид понятий «озеро данных» и «хранилище данных». Архитектуры типа «озеро-хранилище данных» стали популярны примерно с 2020 года, когда компания Databricks начала использовать этот термин. Идея озера-хранилища заключается в том, чтобы избавиться от реляционного хранилища и использовать в архитектуре данных только один репозиторий — озеро данных. Все типы данных — структурированные, полуструктурированные и неструктурированные — попадают в озеро, и все запросы и отчеты выполняются с озером.

Я знаю, о чем вы думаете: «Минуточку. Вы сказали, что озера данных использовали такой же подход и провалились! Что изменилось?» Ответ, как показано на рис. 2.5, заключается в наличии программного слоя Delta Lake для хранения транзакций, который работает поверх существующего озера данных и делает его более похожим на реляционную базу данных. Существующие варианты систем с открытым исходным кодом для этого слоя: Delta Lake, Apache Iceberg и Apache Hudi. Все они будут рассмотрены более подробно в главе 12.

Сетка данных (data mesh)

Термин «сетка данных» впервые появился в мае 2019 года в блоге Жамак Дегани (Zhamak Dehghani) (<https://martinfowler.com/articles/data-monolith-to-mesh.html>), основательницы и руководителя компании Nextdata, а также автора книги «Data

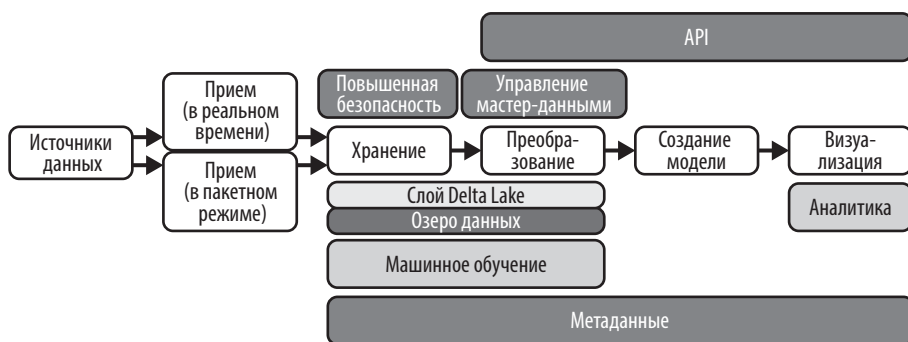


Рис. 2.5. Озеро-хранилище данных

Mesh: Delivering Data-Driven Value at Scale»¹. В декабре 2020 года Дегани в своей статье (<https://martinfowler.com/articles/data-mesh-principles.html>) уточнила, что такое сетка данных, и изложила четыре ее основополагающих принципа. С тех пор архитектуры сеток данных стали чрезвычайно популярны, о них говорят в огромном количестве блогов, презентаций, конференций и СМИ. Упоминание о них появилось даже в ежегодном обзоре Gartner Hype Cycle for Data Management (Цикл зрелости технологий для управления данными от Gartner) (<https://oreil.ly/n4Lzr>). Архитектура сетки данных может быть во многом привлекательна, но, несмотря на невероятный ажиотаж вокруг нее, она подходит лишь для небольшого числа сценариев использования.

Современные архитектуры «хранилище данных», «сетка данных» и «озеро-хранилище данных» подразумевают *централизацию* данных: копирование оперативных данных в централизованное место хранения, которым владеют ИТ-специалисты, в рамках архитектуры, которую они контролируют, и где ИТ-отдел затем создает аналитические данные (левая часть рис. 2.6). Такой централизованный подход влечет за собой три основные проблемы: владение данными, качество данных и организационное/техническое масштабирование. Цель сетки данных — решить эти проблемы.

В сетке данных данные хранятся в нескольких доменах компании, таких как производство, продажи и поставщики (правая часть рис. 2.6). У каждого домена есть своя собственная мини ИТ-команда, которая владеет данными, очищает их, создает аналитические данные и делает их доступными. Каждый домен также имеет собственную вычислительную инфраструктуру и инфраструктуру хранения данных. В результате получается *децентрализованная* архитектура, в которой данные, люди и инфраструктура масштабируются: чем больше доменов, тем больше людей и инфраструктуры. Система может обрабатывать больше данных, и ИТ-отдел больше не является узким местом.

¹ Дегани Ж. «Data Mesh. Новая парадигма работы с данными».



Рис. 2.6. Сетка данных

Важно понимать, что сетка данных — это *концепция*, а не технология. Не существует готового решения «сетка данных в коробке», которое можно было бы купить. Внедрение сетки данных предполагает значительные организационные и культурные изменения, к которым готовы очень немногие компании. (На самом деле большинство компаний даже не настолько велики, чтобы рассматривать их в качестве объектов для применения архитектуры «сетка данных»: она в основном подходит крупным организациям.) Для ее создания необходимо определить, какие части существующих технологий могут быть переориентированы на нее, а какие придется создавать заново. Каждый домен получает возможность определить, какие технологии он будет использовать для построения своей части сетки данных. Это может включать в себя создание современного хранилища данных, ткани данных или озера-хранилища данных. В отношении архитектур сетки данных многое можно обсудить, и мы обязательно сделаем это в главах 13 и 14.

Итоги

Итак, мы получили общее представление о типах архитектур данных, и в следующей главе поговорим о том, как выбрать наиболее подходящую архитектуру для конкретного сценария использования. Этот процесс называется «*дизайн-сессия по разработке архитектуры*».

ГЛАВА 3

Дизайн-сессия по разработке архитектуры

Я провел сотни дизайн-сессий по разработке архитектуры и убедился, что они чрезвычайно важны для создания успешного решения по работе с данными. В этой главе вы узнаете, как провести именно такую дизайн-сессию.

Что такое ADS?

Дизайн-сессия по разработке архитектуры (ADS, architecture design session) — это структурированное обсуждение со стейкхолдерами (ключевыми заинтересованными лицами) со стороны бизнеса и технологий, которое проводится техническими экспертами и направлено на определение и планирование высокоуровневого дизайна по сбору данных для конкретных бизнес-возможностей. Первая ADS — это начало процесса создания архитектуры, и поддержка проекта решения по работе с данными потребует множества других обсуждений (включая, вполне возможно, и других ADS). Результатами ADS являются:

- архитектура (или «схема»), которая послужит отправной точкой для создания решения по работе с данными;
- высокоуровневый план действий, который может включать последующие демонстрации, проверки концепции (PoC, proof of concept) и обсуждение продукта.

ADS — это не технический семинар, не технический тренинг, не демонстрация технологий и не обсуждение низкоуровневых требований.

Зачем проводить ADS?

Существует множество причин для проведения ADS. Во-первых, в ее рамках часто собираются вместе многочисленные стейкхолдеры, работающие в одной организации, включая тех, кто до сессии никогда не встречался лично. Даже сам факт личной или виртуальной встречи может стимулировать креативность и решение проблем. Кроме того, в ней могут участвовать заинтересованные лица от бизнеса, которые хорошо в нем разбираются и могут очень подробно описать конкретные желаемые сценарии использования.

ADS также помогает разделить проекты для реальных потребностей бизнеса от так называемых «исследовательских проектов», предназначенных для тестирования какой-либо технологии или продукта. Она позволяет прояснить замысел проекта и, если это действительно исключительно исследование, ограничить выделяемые на него ресурсы.

Более того, ADS предоставляет структурированный фреймворк, позволяющий «мыслить масштабно, начиная с малого». Как уже отмечалось, это предполагает разработку высокоуровневой архитектуры, но должно включать и обсуждение того, как можно начать с малого, чтобы получить быстрые результаты. Например, архитектура может в итоге включать в себя получение данных из десятков источников, но в ходе ADS может быть выбран подход с низким уровнем риска и высокой отдачей, например начать с двух или трех источников данных с высокой ценностью, которые предполагают меньше времени на завершение работы.

На ADS можно пригласить клиентов организации и/или партнерские компании. Когда я готовился провести свою первую ADS, другой архитектор Microsoft назвал этот процесс «бизнес-терапией». Я быстро понял, что он имел в виду. У нас было около дюжины участников из компании-заказчика, большинство из которых никогда не встречались. Мы начали обсуждать их текущую среду, и несколько человек назвали болевые точки, которые тормозили прогресс. К моему удивлению, другой участник из этой компании дал совет, как убрать одну такую точку. Затем они самостоятельно решили еще две проблемы, а я сидел и слушал. (Конечно, я и мои коллеги тоже часто помогаем устранять блокирующие факторы, но было очень познавательно помолчать и дать другим возможность разобраться самостоятельно.)

Вы можете узнать что-то от своих заказчиков и другими способами, особенно если ваша команда по работе с ключевыми клиентами недостаточно тщательно выполнила свою работу. Вы можете задать все интересующие вас вопросы о текущей среде, болевых точках, целях или получить другую необходимую информацию.

Проведение ADS для заказчика также снабжает вашу команду по работе с клиентами массой других тем для дальнейших обсуждений, таких как демонстрации, проверки концепции и ответы на технические вопросы. Это отличный способ дать импульс работе с клиентом, который не проявлял особой активности, и ускорить цикл продаж. А если вы работаете с партнером, например с консалтинговой фирмой, попробуйте привлечь его к участию в ADS вместе с заказчиком, чтобы обучить их обоим технологиям. Не забудьте предварительно согласовать этот вопрос с заказчиком, особенно если вы будете обсуждать конфиденциальную информацию.

Перед проведением ADS

В этом разделе предполагается, что вы архитектор, который будет проводить ADS. Но даже если вы не архитектор, я рекомендую вам прочитать этот раздел перед участием в дизайн-сессии.

Подготовка

Выделите хотя бы день на подготовку к ADS. Вам нужно будет продумать техническую сторону семинара. Будет ли он проходить в очном или в онлайн-формате? Если в очном, то есть ли у вас достаточно большая переговорная? Если в онлайн, то какую платформу вы будете использовать? Есть ли в переговорной необходимое оборудование, например магнитно-маркерная доска, аудио/видео и другое оборудование? Кто организует обед?

Если ADS будет проводиться в очном формате и особенно если принимать в ней участие должна большая группа людей, обычно выделяют полный семи-восьмичасовой рабочий день, поскольку из-за организационных сложностей снова собрать всех участников вместе может оказаться затруднительным. Обычно если большая часть технологий для клиента в новинку, то на мероприятие будет достаточно шести-семи часов. Если вы заметите, что участники устали, завершите ADS раньше, потому что с этого момента все, что вы дальше расскажете, будет забыто.

Для ADS в онлайн-формате обычно лучше провести две четырехчасовые сессии в разные дни, чем одну большую в один день. Я стараюсь проводить эти сессии два дня подряд или, по крайней мере, в течение одной недели.

Убедитесь, что вы в курсе бюджета и сроков проекта, а также определите лицо, принимающее решения. Это важно независимо от того, является встреча внутренней или нет, но если у вас есть лонг-лист клиентов, которые хотят принять участие в ADS, это поможет сосредоточиться на самых полезных участниках. Если ADS проводится для конкретного клиента, изучите его последний корпоративный отчет, чтобы получить более широкое представление о его интересах и проблемах.

Я также рекомендую предварительно провести внутреннее обсуждение с командой этого клиента. Оно, скорее всего, затронет следующие темы:

- Подробности о клиенте, предпосылки и контекст проекта.
- Проблемы с текущей архитектурой клиента.
- Как можно помочь клиенту.
- Цели клиентской команды и ожидаемый результат от встречи (например, чтобы клиент запросил проверку концепции).
- Насколько хорошо клиент знаком с вашими продуктами.
- Как вы будете проводить ADS и какого результата ожидаете от клиентской команды.

Результатом этого обсуждения станет электронное письмо команде по работе с клиентами, в котором будут подведены итоги встречи, представлены предварительный вариант повестки дизайн-сессии для отправки клиенту, схема рассадки (если ADS будет проводиться очно) и напоминание команде о необходимости предварительной встречи с клиентом.

Обязательно сообщите клиенту, что основная цель этой предварительной встречи с ним — максимально подготовиться к ADS. Объясните, что полученная информация позволит при необходимости уточнить темы или привлечь профильных экспертов (SME, subject matter experts) для помощи в тех областях, с которыми вы не знакомы. Скажите, что вы хотите обсудить повестку мероприятия и вместе позаботитесь о том, чтобы в нее не вошли вопросы, не относящиеся к делу (например, если основное внимание на ADS уделяется архитектуре данных, вопросов безопасности или DevOps лучше не касаться).

Предварительная встреча с клиентом должна строиться примерно так.

Что я понимаю на данный момент

Сначала кратко изложите свое понимание проблемы или возможности, чтобы клиент мог скорректировать ваши ошибочные представления (если таковые имеются).

Расскажите об участниках

Определите, кто будет присутствовать на мероприятии и какие роли они играют в организации. Это поможет выбрать степень профессионального погружения при обсуждении технологий.

Что будет происходить

Предложите план мероприятия и обсудите его с клиентом, но заверьте его, что при необходимости план можно изменить. Объясните, как именно вы будете проводить ADS, и озвучьте соответствующие ожидания. Например, предупредите клиента, что вы ожидаете полной сосредоточенности — никаких звонков и сообщений во время мероприятия для всех участников, включая представителей клиента.

Что вы хотели бы нам показать?

Это дисквери-фаза проекта (фаза открытия)¹. Попросите клиента прислать вам любые схемы архитектуры текущего решения, возможно, его собственные проекты архитектуры будущего решения, а также любые документы, описывающие его текущее решение и планы на будущее.

Какой, по вашему мнению, должна быть успешная ADS?

Спросите: «Есть ли что-то еще, о чем нам следует знать?» Этот вопрос может выявить новые вопросы или цели, которые иначе могли быть упущены.

¹ Дисквери (discovery) в ИТ-сфере — это этап исследования и анализа продукта, направленный на выявление реальных потребностей пользователей, оценку бизнес-возможностей и минимизацию рисков на ранних стадиях разработки. — *Примеч. пер.*

На рис. 3.1 показан пример повестки сессии ADS.

Обязательно включите в повестку дня запланированные перерывы — и не пропускайте их. Может возникнуть соблазн не отвлекаться, если вы втянулись в обсуждение, но лучше так не делать. Попросите кого-нибудь из вашей клиентской команды напомнить о необходимости сделать перерыв, если вы позабудете о нем.

Название вашей компании	Дизайн-сессия по разработке архитектуры Современное хранилище данных Повестка дня	
9:00–9:15	Приветствие и представление участников	Имя архитектора
	Обзор и краткое описание целей и результатов, которые КОМПАНИЯ X хочет достичь в рамках данного стратегического совещания	
9:15–10:30	Предыстория/дискавери/обзор текущего состояния	Все
	Анализ текущей платформы для работы с данными и формирование концепции будущего продукта	
10:30–12:00	Общие представления о современном хранилище данных	Имя архитектора
	Высокоуровневое обсуждение современной архитектуры хранилища данных. Прием, хранение, преобразование, создание модели, визуализация данных	
12:00–12:30	Обед	
12:30–2:00	Концепция озера данных	Имя архитектора
	Как организовать озеро данных и управление данными	
2:00–3:45	Обсуждение сценариев использования продуктов	Имя архитектора
	Какие продукты следует использовать, каковы их сценарии применения и как будет выглядеть окончательное решение	
3:45–4:00	Подведение итогов дня	Все
	Подведение итогов обсуждений и фиксирование по пунктам последующих действий	

* При необходимости можно добавить дополнительные перерывы

Рис. 3.1. Примерная повестка дня для сессии ADS, посвященной проекту современного хранилища данных

Заранее выделите время, чтобы изучить все тонкости использования инструментов для работы с магнитно-маркерной доской (например, Surface Hub¹), чтобы не выяснять это во время ADS. Представьте, что вы находитесь на сессии ADS, и используйте доску, чтобы записать цели, болевые точки и вопросы для дальнейшего обсуждения, а также нарисовать схему архитектуры на основе предположений о том, как клиент будет отвечать на ваши вопросы. Вы даже можете попросить коллегу сыграть роль клиента, чтобы помочь вам.

Приглашение участников

Круг участников ADS будет несколько различаться в зависимости от того, является ли клиент внутренним (отдел вашей компании) или внешним (внешняя компания, с которой вы сотрудничаете).

Со стороны клиента (или, если это ADS для внутреннего клиента, со стороны отдела, для которого вы проводите ADS) в число участников должны входить:

- спонсоры (по крайней мере, один от бизнеса и один от ИТ-отдела);
- представители бизнеса;
- технические специалисты;
- менеджер проекта;
- консультанты, архитекторы, разработчики, специалисты по инфраструктуре или эксплуатации, по мере необходимости.

От своей команды включите:

- архитектора, который будет руководить проведением сессии и следить за тем, чтобы ADS достигла поставленных целей;
- от клиентской команды — ответственного исполнителя, аккаунт-менеджера и/или архитектора облачных решений;
- профильных экспертов (SME, subject matter expert) для предоставления подробных консультаций по конкретным тематикам;
- как минимум одного человека, чтобы делать заметки и фиксировать основные моменты (возможно, члена команды по работе с клиентами).

Если клиент захочет поговорить о предметной области, в которой вы плохо разбираетесь, в большинстве случаев лучше всего пригласить эксперта SME, а не пытаться изучить тему самостоятельно, особенно если ваше время ограничено. Обязательно поблагодарите SME и отправьте письмо с благодарностями также его руководителю. Когда SME присутствует на ADS, выигрывают все: клиент рад

¹ Surface Hub — это многофункциональная цифровая интерактивная доска, платформа для совещаний и вычислительное устройство для совместной работы. — *Примеч. пер.*

получить совет эксперта, команда по работе с клиентами удовлетворена, потому что доволен их клиент, а вы можете чему-то научиться у SME.

Если вы новичок в проведении мероприятий, возможно, вам стоит пригласить ментора, который поможет вам во время ADS (ответит на вопросы, на которые вы не сможете ответить), а после даст обратную связь о том, что получилось, а что можно было бы сделать лучше.

Иногда я составляю программу так, чтобы часть людей присутствовала только на определенных мероприятиях. Например, посвящаю утро бизнесу и дискавери (анализу потребностей), а техническую сторону оставляю на вторую половину дня, когда будут более подробно рассматриваться предлагаемая архитектура и технологии. В этом случае руководители высшего звена, скорее всего, примут участие в утренней сессии, а технические специалисты — в вечерней. Однако желающие могут участвовать во всех мероприятиях — возможно, некоторые руководители высшего звена заинтересуются обсуждением технических тонкостей, а некоторые технические специалисты захотят послушать о дискавери-фазе проекта.

Проведение ADS

Помните, что вы — главный, поэтому именно вам предстоит задавать тон и удерживать разговор в нужном русле. Если кто-то поднимает вопрос, не относящийся к теме, скажите, что мы можем обсудить его позже; затем запишите его на доске на будущее. Аналогично, если вы или кто-то другой соглашаетесь продолжить обсуждение определенного вопроса, добавьте его на доску. Пусть заметки делает аккаунт-менеджер — а вы сосредоточьтесь на разговоре с клиентом и работе с доской.

В середине сессии сверьтесь с повесткой дня и целями. Если вы отстаете, скажите клиенту, что встреча должна проходить в соответствии с планом, чтобы успеть обсудить все намеченные вопросы. Если вам покажется, что вы не успеваете достичь всех намеченных целей, попросите клиента изменить приоритеты, чтобы оставить неохваченными вопросы с самым низким приоритетом.

Представление участников

В начале ADS попросите всех представиться, назвав свое имя, роль в проекте и то, что они знают (если знают) о технологии, которую вы будете обсуждать. Затем, что, возможно, самое важное, спросите, чего они ожидают от ADS (или хотят узнать). Запишите эти вопросы на доске и определите их приоритетность. Если кто-то не участвовал в предварительном обсуждении, разъясните им, о чем пойдет речь на ADS.

Используйте это время, чтобы объяснить, как и для чего была организована ADS, а также чтобы установить основные правила на оставшуюся часть мероприятия.

Например, скажите, что разошлете всем копию итогов с доски, чтобы им не нужно было ее фотографировать, что это будет интерактивная сессия и вы приветствуете многочисленные вопросы, что повестка сессии — это всего лишь примерный план и можно обсудить также другие темы, которые, по мнению участников, будут полезны, и что можно будет впоследствии провести дополнительные встречи для рассмотрения тех вопросов, на которые не хватит времени в рамках данной сессии (поэтому во время важных обсуждений торопиться не стоит).

Дискавери

Дискавери — это час или два в начале ADS на обсуждение таких вопросов, как:

- текущие болевые точки клиента;
- текущая технология и архитектура, используемая клиентом;
- архитектура, которую в будущем предполагает использовать клиент (если таковая обсуждалась);
- любые решения, которые клиент уже принял относительно технологий, продуктов или инструментов, используемых или планируемых к использованию;
- текущие и будущие сценарии использования;
- подробная информация о бизнесе клиента.

Большую часть времени должен говорить клиент, особенно на начальном этапе ADS. Вы ничего не узнаете о нем, если будете говорить только вы. Я всегда начинаю с того, что задаю массу вопросов и получаю ответы, прежде чем перейти в «режим обучения», когда говорю долго. Но даже в этом случае я всегда делаю паузы. Возможно, вам стоит заранее предупредить слушателей, что вы будете периодически останавливаться, чтобы они могли задать вопросы.

Хороший архитектор задает много вопросов. Как говорится, «ты не знаешь того, чего ты не знаешь». Опытные архитекторы имеют представление обо всех доступных архитектурах, методах и инструментах и следят за постоянно меняющимся ландшафтом технологий и продуктов. У них есть опыт задавать вопросы, которые помогут им выбрать правильные технологии и продукты. Дискавери — лучший способ сузить круг предлагаемых продуктов до приемлемого количества, которое можно рассмотреть, а затем подобрать оптимальную архитектуру и продукты для конкретного сценария использования.

В качестве иллюстрации представьте, что вы продаете медицинские инструменты из фургона, где таких инструментов тысячи. (Обычно так не делается, но просто вообразите.) Вы же не станете предлагать врачу перебрать все инструменты, чтобы найти те, которые ему нужны. Вместо этого вы зададите ему массу вопросов — о его специализации, о том, для каких целей он будет их применять, об имеющемся бюджете и т. д. — и, основываясь на этих ответах, покажете ему товары, из которых он сможет выбрать нужные. Архитекторы делают то же самое, и этап дискавери на

ADS — это отличная возможность задать все вопросы, пока вы вместе с нужными людьми.

Я поделюсь с вами еще одним отличным советом, который получил в начале своей карьеры: как только вы начинаете задавать вопросы, постарайтесь отстраниться от мыслей о решении. То есть *НЕ* говорите об архитектурах или продуктах, пока не закончите задавать все вопросы на этапе дискавери. Когда я только начинал проводить ADS, я сразу же переходил к демонстрации продукта Microsoft, как только клиент упоминал, что хочет создать хранилище данных. Мне следовало бы подождать и сначала задать все вопросы, чтобы убедиться, что я четко понимаю, чего хочет клиент. В результате я иногда обнаруживал, что клиенту на самом деле вообще не нужно хранилище данных, а значит, мои усилия по обучению его продукту хранилища данных были напрасны.

Вопросник для сессии ADS

Планируя сессию ADS, ознакомьтесь со списком вопросов, который я разработал в ходе проведенных многочисленных ADS. В начале встречи я уделяю час или два на дискавери. Обратите внимание, что ни один из вопросов не касается какого-либо конкретного продукта. Это сделано для того, чтобы вы могли начать создавать архитектуру, не привязываясь к конкретному поставщику. Позже в ходе ADS вы можете спросить, какие продукты клиент использует и какого облачного провайдера он предпочитает, а затем подбирать продукты в соответствии с архитектурой.

Использует ли ваш бизнес облачные технологии?

Сейчас вам почти всегда ответят «да». Если ответ — «нет», выясните причину и оцените, сможете ли вы преодолеть источник сопротивления. Многим таким клиентам просто нужно лучше разъяснить все преимущества облака (см. главу 16). У других сформировалось предвзятое отношение к облачным технологиям, например, они якобы «недостаточно безопасны», хотя это давно уже не так.

Архитектура данных, которую вы планируете, — это новое решение или миграция?

Если речь идет о миграции, то часть разговора необходимо посвятить тому, как перенести существующее решение в новую среду.

Каковы квалификация и набор навыков у инженеров?

Понимание, с какими продуктами и технологиями знакомы инженеры данных, влияет на то, какие продукты я рекомендую. Если они уже знакомы с инструментом, им потребуется меньше времени на обучение, что, скорее всего, позволит сократить расходы.

Будете ли вы использовать нереляционные данные?

Другими словами, насколько разнообразными будут данные? Если нереляционные данные будут поступать хотя бы из одного источника, клиенту определенно понадобится озеро данных, поэтому сейчас самое время познакомить его с технологиями и дизайном озер данных.

Какой объем данных необходимо хранить?

Какой объем хранилища необходим? Объем данных будет сильно влиять на производительность, поэтому, если он велик, необходимо обсудить, как спроектировать архитектуру (например, структуру папок озера данных), чтобы избежать проблем с производительностью, таких как медленное выполнение запросов и создание отчетов.

Будете ли вы использовать потоковые данные?

Какая требуется скорость? Источники потоковых данных, такие как устройства IoT, влияют на типы продуктов, необходимых для поддержки такого рода данных.

Будете ли вы использовать дашборды и/или узкоспециализированные запросы?

Знание того, как будут использоваться данные, повлияет не только на типы рекомендуемых продуктов, но и на требования к производительности системы. Например, для формирования дашбордов необходимо миллисекундное время отклика, чтобы конечные пользователи могли проводить анализ данных (slice and dice) без заметных задержек.

Будете ли вы использовать пакетные и/или интерактивные запросы?

Убедитесь, что вы понимаете, какие типы запросов понадобятся для работы с данными. Это повлияет на тип используемого хранилища и вычислительные ресурсы, необходимые для обеспечения приемлемой производительности.

Как быстро должны создаваться отчеты? Существуют ли соглашения об уровне обслуживания (SLA) с конкретными требованиями?

От того, насколько быстро требуется создавать отчеты — за миллисекунды или за минуты, зависят архитектура, используемые продукты и количество копий данных, необходимых для достижения приемлемого уровня производительности. Например, может потребоваться агрегирование данных.

Будете ли вы использовать эти данные для предиктивной аналитики или машинного обучения?

Если да, обсудите продукты, которые будут использоваться для машинного обучения (ML) или аналитики, и способы сбора данных, которые облегчат обучение ML-моделей.

Каковы требования к обеспечению высокой доступности и/или к аварийному восстановлению (например, целевое время восстановления и целевая точка восстановления)?

Большинство облачных провайдеров обеспечивают высокую доступность, необходимую среднему клиенту, однако поддержка специфических требований высокого уровня может быть связана с изменением архитектуры. Если требуется аварийное восстановление, то в зависимости от целевой точки восстановления (RPO, recovery point objective) может потребоваться существенно изменять архитектуру. Этот вопрос — хороший повод для клиента задуматься об аварийном восстановлении, если в текущем решении оно не предусмотрено.

Нужно ли вам управление мастер-данными?

Управление мастер-данными (MDM, master data management) подразумевает создание единой мастер-записи для каждого человека, места или предмета в бизнесе, собранной из внутренних и внешних источников данных и приложений. Эти записи в дальнейшем можно использовать для создания более точных отчетов, дашбордов, запросов и моделей машинного обучения. Я подробно объясню, что такое MDM, в главе 6. Если MDM необходимо, вам придется показать, как оно вписывается в архитектуру, и порекомендовать продукты.

Существуют ли ограничения по безопасности при хранении данных в облаке (например, определенные в контрактах с клиентами)?

Это очень важный вопрос: контракты могут ограничивать возможности организации по работе с данными клиентов. Например, если клиенты хранят данные в определенных странах и эти данные не могут покидать пределов страны, вам придется разрабатывать решение с несколькими озерами данных. Точно так же, если они не могут хранить в облаке персональную информацию (PII, personally identifiable information), вам нужно будет обсудить, как и когда анонимизировать эти данные.

Требуется ли это решение круглосуточного доступа клиентов?

Если да, то архитектура должна минимизировать или исключить время простоя для каждого из компонентов. Например, если клиент использует реляционное хранилище данных, решение должно исключать необходимость окна обслуживания для загрузки и очистки данных.

Каково ожидаемое количество пользователей, одновременно обращающихся к решению в часы максимальной загрузки? И сколько пользователей будут обращаться в среднем?

Это важно знать, поскольку некоторые продукты ограничивают количество одновременных обращений пользователей. Если количество одно-

временных пользователей превышает этот лимит или близко к нему, при проектировании следует избегать таких продуктов или комбинировать их с другими, чтобы свести к минимуму вероятность превышения лимита.

Каков уровень квалификации конечных пользователей?

Ответ на этот вопрос будет определять, какие продукты вы порекомендуете клиентам. Если уровень квалификации низкий, можно рекомендовать по-code/low-code продукты. Например, если я упомяну Spark и увижу непонимающий взгляд клиента, я даже не стану говорить о продуктах типа Spark и сосредоточусь на более простых. Если уровень квалификации клиента высок, то можно рассмотреть и варианты с high-code опциями.

Каков бюджет проекта?

Бюджет проекта, несомненно, оказывает огромное влияние на архитектуру и продукты, которые вы выбираете. Если у вас большой бюджет, ограничений на архитектуру нет — вы можете проектировать все, что захотите. Однако чаще бюджет клиентов довольно скромнен. Это вынуждает идти на компромиссы: например, сокращать финансовые расходы за счет снижения производительности, используя меньшее количество вычислений (поскольку загрузка данных и создание отчетов будут в этом случае занимать больше времени).

Каковы планируемые сроки?

Если сроки длинные, как обычно бывает при создании решения для хранения данных, пообщайтесь с провайдером облачных услуг и узнайте, какие продукты находятся в разработке, но еще не анонсированы. Тогда вы сможете рассказать клиенту о том, что может скоро появиться на рынке. Очень неприятно через 4–5 месяцев после начала разработки узнавать, что провайдер облачных услуг собирается выпустить более совершенный продукт.

Где размещаются исходные данные — в облаке и/или локально?

Если некоторые источники данных находятся на локальном сервере, вам понадобится конвейер для передачи данных из локального источника в облако. В зависимости от объема данных это может потребовать изменения архитектуры, например использования продукта, позволяющего создать более мощный конвейер.

Какой объем данных необходимо импортировать в решение ежедневно?

Если каждую ночь необходимо загружать большое количество данных, их передача может занять слишком много времени. Вместо того чтобы загружать много больших файлов ночью, возможно, их следует загружать чаще, например каждый час. Кроме того, можно рассмотреть некоторые

изменения в архитектуре, которые могут ускорить загрузку, например применить параллельную загрузку данных, сжатие данных перед загрузкой или использовать продукты, которые обеспечивают более быструю передачу файлов.

Каковы текущие болевые точки или трудности, связанные с производительностью? Масштабированием? Хранением? Конкурентностью? Временем выполнения запросов?

Вам следует быть в курсе этих проблем, чтобы разработать архитектуру, которая их решит. Никто не хочет тратить время и деньги на решение, которое в результате приведет к тем же болевым точкам, поэтому вам придется убедить участников ADS, что ваше решение избавит их от имеющихся проблем.

Готовы ли вы использовать инструменты сторонних разработчиков и/или инструменты с открытым исходным кодом?

Очень немногие клиенты будут полностью привязаны к определенному облачному провайдеру, используя только его продукты. Поэтому, когда вы закончите описывать высокоуровневую архитектуру и перейдете к обсуждению продуктов, вы можете порекомендовать инструменты сторонних производителей и инструменты с открытым исходным кодом, с которыми инженеры уже знакомы.

Готовы ли вы использовать продукты, находящиеся на этапе закрытого (private preview) или открытого (public preview) превью?

Перед тем как выпустить продукт в общий доступ (GA, general availability), его делают доступным для ограниченной группы приглашенных клиентов, а затем выпускают открытую предварительную версию (которую может использовать любой желающий). Это делается для того, чтобы убедиться, что продукт работает стабильно и без багов. К минусам использования продукта до стадии GA можно отнести ограниченную функциональность, возможные баги, отсутствие поддержки, непредсказуемые изменения и неопределенность в отношении цены. Если компания готова работать с продуктами на стадии закрытого или открытого превью, она может получить преимущество перед другими при использовании нового продукта или функции.

В некоторых компаниях существует строгая политика, запрещающая использование продуктов до стадии GA. Однако такая политика может применяться только к продуктам, которые компания выпустила на продакшен, а значит, продукты в стадии превью, которые станут общедоступными к моменту выпуска решения на продакшен, использовать можно. (Построение архитектуры данных занимает много времени, поэтому такое случается часто.)

Каковы требования к безопасности? Нужен ли цифровой суверенитет данных?

Ответы на эти вопросы могут во многом повлиять на архитектуру. Возможно, вам потребуется встроить шифрование или дополнительные функции безопасности. Сюда же можно отнести и требование иметь несколько озер данных из-за того, что данные не должны покидать пределов страны.

Создаст ли проблему перемещение данных?

Перемещение данных — это процесс извлечения данных из систем-источников и переноса их в хранилище или озеро данных. Если данные поступают из множества разных источников или одновременно из локальных и облачных источников, архитектурные решения должны это учитывать. Например, системе может потребоваться использовать программное обеспечение для репликации, чтобы быстрее извлекать данные из исходной системы в озеро данных.

Насколько необходима бизнес-аналитика самообслуживания?

Если конечные пользователи обладают ограниченными техническими навыками или не имеют их вовсе, но хотят создавать отчеты самостоятельно, то архитектура должна предусматривать преобразование данных в удобный для использования формат. Это может означать применение реляционного хранилища данных (см. главу 4) и/или схемы «звезда» (см. главу 8). Если у всех конечных пользователей технические навыки высоки, то оптимальной архитектурой может стать озеро-хранилище данных (см. главу 12).

Работа с маркерной доской

Используйте маркерную доску или флипчарт, а не слайды презентации. ADS — это выяснение потребностей; она предполагает много обсуждений с использованием доски. Если слайдов слишком много, то ADS превращается в очередную скучную презентацию. На доске должны быть показаны примерная схема архитектуры, а также описание целей, болевых точек и дополнительных вопросов. Рис. 3.2 иллюстрирует, как может выглядеть итоговая доска сессии ADS, посвященной современному хранилищу данных.

На доске представлена не только архитектура, но и приоритетные цели, болевые точки, то, что можно отложить на потом, и другие вопросы, которые необходимо отслеживать.

Запишите все болевые точки на доске и обязательно отмечайте их по мере решения. К концу ADS все эти точки должны быть устранены в архитектуре, которую вы обсуждали. Если этого не произошло, запланируйте повторную встречу с клиентом,

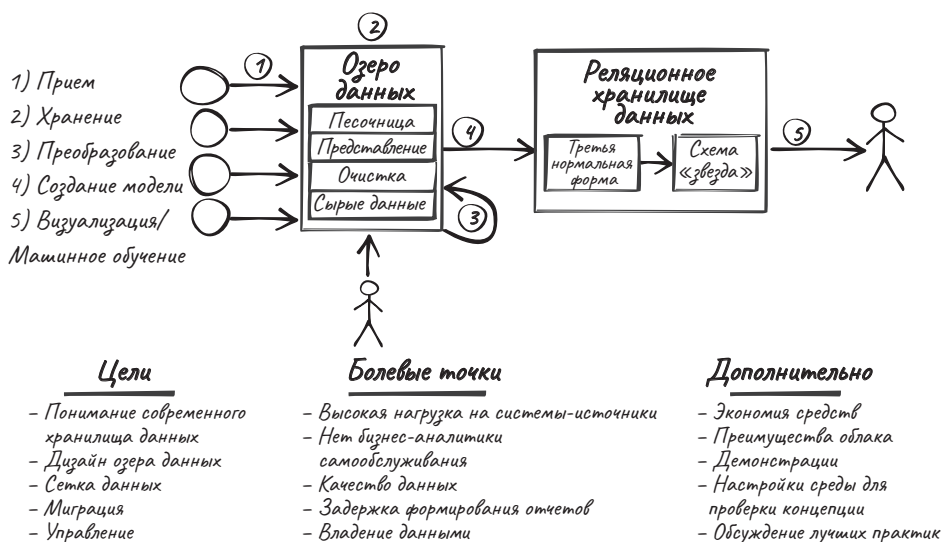


Рис. 3.2. Типичный вид доски после завершения ADS

чтобы рассмотреть незатронутые проблемы. То же справедливо и для целей: если вы не можете достигнуть их все на одной встрече, назначьте повторную.

Зарезервируйте последние 30 минут ADS для рассмотрения дополнительных вопросов.

После проведения ADS

Обсудите с клиентской командой результаты ADS. Спросите, что прошло хорошо и что можно было бы сделать лучше. Это часть реализации установки на профессиональный рост и отличный способ получить обратную связь и повысить качество проведения ADS-сессий. Отправьте результаты ADS всем заинтересованным лицам. Можно подготовить общее резюме для высшего руководства и более подробный отчет для непосредственных участников проекта. Наконец, соберите все материалы, которые вы использовали для ADS, в цифровую папку; ее можно будет использовать в качестве справочного материала для будущих ADS. Например, я храню все свои предыдущие записи с доски; я пришел к выводу, что с их помощью удобно планировать сессии ADS, похожие на те, что уже проводились.

Сразу после проведения ADS отправьте клиенту по электронной почте следующую информацию.

Итоговый документ

Кратко изложите основные моменты, обсуждавшиеся в ходе ADS.

Физическая архитектура

Если вы использовали цифровую доску, экспортируйте конечный результат в файл, который можно отправить заинтересованным лицам, в том числе клиенту.

Последующие действия

Включите сюда все дальнейшие шаги, о которых вы договорились, например «Встретимся в следующий вторник, чтобы обсудить проверку концепции» или «Клиент пришлет по электронной почте диаграмму своей текущей архитектуры».

Вопросы, которые можно отложить на потом, и дополнительные вопросы

Вы отслеживали их на доске. Теперь в электронном письме вы можете описать их подробнее и указать ответственных по каждому пункту. Это даст клиенту возможность прояснить все, что вы не совсем правильно поняли.

Анкетирование

При проведении очной ADS лучше всего раздать каждому участнику анкету для обратной связи в конце сессии. Тогда большинство, если не все, заполнят ее — особенно если в шутку пригрозить, что они не смогут уйти, пока не сделают это! Лучше озвучить эту просьбу во время последнего перерыва в течение дня, а не в самом конце ADS. Если это ADS в онлайн-формате, вы можете разместить ссылку на анкету в чате совещания незадолго до окончания сессии с просьбой «скопировать ссылку до выхода из конференции, чтобы поделиться обратной связью». Скажите участникам, что заполнение анкеты займет всего две минуты (и убедитесь, что это действительно так). Вы также можете включить ссылку на анкету в электронное письмо с итогами совещания.

Советы по проведению ADS

Несмотря на то что это случалось довольно редко, мне приходилось иметь дело с людьми, решившими поспорить со мной во время ADS. Иногда человек, присутствующий на совещании, боится перемен, опасается, что новая архитектура лишит его работы (особенно когда ADS касается миграции в облако), или просто считает, что он знает все лучше всех. Если вы встретились с таким человеком, не вступайте с ним в спор, даже если вы правы. Просто ответьте что-нибудь, что разрядит обстановку и вернет ADS в нужное русло, например: «Это отличная мысль, и я хотел бы обсудить ее подробнее, но нам придется сделать это в другое время и в другой обстановке, чтобы не отклоняться от сегодняшней темы».

Используйте юмор. Не обязательно рассказывать анекдоты, но, если это уместно, попробуйте отпускать забавные комментарии, чтобы разрядить обстановку

и удерживать внимание всех присутствующих. Сессия ADS может оказаться долгой, а смех — лучший способ зарядить людей энергией.

Будьте скромны — не стройте из себя всезнайку. Людям это не нравится. Они предпочитают тех, кто признает, что знает далеко не все. Если вам задали вопрос, на который вы не можете ответить, просто скажите, что не знаете, но выясните и вскоре свяжетесь с ними. Вы можете сами прояснить этот вопрос во время обеда и ответить в тот же день. Или найдите коллегу, который знает ответ, и пригласите его, чтобы он ответил на этот и возможные дополнительные вопросы. Никогда не пытайтесь выдумывать или подтасовывать ответ. Вас могут поймать, и даже если нет, это просто некорректно по отношению к клиенту или коллеге.

Конечно, клиенты не единственные, кто может все испортить. Нередки случаи, когда аккаунт-менеджер начинает длинную речь, надеясь хорошо зарекомендовать себя перед клиентом, не добавляя при этом ничего важного по сути ADS. Если это происходит, отведите его в сторону во время перерыва и в дружеской форме вежливо попросите его говорить покороче. Скажите ему, что вы обязательно зададите соответствующие вопросы, на которые он сможет ответить во время ADS, чтобы продемонстрировать свою компетентность перед клиентом.

Старайтесь правильно оценивать обстановку. Вы знаете, кто принимает решения, поэтому сосредоточьтесь на них и привлекайте их внимание. Следите за языком их тела: если они перестают смотреть на вас и начинают читать электронную почту, значит, им скучно. В этом случае переключитесь на другую тему или сделайте перерыв.

Научитесь перестраиваться на ходу. Я редко полностью следую повестке дня; часто появляются другие темы, гораздо более интересные и полезные для клиента. Сложность будет заключаться в том, что эти темы вам незнакомы! Со временем вы приобретете достаточный объем знаний, чтобы быстро переключаться и подхватывать практически любую. Мне на это потребовалось около двух лет почти ежедневных ADS, так что наберитесь терпения!

И наконец, тренируйте выносливость. Когда я только начинал заниматься ADS, которые длятся весь день, мне было сложно держаться бодрячком шесть-восемь часов, не говоря уже о голосе, и когда я возвращался домой, просто падал без сил. На ADS мне приходилось говорить гораздо больше, чем я привык, и мне потребовалось время, чтобы перестать срывать голос! Пройдет несколько месяцев, прежде чем вы научитесь оставаться энергичным и говорить в течение всего дня. Мои любимые советы, как сохранить бодрость духа и четкость речи: спать не менее восьми часов накануне, поддерживать водный баланс в течение дня, то есть пить воду каждые несколько минут, есть здоровую пищу и регулярно заниматься спортом в течение недели.

Если вы проводите ADS в удаленном или гибридном формате, следует учесть несколько дополнительных советов.

- Включите автоматические субтитры в своей программе для конференций. Это не только сделает сессию более доступной для всех, но и поможет вам следить за ходом обсуждения и избавит от необходимости просить людей повторять свои реплики.
- Войдите в конференцию с двух разных устройств: одно — для демонстрации экрана и работы с доской, другое — для общения (большинство программ такую функцию поддерживает). В идеале проводите конференцию с компьютера с двумя или тремя большими мониторами. В этом случае вам не придется постоянно сворачивать окна или перемещать их и вы всегда сможете отслеживать поведение клиента, пока работаете с доской и говорите.
- Я рекомендую организовать в используемой программе для конференций закрытый чат с командой по работе с клиентами (и только с ней). Это отличный способ общаться с командой так, чтобы клиент вас не слышал, — например, попросить изучить вопрос, на который вы не смогли ответить, вступить в разговор, когда это необходимо, или предоставить вам подробную информацию о собеседнике.
- Я также рекомендую следующие инструменты.
 - Если вы используете Microsoft Teams, включите функцию Speaker Coach, которая прослушивает ваше выступление и предоставляет закрытую обратную связь в реальном времени с предложениями по улучшению, а также резюме по окончании. Эти замечания видны только вам в реальном времени, и они не сохраняются в расшифровке записи встречи.
 - Мне также нравится ZoomIt (<https://oreil.ly/g8NTJ>), инструмент для масштабирования экрана, аннотирования и записи для технических презентаций и демонстрации приложений. Он незаметно запускается в трее и активируется с помощью настраиваемых горячих клавиш. Инструмент позволяет увеличивать отдельные области экрана, перемещаться по ним и рисовать на увеличенном изображении. Это отличный способ привлечь внимание к определенным частям презентации — гораздо лучше, чем использовать курсор, который может быть плохо заметен.

Итоги

ADS — это важнейшая часть процесса разработки архитектуры данных. Она помогает согласовать проектные решения с бизнес-целями, устранить потенциальные риски и проблемы, оптимизировать затраты и ресурсы, а также наладить совместную работу заинтересованных участников.

Конечный результат ADS — создание хорошей или даже отличной архитектуры данных. Хорошей считается архитектура, которая является надежной и масштабируемой и эффективно поддерживает системы, использующие большие данные. Переход архитектуры данных от хорошей к *отличной* означает полное

удовлетворение этих потребностей с учетом отзывов всех пользователей в цепочке извлечения ценности из данных, чтобы общая стратегия работы с данными соответствовала целям организации. Создание решения для данных — это ориентированный на пользователя путь проектирования и обратной связи, и для этого требуются стратегия и планирование, которые можно разработать только на ADS.

Общие понятия архитектуры данных

https://t.me/IT_Portal

Прежде чем перейти к рассмотрению архитектур данных, важно убедиться, что вам хорошо знакомы концепции, которые могут быть использованы в рамках конкретной архитектуры. Я считаю, что многие из них истолковываются неверно, и надеюсь с этим разобраться. Поэтому в следующих главах описываю более 20 таких понятий. По крайней мере, эти главы будут полезны для тех, кто хочет освежить в памяти эти понятия. Не утверждаю, что все мои определения являются общепринятыми, но как минимум они помогут всем прийти к единому пониманию, что облегчит обсуждение архитектур.

Я отнес реляционное хранилище данных и озеро данных к концепциям, а не к архитектурам. Когда-то давно, когда они были единственными продуктами, используемыми в решениях, их можно было считать архитектурами данных. Однако сейчас они почти всегда используются в сочетании с другими продуктами. Например, много лет назад существовали продукты для реляционных хранилищ данных, которые включали в себя реляционное хранилище, вычислительные системы, программы для операций ETL (извлечение, преобразование и загрузка) и программы для создания отчетов — в общем, все, что вам нужно, от одного поставщика. Сейчас, чтобы создать законченную архитектуру данных, необходимо объединить несколько продуктов, возможно, от разных поставщиков, при этом каждый продукт будет иметь определенную направленность (к примеру, ETL).

ГЛАВА 4

Реляционное хранилище данных

К середине 2000-х годов я уже много лет пользовался реляционными базами данных, но никогда не сталкивался с реляционными хранилищами данных. Я работал администратором БД в компании, которая использовала пакет бухгалтерских программ для управления своими финансовыми операциями. Отчетность, формируемая этим пакетом, была ограниченной и медленной. Поэтому компания искала возможности для повышения производительности, создания дашбордов для анализа данных по различным параметрам и объединения финансовых данных с данными из собственного приложения, чтобы лучше понимать свой бизнес.

Мой работодатель нанял консалтинговую компанию для создания какой-то штуки под названием «реляционное хранилище данных» и, приняв решение, изменившее ход моей карьеры, попросил меня помочь. Мы создали дашборды, которые сэкономили конечным пользователям уйму времени и добавили бизнес-информацию, о которой они раньше даже не догадывались. Когда я увидел восторг на их лицах, я понял, что нашел свое новое призвание. Я круто изменил свою карьеру, сосредоточившись на хранилищах данных, и ни разу не пожалел об этом.

Что такое реляционное хранилище данных?

Реляционное хранилище данных (RDW, relational data warehouse) — это место, где можно централизованно хранить и управлять большими объемами структурированных данных из разных источников для составления отчетов по ретроспективному и трендовому анализу, чтобы компания могла принимать более обоснованные бизнес-решения. Оно называется *реляционным* потому, что базируется на *реляционной модели* — широко распространенном подходе к представлению и организации данных в базах данных. В реляционной модели данные организованы в таблицы (также известные как отношения (relations), отсюда и название). Эти таблицы состоят из строк и столбцов, где каждая строка представляет сущность (например, клиента или продукт), а каждый столбец — атрибут этой сущности (например, название, цена или количество). Оно называется *хранилищем данных*, поскольку собирает, хранит и управляет огромными объемами структурированных данных из разных источников, таких как транзакционные базы данных, прикладные системы и внешние каналы данных.

Не все хранилища данных основаны на реляционной модели. К *нереляционным хранилищам* относятся такие типы, как колоночные, NoSQL и графовые хранилища данных. Однако реляционные хранилища гораздо более популярны и широко распространены, прежде всего потому, что реляционные БД на протяжении десятилетий были доминирующей парадигмой управления данными. Реляционная модель хорошо подходит для структурированных данных, которые обычно встречаются в бизнес-приложениях. Она также популярна благодаря широкому распространению языка SQL, который уже много лет является стандартом для реляционных хранилищ данных.

RDW выступает в качестве центрального репозитория для многих предметных областей и содержит *единую версию истины* (SVOT, single version of truth). SVOT — это важнейшее понятие в области хранилищ данных, оно относится к практике создания единого, согласованного представления данных организации. Это означает, что все данные в хранилище находятся в стандартизированном, структурированном формате и представляют собой единую, точную версию информации. Такой подход гарантирует, что все пользователи получают доступ к одной и той же информации, исключая любые расхождения или несоответствия и устраняя разрозненность данных. В свою очередь, это повышает эффективность принятия решений и взаимодействия, а также результативность работы всей организации. Кроме того, снижается риск ошибок и недопониманий, которые могут возникнуть при работе с разрозненными и несогласованными источниками данных.

Представьте, что у вас нет хранилища данных и вы создаете отчеты напрямую из нескольких систем-источников и, возможно, даже из каких-то файлов Excel. Если у того, кто просматривает отчет, возникнет сомнение в точности данных, что вы ему ответите? «Истина» может быть разбросана по многим системам-источникам, и будет трудно отследить, откуда взялись данные. Кроме того, некоторые отчеты будут выдавать разные результаты для одних и тех же данных — например, если в двух отчетах используется сложная логика для извлечения данных из нескольких источников, и эта логика обновляется неправильно (или вообще не обновляется). Размещение всех данных в одном централизованном месте хранения означает, что хранилище данных является единым источником истины; на любые вопросы по данным отчета можно получить ответ из хранилища данных. Поддержание SVOT необходимо организациям, стремящимся использовать весь потенциал своих данных.

Если хранилище данных (DW, data warehouse) используется всей компанией, его часто называют *корпоративным хранилищем данных* (EDW, enterprise data warehouse). Это более полная и надежная версия хранилища, предназначенная для удовлетворения потребностей всей организации. В то время как стандартное хранилище может обслуживать только несколько бизнес-подразделений, и для всей организации их будет множество, то EDW использует более широкий спектр источников и типов данных для обслуживания сразу всех бизнес-подразделений. EDW обеспечивает единое, унифицированное представление всех данных организации.

Рис. 4.1 иллюстрирует одну из основных причин, по которой необходимо иметь хранилище данных. На диаграмме слева показано, как сложно создавать отчет, используя данные из нескольких приложений, если у вас нет единого хранилища. Каждый отдел составляет отчет, который собирает данные из всех баз данных, связанных с отдельными приложениями. Выполняется слишком много запросов, что неизбежно приводит к проблемам с производительностью и появлению некорректных данных. Начинается полный хаос. На диаграмме справа видно, что, когда все данные приложений скопированы в общее EDW, каждому отделу становится очень легко создавать отчет без ущерба для производительности.

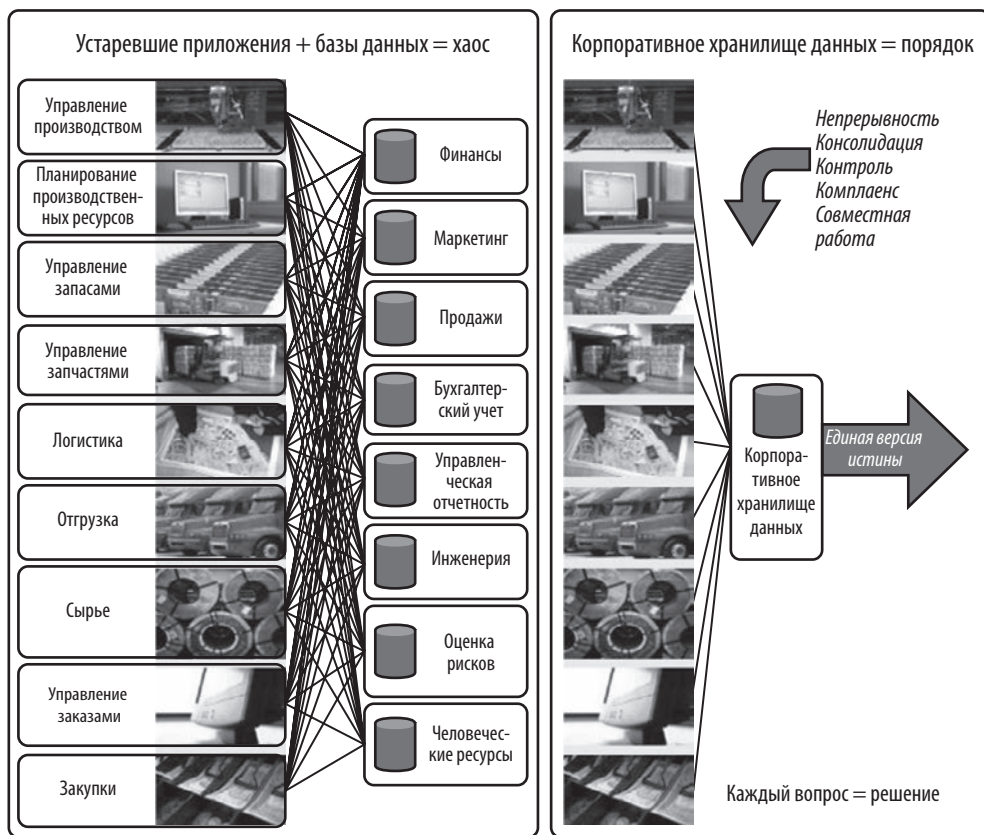


Рис. 4.1. До и после создания корпоративного хранилища данных

Как правило, для хранилища данных создаются конвейеры данных, которые выполняют три этапа — извлечение, преобразование и загрузку (ETL).

1. Конвейер извлекает данные из исходных систем, таких как базы данных и неструктурированные файлы.

2. Затем извлеченные данные преобразуются или обрабатываются так, чтобы соответствовать требованиям целевой системы (в данном случае — хранилища данных). Этот процесс может включать в себя очистку, фильтрацию, агрегирование или объединение данных из нескольких источников.
3. Преобразованные данные загружаются в хранилище данных. Администратор базы данных может переименовать саму базу данных и ее поля более осмысленным образом, чтобы конечным пользователям было проще и быстрее создавать отчеты.

Чем хранилище данных не является

Теперь, когда вы знаете, что такое хранилище данных, проясним его назначение, рассмотрев решения, которые не следует считать хранилищем данных (хотя я не раз замечал, что некоторые так делают).

Префикс DW

Хранилище данных — это не просто копия исходной базы данных из действующей системы с добавлением префикса DW к имени файла. Например, предположим, что вы скопировали базу данных под названием *Finance*, содержащую 50 операционных таблиц, и назвали копию *DW_Finance*, а затем используете эти 50 таблиц для создания отчетов. Это приведет к тому, что хранилище данных будет предназначено для *оперативных* данных, в то время как вам нужно было, чтобы оно предназначалось для *аналитических*. Для аналитических данных выше производительность чтения и можно создавать модели данных, упрощающие конечным пользователям отчетность. (Более подробно я расскажу об этом в следующем разделе.)

Представления с объединениями

Хранилище данных — это не копия нескольких таблиц из разных систем-источников, объединенных в SQL-представление (когда *объединение* выполняется с помощью SQL-оператора *UNION*, который объединяет результаты двух или более операторов *SELECT* в один набор результатов). Например, если вы скопировали данные из трех систем-источников, каждая из которых содержит клиентов, то в итоге в хранилище данных появятся три таблицы с именами *CustomerSource1*, *CustomerSource2* и *CustomerSource3*. Поэтому вам нужно будет создать представление под названием *CustomerView*, в котором оператор *SELECT* объединяет таблицы *CustomerSource1*, *CustomerSource2* и *CustomerSource3*. Этот процесс нужно будет повторить и для других таблиц, таких как товары и заказы.

Вместо этого данные из трех таблиц следует скопировать в одну таблицу в хранилище данных, что потребует дополнительной работы по созданию модели данных, которая будет подходить для всех трех таблиц. Скорее всего, на этом этапе вы используете управление мастер-данными (MDM), о котором

рассказывается в главе 6, чтобы избежать дублирования и повысить доступность и производительность.

Свалка для мусора

Хранилище данных — это не свалка таблиц. Часто так бывает, когда у компании нет хранилища данных, а конечный пользователь хочет создать отчет на основе подмножества данных из нескольких систем-источников. Чтобы быстро помочь им, ИТ-специалист не задумываясь создает хранилище данных, копируя в него данные из этих двух систем-источников. Затем другие пользователи, оценив преимущества первого, захотят получать дополнительные данные из тех же систем-источников и еще из нескольких других для создания собственных отчетов. И снова ИТ-специалист быстро копирует запрошенные данные в хранилище. Этот процесс повторяется снова и снова, пока оно не превращается в беспорядочную мешанину баз данных и таблиц.

Многие хранилища данных начинаются как одноразовые решения для пары пользователей, а затем превращаются в полноценные, но плохо спроектированные решения для всей компании. Есть способ лучше.

Когда поступает первый запрос от конечного пользователя, оцените потребности вашей компании в отчетности. Выясните, является ли запрос действительно разовым или он должен стать началом создания EDW. Если да, то это ваш шанс показать высшему руководству, почему вашей компании нужно хранилище данных. Если оно действительно нужно, будьте непреклонны: вам потребуется достаточно времени для разработки хранилища данных, которое сможет поддерживать множество источников данных и конечных пользователей. (Используйте аргументы из раздела «Почему стоит использовать реляционное хранилище данных?», представленные на стр. 77, для обоснования своего решения.)

Нисходящий подход к проектированию (сверху вниз)

Реляционное хранилище данных (RDW) требует большой предварительной работы, чтобы доставить данные туда, где их можно будет использовать для создания отчетов. Методология, когда вся эта работа выполняется заранее, называется *нисходящим подходом* или проектированием «сверху вниз». Этот подход хорошо работает для ретроспективных отчетов, которые нужны, чтобы определить, что произошло (*описательная*, или *дескриптивная, аналитика*) и почему (*диагностическая аналитика*). При нисходящем подходе вы сначала определяете общий план, дизайн и архитектуру хранилища данных, а затем разрабатываете конкретные компоненты. Этот метод подразумевает в первую очередь формирование общекорпоративного видения и понимания стратегических целей и информационных потребностей организации до начала разработки хранилища данных.

Описательная и диагностическая аналитика — два важных вида анализа данных, которые широко используются в бизнесе. Описательная аналитика предполагает

анализ данных для описания прошлых или текущих событий, зачастую просто с помощью сводной статистики или визуализации данных. Этот вид анализа используется для понимания того, что происходило в прошлом, а также для выявления закономерностей или тенденций в данных, которые могут помочь в процессе принятия решений.

Диагностическая аналитика используется для исследования причин прошлых событий, как правило, путем изучения взаимосвязей между различными переменными или факторами. В рамках этого вида анализа можно выявить первопричины проблем или определить события и факторы, которые могут повлиять на эффективность бизнеса.

Предположим, компания хочет проанализировать данные о продажах за прошлый год. Описательная аналитика будет включать в себя расчет сводных статистических данных, таких как общая выручка с продаж, средний объем продаж в день и продажи по категориям продуктов, чтобы понять общую картину. Диагностическая аналитика, со своей стороны, будет изучать взаимосвязи между факторами (например, между продажами и маркетинговыми расходами или сезонностью и демографическими характеристиками клиентов), чтобы понять, почему продажи колебались в течение года. Комбинируя оба подхода, компании могут обеспечить более глубокое понимание своих данных и принимать более обоснованные решения.

На рис. 4.2 показана архитектура типичного RDW. ETL используется для загрузки данных из различных источников в RDW, где можно создавать отчеты и проводить анализ данных.

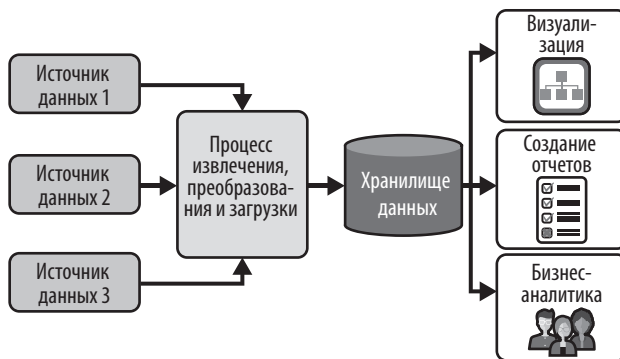


Рис. 4.2. Архитектура хранилища данных

Нисходящий подход обычно включает следующие шаги.

1. *Предварительная формулировка нескольких гипотез*

Начните с четкого понимания корпоративной стратегии. Затем убедитесь, что вы знаете, на какие вопросы хотите получить ответы с помощью данных.

2. *Определение бизнес-требований*

Выясните цели, задачи и ключевые показатели эффективности организации. Соберите и проанализируйте информационные потребности различных отделов и пользователей. На этом шаге также можно определить требования к формированию отчетов.

3. *Дизайн архитектуры хранилища данных*

На основе бизнес-требований создайте высокоуровневую архитектуру хранилища данных, включая его структуру, модели данных и процессы интеграции данных. Это будут ваши технические требования.

4. *Разработка модели данных*

Спроектируйте подробную модель данных для хранилища данных, учитывая взаимосвязи между различными сущностями данных и детализацию данных.

5. *Создание архитектуры*

Разработайте соответствующие базы данных, схемы, таблицы и поля для хранилища данных. Это описанный выше подход «схема при записи» (schema-on-write).

6. *Разработка ETL*

Разработайте процессы ETL для извлечения данных из различных систем-источников, преобразования их в нужный формат и загрузки в хранилище данных.

7. *Разработка и развертывание BI-инструментов и приложений*

Создайте BI-инструменты и приложения, которые позволят пользователям получать доступ к данным, анализировать их и создавать отчеты на основе данных, находящихся в хранилище данных.

8. *Тестирование и доработка хранилища данных*

Проведите тестирование для обеспечения качества, высокой производительности и надежности данных. Внесите все необходимые изменения для оптимизации системы.

9. *Поддержка и расширение хранилища данных*

По мере роста потребностей организации обновляйте и расширяйте хранилище данных в соответствии с этими потребностями.

Нисходящий подход имеет ряд преимуществ, таких как комплексное представление о потребностях организации в данных, лучшая согласованность данных и более эффективное управление. Однако он также может оказаться более трудоемким и ресурсоемким, требуя больше времени на извлечение ценности из данных, чем восходящий подход («снизу вверх»), используемый в озере данных, речь о котором пойдет в главе 5. Современная архитектура хранилища данных, описываемая в главе 10, сочетает в себе оба этих подхода — нисходящий и восходящий.

Почему стоит использовать реляционное хранилище данных?

RDW значительно упрощает создание BI-решений любого типа, поскольку BI-решения могут извлекать данные прямо из RDW без необходимости создавать сложную логику для получения данных из нескольких систем-источников. Кроме того, не придется очищать или объединять данные, поскольку RDW уже сделает это. BI-решение, созданное на основе RDW, может представлять собой витрину данных (которая содержит подмножество данных RDW для определенной группы людей, как описано в главе 6), агрегировать данные для ускорения выполнения запросов и формирования отчетов и даже быть пригодным для использования в Microsoft Excel. Суть в том, что RDW предоставляет прочный фундамент, на который можно опираться.

Рассмотрим подробно некоторые из основных преимуществ использования RDW.

Снижение нагрузки на рабочую систему

Возможно, вы уже сталкивались с такой проблемой: разгневанный пользователь звонит и жалуется, что добавление заказа через приложение занимает целую вечность. Вы начинаете разбираться, и выясняется, что другой пользователь одновременно запускает процесс формирования отчета, который и отнимает все ресурсы сервера, где находится приложение. Такая ситуация особенно распространена, когда конечным пользователям разрешено создавать специальные запросы и они используют плохо написанный SQL-код.

Скопировав базу данных приложения для оформления заказов в хранилище данных и оптимизировав ее, вы можете заставить все отчеты и специальные запросы обращаться к хранилищу и полностью избежать этой проблемы, особенно если пользователю необходимо запустить отчет, который работает с несколькими базами данных приложения.

Оптимизация доступа на чтение

Базы данных приложений обычно оптимизированы для поддержки всех CRUD-операций в равной степени, поэтому чтение данных будет не таким быстрым, как могло бы быть. С другой стороны, хранилище данных — это система типа «однократная запись, многократное считывание», то есть оно будет использоваться в основном для чтения данных. Поэтому его можно оптимизировать под доступ на чтение, особенно при длительном последовательном сканировании диска, которое часто происходит при выполнении отчетов или запросов. Существует множество методов работы с базами данных, которые можно использовать для ускорения доступа на чтение в хранилище данных, в некоторых случаях в ущерб доступу на запись, что нас в принципе не волнует.

Интеграция нескольких источников данных

Возможность интегрировать множество источников данных для создания более полезных отчетов является одним из наиболее важных аргументов в пользу создания хранилища данных. Размещение всех данных в одном месте вместо их распределенного хранения в различных базах данных не только упрощает создание отчетов, но и значительно повышает производительность этого процесса.

Создание точных ретроспективных отчетов

Без хранилища данных конечные пользователи приложений обычно запускают создание всех своих отчетов в определенный день каждого месяца (обычно в последний). Затем они сохраняют их на диске, чтобы иметь копии, к которым можно обратиться в будущем. Например, пользователь хочет просмотреть отчет за несколько месяцев, в котором указаны продажи клиентам в разбивке по штатам. Однако один из клиентов недавно переехал в другой штат. Если пользователь выведет текущий отчет, то продажи этому клиенту попадут в новый штат (поскольку его запись в базе данных была обновлена в соответствии с новым штатом), а не в старый, что неверно. Следовательно, пользователь должен обратиться к сохраненному старому отчету, а не выводить текущий.

Хранилище данных может позаботиться об этом, отслеживая, когда клиент переезжает (записывая историю местонахождения клиента с датами начала и окончания его пребывания), а также любые другие поля, которые необходимо отслеживать (например, работодатель или доход). Поэтому пользователь может вывести отчет сегодня, но попросить извлечь данные на определенную дату в прошлом, и отчет будет точным. Более того, отпадает необходимость сохранять файлы отчетов каждый месяц.

Реструктуризация и переименование таблиц

Во многих базах данных приложений имена таблиц и полей очень сложны для понимания, особенно в старых продуктах ERP и CRM (например, такие наименования таблиц, как T116, и имена полей вроде RAP16). В хранилище данных можно скопировать данные из этих исходных таблиц в таблицу с более осмысленными названиями (например, Customer (Клиент) вместо T116). Вероятно, вы также сможете подобрать лучшую модель данных для всех таблиц. Конечным пользователям будет гораздо проще создавать отчеты, если им не придется расшифровывать загадочные названия таблиц и полей.

Защита от обновлений приложения

Представьте, что у вас нет хранилища данных, а пользователи создают отчеты на основе базы данных приложения. Все работает нормально, и вдруг многие отчеты начинают выдавать ошибки. Оказывается, приложение было обновлено и установлена новая версия, которая переименовала кучу таблиц и полей. Так что теперь вам нужно просмотреть сотни отчетов и вручную переименовать

измененные таблицы и поля. На это могут уйти месяцы, и множество пользователей будут недовольны. Но даже после этого в отчетах, которые вы пропустили при этой ручной работе, будут сохраняться ошибки.

Хранилище данных может защитить вас от этого. После обновления приложения потребуется обновить только ETL, который копирует данные из баз данных приложения в хранилище, и это несложно. Отчеты менять не нужно. Конечные пользователи не увидят новых данных до тех пор, пока ETL-процесс не будет исправлен, но в их отчетах не будет ошибок.

Повышение уровня безопасности

Без хранилища данных ИТ-команде пришлось бы предоставлять каждому конечному пользователю защищенный доступ к каждой базе данных приложения, которая им необходима для создания отчетов. Таких баз данных могут быть десятки; процесс предоставления доступа может занять несколько недель, и иногда у пользователя все равно не будет доступа ко всему, что ему нужно. С использованием хранилища данных каждому конечному пользователю нужен доступ только к соответствующим таблицам, и предоставить его гораздо проще и быстрее.

Хранение исторических данных

Многие рабочие системы ограничивают объем хранимых исторических данных (например, данные только за последние три года). Это делается для экономии места и повышения производительности, а в некоторых случаях — для соблюдения нормативных требований. Старые данные обычно удаляются ежегодно или ежемесячно. С другой стороны, хранилище данных может хранить всю историю, поэтому вам не придется беспокоиться о том, что вы запустите отчет за прошлые годы и не найдете никаких данных.

Управление мастер-данными (MDM)

При сборе данных из нескольких систем-источников часто возникает необходимость использовать MDM для удаления дублирующихся записей для таких объектов, как клиенты, продукты и активы. (Более подробное описание MDM см. в главе 6.) Хранилище данных идеально подходит для MDM. Кроме того, многие инструменты MDM позволяют создавать иерархии (например, компания → отдел → сотрудник), что повышает ценность MDM.

Повышение качества данных путем устранения дефектов в системах-источниках

Вы обнаружите, что многие данные, получаемые из разных систем-источников, нуждаются в очистке, несмотря на то что говорят владельцы приложений (я много раз слышал: «У нас чистые данные», а потом оказывалось, что это не так). Например, приложение для оформления заказов может потребовать дату рождения клиента, и, если человек, вводящий данные, не знает ее, он может

ввести дату в будущем или дату более чем 100-летней давности, просто чтобы иметь возможность завершить заказ. Или, может быть, приложение не проверяет корректность двухбуквенного кода штата. В системе-источнике всегда найдется десяток подобных дефектов. Вы можете не только очистить данные в хранилище, но и сообщить тем, кто обслуживает приложения, об ошибках в их системах, чтобы они могли их исправить. Таким образом, вы поможете предотвратить ввод неверных данных в будущем.

Исключение участия ИТ-специалистов в создании отчетов

Это относится к бизнес-аналитике самообслуживания, о которой говорилось в главе 3: правильное хранилище данных исключит необходимость привлечения ИТ-специалистов к созданию отчетов и оставит эту задачу конечному пользователю. Без такого узкого места, как ограниченные ИТ-ресурсы, отчеты и дашборды можно будет создавать быстрее. А ИТ-специалисты будут благодарны за то, что могут работать над более интересными проектами, чем создание отчетов!

Недостатки использования реляционного хранилища данных

Во всем есть плюсы и минусы, поэтому перечислим недостатки, которые следует учитывать при создании RDW.

Сложность

Хранилища данных могут быть сложными и требующими много времени на проектирование, создание и обслуживание. Необходимые для этого специальные навыки и ресурсы могут привести к увеличению общих затрат.

Высокая стоимость

Внедрение хранилища данных может оказаться дорогостоящим и требующим значительных инвестиций в аппаратное и программное обеспечение, а также в персонал. Текущее обслуживание и обновления также могут привести к увеличению общей стоимости.

Проблемы интеграции данных

Интеграция данных из разных источников может представлять собой довольно сложную задачу, поскольку при этом приходится иметь дело с разными форматами и структурами данных, а также решать проблемы с их качеством. Это влечет за собой затраты времени и ресурсов на очистку и предварительную обработку данных. Кроме того, некоторые данные, например потоковые с IoT-устройств, слишком сложны для непосредственной загрузки в RDW, поэтому возможность аналитики этой информации теряется.

Длительный процесс преобразования данных

Для загрузки данных в хранилище может потребоваться их преобразование в соответствии с моделью данных хранилища. Этот процесс может занять много времени, а ошибки могут привести к неправильному анализу.

Задержка при извлечении данных

Поскольку хранилища данных предназначены для работы с большими объемами данных, процесс обработки может идти медленнее, чем в других базах данных. Это может привести к задержке при извлечении данных, когда данные в хранилище окажутся неактуальными и не будут соответствовать последним изменениям в источнике.

Окно обслуживания

При использовании RDW обычно необходимо окно обслуживания. Загрузка и очистка данных требуют больших затрат ресурсов, и, если пользователи одновременно попытаются формировать отчеты, производительность значительно снизится. Поэтому пользователи не должны иметь доступа в хранилище на время обслуживания, что исключает круглосуточный (24/7) доступ. Если в течение периода обслуживания возникнут проблемы, например сбой ETL-задания, возможно, придется продлить этот период. Если в это время пользователи попытаются сформировать отчеты, а доступ по-прежнему будет заблокирован, вы столкнетесь с недовольством людей, которые не могут выполнять свою работу.

Ограниченная гибкость

Хранилища данных предназначены для поддержки конкретных видов анализа, что может ограничить их возможности при применении других видов обработки или анализа данных. Для выполнения определенных задач может потребоваться интеграция дополнительных инструментов или систем с хранилищем.

Проблемы с безопасностью и конфиденциальностью

Централизованное хранение больших объемов чувствительных данных может повысить риск утечки данных и нарушения конфиденциальности, что требует принятия строгих мер безопасности.

Наполнение хранилища данных

Поскольку исходные таблицы, поступающие в хранилище данных, со временем меняются, хранилище должно отражать эти изменения. Это звучит достаточно просто, но требует принятия множества решений: как часто следует *извлекать* (или вытягивать (pull)) данные, какой метод извлечения использовать, как физически извлекать данные и как определять, какие данные изменились с момента последнего извлечения. Я кратко расскажу о каждом из этих действий.

Как часто извлекать данные

Частота обновления хранилища данных во многом зависит от частоты обновления систем-источников и от того, насколько актуальной должна быть отчетность, необходимая конечному пользователю. Часто конечным пользователям не нужны данные за текущий день и достаточно данных на конец предыдущего дня. В этом случае можно запускать задания по извлечению данных из систем-источников с помощью инструментов ETL каждую ночь после завершения обновления баз данных источников, создавая ночное окно обслуживания для переноса всех данных. Если конечным пользователям требуются данные, обновляемые в течение дня, то необходимо извлекать данные чаще, например каждый час.

Одним из факторов, которые следует учитывать, является объем данных при каждом извлечении. Если он очень велик, обновление хранилища может занять слишком много времени, поэтому лучше разбить этот процесс на более мелкие фрагменты и выполнять извлечения и обновления чаще (например, каждый час, а не раз в день). Кроме того, передача больших объемов данных из источников в хранилище может занять слишком много времени, особенно если исходные данные находятся в локальной сети, а пропускная способность канала передачи данных из системы-источника в интернет невысока. Это еще одна причина, по которой вам, возможно, захочется перейти от длительной ночной передачи данных к более частым и коротким передачам в течение дня.

Методы извлечения

Существуют два метода извлечения данных из систем-источников. Рассмотрим каждый из них в отдельности.

Полное извлечение

Все данные полностью извлекаются из одной или нескольких таблиц системы-источника. Этот метод лучше всего подходит для небольших таблиц. Поскольку такое извлечение задействует все данные, доступные на данный момент в источнике, отслеживать изменения не требуется, что делает этот метод очень простым в реализации. Исходные данные предоставляются как есть, и вам не нужна никакая дополнительная информация (например, метки времени).

Инкрементное извлечение

Извлекаются только те данные, которые изменились с указанного момента (например, с момента последнего извлечения или с окончания финансового периода), а не вся таблица. Этот метод лучше всего подходит для больших таблиц и работает только в том случае, если можно идентифицировать всю измененную информацию (об этом ниже).

Для большинства систем-источников используется комбинация этих двух методов.

Независимо от того, выполняете ли вы полное или инкрементное извлечение, существуют два способа извлечения данных: онлайн и офлайн.

При онлайн-извлечении процесс подключается непосредственно к системе-источнику для доступа к исходным таблицам или к промежуточной системе, которая хранит изменения данных в предварительно заданном виде (например, в логах транзакций или таблицах изменений).

Однако прямой доступ к исходной системе не всегда возможен. В этом случае данные размещаются вне исходной системы-источника и создаются с помощью процедуры извлечения из исходной системы (например, мейнфрейм выполняет процедуру извлечения из таблицы и помещает данные в папку в файловой системе). Извлеченные данные обычно помещаются в простой неструктурированный файл определенного универсального формата (например, CSV или JSON).

Как определять, какие данные изменились с момента последнего извлечения

К сожалению, во многих системах-источниках может быть сложно определить недавно измененные данные и выполнить инкрементное извлечение. Ниже приведены несколько способов идентификации недавно измененных данных и реализации инкрементного извлечения. Эти способы могут работать в сочетании с рассмотренными методами извлечения данных. Некоторые из них основаны на использовании определенных характеристик систем-источников; другие могут потребовать внесения изменений в источник. Владельцы систем-источников должны тщательно оценить каждую методику перед ее внедрением.

Метки времени

Наиболее предпочтительный и самый простой в реализации вариант. В таблицах некоторых действующих систем есть столбцы меток времени, содержащие время и дату последнего изменения заданной строки, что позволяет легко идентифицировать последние данные. В реляционных базах данных столбцу метки времени часто присваивается тип данных `time stamp` или `datetime`, а также соответствующее название столбца, например `Timestamp` или `Last Modified`. Приложение-источник будет самостоятельно заполнять этот столбец. В противном случае должна быть возможность настроить реляционную базу данных так, чтобы по умолчанию при сохранении записи использовалась текущая дата, или можно добавить триггеры базы данных для автоматического заполнения этого столбца.

Захват изменений данных

Большинство реляционных баз данных поддерживают функцию *захвата изменений данных* (CDC, `change data capture`), которая записывает все операции `INSERT`, `UPDATE` и `DELETE`, применяемые к таблицам базы данных, и делает доступной запись в таблице о том, что, где и когда изменилось, на основе журнала транзакций реляционной базы данных. Если вам требуется хранилище данных, работающее в режиме, близком к реальному времени, когда изменения

в источнике отражаются в хранилище данных в течение нескольких секунд, CDC может стать отличным вариантом, обеспечивающим такую возможность.

Партиционирование

В некоторых исходных системах используется партиционирование по диапазонам, когда исходные таблицы разделяются по ключу даты, что облегчает идентификацию новых данных. Например, если вы извлекаете данные из таблицы заказов, партиционированной по дням, легко определить данные за текущий или предыдущий день.

Триггеры базы данных

Можно добавить триггер для `INSERT`, `UPDATE` и `DELETE` в одну таблицу, чтобы эти триггеры записывали информацию об изменении записи в так называемую таблицу изменений. Это похоже на захват изменений данных, поэтому используйте CDC, если ваш продукт базы данных поддерживает его; в противном случае применяйте триггеры.

Оператор MERGE

Наименее предпочтительный вариант — выполнить полное извлечение из системы-источника в промежуточную область в хранилище данных, а затем сравнить эту таблицу с предыдущим полным извлечением из системы-источника с помощью оператора `MERGE`, чтобы идентифицировать измененные данные. В этом случае потребуется сравнить все поля источника со всеми полями назначения (или использовать хеш-функцию). Такой подход, скорее всего, не окажет существенного влияния на систему-источник, но может создать значительную нагрузку на хранилище данных, особенно если объемы данных велики. К этому варианту обычно прибегают в последнюю очередь, если другие способы невозможно реализовать.

Слухи о смерти реляционных хранилищ данных сильно преувеличены

Примерно в начале 2010-х годов ИТ-специалисты начали задаваться вопросом: «Живы ли еще реляционные хранилища данных?» Многие расценили это как сомнение, нужны ли они еще бизнесу. Как отмечается в этой главе, все-таки нужны. Но на самом деле вопрос касается архитектуры хранилищ данных — можно ли использовать только озеро данных или необходимы и озеро данных, и реляционное хранилище?

Когда озера данных только появились, они были построены на базе технологии Apache Hadoop, и именно поставщики Hadoop объявили, что эра реляционных хранилищ закончилась. «Просто поместите все свои данные в озеро данных и избавьтесь от реляционного хранилища», — советовали они. Как уже говорилось в главе 2, все проекты, которые пытались это сделать, потерпели неудачу.

В течение многих лет я придерживался мнения, что реляционные хранилища будут нужны всегда, потому что все озера данных создавались на основе Hadoop и имели слишком много ограничений. Но когда появились такие решения, как Delta lake (см. главу 12), и озера данных стали использовать более совершенные и простые продукты, чем Hadoop (см. главу 16), я пришел к выводу, что в некоторых сценариях можно обойтись без реляционного хранилища. Так работает архитектура озера-хранилища данных, которая будет рассмотрена в главе 12.

Однако во множестве сценариев использования реляционное хранилище необходимо. И хотя технологии озера данных будут продолжать совершенствоваться, тем самым снижая или совсем исключая сомнения по поводу отказа от RDW (см. главу 12), мы никогда полностью от него не откажемся. На мой взгляд, на то есть три причины. Во-первых, создавать отчеты из озера данных все еще сложнее, чем из хранилища. Во-вторых, RDW продолжают удовлетворять информационные потребности пользователей и приносить пользу. В-третьих, многие люди используют реляционные хранилища, полагаются на них и доверяют им и не хотят заменять их озерами данных.

Озера данных предлагают богатый источник данных для специалистов data science и потребителей самообслуживаемых данных («продвинутых пользователей»), и они вполне удовлетворяют потребностям в сферах аналитики и работы с Big Data. Но не всем специалистам, работающим с данными и информацией, требуется быть продвинутыми пользователями. Большинству по-прежнему нужны хорошо интегрированные, систематически очищаемые, легкодоступные реляционные данные, включающие журнал истории событий, где фиксируется развитие событий за определенный период времени. А для этих целей лучше всего подходит хранилище данных.

Итоги

В этой главе мы рассмотрели первое массовое технологическое решение для централизации данных из нескольких источников и создания отчетов: реляционное хранилище данных (RDW). RDW произвело революцию в способах управления данными на предприятиях и в организациях, предоставив централизованный репозиторий для хранения и извлечения данных. Это позволило более эффективно управлять данными и анализировать их. Благодаря возможности хранить и упорядочивать данные структурированным образом RDW дают возможность пользователям быстро и легко создавать сложные запросы и отчеты, предоставляя ценную информацию и помогая принимать критически важные решения.

Сегодня RDW остается основополагающим компонентом многих архитектур данных и может использоваться в самых разных сферах — от финансов и здравоохранения до розничной торговли и производства. В следующей главе рассматривается технология, которая должна занять важнейшее место для централизации данных и создания отчетов на их основе, — озеро данных.

Озеро данных

Big Data появились и начали беспрецедентный рост в начале 2010-х годов в связи с увеличением количества источников, которые выдают полуструктурированные и неструктурированные данные, таких как датчики, видео и социальные сети. Полуструктурированные и неструктурированные данные обладают феноменальной ценностью — только подумайте, сколько информации содержится в электронных письмах клиентов за несколько лет! Однако реляционные хранилища данных в то время могли работать только со структурированными данными. Кроме того, они с трудом справлялись с большими объемами данных или с данными, поступающими достаточно часто, поэтому они не подходили для хранения таких типов данных. Это побудило ИТ-отрасль найти другое решение — озеро данных. Озера данных могут легко обрабатывать полуструктурированные и неструктурированные данные и управлять данными, которые часто поступают в систему.

Много лет назад я беседовал с аналитиками из крупной розничной сети, которые хотели получать данные из Twitter, чтобы узнать, что покупатели думают об их магазинах. Они знали, что покупатели не решатся высказывать претензии сотрудникам магазина, но быстро выложат жалобы в Twitter. Я помог им загрузить данные из Twitter в озеро данных и оценить настроение комментариев покупателей, классифицировав их как положительные, нейтральные или отрицательные. Когда аналитики прочитали негативные комментарии, то обнаружили необычно большое количество жалоб на примерочные — те были слишком маленькими, переполненными и недостаточно уединенными. В качестве эксперимента компания решила переделать примерочные в одном из магазинов. Через месяц после переделки аналитики обнаружили массу положительных комментариев о примерочных в этом магазине, а также зафиксировали 7%-ный рост продаж. Это побудило компанию переделать примерочные во всех своих магазинах, в результате чего продажи по всей стране выросли на 6 %, а прибыль увеличилась на миллионы. И все это благодаря озеру данных!

Что такое озеро данных?

Термин «*озеро данных*» — это метафора, описывающая концепцию хранения огромных объемов необработанных данных в их естественном формате. Подобно тому как озеро вмещает в себя воду, не меняя ее природу, так и озеро данных хранит в себе данные без необходимости их предварительного структурирования или обработки.

Кроме того, как в озерах обитают различные виды диких животных и растений, так и в озере данных хранятся различные типы данных (структурированные, полуструктурированные и неструктурированные) — в отличие от хранилища данных, где данные более структурированы и обработаны, как товары в бутылках или упаковках на складе.

Как только данные попадают в озеро, их необходимо очистить, объединить и, возможно, агрегировать, чтобы сделать полезными. Для этого к озеру данных необходимо подключить *вычислительные ресурсы* (то есть вычислительные мощности, необходимые для управления, обработки и анализа данных), которые будут извлекать и преобразовывать данные, а затем возвращать их обратно в озеро.

Почему следует использовать озеро данных?

Существует множество причин, по которым стоит использовать озеро данных, особенно в связке с RDW. Как уже упоминалось, озеро данных позволяет быстро сохранять данные без предварительной обработки — подход, называемый «схема при чтении» (см. главу 2). Это дает возможность гораздо быстрее получать доступ к данным, благодаря чему опытные пользователи смогут оперативно формировать отчеты (для скорейшего возврата инвестиций, ROI (return on investment)), а специалисты data science получают быстрый доступ к данным для машинного обучения моделей. Это также полезно для исследования данных. Если конечный пользователь просит вас скопировать исходные данные в хранилище, вы можете сперва быстро скопировать их в озеро данных и исследовать, чтобы убедиться в их ценности, прежде чем тратить силы на создание схемы данных в хранилище и написание ETL.

В главе 4 я уже упоминал, что обычно при работе с хранилищем (DW) используют ночное окно обслуживания, когда вы отключаете конечных пользователей, чтобы загрузить исходные данные в промежуточные таблицы хранилища. Затем данные очищаются и копируются в рабочие таблицы. Однако если при этом возникнут проблемы и обслуживание займет больше времени, чем ожидалось, то утром, когда конечные пользователи попытаются получить доступ к данным, они окажутся заблокированными. Лучше проводить преобразование данных в озере данных, а не в хранилище. Это дает следующие преимущества.

- Экономия средств, поскольку вычисления в хранилище обычно намного дороже, чем другие типы вычислений, которые можно использовать для данных в озере.
- Очень высокая производительность преобразований (при их необходимости) за счет множественных вычислений, каждое из которых может обращаться к разным папкам с данными и работать с ними параллельно (или параллельно с другими процессами работать с одними и теми же данными).
- Возможность уточнения данных гораздо большим количеством способов, чем в DW, благодаря гибкости использования разных типов вычислительных опций

для данных в озере. Например, можно использовать библиотеки кода с готовыми процедурами для выполнения преобразований, которые очень сложно или невозможно выполнить в хранилище данных с помощью языка SQL.

- Отсутствие необходимости в окне обслуживания хранилища, что позволяет использовать DW для запросов круглосуточно (24/7). Вы также избежите конкуренции пользователей за вычислительные ресурсы, когда попытаетесь запустить отчеты во время преобразования данных. Поскольку преобразования данных требуют много ресурсов, это реально замедляет оба процесса.

Озеро данных — это недорогой способ хранения неограниченного объема данных. В отличие от локального хранилища, облачные провайдеры предоставляют неограниченное пространство хранения, поэтому вам никогда не придется беспокоиться о том, что в озере данных закончится место. Кроме того, облачные хранилища относительно дешевы, а большинство облачных провайдеров предлагают несколько уровней хранения, что позволяет их клиентам экономить еще больше.

Озеро данных дает возможность собирать любые данные «на всякий случай», если вдруг они понадобятся в будущем. Такое *накопление данных* редко осуществляется в рамках DW, поскольку хранение данных в DW обычно обходится намного дороже, чем в озере данных, и в случае большого объема данных может повлиять на производительность. Кроме того, поскольку хранение данных в озере стоит дешево, данные редко удаляются, если только это не обусловлено нормативными требованиями.

Поскольку озеро данных выступает в качестве централизованного места хранения для всех субъектов данных, оно может быть «единой версией истины». Если все без исключения данные попадают в озеро данных до того, как будут скопированы в другие расположения, озеро данных может стать тем местом, куда можно вернуться, если возникнут вопросы к точности данных в других местах их использования, например в запросах, отчетах и дашбордах. Озеро данных также позволяет любому конечному пользователю легко получать доступ к любым данным из любого места и использовать их многократно для любых потребностей аналитики и сценариев использования по своему усмотрению. Когда вы используете озеро данных вместе с хранилищем данных, озеро данных становится единой версией истины вместо хранилища. Кроме того, озеро данных подходит для размещения потоковых данных, например от IoT-устройств, которые практически невозможно размещать в DW.

Озеро данных также выступает в роли онлайн-архива. Например, данные о заказах за последние три года можно хранить в DW, а данные о заказах старше трех лет — в озере данных. Большинство технологий, используемых для создания хранилищ данных, позволяют обращаться к данным и из озера данных с помощью SQL-запросов, поэтому пользователи по-прежнему смогут использовать данные из озера данных в отчетах (за счет более низкой производительности). Это позволит экономить средства и избежать нехватки места для хранения данных в DW.

Озеро данных — это также место, где можно создавать резервные копии данных из хранилища на случай, если потребуется восстановить данные в DW из-за случайного удаления, повреждения, неправильно составленного условия WHERE в запросе UPDATE и пр.

Кроме того, в озеро данных можно интегрировать данные, структурированные различным образом: от файлов CSV и JSON до документов Word и медиафайлов. Из этих файлов можно будет извлекать данные и получать больше пользы.

В хранилище данных необработанные данные, которые копируются из системы-источника в промежуточные таблицы, а затем преобразуются и загружаются в рабочие таблицы, обычно удаляются через день или два, чтобы сэкономить место в ограниченном реляционном хранилище. Проблема возникает, если ошибка ETL обнаруживается через много дней после выполнения ETL; если ETL-процесс нужно запустить заново, а исходные данные уже удалены, придется вернуться к системе-источнику, что, несомненно, повлияет на его производительность. Выполняя преобразования в озере данных, где хранение дешевле, вы можете хранить долгую историю необработанных данных и никогда не возвращаться к системе-источнику.

Восходящий подход к проектированию (снизу вверх)

Поскольку озеро данных представляет собой схему при чтении, то для того, чтобы начать использовать данные, не требуется много предварительной работы. Это очень удобно, если вы еще не знаете, какие вопросы задавать данным, — вы можете быстро изучить данные, чтобы определиться с вопросами. Это *восходящий подход*, или *подход «снизу вверх»*, показанный на рис. 5.2, когда вы собираете данные еще до создания теорий или гипотез. Он сильно отличается от нисходящего подхода («сверху вниз») при создании реляционного хранилища данных (см. рис. 5.1).



Рис. 5.1. Нисходящий подход



Рис. 5.2. Восходящий подход

Этот подход позволяет специалистам data science, которые обычно используют программы, работающие с данными, представленными в файлах, применять машинное обучение на основе данных из озера данных, чтобы определить, что произойдет (предиктивная аналитика) и как сделать так, чтобы это произошло (предписывающая аналитика).

Предиктивная аналитика использует данные, статистические алгоритмы и методы машинного обучения для прогнозирования будущих результатов на основе исторических данных. Она включает различные статистические методы, такие как интеллектуальный анализ данных (или дата-майнинг), предиктивное моделирование и машинное обучение, для анализа текущих и исторических фактов с целью составления прогнозов относительно будущих или других неизвестных событий. Это позволяет вам не просто реагировать, а действовать проактивно. Например, предиктивная аналитика может использоваться в здравоохранении для прогнозирования показателей повторной госпитализации пациентов; в розничной торговле для прогнозирования будущих продаж или в банковской сфере для прогнозирования невозврата кредитов.

Предписывающая (прескриптивная) аналитика идет на шаг дальше, чем предиктивная. Она использует алгоритмы оптимизации и моделирования, чтобы дать рекомендации по возможным результатам. Предписывающая аналитика не только предсказывает, что произойдет в будущем, но и предлагает действия, которые следует предпринять, чтобы повлиять на эти результаты. Ее цель состоит в том, чтобы предоставить рекомендации на основе прогнозируемых сценариев будущего для оптимизации процесса принятия решений. Предписывающая аналитика может предложить варианты решений, как воспользоваться будущей возможностью или снизить будущий риск, и проиллюстрировать последствия каждого варианта. Например, в логистике предписывающая аналитика может использоваться для поиска наилучших маршрутов доставки и даже предлагать альтернативные маршруты в случае неожиданного перекрытия дорог.

Поначалу озера данных использовались в основном для предиктивной и предписывающей аналитики, чтобы избежать трудностей, связанных с попытками провести расширенную аналитику с помощью традиционного хранилища данных. Но теперь озера данных используются гораздо шире, о чем говорится в разделе «Почему следует использовать озеро данных?» на стр. 87.

Если при исследовании вы обнаружите ценные данные в озере данных и захотите сделать их легкодоступными для конечных пользователей, вы всегда сможете провести моделирование данных позже, скопировав их в реляционное хранилище. *Моделирование данных* похоже на создание наброска примерной организации и понимания данных. Оно помогает определить, какие данные важны, как они связаны между собой и как их следует хранить и использовать в RDW. Озера данных имеют ограниченный набор инструментов для моделирования данных, в то время как в DW инструменты моделирования существуют уже много лет.

Лучшие практики проектирования озер данных

Проектирование озера данных должно занимать много времени. Я часто сталкиваюсь с тем, что компании не уделяют достаточно внимания проработке всех сценариев использования и впоследствии вынуждены перепроектировать и перестраивать их. Поэтому убедитесь, что вы продумали все источники данных, которые используете сейчас и будете использовать в дальнейшем, и понимаете объем, тип и скорость передачи данных. Затем изучите всю информацию, которую сможете найти, о проектировании озер данных и выберите подходящий для вашего сценария вариант.

Озеро данных, как правило, не навязывает определенную структуру поступающим в него данным; на самом деле это одна из его ключевых особенностей. Этим оно отличается от традиционной базы данных или хранилища данных, которые требуют, чтобы данные были предварительно структурированы или смоделированы. Однако, чтобы сделать данные пригодными для использования и не допустить превращения озера данных в «болото данных» (неорганизованное и неуправляемое скопление данных), важно применять определенные методы организации и управления. В этом разделе представлены некоторые лучшие практики для начала работы.

Первый метод заключается в логическом разделении озера данных на несколько слоев (иногда называемых зонами), соответствующих возрастающим уровням качества данных, как показано на рис. 5.3. В противном случае все файлы будут находиться в одной папке, что значительно ухудшит производительность, управляемость и безопасность.

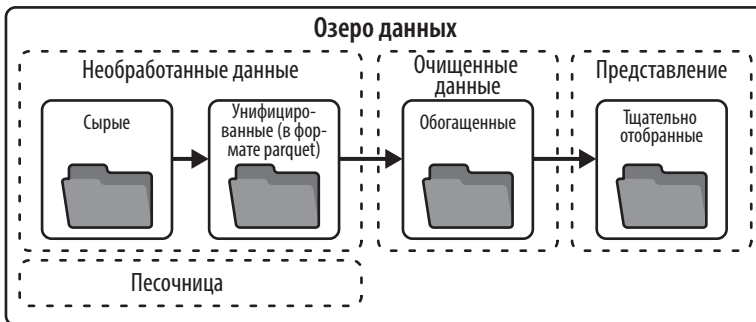


Рис. 5.3. Уровни озера данных

Чтобы лучше понять структуру и функции озера данных, разберем по отдельности все уровни, на каждом из которых повышаются качество данных и удобство их использования. Эти уровни представляют собой иерархическую структуру, начиная с сырых, необработанных данных и заканчивая тщательно переработанной

и готовой для применения информацией. Вот более подробное описание каждого уровня.

Уровень необработанных данных (raw layer)

Необработанные события, хранящиеся для составления исторических справок, обычно сохраняются навсегда (неизменяемые). Уровень необработанных данных — это своего рода резервуар, в котором хранятся данные в их естественном и исходном состоянии (без каких-либо преобразований). Он не фильтруется и не очищается. Этот уровень также называют *бронзовым* или *промежуточным слоем* (staging layer), а иногда *зоной приземления* данных (landing area).

Уровень унифицированных данных (conformed layer)

Часто в необработанном слое хранятся данные разных типов, таких как CSV, JSON и Parquet (распространенный формат файлов для хранения данных, оптимизированный под эффективную обработку Big Data). На уровне унификации все типы преобразуются в один формат, обычно в Parquet. Представьте себе, что озеро состоит из двух резервуаров — один с соленой, а другой с пресной водой — и с помощью опреснения соленая вода превращается в пресную. Этот слой также называют *базовым* или *уровнем стандартизированных данных*.

Уровень очищенных данных (cleansed layer)

Здесь необработанные события преобразуются в датасеты, пригодные для непосредственного использования (данные очищаются, интегрируются и консолидируются). Считайте, что уровень очищенных данных — это фильтрационный слой. Он удаляет примеси и может обогатить данные. Его цель состоит в том, чтобы сделать хранимые файлы единообразными с точки зрения их кодирования, схемы, формата и типа данных (например, строки и целые числа), а также содержимого. Этот слой также называют *серебряным, слоем трансформации данных, слоем переработанных, интегрированных, обработанных или обогащенных данных*.

Уровень представления данных (presentation layer)

К очищенным данным применяется бизнес-логика, чтобы получить данные, готовые к потреблению конечными пользователями или приложениями, обычно в формате, удобном для понимания и использования. Преобразование может включать в себя агрегирование или обобщение данных, а также приведение файлов к определенной схеме для использования инструментами создания отчетов (например, схемы «звезда», рассматриваемой в главе 8), часто с добавлением информации о данных (метаданных) в каждый файл. Этот уровень также называют *уровнем приложения, рабочим пространством, доверенным, золотым, безопасным, готовым к продакшену, управляемым, отобранных данных, обслуживаемым, аналитическим* или *уровнем потребления*.

Уровень песочницы (sandbox layer)

Этот необязательный слой используется, чтобы «поиграть» с данными. Как правило, песочница нужна специалистам data science. Обычно это копия слоя необработанных данных, где данные можно не только читать, но и изменять. Можно создать несколько слоев песочницы. Этот уровень также называют *исследовательским уровнем, слоем разработки или рабочим пространством data science*.

Это лишь один из способов организации уровней в озере данных. Я видел множество других отличных проектов, которые включают дополнительные уровни или комбинируют некоторые, например уровни унифицированных и очищенных данных.

В качестве примера использования этих уровней рассмотрим компанию розничной торговли, где данные поступают из разных источников, таких как POS-системы (point-of-sale — точка продаж), онлайн-продажи, отзывы клиентов и системы инвентаризации. Четыре уровня будут содержать данные разного качества, как показано ниже.

- Уровень необработанных данных
 - Необработанные логи POS-систем, включающие все детали транзакции (например, метку времени, приобретенные товары, их количество, цены, общую сумму, идентификатор кассира и местоположение магазина).
 - Данные онлайн-продаж с сайта или приложения, такие как идентификатор пользователя, приобретенные товары, их количество, цены, общая сумма и метки времени.
 - Инвентаризационные данные с различных складов и магазинов, включая товары, их количество и даты пополнения запасов.
 - Необработанные отзывы покупателей, собранные с помощью опросов, обзоров и рейтингов.
- Уровень унифицированных данных
 - Все файлы POS-систем, онлайн-продаж, инвентаризации и отзывов покупателей преобразуются в формат Parquet, если они исходно не представлены в этом формате.
- Уровень очищенных данных
 - Данные POS-систем и онлайн-продаж с удаленными или исправленными ошибками (например, несоответствий в наименованиях товаров, ошибок в количестве и отсутствием меток времени). Кроме того, данные на этом уровне уже преобразованы в общую схему (например, применены общие соглашения об именах и форматы наименования товаров, установлен

общий формат времени или общая система идентификации магазинов (ID магазинов)).

- Данные инвентаризации со стандартизированным наименованием товаров и исправленными ошибками или несоответствиями. Данные также преобразованы и приведены в соответствие с общей схемой, используемой в данных о продажах (названия товаров и идентификаторы магазинов).
 - Очищенные данные из отзывов клиентов (то есть из них удалены нерелевантные или ошибочные ответы, они приведены к стандартным форматам и т. д.). Кроме того, данные преобразованы в общий формат, соответствующий общей системе идентификации магазинов и, возможно, с общими элементами, извлеченными из отзывов для анализа.
- Уровень представления данных
- Сводный отчет о продажах, показывающий общий объем продаж по магазину, региону или дням/месяцам/годам, а также, возможно, продажи с разбивкой по товарам или категориям.
 - Отчет об инвентаризации, показывающий текущий уровень запасов по каждому товару в каждом магазине или на складе, а также графики пополнения запасов.
 - Отчет об отзывах покупателей, обобщающий отзывы, возможно, с анализом тональности высказываний клиентов по каждому магазину или продукту.

Еще одна хорошая практика — создать структуру папок, обычно свою для каждого уровня, которую можно организовать разными способами в зависимости от потребностей. Вот несколько примеров.

Разделение данных

Организация данных на основе источника, бизнес-подразделения или типа данных облегчает специалистам data science и аналитикам данных поиск и использование подходящих данных.

Контроль доступа

Различные команды или отдельные сотрудники организации могут иметь разные уровни доступа к данным. Структурируя папки на основе ролей пользователей или отделов, организации могут внедрять детализированные политики контроля доступа.

Оптимизация производительности

Организация данных определенным образом может способствовать повышению производительности. Например, некоторые операции по обработке данных или запросов могут выполняться быстрее, если данные сгруппированы по определенным признакам.

Управление жизненным циклом данных

Данные часто имеют свой жизненный цикл — от поступления до архивации или удаления. Для разделения данных по стадиям жизненного цикла можно использовать различные папки.

Управление метаданными

Папки можно использовать для управления метаданными и их отделения от исходных данных. Такое разделение позволяет упростить управление метаданными и ускоряет обнаружение данных.

Требования комплаенса

Во многих отраслях требования комплаенса (соответствия нормативам и правилам) диктуют необходимость хранить определенные типы данных и управлять ими определенным образом. Различные структуры папок могут помочь организациям выполнить эти требования.

Резервное копирование и аварийное восстановление

Наличие различных структур папок может помочь в создании стратегического плана резервного копирования и аварийного восстановления. Определенные папки можно копировать чаще или хранить дольше в зависимости от их важности.

Версионирование данных

Разные папки могут использоваться для управления разными версиями одного и того же набора данных.

Партиционирование данных

Данные можно разбить на части по ключевым атрибутам, чтобы ускорить выполнение запросов.

Потребности в приеме и обработке

В зависимости от источника данные могут нуждаться в разных схемах обработки. Размещение данных в разных папках может помочь в управлении и оптимизации этих процессов.

Для решения перечисленных выше задач можно выделить папки одним из способов:

- по времени (год/месяц/день/час/минута), например, когда данные были скопированы в озеро данных;
- предметной области;
- источнику данных;
- объекту, например по исходной таблице;

- границам зон безопасности, например по отделам или бизнес-зонам;
- целевым приложениям или последующему назначению;
- типу данных, например подробные или сводные;
- политике хранения данных, например временные или редко меняющиеся данные, за расчетный период или на срок действия проекта;
- степени влияния на бизнес или критичности данных, например высокая, средняя или низкая;
- владельцу данных/администратору/эксперту в предметной области;
- вероятности обращения к данным, например недавние или текущие данные по сравнению с историческими данными;
- классификации конфиденциальности, например общедоступная информация, информация только для внутреннего использования, конфиденциальная информация поставщика/партнера, персональные данные, чувствительные данные.

На рис. 5.4 показан пример структуры папок для зон необработанных и очищенных данных.



Рис. 5.4. Структура папок в двух зонах озера данных

В большинстве случаев все эти уровни находятся в рамках одной облачной подписки. Существуют некоторые исключения, например если у вас есть особые требования к биллингу, если вы превышаете лимит подписки или если вам нужны отдельные подписки для разработки, тестирования и продакшена. (Подробнее об исключениях см. в следующем разделе.)

Большинство клиентов создают отдельную учетную запись хранения для каждого уровня (таким образом, три уровня означают три учетные записи) в рамках одной *ресурсной группы* (в контейнере, содержащем все ресурсы, связанные с данным

решением). Это позволяет изолировать уровни озера данных, помогая сделать производительность более предсказуемой, и использовать различные опции и функциональность на уровне учетной записи хранения.

Типичные функции, настраиваемые для каждой учетной записи хранения, доступны у большинства облачных провайдеров (в данном разделе я рассматриваю конкретно Azure), включают управление жизненным циклом, которое позволяет сократить расходы за счет автоматизации задач управления данными (например, перемещение данных между уровнями хранения или удаление данных, когда они больше не нужны). Также можно установить правила брандмауэра, чтобы разрешить доступ только определенным доверенным группам или лицам или предотвратить превышение лимита хранения или лимита пропускной способности учетной записи.

Большинство озер данных используют тарифные уровни доступа к облачному хранилищу: для слоев необработанных и унифицированных данных используется архивный тариф, для слоя очищенных данных — холодный, а для слоев представления данных и песочницы — горячий. Каждый тариф предлагает разное соотношение стоимости хранения и стоимости доступа, при этом горячий является самым дорогим для хранения, а архивный — самым дорогим для извлечения.

Я рекомендую внедрить аудит или проверку целостности данных, чтобы убедиться, что они корректно перемещаются по уровням озера данных. Например, если вы работаете с финансовыми данными, можно создать запрос, который суммирует заказы за день (количество строк и общий объем продаж) и сравнивает эти значения с исходными, чтобы убедиться, что значения равны на всех уровнях данных. Если они не равны, это указывает на проблему с конвейерами, созданными для перемещения и преобразования данных. Потребуется устранить ее и запустить конвейеры заново.

Несколько озер данных

Идеальное решение — одно большое озеро, используемое для всех данных. Такой подход упростил бы поиск и объединение данных для запросов или отчетов. Однако по многим причинам вместо простого деления одного озера данных на части может потребоваться создать несколько отдельных озер данных (рис. 5.5). Большинство сценариев использования, которые я обсуждаю далее, невозможны (или, по крайней мере, гораздо менее удобны), если озера данных не будут физически разделены.

Преимущества

Я сгруппировал причины, по которым создание отдельных озер данных является преимуществом, в пять разделов, после чего расскажу о недостатках.

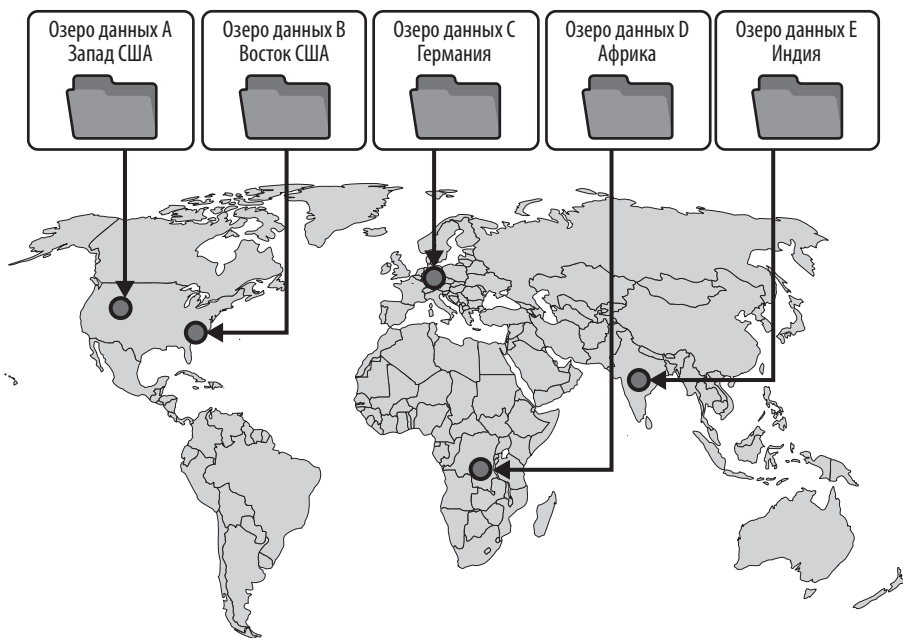


Рис. 5.5. Отдельные озера данных по всему миру

Организационная структура и владение данными

Наличие нескольких озер данных может быть выгодно по многим причинам, большинство из которых связано с организационной структурой и владением данными. Например, организационная структура компании может предполагать или требовать, чтобы каждое структурное подразделение владело только собственными данными. Такая функция обычно характерна для сетки данных (см. главу 13).

Другая причина заключается в том, что разным командам или отделам могут потребоваться свои озера данных для определенных сценариев использования или проектов.

Кроме того, компании может быть выгодно иметь озеро данных, ориентированное на источник данных, а также озеро данных, ориентированное на потребителя. Данные из определенного источника могут быть собраны в озеро с минимальными преобразованиями. Это хорошо подходит для пользователей, которые понимают исходные данные, например для тех, кто работает в сфере производства, когда данные поступают непосредственно из производственного процесса; однако людям, не связанным с производством, эти данные могут показаться слишком сложными. В таких случаях можно скопировать данные в озеро, ориентированное на потребителя, и преобразовать их, чтобы сделать более понятными. Таким образом, наличие

отдельных озер данных, ориентированных и на источник, и на потребителя, может упростить процесс обработки данных.

Комплаенс, управление и безопасность

На решение о создании нескольких озер данных существенно влияет не один, а целый ряд факторов, подпадающих под широкую категорию «комплаенс, управление и безопасность». Например, при развертывании в ряде регионов может возникнуть необходимость в создании нескольких озер данных, если требования к местонахождению или цифровому суверенитету данных в разных регионах отличаются. Например, если страна происхождения данных — Китай, то данные не могут быть экспортированы за пределы страны, поэтому для них требуется уникальное озеро данных в этом регионе.

Еще один важный момент: несколько озер данных позволяют отделить более чувствительные или конфиденциальные данные от менее чувствительных. Более строгие и жесткие меры контроля безопасности могут применяться именно к озерам чувствительных данных, что повышает общую безопасность данных.

Если некоторые сотрудники имеют повышенные привилегии доступа, отдельные озера данных позволяют ограничить сферу действия этих привилегий только тем озером данных, с которым работает конкретный специалист. Это способствует созданию более безопасной среды данных.

Наконец, различные требования к управлению и комплаенсу могут стать стимулом для создания нескольких озер данных. Такие нормативные акты, как Общий регламент по защите данных (GDPR, General Data Protection Regulation) и Федеральный закон США о переносимости и подотчетности медицинского страхования (HIPAA, Health Insurance Portability and Accountability Act), требуют применения уникальных стандартов и практик управления данными. Используя несколько озер данных, организации имеют возможность управлять своими данными в соответствии с конкретными требованиями к управлению и комплаенсу, которые применяются к каждому отдельному набору данных. Это особенно важно в жестко регулируемых секторах бизнеса.

Облачная подписка, ограничения на сервисы и политики

Наличие нескольких озер данных также может быть полезно по причинам, связанным в первую очередь с облачной подпиской, ограничениями на сервисы и политиками. Например, наличие нескольких озер может помочь обойти лимиты, квоты и ограничения, установленные провайдерами облачных услуг на подписки, например ограничение на максимальное количество учетных записей хранения на одну подписку. В подобных сценариях, если бы вы работали только с одним озером данных, ваши потребности могли бы превысить эти ограничения.

Более того, несколько озер данных позволяют реализовывать разные облачные политики. Провайдеры облачных услуг, как правило, предлагают сотни вариантов

политик, которые можно индивидуально настраивать для каждого озера данных. Такая гибкость помогает обеспечить соответствие политике компании. В качестве примера можно привести предписание, согласно которому определенные учетные записи хранения должны иметь шифрование инфраструктуры — правило, которое может различаться для разных озер данных в рамках компании.

Наконец, поддержка нескольких озер данных, каждое из которых имеет собственную облачную подписку, может упростить отслеживание затрат для выставления счетов. Такой уровень детализации в отслеживании расходов дает явное преимущество по сравнению с альтернативными методами, предоставляемыми облачными сервисами, такими как использование тегов для каждого ресурса.

Производительность, доступность и аварийное восстановление

Улучшение производительности, доступности и аварийного восстановления также является веским основанием использования нескольких озер данных. В частности, для уменьшения задержки доступа к данным. Если несколько конечных пользователей по всему миру обращаются к одному озеру данных, то те, кто находится далеко, могут обнаружить, что сервис работает медленно. Если расположить озеро данных в том же регионе, что и конечный пользователь или приложение, запрашивающее данные, можно значительно сократить время, необходимое для доступа к данным.

Поддержка нескольких озер с копиями данных в разных регионах значительно расширяет возможности аварийного восстановления. Если озеро данных в одном регионе становится недоступным, конечные пользователи могут быть перенаправлены в другой регион, где хранятся такие же данные.

Наличие нескольких озер данных обеспечивает гибкость при реализации стратегий восстановления данных и аварийного восстановления для разных типов данных. Критически важные данные, требующие постоянной доступности, могут быть размещены в озере, данные которого реплицируются в нескольких озерах в разных регионах, в то время как менее важные данные, для которых допустимы периоды недоступности, могут храниться в одном озере данных, что позволит значительно сократить расходы.

Наконец, при наличии нескольких озер данных можно реализовать разные уровни обслуживания для разных типов данных. Например, можно выделить одно озеро данных для хранения и обработки высокоприоритетных данных, для которых требуется доступ с низкой задержкой и высокой доступностью, и другое озеро данных для менее приоритетных данных, для которых приемлемы более высокие задержки и более низкая доступность. Такая стратегия оптимизирует стоимость и производительность инфраструктуры управления данными, позволяя использовать менее дорогие ресурсы хранения и обработки для низкоприоритетных данных.

Сохранность данных и управление средами

Несколько озер данных позволяют обеспечить сохранность данных и повысить эффективность управления средами. Например, разделив озера данных,

предназначенные для сред разработки, тестирования и продакшена, организации смогут гарантировать, что каждая среда будет иметь свое изолированное пространство для хранения и обработки данных. Это минимизирует риск взаимного влияния или конфликтов между различными этапами жизненного цикла данных.

Наличие нескольких озер данных позволяет применять различные политики при сохранении данных. Правовые или нормативные требования часто диктуют необходимость сохранять данные в течение определенного периода времени. Если у вас есть отдельные озера данных для разных типов данных, вы можете легко применять различные политики хранения, адаптированные к этим категориям. Такой подход позволяет эффективно управлять хранением данных, обеспечивая соответствие различным нормативным требованиям и одновременно оптимизируя ресурсы хранения.

Недостатки

Использование нескольких озер данных может повысить сложность и стоимость инфраструктуры управления данными, потребовать больше ресурсов и более высокого уровня квалификации специалистов для их обслуживания. Поэтому, если у вас есть выбор, важно взвесить все «за» и «против». Однако в некоторых случаях выбора не будет; например, если вы проектируете систему, где необходимо обеспечить цифровой суверенитет данных, вам придется иметь несколько озер данных.

Для правильной передачи данных между несколькими озерами с сохранением их согласованности могут потребоваться дополнительные инструменты интеграции и управления (такие, как, например, Azure Data Factory, Informatica или Apache NiFi). Наконец, наличие нескольких озер данных создает дополнительные проблемы с производительностью при необходимости объединения данных, когда в запросе или отчете требуются данные из нескольких озер. Если озера физически расположены очень далеко друг от друга, возможно, даже в разных частях света, копирование всех данных в одно местоположение может занять много времени.

Итоги

В этой главе было детально рассмотрено озеро данных — крупное решение для хранения и управления данными, позволяющее держать необработанные данные в их исходном формате. Мы обсудили роль озера данных и принципы его проектирования, а также то, почему в определенных сценариях стоит рассмотреть возможность создания нескольких озер. В отличие от реляционных хранилищ, озера позволяют быстро сохранять данные без предварительной подготовки или преобразования. Это повышает скорость и удобство приема данных, что особенно полезно в современную эпоху Big Data.

Мы поговорили о преимуществах использования озера данных, с акцентом на гибкость и скорость, которые оно предлагает при хранении данных. Это, а также способность принимать данные разнообразных типов (структурированные,

полуструктурированные и неструктурированные) составляет его ключевые преимущества, особенно в сценариях, когда крайне важен быстрый и масштабируемый сбор разнообразных данных.

В этой главе также был представлен восходящий подход («снизу вверх») — важная методика, при которой сбор данных осуществляется еще до выдвижения теорий или гипотез. В отличие от традиционной стратегии проектирования «сверху вниз» (нисходящий подход), эта методика способствует более гибкому процессу принятия решений, ориентированному на данные.

Значительная часть этой главы была посвящена проектированию озера данных и рассмотрению концепции логического разделения озера данных на несколько уровней. Такая многоуровневая структура помогает организованно управлять данными, прокладывая путь к эффективному извлечению и анализу данных, а также помогает в обеспечении безопасности данных, контроля доступа к ним и в управлении их жизненным циклом.

Глава завершается рассмотрением причин, по которым организация может выбрать решение с несколькими озерами данных. Потенциальные преимущества такого выбора включают повышение безопасности, более точное соблюдение нормативных требований и более эффективное управление данными в различных бизнес-подразделениях или конкретных сценариях использования.

В главе 6 мы перейдем к рассмотрению систем хранения данных.

Решения для хранения данных и процессы работы с данными

В цифровую эпоху данные стали хлебом насущным для всех компаний. Но, как известно любому опытному специалисту data science, просто иметь данные недостаточно. Настоящая ценность состоит в том, насколько эффективно осуществляются управление ими, их хранение и обработка. Эта глава представляет собой исчерпывающее руководство по тому, как не потеряться в запутанном мире управления данными. Мы подробно рассмотрим, как конкретные решения для хранения данных и процессы функционируют в разных архитектурах данных.

Компоненты управления данными подобны шестеренкам в сложном механизме: каждый из них выполняет свою уникальную роль, но все работают синхронно для достижения общей цели. Я начну с наиболее практичных решений для хранения данных: витрин данных, хранилищ оперативных данных и концентраторов данных. Вторая половина главы, «Процессы работы с данными», посвящена широкому спектру процессов, связанных с эффективным управлением и использованием данных, и в ней рассматриваются такие понятия, как управление мастер-данными, виртуализация данных, каталоги данных и маркетплейсы данных.

Например, если витрины предоставляют данные в виде, удобном для конкретных подразделений компании, то хранилища оперативных данных предлагают более развернутое представление бизнес-операций в режиме реального времени. Со своей стороны, управление мастер-данными обеспечивает согласованность и надежность данных, а виртуализация данных — единое абстрактное представление данных из различных источников. Каталог данных служит справочным руководством, облегчающим поиск данных, а маркетплейс данных открывает возможности для обмена данными или даже их монетизации. В то же время концентратор данных выступает в качестве ядра (центральной точки) системы, организуя данные из различных источников и управляя ими для унификации доступа. Каждый из этих компонентов может выполнять свои функции самостоятельно, однако, будучи интегрированными, они способны сформировать мощную, целостную стратегию работы с данными, способствующую принятию обоснованных решений, повышению операционной эффективности и улучшению конкурентных преимуществ бизнеса.

Неважно, матерый ли вы профессионал в области data science, начинающий специалист или тот, кто принимает стратегические решения, понимание этих ключевых

компонентов и умение гармонично их сочетать помогут раскрыть истинный потенциал данных вашей организации.

Решения для хранения данных

Итак, приступим. Сейчас речь пойдет не только о хранении и обработке данных, но и о том, как заложить фундамент для перехода к принятию решений на основе данных.

Витрины данных

Витрина данных (data mart), схематически показанная на рис. 6.1, представляет собой разновидность хранилища данных. Обычно она разрабатывается как специализированное хранилище данных, оптимизированное для удовлетворения потребностей конкретного подразделения или отдельного направления организации (например, финансов, кадров или маркетинга). Благодаря своей более узкой направленности витрина данных может предоставить пользователям информацию в более упорядоченном и доступном виде, чем DW.

Пользователям может быть тяжело ориентироваться в больших и сложных хранилищах данных, содержащих огромные объемы информации. Витрины данных решают эту проблему, извлекая и консолидируя соответствующие данные из DW, представляя их в более удобочитаемой форме. Таким образом, пользователи внутри одного подразделения могут легко находить и извлекать нужные им данные.

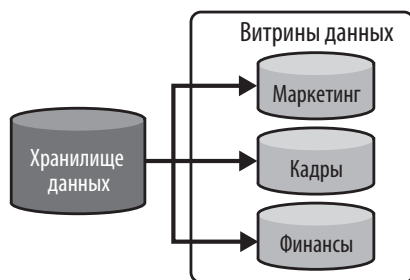


Рис. 6.1. Витрины данных

Одним из ключевых преимуществ витрин данных является то, что их могут самостоятельно создавать и поддерживать отдельные подразделения, не полагаясь исключительно на IT-отдел. Кроме того, каждая витрина данных может быть структурирована по-разному, чтобы соответствовать потребностям подразделения, которое она обслуживает. Это позволяет бизнес-пользователям лучше контролировать данные; они могут адаптировать их к своим уникальным потребностям и использовать структуры данных, наиболее подходящие для их предметной области.

Витрины данных, ориентированные на конкретные подразделения, предоставляют более быстрый и удобный доступ к данным, которые нужны пользователям для их повседневной работы и процессов принятия решений. Они могут выполнять аналитические запросы, создавать отчеты и получать информацию об эффективности работы своего отдела, не прибегая к навигации по всему DW.

Более того, витрины данных способствуют улучшению управления данными и повышению безопасности. Каждое подразделение может самостоятельно определять и применять политики доступа к своей витрине данных, обеспечивая доступ к определенным наборам данных только уполномоченным лицам. Это помогает защитить конфиденциальную информацию и обеспечить соответствие регламенту по защите персональных данных.

Разобравшись с витринами данных с их строго ориентированной на нужды конкретного подразделения информацией, перейдем к рассмотрению хранилищ оперативных данных.

Хранилища оперативных данных

Хранилище оперативных данных (ODS, operational data store) имеет более широкую сферу применения, чем витрина данных, и обеспечивает представление данных в масштабах всего предприятия в режиме реального времени. В то время как витрины данных предлагают ценные, детальные сведения в конкретных областях, ODS представляет комплексный обзор оперативных данных организации. Каждый инструмент — взгляд со своей точки зрения; если витрина данных позволяет смотреть на данные через микроскоп, то ODS — через широкоугольный объектив.

Цель ODS — интегрировать корпоративные данные из разных источников, чтобы обеспечить оперативную отчетность в реальном (или почти в реальном) времени. Это не обычное хранилище данных. Как следует из названия, в нем хранятся оперативные, а не аналитические данные. Оно предоставляет свежие данные гораздо быстрее, чем DW: каждые несколько минут, а не ежедневно.

ODS может подойти для вашей организации, если данные, которые вы в настоящее время используете для отчетности в системах-источниках, слишком ограничены или если вам нужен более мощный инструмент отчетности, чем тот, который предлагают системы-источники. Если на данный момент только несколько человек имеют доступ к системам-источникам, но вы хотите дать возможность и другим сотрудникам формировать отчеты, то ODS в этом случае может стать хорошим решением.

Обычно данные в ODS структурированы аналогично данным в системах-источниках. Однако в процессе интеграции ODS-решение может очищать, нормализовать данные и применять к ним бизнес-правила для обеспечения их целостности. Такая интеграция происходит довольно часто в течение дня на самом низком уровне детализации. Как правило, оптимизировать ODS для ретроспективного и трендового анализа не нужно, поскольку это делается в хранилище данных.

Данные в ODS ближе к данным реального времени, чем те, что содержатся в хранилище данных. Вы можете использовать ODS в качестве одной из промежуточных областей хранения для DW, по крайней мере для тех данных, которые оно поддерживает (см. рис. 6.2). Я рекомендую сначала синхронизировать и очищать эти данные, заполняя ODS практически в режиме реального времени, чтобы те, кто принимает оперативные и тактические решения на основе этих данных, могли извлечь из них пользу. Затем эти данные также можно будет использовать для заполнения DW, которое обычно менее требовательно к циклам загрузки. Вам по-прежнему понадобится промежуточная область хранения для DW-данных, которые не размещаются в ODS, однако, следуя этой рекомендации, вы сократите количество потоков ETL и размеры промежуточной области хранения. Это поможет повысить производительность без ущерба для ценности данных и их функциональности.

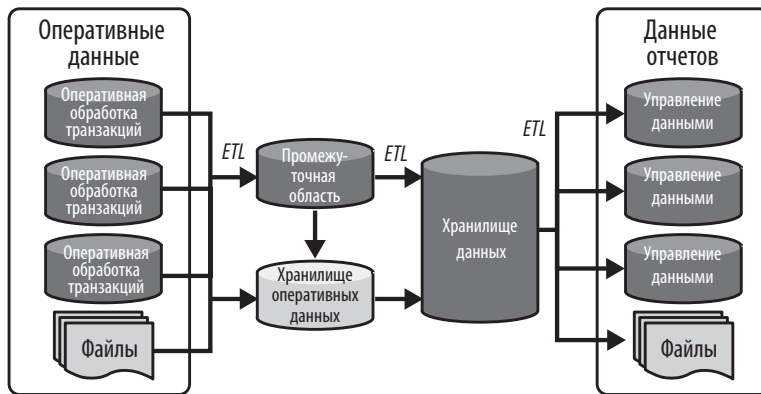


Рис. 6.2. Хранилище оперативных данных

В таблице 6.1 приведены наиболее важные различия между ODS и хранилищем данных.

Таблица 6.1. Различия между ODS и хранилищем данных

	Хранилище оперативных данных (ODS)	Хранилище данных (DW)
Сценарии применения	Детализированные, низкоуровневые запросы к подробным оперативным данным	Сложные запросы к сводным или агрегированным аналитическим данным
Цель	Сбор и передача оперативной информации; составление отчетов на основе текущих или поступающих в режиме, близком к реальному времени данных	Составление отчетов на основе ретроспективного и трендового анализа, проведенных на большом объеме данных

	Хранилище оперативных данных (ODS)	Хранилище данных (DW)
Продолжительность накопления данных	Содержит короткий период сбора данных	Содержит всю историю данных организации
Процесс принятия решений	Поддержка оперативных и тактических решений на основе текущих или близких к реальному времени данных	Обеспечивает обратную связь по стратегическим решениям, что приводит к общему улучшению системы
Частота загрузки данных	Каждые несколько минут или каждый час	Ежедневно, еженедельно, ежемесячно или ежеквартально

Сценарий использования

Допустим, вы создаете отчеты о товарных запасах для вымышленной розничной сети под названием ShoesForLess. Если бы у вас был только один розничный магазин с одной базой данных для отслеживания заказов, вы могли бы просто запустить отчет о товарных запасах по этой БД. Но поскольку у вас много розничных магазинов, каждый из которых отслеживает заказы в своей БД, единственный способ получить актуальные точные отчеты о запасах — это объединить данные о заказах из этих баз данных. Это важно, чтобы информировать покупателей о том, какие товары отсутствуют на складе, а также для быстрого пополнения запасов, когда они почти на исходе. Хранилище данных в таком случае не будет достаточно оперативным, то есть не сможет работать в режиме реального времени.

Теперь предположим, что представитель отдела по работе с клиентами получает звонок от покупателя, который спрашивает, есть ли в наличии обувь определенного фасона и размера. Представитель запрашивает в ODS данные о наличии такого типа обуви (например, «мужские кроссовки, размер 10») в режиме реального времени и об их количестве на складе.

С другой стороны, если менеджер по маркетингу магазина хочет понять тенденции продаж данного типа обуви за последний год, он может использовать хранилище данных. Он запускает агрегированный запрос, который суммирует продажи товара «мужские кроссовки» за каждый месяц в течение прошлого года. Это дает высокоуровневое представление о тенденциях, на основе которого можно принимать стратегические решения по маркетингу и запасам продукции.

Концентраторы данных

Концентратор данных (data hub) — это централизованная система хранения и управления данными, которая помогает организации собирать, интегрировать, хранить, упорядочивать данные и обмениваться данными из разных источников, а также обеспечивает удобный доступ к инструментам аналитики, отчетности и принятия решений. Что особенно важно, концентраторы данных избавляют от необходимости использовать сотни или тысячи интерфейсов типа «точка-точка» для обмена данными между системами. Хотя основной (и зачастую единственной) целью хранилищ и озер данных является предоставление данных для аналитики,

концентраторы данных также ценны как средство связи и обмена данными между действующими системами (рис. 6.3).

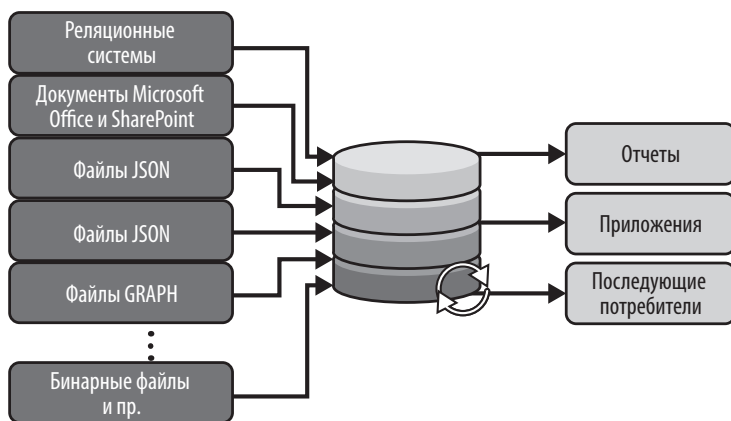


Рис. 6.3. Концентратор данных

Концентраторы данных часто используются в сочетании с озерами и хранилищами данных для создания комплексной инфраструктуры управления данными. Как мы уже выяснили в главах 4 и 5, в озерах данных хранятся большие объемы сырых, необработанных данных, а в хранилищах данных — структурированные, обработанные данные, оптимизированные для аналитики. Концентратор данных дополняет эти системы, выступая в качестве посредника для обмена данными, совместной работы и управления. Чтобы прояснить ситуацию, рассмотрим, как концентратор данных соотносится с другими системами, описанными в этой главе, и что он представляет собой в сравнении с ними. В таблице 6.2 приводится краткое сравнение различных решений.

Таблица 6.2. Сравнение функциональных возможностей различных решений для работы с данными

Характеристика	Концентратор данных	Озеро данных	Реляционное хранилище данных	Каталог данных	Маркетплейс данных
Основная функция	Хранение, интеграция, распространение данных	Хранение, обработка, анализ данных	Хранение, обработка, анализ данных	Управление метаданными, обнаружение данных	Покупка, продажа, обмен данными
Хранение данных	Сырые или слабо обработанные данные	Сырые и обработанные данные	Высокоструктурированные, обработанные данные	Только метаданные	Различные форматы в зависимости от поставщика данных

Характеристика	Концентратор данных	Озеро данных	Реляционное хранилище данных	Каталог данных	Маркетплейс данных
Структура данных	Структурированные, полуструктурированные, неструктурированные	Структурированные, полуструктурированные, неструктурированные	Структурированные	Только метаданные	Структурированные, полуструктурированные, неструктурированные
Обработка данных	Ограниченная обработка, в основном интеграция и распространение	Обработка и аналитика по запросу	Обширный ETL, сложные запросы	Отсутствует	Зависит от поставщика данных
Обмен данными	Встроенные инструменты	Сторонние инструменты	Сторонние инструменты	Отсутствуют	Встроенные инструменты
Владение данными	В рамках организации или смежных групп	В рамках организации или смежных групп	В рамках организации или смежных групп	В рамках организации или смежных групп	В рамках нескольких организаций
Сценарии использования	Прием, интеграция, распространение данных	Аналитика больших данных, машинное обучение, искусственный интеллект	Сложная аналитика, создание отчетов, принятие решений	Обнаружение данных, происхождение данных, применение политик управления данными	Монетизация данных, получение новых наборов данных, обнаружение данных

Концентратор данных хранит копии исходных данных и ориентирован на управление данными и их распространение, в то время как каталог не хранит реальные данные и в первую очередь занимается управлением метаданными, отслеживанием происхождения данных, применением политик управления данными и облегчением обнаружения данных.

Концентратор данных может служить источником для маркетплейса данных, который не хранит данные. Концентратор ориентирован на управление данными и распространение данных внутри организации, а не на коммерческие сделки на маркетплейсе данных. Это центральная платформа, которая использует больше систем хранения, чем озеро данных или реляционное хранилище. Озера данных обычно применяют для хранения данных объектное хранилище, а не реляционное; концентраторы данных могут сочетать такие системы хранения, как реляционные базы данных, базы данных NoSQL и озера данных.

Как и озеро данных, концентратор поддерживает структурированные, полуструктурированные и неструктурированные данные, как показано на рис. 6.3, в то время как реляционное хранилище — только структурированные данные. Концентраторы чаще всего предоставляют встроенные механизмы для каталогизации данных, определения их происхождения и контроля доступа, в то время как озеро данных часто интегрирует в себя инструменты сторонних производителей или собственные решения.

Что касается обработки, то озеро данных использует вычисления для очистки данных и их последующего помещения обратно в озеро. Реляционные хранилища выполняют масштабную обработку данных, включая операции ETL, чтобы очистить, нормализовать и структурировать данные для аналитических целей. Однако при использовании концентратора данные обычно преобразуются и анализируются вне концентратора с помощью внешних инструментов или приложений.

Подводя итог, можно сказать, что концентратор данных — это гибкое и универсальное решение для приема, управления, интеграции и распространения данных самых разных типов и из разных источников. Озеро данных ориентировано на хранение и обработку больших объемов необработанных данных для использования в аналитике и машинном обучении, а хранилище данных оптимизировано для хранения структурированных данных, используемых в сложной аналитике, при создании отчетов и принятии решений.

Процессы работы с данными

В этой главе мы рассматриваем различные решения для хранения данных и то, какие разнообразные и эффективные способы организации данных и доступа к ним они предоставляют. Однако хранение данных — это лишь одна часть этого пазла. Чтобы по-настоящему использовать весь потенциал данных, мы также должны понимать, как эффективно управлять ими. Итак, теперь мы переходим от рассмотрения решений для хранения данных к тонкостям процессов обработки данных. Вы откроете для себя стратегии и технологии, которые помогут производить манипуляции с данными, управлять ими и извлекать выгоду из активов данных.

Управление мастер-данными

Управление мастер-данными (MDM, master data management) — это набор технологий, инструментов и процессов, позволяющих пользователям создавать и поддерживать согласованные и точные списки мастер-данных (например, списки клиентов, продуктов и поставщиков). Оно предполагает создание единой основной записи (мастер-записи) для каждого человека, места или предмета в бизнесе с данными, полученными из внутренних и внешних источников данных и приложений. Затем можно использовать эти мастер-записи для создания более точных отчетов, дашбордов, запросов и моделей машинного обучения.

Большинство продуктов MDM имеют очень сложные инструменты для очистки, преобразования, слияния и проверки данных, чтобы обеспечить соблюдение стандартов представления данных. Они также позволяют создавать иерархии (например, Компания → Подразделение → Сотрудник), которые можно использовать для улучшения отчетов и дашбордов. Первым шагом является копирование данных, с которыми требуется работать, в продукт MDM. Затем продукт очищает и стандартизирует данные и создает мастер-запись для каждой сущности (например, клиента), которую нужно преобразовать в мастер-данные. Он автоматически удаляет большинство дубликатов, но часть работы придется выполнить вручную, просматривая записи, которые могут быть дубликатами. Затем мастер-записи копируются обратно в озеро или хранилище данных. Эта мастер-запись также становится «эталонным (золотым) источником» или «лучшей версией истины».

Если вы создаете схему «звезда» (см. главу 8), то обрабатываемыми записями будут таблицы измерений, которые затем будут присоединены к таблицам фактов (которые не обрабатываются).

Сценарий использования

Вернемся к розничной сети ShoesForLess, где у каждого магазина есть своя база данных клиентов. Компания собирает данные о клиентах из всех этих баз данных в хранилище данных. Поскольку часть клиентов приобретала товары в нескольких торговых точках, в хранилище данных будут содержаться дубликаты некоторых записей о клиентах.

Если имя покупателя написано одинаково в обеих записях, отфильтровать дубликаты легко. Если же в одной записи имя написано неправильно или введено по-другому (например, в одной записи есть отчество, дефис или суффикс типа *Junior* или *Senior*, а в другой нет), фильтр не распознает эти имена как дубликаты (такие записи называются *нечеткими дубликатами*).

Если вы не внедрите MDM, то, когда конечный пользователь впервые получит отчет из хранилища данных, он увидит, что одни и те же клиенты встречаются в нем несколько раз, и, скорее всего, усомнится в точности отчета. Так вы сразу потеряете его доверие, а вернуть его будет очень сложно. MDM — вот решение!

Виртуализация и федерализация данных

Виртуализация данных известна также как *логическое хранилище данных*, *виртуальное хранилище данных* или *децентрализованное хранилище данных*. Эта технология позволяет получать доступ к данным разных форматов и полученным из разных источников и объединять их, создавая логическое представление данных без копирования или перемещения реальных данных. Она обеспечивает единую точку доступа ко всем данным, позволяя проводить интеграцию данных в режиме реального времени и сокращая время и затраты, связанные с традиционными подходами к интеграции данных, такими как ETL или ELT (см. главу 9).

Федерализация данных и виртуализация данных — очень похожие понятия, которые часто используются как взаимозаменяемые, и вы вряд ли попадете впросак, если даже будете использовать их оба. Однако между ними есть некоторые различия. В процессе *федерализации* данных участвует зонтичная головная организация, состоящая из более мелких, полностью или частично автономных подгрупп организаций, которые контролируют все или часть своих операций. (Система федеральных органов власти и органов власти штатов США — хороший пример федерализации.) Как и виртуализация, федерализация данных создает единое представление данных из нескольких источников. Однако виртуализация — это более широкое понятие, включающее в себя федерализацию, а также преобразование данных, кэширование, безопасность и управление.

Наиболее удачные инструменты виртуализации данных предоставляют функции, которые помогают повысить производительность, такие как оптимизация запросов, поддержка операций *pushdown* для запросов и кэширование. Инструменты с такими функциями можно назвать *инструментами виртуализации данных*, а инструменты без них — *инструментами федерализации данных*.

Виртуализация данных может заменить хранилище данных (см. главу 4) или ETL-процессы/перемещение данных. Рассмотрим, каким образом.

Виртуализация данных в качестве замены хранилища данных

В некоторых случаях, особенно когда источников данных много или они постоянно меняются, виртуализация данных может стать альтернативой хранилищу данных. При использовании такого подхода не требуются копирование и хранение данных в централизованном месте.

На рис. 6.4 показано, как работает большинство продуктов виртуализации данных. Таблица-грид¹ (шаг 1) содержит результаты запроса конечного пользователя — список адресов. Когда конечный пользователь прокручивает грид, чтобы увидеть больше данных, грид запрашивает элементы, которые в данный момент не загружены из движка виртуализации данных (шаг 2). Если у движка есть кэш доступных данных, он немедленно возвращает данные из кэша, и грид просто обновляется для отображения данных. Если вы запрашиваете данные, которых нет в кэше, движок виртуализации данных передает запрос удаленному источнику данных (шаг 3), где он обрабатывается. Свежеизвлеченные данные попадают в движок (шаг 4), а затем передаются в грид. В обоих случаях грид всегда оперативно реагирует на действия пользователей.

Основное преимущество решения с виртуализацией данных заключается в том, что оно оптимизировано с точки зрения скорости выхода на рынок. Его можно создать за малую долю того времени, которое необходимо для построения хранилища данных, поскольку не нужно проектировать и создавать хранилище данных и ETL для копирования данных в него и не нужно тратить слишком много времени

¹ Грид — таблица с вертикальной полосой прокрутки. — *Примеч. пер.*

на его тестирование. Копирование данных (как в случае с хранилищем) означает более высокие затраты на хранение и управление данными, больше ETL-поток, которые нужно создавать и поддерживать, и больше несоответствий в данных, поэтому использование виртуализации может сэкономить уйму денег.

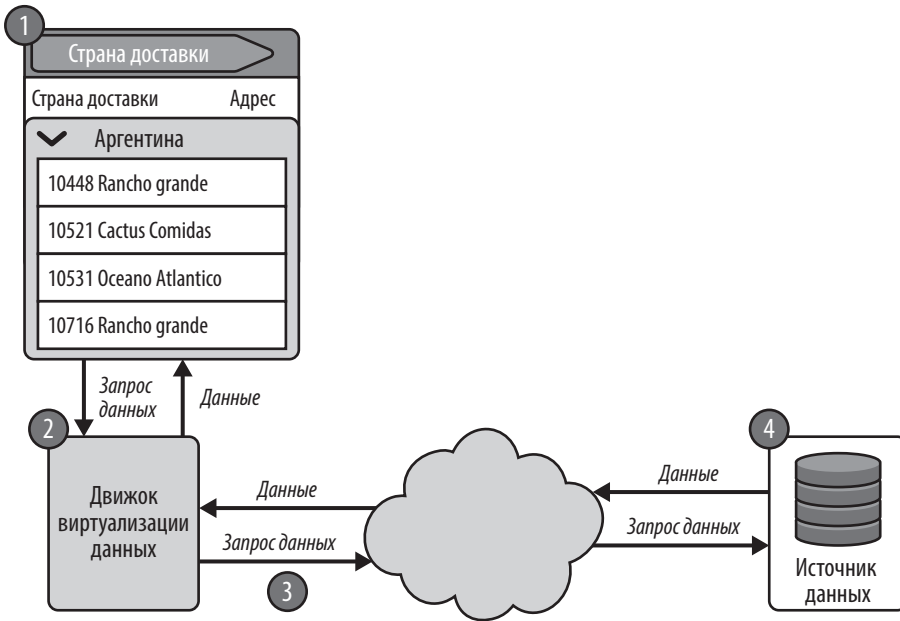


Рис. 6.4. Виртуализация данных

Виртуализация данных поддерживает *федеративные запросы*, когда создается один запрос, который извлекает данные из нескольких источников и форматов и объединяет их в единый набор результатов¹.

Однако у виртуализации данных и федеративных запросов есть ряд серьезных недостатков. Если вы рассматриваете решение с виртуализацией данных в качестве замены хранилища данных, я рекомендую ответить на следующие вопросы.

- Какой уровень производительности мне нужен? Можно ли использовать это решение для создания дашборда, для которого время отклика должно быть меньше секунды, или оно больше подходит для оперативного создания отчетов?

¹ Разница между федерализацией данных и федеративными запросами заключается в том, что федерализация данных в большей степени относится к архитектуре системы, а федеративные запросы — это механизм, используемый для извлечения информации из этой системы. На практике, если у вас есть система федерализации данных, вы используете федеративные запросы для получения данных.

- Насколько сильно решение с виртуализацией данных повлияет на производительность системы-источника? Будет ли мой запрос выполняться медленнее при условии, что другой конечный пользователь выполнит запрос к тому же источнику данных, который задействует все ресурсы сервера? Будет ли для запроса выполняться операция *pushdown*?¹
- Потребуется ли мне устанавливать какие-либо программы на каждый сервер, содержащий источник данных, который я хочу использовать?
- Использует ли это решение индексирование, определенное источником данных, или создает собственные индексы?² Использует ли оно статистику базы данных из каждого источника данных для запросов?³
- Можно ли ограничить доступ пользователей к каждому из источников данных в целях безопасности?
- Как решение обрабатывает «нечеткие дубликаты»?
- Где и как будет производиться очистка данных?
- Перестанет ли работать система формирования отчетов при изменении системы-источника?
- Какова стоимость программного обеспечения для виртуализации данных и насколько сложно его использовать?

Помимо причин, раскрываемых ответами на эти вопросы, существуют и другие веские основания для использования физического хранилища данных, о котором говорилось в главе 4, например необходимость создавать точные ретроспективные отчеты, возможность переименовывать таблицы и хранить исторические данные.

¹ Под термином *pushdown* применительно к виртуализации данных понимается возможность передавать определенные части запроса, такие как фильтрация или агрегирование, самим источникам данных. Это позволяет уровню виртуализации данных разгрузить задачи обработки и получать только релевантные результаты. Преимущества заключаются в повышении производительности и минимизации передачи данных; недостаток — в использовании вычислительных ресурсов на сервере, где находится источник данных.

² *Индексы в базах данных* — это тип структуры данных, который повышает скорость операций поиска данных в таблице базы данных. Они работают во многом подобно индексам в книгах, предоставляя пользователям быстрый доступ к нужным данным.

³ *Статистика базы данных* — это набор информации, которую система управления базами данных (СУБД) хранит о данных, содержащихся в базе. Эта статистика помогает СУБД оптимизировать производительность запросов, сообщая, как она определяет наиболее эффективный способ выполнения данного запроса.

Виртуализация данных как замена ETL-процессов или перемещения данных

Если вы создаете хранилище данных, нужно ли перемещать все исходные данные в хранилище или лучше оставить их там, где они есть, и создать уровень виртуализации поверх некоторых из них?

Наиболее частая причина перемещения данных — если планируется однократно агрегировать или преобразовать их, а затем многократно запрашивать результаты. Другая причина — если надо часто объединять наборы данных из нескольких источников и требуется сверхпроизводительность. В таких случаях я обычно рекомендую в качестве решения хранилище данных.

Однако иногда специальные запросы могут не нуждаться в сверхбыстром выполнении. И конечно, хранилище данных может использовать перемещение данных для одних таблиц и виртуализацию данных — для других. В таблице 6.3 проводится сравнение перемещения и виртуализации данных.

Таблица 6.3. Сравнение перемещения и виртуализации данных

	Затраты на разработку и эксплуатацию	Время, затрачиваемое на работу с решением	Безопасность	Актуальность и качество данных	Соответствие нормативным требованиям
Перемещение данных	Создание и поддержка ETL-заданий требуют больших затрат; дублирование расходов на хранение данных	Требуется время на создание и запуск заданий	Создание копий данных делает их более уязвимыми для злоумышленников	Конвейеры ETL делают данные «не-свежими»; процессы ETL могут вносить ошибки в данные	Перемещение данных в пределах соответствия нормативным требованиям и за его пределы может привести к проблемам управления данными
Виртуализация данных	Сокращение расходов на текущее обслуживание и управление изменениями; минимизация затрат на хранение данных	Позволяет быстро выполнять итерации и прототипирование	Данные хранятся в одном безопасном месте, что минимизирует поверхность атаки	Запрашиваемые данные всегда «свежие» и точные, как в источнике	Небольшое количество копий данных и меньшее количество перемещений помогают строго соблюдать нормативные требования

Виртуализация данных обеспечивает ряд преимуществ. Во-первых, она полностью отслеживает происхождение данных, позволяя четко проследить их путь

от источника до уровня представления. Во-вторых, включение дополнительных источников данных происходит бесшовно, поэтому изменять пакеты преобразования данных или промежуточные таблицы не требуется. Наконец, программное обеспечение для виртуализации данных представляет все данные через единый SQL-интерфейс. Это позволяет пользователям легко получать доступ к данным независимо от их источника, который может представлять собой неструктурированные файлы, электронные таблицы, мейнфреймы, реляционные базы данных или другие типы репозитория данных.

Хотя таблица 6.3 демонстрирует преимущества виртуализации данных по сравнению с перемещением данных, этих преимуществ может оказаться недостаточно, чтобы компенсировать потери в производительности и другие недостатки отказа от использования реляционного хранилища данных, перечисленные в главе 4. Также следует помнить, что выбранный инструмент виртуализации может не поддерживать все источники данных.

Ниже приведено несколько аргументов в пользу выбора виртуализации данных.

Существуют нормативные ограничения на перемещение данных.

Как вы узнали из подраздела «Комплаенс, управление и безопасность» главы 5, китайские правила цифрового суверенитета данных гласят, что данные, хранящиеся в Китае, должны оставаться в Китае и могут быть запрошены только лицом, находящимся там в данный момент. Но предположим, что вы находитесь в Соединенных Штатах и пытаетесь запросить данные клиента в Китае из своего приложения. Приложение может извлекать метаданные о данных клиентов в Китае, но не может получить сами данные. Вы можете использовать программы для виртуализации, чтобы хранить данные в пределах страны, позволяя запрашивать их только лицам, находящимся в стране. Любой человек за пределами Китая, пытающийся запросить данные, либо получит сообщение об ошибке, либо просто не увидит никаких результатов.

Конечный пользователь, находящийся вне компании, хочет получить доступ к данным клиента.

Вместо того чтобы отправлять пользователю копию данных, он может использовать программу для виртуализации для запроса этих данных. Это позволяет контролировать, кто к каким данным получает доступ, и знать, к какому объему данных они получили доступ.

В базе данных вне хранилища данных существуют ссылочные данные, которые часто меняются.

Вместо того чтобы снова и снова копировать их в хранилище, можно использовать виртуализацию для запроса этих данных при объединении данных внешней базы с данными хранилища.

Вы хотите запрашивать данные из разных источников хранения данных и объединять их.

Программы для виртуализации обычно поддерживают множество источников данных (например, озеро данных, реляционную базу данных и базу данных NoSQL) и хорошо справляются с этой задачей.

Вы хотите, чтобы пользователи могли самостоятельно проводить аналитику посредством виртуальной песочницы.

Виртуальная песочница — это безопасная и изолированная среда, в которой пользователи могут изучать, анализировать и манипулировать данными, не затрагивая базовые производственные данные или системы. Например, допустим, вы используете виртуальную песочницу для анализа как данных клиентов, так и сторонних данных. Вместо того чтобы копировать эти данные в виртуальную песочницу, можно использовать виртуализацию для запроса их внутри виртуальной песочницы. Это минимизирует количество копий данных и избавит от необходимости создавать ETL.

Вы хотите быстро создать решение для перемещения исходных данных.

Вместо того чтобы использовать традиционный инструмент ETL для копирования исходных данных в централизованное место хранения, а затем создавать на их основе отчеты, можно просто формировать отчеты с помощью программы для виртуализации. Она будет выполнять запросы к исходным данным и автоматически выполнять централизацию. Это означает, что решение получится создать гораздо быстрее, поскольку инструмент ETL (который может оказаться очень медленным) для этого не понадобится.

Каталоги данных

Каталог данных (data catalog) — это централизованный репозиторий, обычно размещаемый в облаке, в котором хранятся и систематизируются метаданные обо всех источниках данных организации, таблицах, схемах, столбцах и других активах данных, таких как отчеты и дашборды, процессы ETL и скрипты SQL. Он функционирует как единый источник истины для пользователей, позволяющий обнаруживать, понимать и управлять данными, расположенными в базах данных приложений, озерах данных, реляционных хранилищах данных, хранилищах оперативных данных, витринах данных и любых других формах хранения.

Каталог обычно содержит следующую информацию о каждом активе данных.

- Информация об источнике, например расположение, тип и сведения о подключении.
- Если речь идет о реляционном хранилище, информация о таблице и схеме (например, структура, связи и организация) и информация о столбцах (например, типы данных, форматы, описания и связи между столбцами).

- Если речь идет о хранилище объектов, например об озере данных, — свойства файла (хранилище, имя папки, имя файла).
- Сведения о происхождении данных (как данные поступили из источника), включая информацию о любых преобразованиях, агрегации и интеграции, которым подвергались данные.
- Подробности об управлении данными и комплаенсе, такие как качество данных, право владения и политики.
- Инструменты поиска и обнаружения, позволяющие пользователям искать, фильтровать и находить нужные данные.

Каталог помогает эффективнее управлять данными, давая представление об их ландшафте. Это, в свою очередь, помогает наладить совместную работу команд и принимать более обоснованные решения. Он также помогает обеспечить качество данных, безопасность и комплаенс, облегчая отслеживание происхождения данных, соблюдение политик в отношении данных и давая четкое представление об активах данных.

На рынке представлено множество продуктов для создания каталогов данных, некоторые из них существуют уже почти десять лет. Наиболее популярные — Enterprise Data Catalog от компании Informatica, Collibra Data Catalog и Microsoft Purview. Эти продукты имеют дополнительные функции, такие как управление данными и комплаенс, бизнес-глоссарии и метрики качества данных.

Маркетплейсы данных

Маркетплейс данных (data marketplace), иногда называемый *биржей данных* (data exchange), — это онлайн-платформа, на которой поставщики и потребители данных собираются вместе, чтобы покупать, продавать датасеты и обмениваться ими¹.

Такие маркетплейсы призваны помочь потребителям данных (например, компаниям, исследователям и разработчикам) находить, оценивать и приобретать данные для таких целей, как анализ, машинное обучение, бизнес-аналитика и принятие решений. Поставщиками обычно являются организации, правительства и частные лица, которые собрали или создали ценные датасеты.

Маркетплейс данных обычно включает в себя каталог данных. Многие маркетплейсы оценивают и улучшают качество датасетов, чтобы гарантировать их точность, полноту, согласованность и простоту использования. Некоторые из них предоставляют инструменты и услуги для очистки, преобразования и обогащения данных, что позволяет пользователям сэкономить время и силы на подготовку

¹ Иногда, однако, маркетплейс данных создается внутри компании и просто используется для помощи его пользователям в поиске и обмене данными.

данных к анализу. (Это особенно важно при интеграции данных из нескольких источников или при работе с неполными или противоречивыми данными.) После того как потребитель совершает покупку, маркетплейс может предложить инструменты, помогающие получить доступ к данным и интегрировать их в свои рабочие процессы, приложения или системы. Некоторые из них даже позволяют пользователям кастомизировать датасеты, объединяя данные из нескольких источников или фильтруя их по определенным критериям, чтобы они могли получить доступ именно к тем данным, которые им нужны.

Маркетплейсы данных должны иметь надежные меры безопасности для защиты пользовательских данных, обеспечения соблюдения правил защиты данных и сохранения конфиденциальности чувствительной информации. Некоторые из них предлагают способы легкого маскирования личных или конфиденциальных данных. Система цен и лицензионные соглашения на маркетплейсах данных четко определяют условия использования данных и обеспечивают соблюдение правовых норм как для поставщиков, так и для потребителей данных.

Многие маркетплейсы данных также предлагают встроенные инструменты аналитики и визуализации данных, которые позволяют анализировать и визуализировать (или просто просматривать) данные непосредственно в платформе. Другие распространенные функции и инструменты:

- рейтинги и обзоры датасетов;
- рекомендации по другим датасетам, которые могут быть интересны;
- профили покупателей для удобства повторного заказа, списки избранного и т. д.;
- возможности совместной работы и общения (доступ в сообщество);
- обучение и поддержка.

С точки зрения поставщиков, маркетплейсы данных предлагают способ монетизации активов данных, создавая новые источники дохода. Это может быть особенно полезно для организаций, которые собирают большие объемы данных, но не имеют ресурсов и опыта для их самостоятельного анализа и монетизации.

Маркетплейсы данных становятся все более популярными по мере того, как растет спрос на аналитику, основанную на данных, и доступны все больше разнообразных датасетов. Для создания маркетплейсов данных существует не так много продуктов, как для каталогов данных, поскольку маркетплейсы начали появляться только в начале 2020-х годов. Наиболее популярными являются Snowflake Marketplace, Datarade и Informatica Cloud Data Marketplace. Как поставщики, так и потребители получают пользу от более эффективного и прозрачного обмена данными, что может способствовать появлению новых продуктов и услуг, использующих большие данные.

Итоги

В этой главе были рассмотрены различные подходы к хранению данных, каждый из которых играет ключевую роль в современной бизнес-среде, использующей данные.

Мы обсудили хранение данных, включая витрины данных, хранилища оперативных данных и концентраторы данных, а затем перешли к процессам работы с данными. Управление мастер-данными (MDM, master data management) — это важнейший подход, который сосредоточен на управлении основными бизнес-сущностями для обеспечения точности, единообразия и согласованности данных в разнообразных системах и процессах. Затем мы перешли к описанию виртуализации данных, каталогов данных и маркетплейсов данных.

Понимая эти стратегии и их функциональные возможности, вы сможете лучше согласовывать архитектуру данных с потребностями своей организации и извлекать большую бизнес-ценность из данных.

Подходы к проектированию данных

В этой главе рассматриваются подходы к проектированию, направленные на обработку и структурирование данных, и то, как эти методы помогают создавать мощные, адаптируемые и надежные системы управления данными. Проще говоря, мы расскажем о стратегиях, которые помогают решить, как следует хранить, обрабатывать и предоставлять доступ к данным, чтобы повысить скорость и надежность создаваемых систем данных.

Чтобы внести ясность, разберемся, что такое проектирование, а также моделирование данных, о котором пойдет речь в главе 8. *Проектирование данных* сродни проектированию города. Нужно решить, где будут располагаться здания, какие дороги соединят разные части города и как будет двигаться транспорт. *Моделирование данных*, в свою очередь, больше похоже на проектирование отдельных зданий — оно касается расположения комнат, решений о том, как они будут связаны между собой, и определения, для чего нужна каждая комната.

В этой главе мы рассмотрим различные виды проектирования данных. Мы сравним и противопоставим такие методы, как OLTP и OLAP, которые по сути являются разными способами обработки и анализа данных. Мы также изучим такие концепции, как SMP и MPP, которые представляют собой стратегии более эффективной обработки данных. Затем мы узнаем о лямбда- и капша-архитектурах; это схемы обработки больших объемов данных в режиме реального времени. Наконец, мы поговорим о подходе под названием полиглотное хранение (polyglot persistence), который позволяет использовать разные технологии хранения данных в одном приложении.

Я не ставлю цели продвинуть какой-то один из этих подходов как лучшее универсальное решение, а стремлюсь помочь вам оценить сильные и слабые стороны каждого из них. Так вы сможете выбирать или комбинировать подходящие методы в зависимости от конкретных потребностей. К концу этой главы вы будете иметь более четкое представление о том, как создавать эффективные системы данных, способные адаптироваться к меняющимся потребностям и технологическому прогрессу.

Оперативная обработка транзакций (OLTP) и оперативная аналитическая обработка (OLAP)

*Оперативная обработка транзакций*¹ (OLTP, online transaction processing) — это тип информационной системы или приложения, когда онлайн-транзакции создания, чтения, обновления и удаления (CRUD) (см. главу 2) обрабатываются в режиме реального времени. Системы OLTP разработаны для поддержки высокого уровня конкурентности, это означает, что они могут обрабатывать большое количество транзакций одновременно. Обычно они используют реляционную модель (см. главу 8) и оптимизированы под малую задержку в доступе к данным, то есть могут обрабатывать транзакции очень быстро. В качестве примера можно привести приложения для точек продаж (POS-приложения), сайты электронной коммерции и решения для онлайн-банкинга.

В системе OLTP транзакции обычно обрабатываются с высокой скоростью и строго определенным образом. Например, транзакцией считается покупка в онлайн-магазине. В результате транзакции со счета покупателя списывается сумма, равная стоимости покупки, количество товаров в магазине уменьшается, а финансовая отчетность магазина обновляется, чтобы отразить продажу. В системах OLTP для хранения и управления данными используются различные СУБД, например Microsoft SQL Server и Oracle.

Системы OLTP часто противопоставляются *системам оперативной аналитической обработки*² (OLAP, online analytical processing), которые используются для анализа данных и создания отчетов с целью поддержки бизнес-аналитики и принятия решений. Системы OLAP оптимизированы под высокую производительность выполнения запросов, что позволяет конечным пользователям легко и быстро анализировать данные с разных точек зрения методом slice and dice (деления на фрагменты) и создавать отчеты и дашборды гораздо быстрее, чем в системах OLTP. Можно считать, что OLAP-система работает по принципу «однократная запись, многократное считывание» (в отличие от CRUD). Часто несколько баз данных OLTP используются в качестве источников, данные из которых поступают в хранилище данных, а уже затем наполняют базу данных OLAP (рис. 7.1). Однако иногда можно создать OLAP-базу данных непосредственно на основе OLTP-источников.

База данных OLAP обычно состоит из одного или нескольких OLAP-кубов. *OLAP-куб* (OLAP cube) — это место, где данные предварительно агрегируются, то есть уже обобщены и сгруппированы по определенным измерениям; конечные пользователи могут быстро получить доступ к ним по нескольким измерениям и уровням детализации, не дожидаясь завершения длительных запросов. Создание

¹ Встречается также термин «обработка транзакций в реальном времени». — *Примеч. ред.*

² Встречается также термин «интерактивная аналитическая обработка». — *Примеч. ред.*

OLAP-куба обычно предполагает использование *многомерной модели*, в которой для представления данных используется схема «звезда» или «снежинка». Более подробно эти схемы рассматриваются в главе 8.

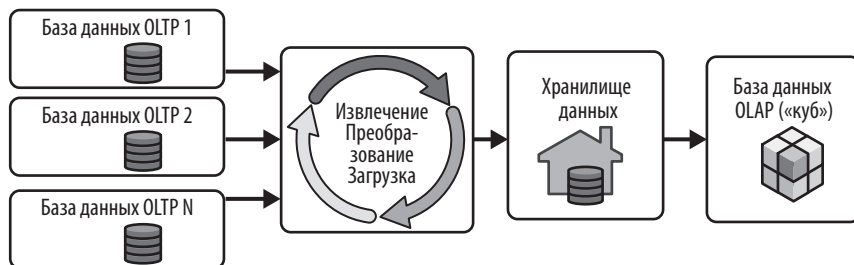


Рис. 7.1. Архитектура базы данных OLAP

Например, крупный ритейлер может использовать базу данных OLAP с OLAP-кубами для быстрого анализа данных о продажах. Эти предварительно агрегированные данные упорядочены по времени, что позволяет анализировать тренды продаж, будь то годовые, ежедневные или промежуточные. Данные также сортируются по местоположению, что облегчает географическое сравнение, например для оценки продаж товаров в разных городах. Кроме того, данные категоризируются по товарам, это упрощает отслеживание эффективности продаж отдельных позиций. Если региональному менеджеру нужно быстро просмотреть, как определенная категория товаров показала себя в его регионе в последний праздничный сезон, в OLAP-кубе уже подготовлены соответствующие данные. Менеджеру не придется ждать, пока длинный запрос отследит каждую отдельную продажу: соответствующие данные — отсортированные по товарам, регионам и времени — уже доступны. Это ускоряет процесс принятия решений.

Вы можете взять другую, более современную модель, называемую *табличной моделью данных* (tabular data model), которая обеспечивает такую же высокую производительность запросов, как и OLAP-куб. Табличная модель использует для представления данных табличную структуру, похожую на таблицу реляционной БД. Это делает ее более гибкой и простой, чем многомерная модель. В нашей книге многомерная модель называется OLAP-кубом, а табличная — табличным кубом, но имейте в виду, что на практике термин «куб» может использоваться для обозначения любой из этих моделей.

В таблице 7.1 приведено сравнение вариантов проектирования OLTP и OLAP.

OLAP-базы данных и хранилища данных (см. главу 4) — это связанные, но разные концепции, которые часто используются совместно. С помощью многомерной или табличной модели OLAP-базы данных позволяют анализировать данные, находящиеся в хранилище данных, более гибким и интерактивным способом, чем

выполнение традиционных SQL-запросов к хранилищу, содержащему большие объемы данных. Многомерные и табличные модели часто считаются *семантически-ми* слоями или моделями, это означает, что они обеспечивают уровень абстракции над схемами в хранилище данных.

Таблица 7.1. Сравнение систем OLTP и OLAP

	OLTP	OLAP/таблицы
Тип обработки	Транзакционная	Аналитическая
Характер данных	Оперативные данные	Консолидированные данные
Направленность	Ориентированность на применение	Ориентированность на предмет
Назначение	Работает над текущими бизнес-задачами	Помогает в принятии решений
Частота транзакций	Частые транзакции	Периодические транзакции
Виды операций	Короткие онлайн-транзакции: вставка, обновление, удаление	Множественные последовательные сканирования
Дизайн данных	Нормализованная база данных (3NF)	Денормализованная база данных
Где обычно используется	В розничных продажах и других системах финансовых операций	В области добычи данных (майнинг данных), в продажах и маркетинге
Время отклика	Мгновенное	От нескольких секунд до нескольких часов
Сложность запросов	Простые и мгновенные запросы	Сложные запросы
Шаблон использования	Повторяющееся использование	Использование на разовой основе
Характер транзакций	Короткие, простые транзакции	Сложные запросы
Размер базы данных	Гигабайты	Терабайты

Оперативные и аналитические данные

Оперативные данные — это данные, поступающие в режиме реального времени, которые используются для управления повседневными операциями и процессами. Они собираются, хранятся и обрабатываются системами OLTP. С их помощью можно сделать «мгновенный снимок» (снимок) текущего состояния бизнеса, чтобы убедиться, что операции проходят гладко и эффективно. Оперативные данные, как правило, имеют большой объем и помогают быстро принимать решения.

Аналитические данные являются результатом сбора и преобразования оперативных данных. Это ретроспективный взгляд на данные, которые поддерживаются и используются OLAP/табличной системой и хранилищами данных. С помощью

инструментов аналитики данных можно проводить анализ, чтобы понять тенденции, закономерности и взаимосвязи во времени. Аналитические данные оптимизированы для создания отчетов и визуализаций, а также для построения моделей машинного обучения. Как правило, они предоставляют картину данных за более длительный период, чем оперативные, имеют меньший объем, а также консолидированы и агрегированы. Аналитические данные чаще всего поступают в пакетном режиме и требуют больше времени на обработку, чем оперативные.

Обобщая, можно сказать, что оперативные данные используются для мониторинга и управления бизнес-процессами в режиме реального времени, а аналитические — для получения информации и принятия решений в течение более длительного периода времени. Оба типа данных необходимы для эффективного управления бизнесом, и они дополняют друг друга, формируя целостную картину операционной деятельности и результатов организации.

Можно считать, что OLTP — это технология, используемая для работы с оперативными данными, а OLAP/табличные системы совместно с хранилищами данных — технология для работы с аналитическими данными.

Симметричная многопроцессорная обработка и массово-параллельная обработка

Некоторые из первых реляционных баз данных использовали *симметричную многопроцессорную* (SMP) архитектуру, при которой машинная обработка данных выполняется несколькими процессорами, совместно использующими диск и память, и все это на одном сервере — например, SQL Server и Oracle (рис. 7.2, пунктирный блок слева). Чтобы повысить производительность такой системы, ее вертикально масштабируют, увеличивая количество процессоров и памяти на сервере. Это хорошо работает для баз данных OLTP, но не слишком практично для среды типа «однократная запись, многократное считывание» хранилища данных.

По мере того как в 1990-х годах хранилища данных становились все более популярными и в них стали поступать огромные объемы данных, производительность стала большой проблемой. Чтобы решить ее, появился новый тип архитектуры баз данных. При *массово-параллельной* (MPP) архитектуре база данных состоит из нескольких серверов, каждый из которых имеет несколько процессоров, и (в отличие от SMP) каждый процессор имеет собственную память и диск. Это позволяет горизонтально (а не вертикально) масштабировать базу данных, добавляя новые серверы.

MPP-серверы распределяют часть данных из БД по дискам каждого сервера (тогда как SMP-базы данных хранят все данные на одном диске). После этого запросы направляются на *управляющий*, или *главный, узел* (control node), разбивающий каждый запрос на несколько подзапросов, которые направляются на каждый сервер, называемый *вычислительным* (computer node) или *рабочим узлом* (worker node)

(рис. 7.2, пунктирный блок справа). Там подзапросы выполняются, а результаты с каждого вычислительного узла направляются обратно на управляющий узел, объединяются и возвращаются пользователю. Именно так работают такие решения, как Teradata и Netezza.

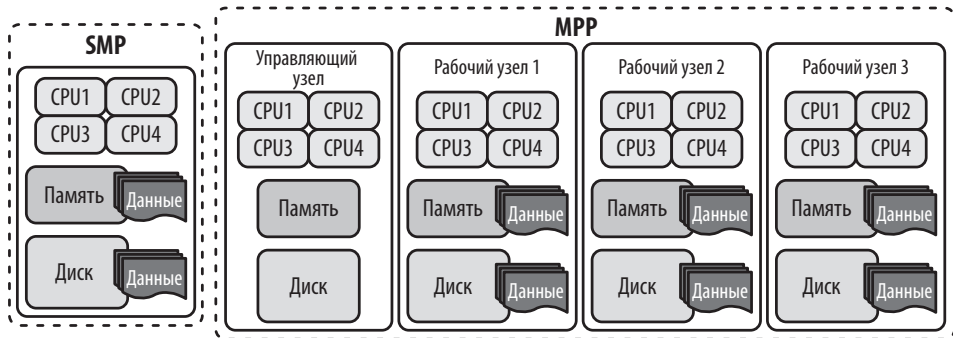


Рис. 7.2. SMP- и MPP-архитектуры баз данных

В порядке аналогии представьте, что у вашей подруги Фионы есть колода из 52 карт и она ищет туза червей. В среднем Фионе требуется около 15 секунд, чтобы найти эту карту. Вы можете «вертикально масштабировать» систему, заменив Фиону на вашего друга Макса, который работает быстрее. При помощи Макса среднее время уменьшится до 10 секунд — это незначительное улучшение. Так работают базы данных SMP.

А теперь представьте, что вы «горизонтально», а не «вертикально» масштабируете систему, заменяя одну Фиону 26 людьми, у каждого из которых в руках только две карты. Теперь среднее время поиска карты составляет всего 1 секунду. Именно так работают базы данных MPP.

Базы данных SMP и MPP начинались как локальные решения, и они до сих пор широко распространены, но сейчас существует множество аналогичных решений в облаке.

Лямбда-архитектура

Лямбда-архитектура — это архитектурный паттерн, предназначенный для обработки больших объемов данных методами пакетной и потоковой обработки в реальном времени. Идея заключается в том, чтобы получить исчерпывающие и точные результаты на основе пакетных данных, балансируя задержку, пропускную способность, и в то же время использовать потоковую обработку в реальном времени для просмотра онлайн-данных (например, датчиков IoT-устройств, ленты Twitter или лог-файлов компьютера). Оба набора результатов можно объединить перед слоем представления (presentation/serving layer).

Лямбда-архитектура устраняет разрыв между классическим «единым источником истины» и чрезвычайно востребованным решением типа «хочу получить это сейчас», работающим в режиме реального времени, объединяя традиционные системы пакетной обработки с инструментами потоковой передачи данных для удовлетворения обеих потребностей.

Архитектура Lambda включает три ключевых принципа.

Двойная модель данных

Одна модель используется для пакетной обработки данных (пакетный уровень), а другая — для обработки в режиме реального времени (поточковый уровень). Это позволяет системе обрабатывать данные как в пакетном режиме, так и в реальном времени и выполнять оба типа обработки масштабируемым и отказоустойчивым способом.

Единое унифицированное представление

Лямбда-архитектура использует единое унифицированное представление (так называемый *слой представления данных*) для отображения конечным пользователям результатов как пакетной обработки, так и обработки в реальном времени. Это дает пользователю возможность видеть полную и актуальную картину данных, даже несмотря на то, что они обрабатываются двумя разными системами.

Несвязанные уровни обработки

Лямбда-архитектура разделяет уровни пакетной обработки и обработки в режиме реального времени, поэтому их можно масштабировать независимо друг от друга, разрабатывать и поддерживать отдельно, что обеспечивает гибкость и простоту разработки.

На рис. 7.3 представлен обобщенный высокоуровневый дизайн лямбда-архитектуры.

Крайний слева на рис. 7.3 — *слой потребления данных* (data consumption layer). Именно здесь данные импортируются из всех систем-источников. Некоторые источники могут передавать данные в потоковом режиме, в то время как другие предоставляют данные не чаще чем ежедневно или ежечасно.

В верхней половине средней части находится *поточковый слой* (stream layer), который также называют *слоем быстрой обработки* (speed layer). Он обеспечивает инкрементное обновление данных, что делает его более сложным из двух средних слоев. В нем ради низкой задержки доступа к данным пожертвовано точностью, причем просматриваются только последние данные. Данные здесь могут отставать всего на несколько секунд, но при этом могут быть неочищенными. Данные на этом слое обычно хранятся в озере данных.

Под ним расположен *пакетный слой* (batch layer), который обрабатывает все данные одновременно и в конечном счете корректирует данные, поступающие на потоковый слой. Это единый источник истины, доверенный слой. Здесь обычно

выполняется множество ETL-процессов, а данные хранятся в традиционном хранилище данных или озере данных. Этот слой создается с использованием заранее определенного графика загрузки данных, обычно ежедневного или почасового, и включает импорт данных, хранящихся в данный момент на потоковом слое.



Рис. 7.3. Обобщенный вид лямбда-архитектуры

С правой стороны на рис. 7.3 находится *слой представления данных* (presentation layer), также называемый *сервисным* (service layer). Его можно считать чем-то типа посредника; принимая запросы, он решает, когда использовать пакетный слой, а когда — потоковый. Обычно по умолчанию используется пакетный слой, так как он содержит надежные данные, но если вы запросите с него данные с точностью до секунды (возможно, установив уведомления об определенных сообщениях в журнале событий, которые указывают на то, что сервер не работает), он возьмет их с потокового слоя. Этот слой должен обеспечивать баланс между получением данных, которым можно доверять, и данных, которые нужны прямо сейчас.

Лямбда-архитектура — отличный выбор для создания распределенных систем, которым необходимо обрабатывать данные как в пакетном режиме, так и в режиме реального времени, например рекомендательных систем и систем обнаружения мошенничества. Однако это не означает, что она подходит к любой ситуации. К потенциальным недостаткам лямбда-архитектуры относятся:

Сложность

Лямбда-архитектура включает в себя двойную модель данных и единое унифицированное представление. Это может оказаться слишком сложным при реализации и поддержке.

Ограниченная обработка в режиме реального времени

Лямбда-архитектура предназначена как для пакетной обработки, так и для обработки в реальном времени, но она может быть не столь эффективна при обработке больших объемов данных в реальном времени, как каппа-архитектура

(рассматривается в следующем разделе), которая специально спроектирована для работы в режиме реального времени.

Ограниченная поддержка обработки с сохранением состояния

Лямбда-архитектура предназначена для обработки данных без сохранения состояния (stateless) и может не слишком подходить для приложений, требующих сохранения состояния (stateful) в течение нескольких событий. Например, рассмотрим розничный магазин с рекомендательной системой, которая предлагает товары на основе шаблонов просмотра и покупок клиентов. Если бы эта система использовала лямбда-архитектуру, которая обрабатывает каждое событие отдельно, не сохраняя состояние, она могла бы не уловить путь клиента к покупке и намерения покупателя. Если покупатель ищет обувь, затем носки, а потом средство для чистки обуви, система без сохранения состояния может неправильно порекомендовать сопутствующие товары, например шнурки или приспособление для хранения обуви, поскольку она не учитывает последовательность событий. Кроме того, она может рекомендовать товары, которые покупатель уже добавил в корзину.

В целом лямбда-архитектуру следует рассматривать как вариант решения, если вам нужно создать распределенную систему, которая может обрабатывать как пакетные данные, так и данные, поступающие в режиме реального времени, и при этом обеспечивать единое унифицированное представление данных. Если же вам нужна система с сохранением состояния или обработка больших объемов данных в реальном времени, лучше рассмотреть каппа-архитектуру.

Каппа-архитектура

В отличие от лямбда-архитектуры, которая предназначена для работы с данными как реального времени, так и с пакетными, каппа-архитектура подходит только для обработки данных реального времени. Как и лямбда-архитектура, она рассчитана на работу с высоким уровнем конкурентности и большими объемами данных. На рис. 7.4 представлен обобщенный высокоуровневый дизайн каппа-архитектуры.

Три ключевых принципа каппа-архитектуры:

Обработка в режиме реального времени

Каппа-архитектура предназначена для обработки в реальном времени, что означает, что события обрабатываются сразу после их получения, а не пакетно позже. Это уменьшает задержку и позволяет системе быстро реагировать на изменение условий.

Единый поток событий

В каппа-архитектуре используется единый поток событий для хранения всех данных, проходящих через систему. Это обеспечивает легкую масштабируемость

и высокую отказоустойчивость, поскольку данные могут быть легко распределены по нескольким узлам.

Обработка без сохранения состояния

В каппа-архитектуре вся обработка выполняется без сохранения состояния. Это означает, что каждое событие обрабатывается независимо, не опираясь на состояние предыдущих событий. Это облегчает масштабирование системы, поскольку нет необходимости сохранять состояние на нескольких узлах.



Рис. 7.4. Обобщенный вид каппа-архитектуры

Слои в каппа-архитектуре точно такие же, как и в лямбда-архитектуре, за исключением того, что в каппа-архитектуре отсутствует пакетный слой.

К потенциальным недостаткам каппа-архитектуры относятся:

Сложность

Каппа-архитектура предполагает использование одного потока событий и обработку без сохранения состояния, что может быть сложнее в реализации и поддержке, чем другие архитектуры.

Ограниченная пакетная обработка

Каппа-архитектура предназначена для обработки данных в режиме реального времени и не слишком хорошо поддерживает пакетную обработку исторических данных. Если вам необходима пакетная обработка, возможно, стоит рассмотреть лямбда-архитектуру.

Ограниченная поддержка специальных запросов

Поскольку каппа-архитектура предназначена для обработки данных в режиме реального времени, она не слишком подходит для выполнения специальных запросов, предполагающих обработку больших объемов исторических данных.

В целом каппа-архитектура отлично подходит для создания распределенных систем, которым необходимо обрабатывать большие объемы данных в реальном

времени и которые должны быть масштабируемыми, отказоустойчивыми и иметь низкую задержку доступа к данным. В качестве примера можно привести стриминговые платформы и системы торговли на финансовых рынках. Однако для выполнения пакетной обработки или поддержки специальных запросов лучше подойдет лямбда-архитектура.

Обратите внимание, что лямбда- и каппа-архитектуры — это высокоуровневые паттерны проектирования, которые можно реализовать в рамках любой из архитектур данных, описанных в части III этой книги. Если вы используете одну из перечисленных там архитектур для создания решения, работающего как с пакетными данными, так и с данными реального времени, то она должна поддерживать лямбда-архитектуру; если же вы используете одну из этих архитектур для создания решения, работающего только с данными реального времени, то она должна поддерживать каппа-архитектуру.

Полиглотное хранение и полиглотные хранилища данных

*Полиглотное хранение*¹ (polyglot persistence) — модный термин, означающий использование нескольких технологий хранения данных для хранения различных типов данных *в рамках одного приложения или системы* в зависимости от того, как эти данные будут использоваться. Данные разных типов лучше всего хранить в разных хранилищах данных. Если коротко, полиглотное хранение означает подбор подходящего инструмента для выбранного сценария использования. Это та же идея, что лежит в основе полиглотного программирования (polyglot programming) (<https://oreil.ly/eowsh>), когда приложения пишутся на смеси языков, чтобы использовать сильные стороны разных языков для решения различных задач.

Напротив, *полиглотное хранилище данных* (polyglot data store) подразумевает существование нескольких хранилищ данных в рамках организации или предприятия. Каждое хранилище оптимизировано для данных определенного типа или сценария использования. Такой подход позволяет организациям задействовать различные хранилища для разных проектов или бизнес-подразделений, а не применять универсальный подход для всей организации.

Например, предположим, вы создаете платформу электронной коммерции, которая будет работать с данными многих типов (корзины покупок, товарные остатки, выполненные заказы и т. д.). Вместо того чтобы держать все это в одной базе данных, что потребовало бы большого количества преобразований, вы можете использовать полиглотное хранение и хранить данные каждого типа в той базе, которая лучше всего для этого подходит. Таким образом, платформа электронной коммерции может выглядеть примерно так, как показано на рис. 7.5.

¹ Или полиглотная персистентность. — *Примеч. ред.*

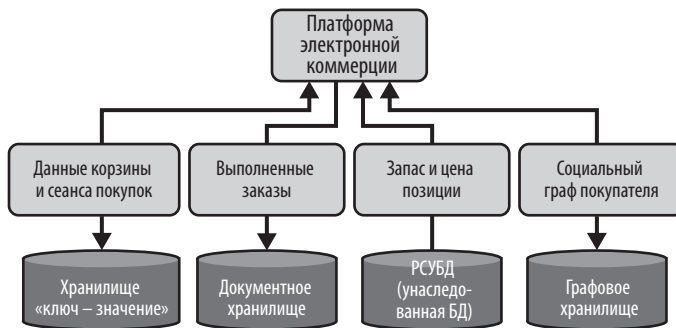


Рис. 7.5. Платформа электронной коммерции с использованием полиглотного хранения

В результате для данных каждого типа применяется наиболее подходящий инструмент. Как видно на рис. 7.5, база данных использует хранилище «ключ — значение» для данных корзины и сеанса покупок (что обеспечивает очень быстрый поиск), документное хранилище для выполненных заказов (что делает быстрым и простым хранение и извлечение данных о заказах), РСУБД для остатков и цен на товары (эти данные лучше всего хранить в реляционной базе данных благодаря их структурированности) и графовое хранилище для социальных графов покупателей (очень сложно хранить данные графов в неграфовом хранилище).

За все это придется заплатить сложностью, поскольку каждое решение для хранения данных означает освоение новой технологии. Но преимущества того стоят. Например, если использовать реляционные базы данных для нереляционных данных, это может значительно замедлить разработку и производительность приложений. Применение соответствующего типа хранилища окупается скоростью работы с данными.

Итоги

В этой главе были рассмотрены различные концепции архитектур и философии проектирования, которые лежат в основе эффективных систем обработки данных.

В начале мы познакомились с двумя основными типами систем обработки данных: оперативной обработкой транзакций (OLTP) и оперативной аналитической обработкой (OLAP). Системы OLTP предназначены для быстрых, надежных и коротких транзакций, как правило, в операционных базах данных, которые обеспечивают ежедневную бизнес-деятельность. В отличие от них системы OLAP, обычно в хранилищах данных, поддерживают сложные аналитические запросы, агрегацию и вычисления, используемые для принятия стратегических решений. Далее мы рассмотрели различия между оперативными и аналитическими данными.

Кроме того, мы обсудили различия между архитектурами симметричной многопроцессорной обработки (SMP) и массово-параллельной обработки (MPP). Затем

подробно рассмотрели две современные архитектуры обработки Big Data: лямбда- и каппа-архитектуру. И наконец, изучили концепции полиглотного хранения и полиглотных хранилищ данных, которые продвигают идею использования технологий баз данных, лучше всего подходящих для определенных потребностей и характеристик рабочей нагрузки конкретных данных.

Начиная со следующей главы, мы переключим внимание с хранения и обработки на принципы и методы моделирования данных — важнейшее связующее звено между необработанными сырыми данными и значимыми аналитическими результатами их обработки. Такие подходы к моделированию, как реляционный и размерный, а также общая модель данных представляют собой базовую структуру, позволяющую эффективно использовать и интерпретировать данные в различных приложениях.

По мере того как мы будем углубляться в эти вопросы, вы увидите, как моделирование данных позволяет эффективно использовать рассмотренные нами решения для хранения и обработки данных, служа шаблонами для преобразования необработанных данных в полезную практическую информацию.

Подходы к моделированию данных

Моделирование данных — это концептуальный высокоуровневый метод проектирования базы данных. Существует несколько типов моделирования данных, включая реляционное и размерное (dimensional) моделирование. Процесс моделирования включает в себя определение данных, которые необходимо хранить, а затем создание структурированного представления этих данных и связей между ними, организованных в таблицы и столбцы. Таблицы и столбцы служат *логическим* представлением базы данных, где *физические* данные, хранящиеся в этих таблицах и столбцах, могут на самом деле находиться в реляционной базе данных или озере данных.

Моделирование данных можно использовать с базой данных любого типа: реляционной, размерной, NoSQL и так далее. Я рекомендую уделять достаточно много времени моделированию данных, чтобы убедиться, что ваша база данных логична, эффективна и проста в использовании. Это обеспечит максимальную производительность и упростит последующие извлечение и анализ данных.

Реляционное моделирование

Реляционное моделирование, разработанное Эдгаром Ф. Коддом в 1970 году (как уже говорилось в главе 2), представляет собой подробную методику моделирования, используемую для проектирования базы данных. Оно включает в себя организацию данных в таблицы и определение отношений между таблицами. В реляционных базах данных и реляционных хранилищах данных каждая таблица состоит из строк (также называемых записями или кортежами) и столбцов (также называемых полями или атрибутами). Каждая строка представляет собой уникальный экземпляр данных, а каждый столбец — определенную часть информации об этих данных.

Ключи

В реляционном моделировании отношения между таблицами, строками и столбцами определяются с помощью первичных и внешних ключей. *Первичный* (primary) *ключ* — это уникальный идентификатор для каждой записи в таблице, гарантирующий, что ни одна из двух строк данных не имеет одинакового значения ключа. *Внешний* (foreign) *ключ* — это столбец в дочерней таблице, который ссылается

на первичный ключ в родительской таблице. Он используется для установления отношения между двумя таблицами, чтобы обеспечить целостность данных. Например, у вас может быть уникальный ключ в таблице товаров под названием **ProductKey** («ключ продукта»), который ссылается на внешний ключ в таблице фактических продаж под названием **ProductKey**. *Естественный (natural) ключ* — это поле, которое уже существует в таблице и является уникальным для каждой строки, например номер социального страхования человека или серийный номер товара. В реляционном моделировании естественный ключ обычно используется в качестве первичного ключа.

Диаграммы «сущность — связь»

Обычно реляционное моделирование начинается с диаграммы «сущность — связь» (entity-relationship), или ER-диаграммы: высокоуровневой визуальной структуры базы данных, которая представляет сущности (данные) и отношения между ними. В ней используются блоки для сущностей и соединительные линии для отношений. На рис. 8.1 показан пример ER-диаграммы¹. После ее создания можно приступить к построению более подробной реляционной модели, которая представляет собой реальные таблицы и столбцы базы данных.

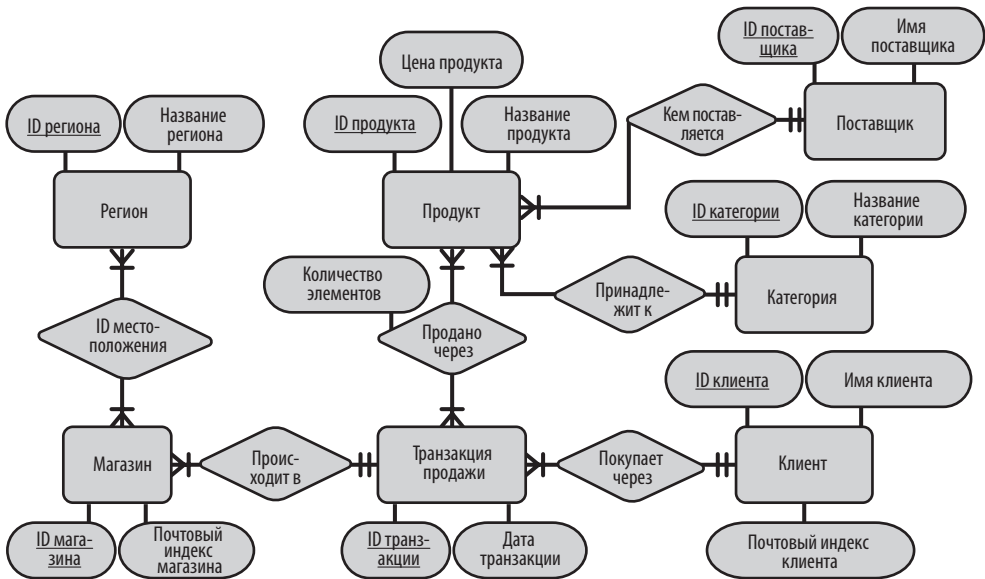


Рис. 8.1. Диаграмма «сущность — связь» (ER-диаграмма)

¹ Здесь и далее на схемах реляционной и размерной моделей все названия таблиц и столбцов переведены на русский язык для удобства читателя. — *Примеч. ред.*

Правила и формы нормализации

Далее применяются *правила нормализации*, которые представляют собой способ декомпозиции сложной базы данных на более мелкие и простые таблицы. При этом минимизируются избыточность и зависимость и повышается целостность данных, что делает базу данных более эффективной и простой в обслуживании и управлении. Это достигается применением все более высоких уровней нормализации.

Говорят, что таблица находится в *первой нормальной форме (1NF)*, если она удовлетворяет всем перечисленным ниже условиям:

- Имеет первичный ключ. Например, в таблице **Students** (Студенты) первичным ключом может быть **StudentID** (ID студента).
- Каждый атрибут в таблице содержит единственное значение, а не список значений. Например, в таблице **Students** в каждом столбце (атрибуте), скажем **Name** (Имя), хранится единственное значение.
- В ней нет повторяющихся групп столбцов. Например, в таблице **Students** не должно быть нескольких столбцов **Course** (Курс), таких как **Course1**, **Course2** и **Course3**. Вместо этого каждая пара «студент — курс» должна иметь собственную строку.

Таблица находится во *второй нормальной форме (2NF)*, если она удовлетворяет всем вышеперечисленным условиям 1NF, а также следующему.

- каждый дополнительный признак (неключевой атрибут) в записи базы данных должен полностью определяться своим уникальным идентификатором (первичным ключом), а не чем-либо еще. Например, в таблице **Students** со столбцами **StudentID**, **Name** и **Major** (Специальность) неключевой атрибут **Major** должен определяться исключительно **StudentID** (первичным ключом), а не атрибутом **Name** или другим атрибутом в таблице.

Таблица находится в *третьей нормальной форме (3NF)* тогда и только тогда, когда она удовлетворяет всем вышеперечисленным условиям 1NF и 2NF, а также следующему:

- каждый неключевой дополнительный атрибут в таблице должен быть связан с основным идентификатором (первичным ключом) напрямую, а не через другой атрибут. Например, в таблице **Students** с атрибутами **StudentID**, **Major** и **DepartmentHead** (Заведующий кафедрой) (где **DepartmentHead** является руководителем **Major**), атрибут **DepartmentHead** не должен зависеть от **Major**, который, в свою очередь, не должен зависеть от **StudentID**.

Большинство реляционных моделей, особенно для баз данных OLTP (см. главу 7), находятся в 3NF. Это должно быть вашей конечной целью.

Реляционная модель использует *нормализованную схему базы данных*, где данные хранятся в одном месте и организованы в несколько таблиц со строго

определенными отношениями между ними. Это помогает обеспечить целостность, но также может сделать запросы более трудоемкими, поскольку для получения нужных данных базе данных, скорее всего, придется объединить несколько таблиц.

На рис. 8.2 показано, как может выглядеть реляционная модель данных о продажах.

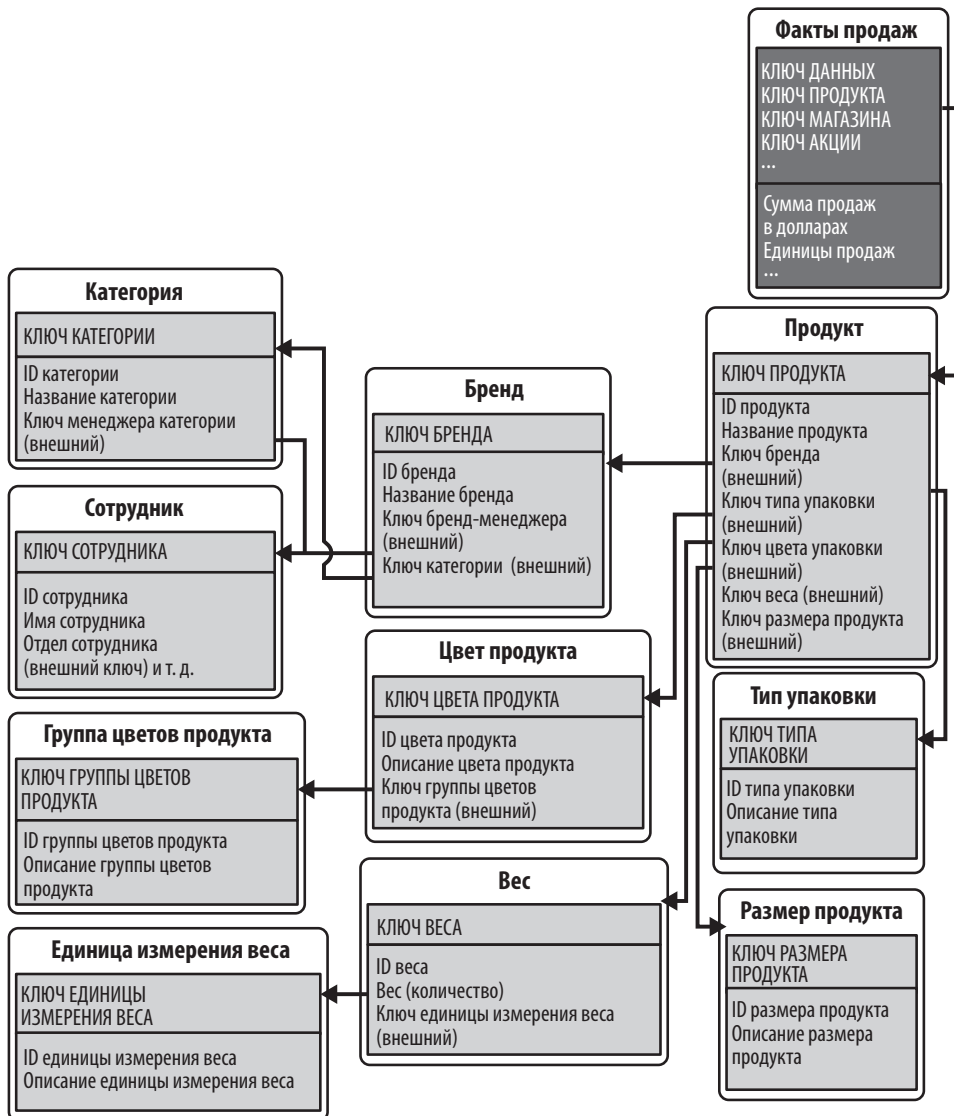


Рис. 8.2. Пример реляционной модели данных о продажах

Отслеживание изменений

Важно отслеживать изменения данных с течением времени в рамках реляционной модели. Чтобы вести запись о прошлых данных и их изменениях, часто используются исторические таблицы. *Историческая таблица* (history table) обычно представляет собой копию исходной таблицы с дополнительными столбцами для отслеживания изменений, такими как метка даты/времени, идентификатор пользователя, а также значения «до» и «после» для каждого столбца в таблице. Исторические таблицы могут быть полезны для аудита, создания отчетов и восстановления данных, помогая обеспечить целостность данных и облегчая понимание того, как они менялись с течением времени.

Размерное моделирование

Размерное моделирование (dimensional modeling) появилось в 1996 году как способ сделать запросы и анализ данных более эффективными путем организации данных в таблицы в виде фактов и измерений. Размерные модели необходимы, когда запросы и создание отчетов по реляционной модели становятся слишком медленными или сложными. Размерные модели обычно используют реляционную модель в качестве источника данных.

Факты, измерения и ключи

В размерных моделях используются *факты* и *измерения*.

Факты

Факты в данном контексте — это фрагменты данных, обычно числовые значения, используемые для измерения чего-либо, например продаж, заказов или выручки. Факты можно агрегировать или суммировать из соображений производительности, например путем вычисления средних величин или подсчета количества записей, так что эти значения уже вычислены и их не нужно вычислять в процессе выполнения запроса.

Измерения

Измерения описывают характеристики данных, такие как время, продукт, клиент или местоположение. Обычно они представлены в виде иерархий, причем каждый уровень иерархии содержит более подробное описание данных, чем предыдущий.

Другой способ понять суть размерного моделирования — вспомнить, что таблицы фактов содержат метрики, а таблицы измерений — *атрибуты* для метрик в таблицах фактов.

Вместо естественных ключей, используемых в реляционном моделировании, в размерном моделировании применяются *суррогатные ключи*: искусственные

значения, созданные специально (и обычно автоматически) для использования в качестве первичного ключа таблицы. Суррогатный ключ часто используется, когда естественный ключ недоступен или не подходит в качестве первичного, что часто происходит при размерном моделировании.

Естественные ключи обычно более содержательны и понятны, чем суррогатные, поэтому они могут упростить использование и поддержку базы данных. Однако у естественных ключей есть и недостатки.

- Они могут быть длиннее и сложнее, чем суррогатные, и поэтому с ними сложнее работать.
- Они часто содержат чувствительную информацию, что может вызвать проблемы с конфиденциальностью.
- Они могут создавать проблемы из-за дублирования и нестандартизированных форматов. Например, вы используете идентификаторы клиентов в качестве естественного ключа. Если ваша компания приобретает другую, которая использует совпадающие идентификаторы клиентов или иной формат идентификаторов (например, символы вместо целых чисел), могут возникнуть конфликты. Главное, что нужно учитывать: дубликаты могут стать проблемой при слиянии или сравнении данных в разных системах, а несоответствие форматов может еще больше усложнить процесс интеграции.

Отслеживание изменений

Для отслеживания изменений в таблице при размерном моделировании используется механизм *медленно меняющихся измерений* (Slowly Changing Dimensions, SCD). SCD подразделяются на три типа.

Тип 1

SCD первого типа перезаписывают существующие данные новыми, при этом старые данные не сохраняются. Это самый простой и распространенный тип SCD, он часто используется, когда изменения данных незначительны или когда старые данные больше не нужны, например при исправлении номера телефона клиента.

Тип 2

SCD второго типа хранят несколько версий данных: новые данные и запись старых данных. Используйте этот тип SCD, когда вам нужно отслеживать изменения данных с течением времени и сохранять старые данные. Например, предположим, что шесть месяцев назад компания проанализировала данные о продажах и обнаружила, что самые высокие продажи совершаются в штате Нью-Йорк. Если с тех пор некоторые клиенты переехали из Нью-Йорка в Нью-Джерси, повторное составление того же отчета без учета этих изменений

покажет неточные, более низкие данные о продажах в Нью-Йорке. Это приведет к ошибочному восприятию исторических данных, свидетельствующих о снижении продаж в Нью-Йорке, что может повлиять на стратегические решения. Таким образом, если клиент переезжает из одного штата США в другой, SCD компании второго типа будет хранить данные как по старому штату, так и по новому.

Тип 3

SCD третьего типа создают новую запись для каждого изменения, чтобы вести полную историю данных. Этот тип SCD является как самым сложным, так и самым гибким. Например, когда клиент переезжает из одного штата США в другой, SCD третьего типа будет хранить всю старую запись и всю новую запись, которая может включать десятки полей, а не только поле штата.

Одно из ключевых различий между историческими таблицами в реляционном моделировании и SCD заключается в их уровне детализации. Исторические таблицы отслеживают изменения данных на уровне отдельных записей, в то время как SCD — на уровне измерений. И если таблицы истории обычно используются для отслеживания изменений в любом типе данных, то SCD предназначены специально для отслеживания изменений в таблицах измерений в размерной модели.

На рис. 8.3 показана размерная модель продаж, полученная из реляционной модели на рис. 8.2.

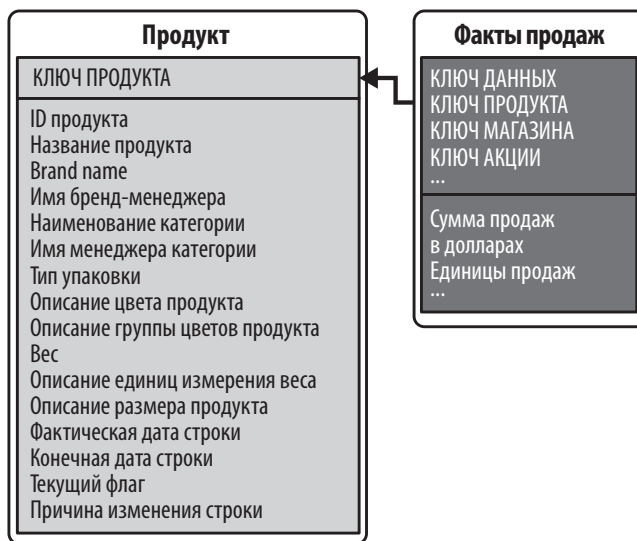


Рис. 8.3. Размерная модель данных о продажах

Денормализация

При размерном моделировании применяется процесс, называемый *денормализацией* (denormalization), когда избыточные копии данных включаются в несколько таблиц. Это позволяет сократить количество таблиц, и запросы выполняются гораздо быстрее, поскольку для этого не требуется объединять много таблиц. Меньшее количество объединений также удобнее для конечных пользователей, создающих отчеты. Однако это означает, что избыточные копии данных должны быть синхронизированы в разных таблицах для обеспечения целостности, а это требует тщательной поддержки. Избыточные данные также требуют больше места для хранения, что может несколько увеличить затраты.

Из рис. 8.3 следует, что поле «Наименование категории» (CategoryName) включено в список ключей продуктов. В результате наименование категории повторяется во всех записях о данном продукте. Напротив, в реляционной модели, представленной на рис. 8.2, в записях о продукте указывается ID категории (CategoryID). В размерной модели, если наименование категории изменится, его придется менять в нескольких записях в таблице продукта; в реляционной модели его потребуется изменить только один раз (в таблице «Категории»).

Чем больше таблиц в реляционной модели, тем более полезной будет размерная модель. Если в реляционной модели всего несколько таблиц, то размерная модель может и не понадобиться.

Схема базы данных — это логическая схема, которая описывает структуру, отношения, ограничения и другие элементы того, как данные организованы и связаны в базе данных. При размерном моделировании используется множество типов схем, например схемы типа «снежинка» и схемы с несколькими таблицами фактов. *Схема «звезда»* — это метод размерного моделирования, когда в центре находится таблица фактов, окруженная несколькими связанными с ней по внешнему ключу таблицами измерений. Диаграмма такой схемы выглядит как звезда (рис. 8.4), отсюда и название.

Подводя итог, можно сказать, что реляционная модель отражает, как работает часть бизнеса, в то время как размерная содержит детали, необходимые для понимания, насколько хорошо работает бизнес. Реляционную модель легче построить, если исходная система уже является реляционной, но размерная модель проще для бизнес-пользователей и, как правило, лучше подходит для аналитических запросов.

Общая модель данных

Общая модель данных (CDM, common data model) — это стандартизированная структура для хранения и упорядочивания данных, которая обычно используется при построении хранилища данных. Она обеспечивает согласованный способ представления данных в таблицах и связей между таблицами, что облегчает понимание данных любой системой или приложением.

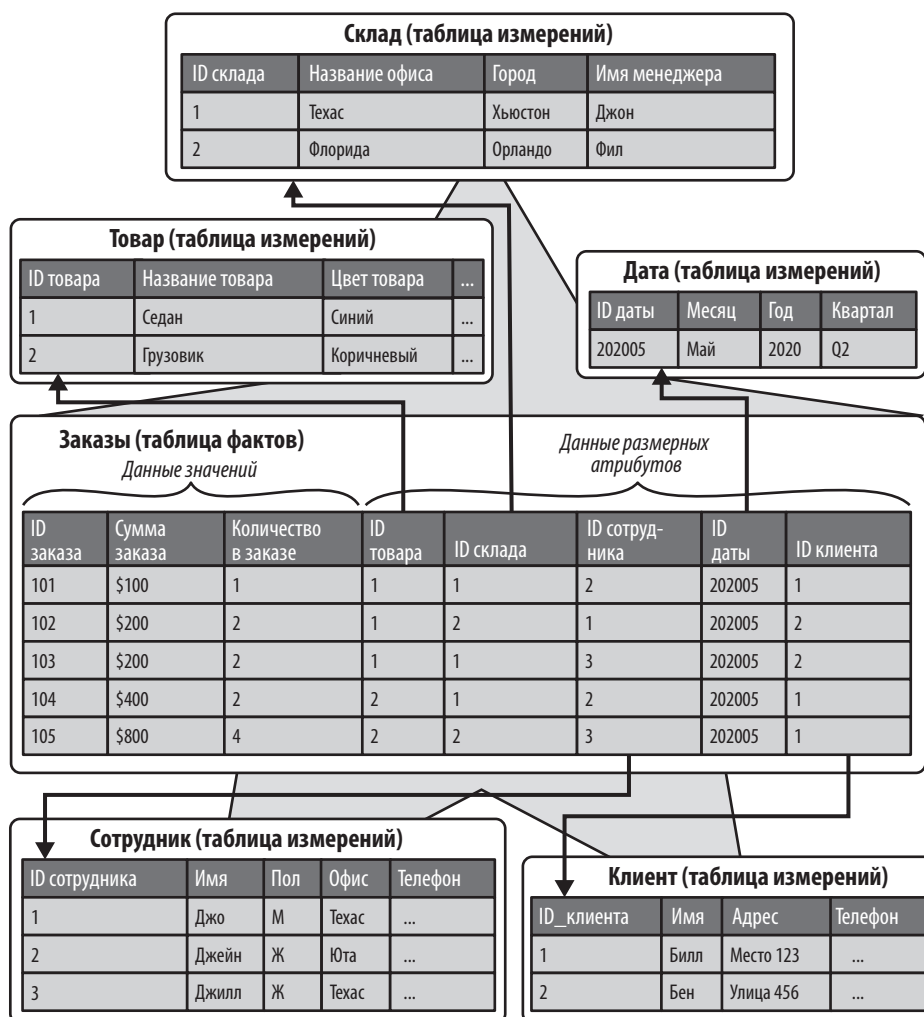


Рис. 8.4. Пример схемы «звезда»

Представьте, что у вас есть множество систем-источников, из которых вы хотите импортировать данные в свое хранилище. Часть из них — это приложения для управления взаимоотношениями с клиентами (CRM-приложения) от разных поставщиков, имеющие разные форматы (то есть таблицы и поля отличаются друг от друга). Вам нужно решить, какой формат таблиц использовать в хранилище данных. Выберите ли вы одну из моделей CRM-поставщиков? Нет. Лучше создать новую модель, которая сможет работать со всеми форматами этих поставщиков.

Эта новая модель (рис. 8.5) — ваша общая модель данных, CDM. Ее создание потребует много сил, но, если вы все сделаете правильно, ваша CDM сможет поддерживать текущие и будущие данные от любого CRM-поставщика. К счастью, вам не придется создавать ее с нуля. Многие облачные провайдеры и поставщики ПО имеют очень надежные CDM для таких отраслей, как банковское дело, здравоохранение и розничная торговля. Если вы сможете настроить готовую модель, то сэкономите много часов на моделировании, снизите риски и обеспечите всем приложениям в вашей организации доступ к данным на общем языке.

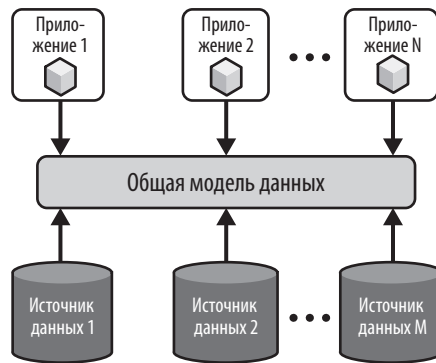


Рис. 8.5. Архитектура общей модели данных

Модель Data Vault

Созданная Даниэлом Линстедтом (Daniel Linstedt) в 2000 году концепция *моделирования Data Vault* — это целая методология, специально разработанная для использования в системах хранилищ данных и бизнес-аналитики. Ее основная цель — обеспечить гибкий, масштабируемый и стандартизированный способ моделирования и управления историческими данными.

Модель Data Vault (показана на рис. 8.6) строится на базе трех типов сущностей.

Хабы (Hubs)

Хабы — это центральные *бизнес-сущности* в модели, представляющие ключевые бизнес-понятия, такие как клиенты или продукты. Хабы обычно моделируются как отдельная таблица с уникальным идентификатором (например, первичным ключом) и набором атрибутов.

Ссылки (Links)

Таблицы-ссылки моделируют *связи* между хабами, как правило, в виде отдельной таблицы. Таблица-ссылка содержит набор внешних ключей, которые ссылаются на первичные ключи связываемых хабов.

Сателлиты (Satellites)

Сателлиты хранят *описательные атрибуты* хаба или таблицы-ссылки, такие как изменения данных с течением времени. Обычно они моделируются как отдельные таблицы и связаны с хабом или таблицей-ссылкой через внешний ключ.

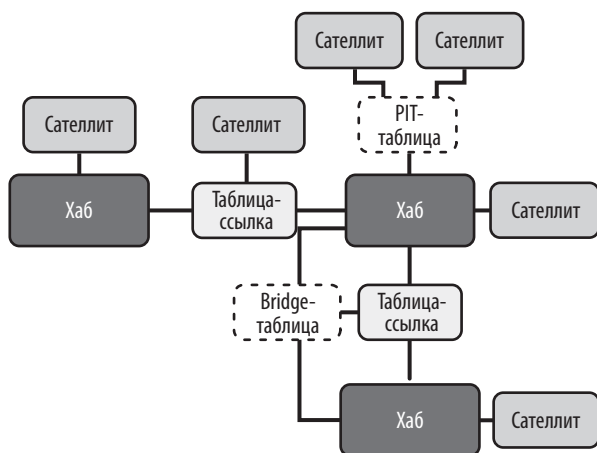


Рис. 8.6. Устройство Data Vault¹

Модели данных, в основе которых лежит Data Vault, позволяют отслеживать происхождение данных, проводить аудит данных и обеспечивать соблюдение правил и стандартов. Они часто используются в сочетании с другими методами моделирования данных, такими как размерное моделирование, для создания комплексной и гибкой архитектуры данных, которую можно легко интегрировать с другими системами и приложениями. В спектре рассмотренных нами моделей Data Vault находится где-то между 3NF-моделью данных и схемой «звезда»; она хорошо подходит для организаций с большими объемами данных и сложными взаимосвязями между данными. Модель Кимбалла (которая будет рассмотрена далее в разделе «Методология Кимбалла — подход „снизу вверх“» на стр. 148) лучше подходит для компаний с более простыми потребностями в данных.

К основным недостаткам методологии Data Vault относятся:

Сложность

Моделирование Data Vault может быть слишком сложной задачей с точки зрения внедрения и поддержки, особенно для организаций, которые не знакомы с этой методикой. Оно требует значительных затрат ресурсов и наличия

¹ PIT-таблица (point-in-time) на рисунке — вспомогательная конструкция в Data Vault, которая упрощает доступ к данным. Bridge-таблица является средством упрощения и ускорения запросов между различными хабами, ссылками и их сателлитами. — *Примеч. ред.*

экспертных знаний, а понимание модели данных и навигация по ней могут быть затруднены без надлежащей документации.

Дублирование данных

Моделирование Data Vault часто приводит к дублированию, поскольку данные хранятся на атомарном уровне детализации. Это может увеличить затраты на хранение и усложнить обеспечение согласованности данных.

Производительность

Моделирование Data Vault может привести к появлению большого количества таблиц и связей, снижая производительность запросов. Для достижения приемлемой производительности могут потребоваться более сложные запросы и стратегии индексирования.

Отсутствие стандартизации

Моделирование Data Vault — это относительно новая и малоиспользуемая технология, поэтому стандартизированный подход пока отсутствует. Найти инженеров для создания и поддержки Data Vault может оказаться непросто.

Методологии хранения данных Кимбалла и Инмона

Билл Инмон (Bill Inmon) считается отцом-основателем хранения данных благодаря своей ранней пионерской работе в этой области, опубликованной в конце 1980-х годов. Первую книгу о хранилищах данных он написал в 1992 году, она сыграла значительную роль в популяризации и продвижении этой концепции¹. Подход Инмона к хранилищам данных часто характеризуется как подход к витринам данных «сверху вниз», использующий реляционное моделирование.



Термины «сверху вниз» и «снизу вверх» здесь не имеют ничего общего с нисходящим и восходящим подходами для реляционных хранилищ данных и озер данных, описанными в главах 4 и 5. Те подходы связаны с продуманной заранее (или нет) работой при построении архитектуры данных, включающей реляционное хранилище и озеро данных. Напротив, подходы Кимбалла и Инмона (помимо всего прочего) посвящены тому, как витрины данных используются при построении архитектуры данных, включающей только реляционное хранилище, без озера данных.

¹ Инмон, вероятно, впервые ввел термин «хранилище данных» примерно в 1985 году; его книга *Building the Data Warehouse* (Wiley, 1992), доступная в Internet Archive (<https://oreil.ly/DkEV->), была первой книгой по этой теме. Однако Барри Девлин (Barry Devlin) и Пол Мерфи (Paul Murphy) опубликовали первое подробное описание архитектуры хранилища данных в том виде, в котором мы его знаем сегодня, в статье *An Architecture for a Business and Information System* (<https://oreil.ly/M50Xo>), IBM Systems Journal, 1988 (доступна на сайте 9sight Consulting (<https://oreil.ly/g4NIX>)).

Ральф Кимбалл (Ralph Kimball), еще один первопроходец в области создания хранилищ данных, популяризировал схему «звезда» при проектировании хранилищ данных в своей первой книге *The Data Warehouse Toolkit: Practical Techniques for Building Dimensional Data Warehouses*¹ (Wiley, 1996). Подход Кимбалла часто характеризуется как подход «снизу вверх», его также называют *размерным моделированием* или *методологией Кимбалла*.

Вокруг методологий построения хранилищ данных Инмона и Кимбалла ведутся многолетние споры. В этом разделе мы более подробно обсудим обе эти методики и развеем некоторые распространенные мифы.

Методология Инмона — подход «сверху вниз»

В методологии Инмона особое внимание уделяется интеграции данных из нескольких источников и их представлению в таком виде, который помогает при принятии решения. Подход «сверху вниз» — это традиционная, строгая и четко определенная методология, которой часто отдают предпочтение в крупных и сложноорганизованных компаниях, которые оперируют большими объемами данных и ценят хорошее управление данными, их качество и соответствие нормативным требованиям. В основном ею занимается технологический отдел; конечные пользователи принимают только пассивное участие.

В этой архитектуре, приведенной на рис. 8.7, данные сначала как можно быстрее извлекаются из каждой исходной OLTP-системы (см. главу 7) во временные реляционные промежуточные таблицы, без преобразований или очистки.

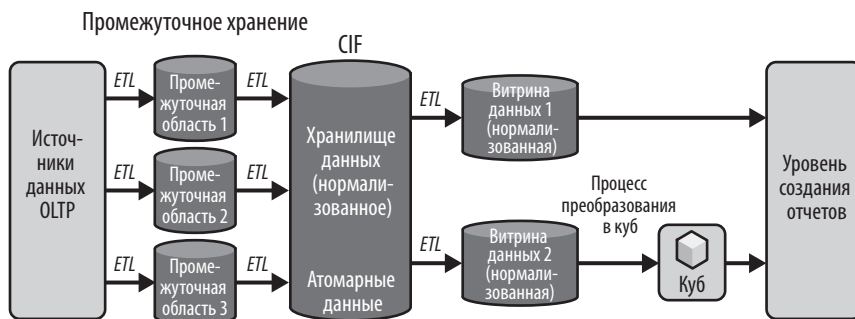


Рис. 8.7. Хранилище данных, спроектированное по методологии Инмона

Преобразование данных может значительно увеличить загрузку процессора и снизить пропускную способность систем-источников, поэтому такой подход позволяет избежать замедления работы систем-источников. Кроме того, промежуточные

¹ Кимбалл Р. «Инструментарий хранения и анализа данных. Полное руководство по размерному моделированию».

таблицы полезны для целей аудита, и, если вам понадобится повторно запустить преобразования, вы можете извлечь данные из этих таблиц, не обращаясь к системе-источнику.

На следующем этапе данные копируются в то, что Инмон называет *корпоративной информационной фабрикой* (CIF, Corporate Information Factory), где все данные централизованы и хранятся на атомарном уровне детализации (настолько детально, насколько это возможно) в третьей нормальной форме. Можно представить это как корпоративное хранилище данных, которое является единой версией истины. В этом разделе я использую термин «*корпоративное хранилище данных*», а не просто «*хранилище данных*», поскольку большинство решений, созданных в первые годы существования хранилищ данных, включали данные из всех сфер деятельности организации. В настоящее время некоторые компании имеют несколько хранилищ данных, но чаще всего решения создаются с использованием только одного «корпоративного» хранилища.

CIF ориентирована на моделирование и проектирование данных, при этом особое внимание уделяется управлению данными и их качеству. На ее основе можно формировать *зависимые витрины данных*: отдельные физические подмножества данных, обычно создаваемые для того, чтобы предоставить каждому подразделению необходимые только ему данные. Витрины данных называются «зависимыми», потому что они зависят только от данных из CIF и не получают никаких данных откуда-либо еще. Такую архитектуру также называют «*ступица и спицы*» (hub-and-spoke) за свою форму. Пользователи могут отправлять запросы к витринам данных для создания отчетов или подмножеств данных — моделям небольших размеров, называемым *кубами* (см. главу 7), — для конкретных целей. Например, аналитикам может понадобиться только объем продаж. Данные в кубе зачастую являются агрегированными.

Методологию Инмона называют нисходящей («сверху вниз»), поскольку она начинается с определения бизнес-потребностей конечных пользователей. Далее следуют моделирование и проектирование данных, затем интеграция и загрузка данных и, наконец, предоставление доступа и создание отчетов. Все данные централизуются в CIF, а затем их подмножества копируются в предметно-ориентированные витрины данных.

CIF недоступна для конечных пользователей; они получают доступ к данным через витрины данных или кубы. Один из ее недостатков заключается в том, что данные дублируются: для CIF, витрины данных и куба. Это приводит к увеличению затрат на хранение, усложняет обслуживание и создает проблемы синхронизации всех трех копий.

Методология Кимбалла — подход «снизу вверх»

Моделирование по методологии Кимбалла («снизу вверх»), или «гибкой» методологии, начинается аналогично методологии Инмона, с извлечения необработанных

данных из каждой из исходных OLTP-систем (см. главу 7) во временные реляционные таблицы без преобразования или очистки. Затем в методологии Кимбалла следует переходить к фазе интеграции данных и обеспечения их качества, за которой следуют фазы моделирования, проектирования и организации доступа к данным. Архитектура такого типа приведена на рис. 8.8.

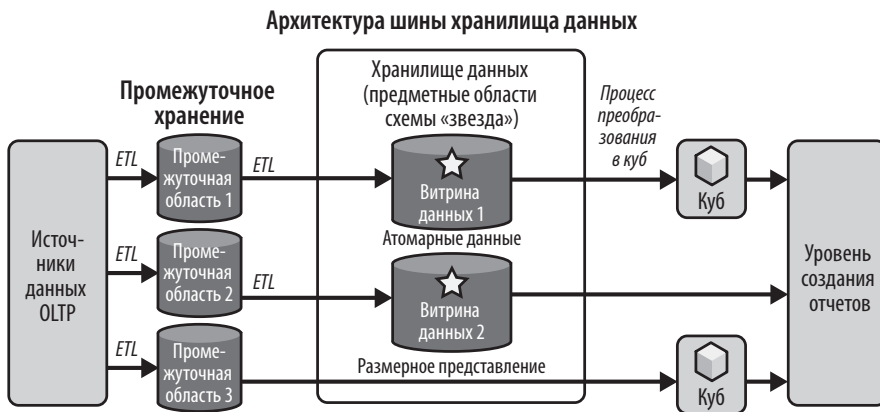


Рис. 8.8. Хранилище данных, спроектированное согласно подходу Кимбалла

Данные из промежуточных таблиц затем копируются в критически важные *независимые витрины данных*, которые обслуживают аналитические потребности подразделений. Витрины данных являются независимыми, поскольку они сфокусированы на конкретной предметной области или бизнес-процессе, в отличие от подхода Инмона (CIF), предполагающего хранение всех данных и управление ими в едином центральном репозитории. Эти независимые витрины данных используют размерную модель, где данные хранятся на атомарном или агрегированном уровне, в зависимости от потребностей пользователей. Витрины данных для обеспечения согласованности данных интегрируются через *шину хранилища данных* (DW bus), иногда называемую *информационной шиной*, которую можно рассматривать как согласованное хранилище данных. Все данные находятся в едином унифицированном представлении и выглядят так, как будто хранятся в одной базе данных в одном централизованном физическом расположении, но на самом деле они находятся в нескольких БД (и зачастую в разных местах). Такие витрины данных децентрализованы, но они не обязательно должны находиться в отдельных физических хранилищах данных — это могут быть несколько БД на одном сервере или даже одна БД, разделенная разными именами схем.

Для обеспечения согласованности между источниками данных витрины данных интегрируются с помощью *согласованных измерений* (conformed dimensions) — измерений, которые были стандартизированы и унифицированы для использования в нескольких таблицах фактов. Например, если измерение «клиент» используется

в нескольких таблицах фактов, имена и идентификаторы клиентов должны быть одинаковыми во всех этих таблицах.

Из витрин данных можно копировать подмножества данных в кубы. Можно даже построить куб непосредственно из промежуточных таблиц, используя размерное представление, если это обеспечит все необходимые типы доступа. *Размерное представление* (dimensionalized view) — это SQL-представление, которое организует данные в размерную модель. Отчеты можно делать из куба или из витрины данных. Методология Кимбалла бизнес-ориентирована, и конечные пользователи являются ее активными участниками. Данные копируются либо дважды — в витрину данных и в куб, либо только один раз, если используется размерное представление.

Выбор методологии

По сути все сводится к двум ключевым различиям. Во-первых, в методологии проектирования по Инмону сначала создается нормализованное хранилище данных, и лишь затем — витрины данных, а в методологии Кимбалла нормализованное хранилище данных отсутствует. Во-вторых, в методологии Инмона используется физическое корпоративное хранилище данных (EDW, enterprise data warehouse), а в методологии Кимбалла — логически согласованное, но не физическое. Обе методологии возникли до появления озер данных, но когда озера данных стали популярны, обе включили их в себя. Методология Кимбалла включает озера данных наряду с витринами данных в шину хранилища данных. Обе методологии, Кимбалла и Инмона, допускают использование озер данных вместо промежуточных реляционных таблиц.

Однако существует множество причин, по которым физическое EDW предпочтительнее, например:

- При наличии физического EDW значительно легче иметь единую версию истины; использование множества витрин данных, составляющих согласованное хранилище данных, может быть слишком сложным для понимания и сбивать с толку пользователей. Данные, физически размещенные в одной базе данных, проще понять.
- Построение модели данных на основе денормализованных таблиц в физическом EDW проще, чем создание ее непосредственно из источника OLTP.
- Нормализованное физическое EDW обеспечивает согласованность данных в масштабах всего предприятия. Это упрощает создание витрин данных, хотя и имеет недостаток в плане дублирования данных.
- Физические EDW требуют меньшего количества обновлений ETL и согласований; при наличии множества источников данных и размерных витрин данных в нескольких базах данных потребуется гораздо больше ежедневных или ежечасных обновлений и согласований, чтобы поддерживать синхронизацию.

- При использовании физического EDW контроль данных осуществляется только в одном месте, поэтому не нужно распылать силы и дублировать данные.

Однако если у вас всего несколько источников данных и вам нужно быстро создавать отчеты, лучше обойтись без физического EDW.

Какую модель использовать? По правде говоря, за прошедшие годы обе эти модели претерпели изменения и теперь довольно похожи. Фактически они могут дополнять друг друга. Вы можете добавить нормализованное EDW в подходе Кимбалла или размерно-структурированные витрины данных в подходе Инмона. Суть в том, что не обязательно выбирать только один подход и строго его придерживаться. Вы должны понимать оба и выбирать из них то, что подходит для вашего сценария. При этом ни одно решение не будет эффективным, если члены вашей команды не обладают хорошими лидерскими качествами, навыками коммуникации и планирования (см. главу 15).

Гибридные модели

Рассмотрим возможности создания гибрида этих двух методологий (рис. 8.9).

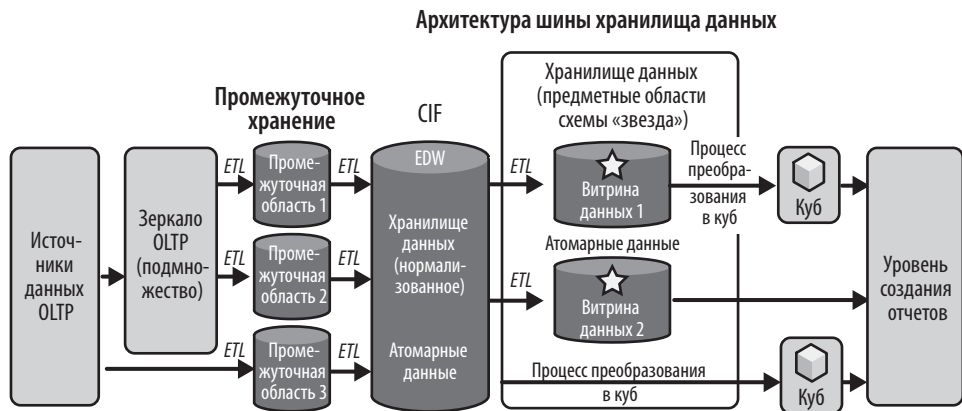


Рис. 8.9. Гибридная модель на основе подходов Кимбалла и Инмона

Моделирование данных в этом случае начинается так же, как и в двух предыдущих, — с извлечения необработанных данных из каждого OLTP системы-источника непосредственно во временные реляционные промежуточные таблицы. Здесь по-прежнему отсутствует преобразование или очистка данных, но добавляется зеркало OLTP (подробнее об этом чуть позже).

Далее данные копируются в CIF, чтобы централизовать их и хранить на атомарном уровне в 3NF, как в методологии Инмона. Из CIF данные копируются в независимые, размерные витрины данных. Некоторые витрины данных хранятся на

атомарном уровне, другие — на агрегированном, в зависимости от аналитических потребностей каждого подразделения.

Из витрин данных или из CIF можно также копировать подмножества данных в кубы. Отчеты можно формировать как из куба, так и из витрины данных, но я рекомендую по возможности делать это из куба. Куб имеет следующие преимущества.

- Он выступает в качестве семантического слоя.
- Он может обслуживать много пользователей одновременно.
- Данные куба агрегируются для повышения производительности.
- В кубе не нужны объединения или отношения.
- Куб может содержать иерархии и KPI.
- Он обладает безопасностью на уровне строк, что повышает конфиденциальность данных, ограничивая доступ пользователей к определенным строкам в базе данных.
- Он предлагает расширенные временные расчеты, которые помогают понять закономерности и тенденции в данных с течением времени, например рост продаж от года к году.

Независимо от того, используете ли вы модель Кимбалла, Инмона или гибридную, по возможности применяйте представления базы данных. *Представление базы данных* (database view) — это виртуальная таблица, основанная на результате выполнения SQL-оператора SELECT. В представлении не хранятся данные, только код SQL; при каждом запросе оно извлекает данные из одной или нескольких таблиц. Использование представлений в процессе извлечения, преобразования и загрузки (ETL) упрощает код внутри ETL, и вам не требуется видеть SQL-код внутри представления, чтобы понять, какие данные оно читает. Представление упрощает запросы к базе данных для отладки, а процесс ETL можно оптимизировать, обновив представление вне кода ETL. Любой может читать, изменять или оптимизировать представления не только в инструменте ETL, но и в других инструментах. Зависимости в представлении (например, таблицы и поля, которые оно использует) можно анализировать и отслеживать с помощью сторонних инструментов. Представления могут выдавать значения по умолчанию и выполнять простые вычисления, а также можно переименовывать поля, чтобы лучше понимать потоки данных. Представления могут отображаться в схеме «звезда», даже если базовая структура гораздо сложнее.

Представления допустимо использовать и в кубе. В этом случае можно переименовать столбцы базы данных, чтобы они соответствовали атрибутам куба. Таким образом все преобразования становятся доступными для администратора базы данных. Использование представлений упрощает обработку быстрых изменений и дает полный контроль за любыми соединениями, отправляемыми в источник, из которого извлекает данные куб. И, как и в ETL, представления в кубе могут

иметь схему «звезда», даже если лежащая в основе структура таковой не является.

Выше я упоминал, что эта гибридная модель включает в себя *зеркало OLTP*, которое представляет собой дубликат исходной OLTP-системы, работающий параллельно с ней. Создание зеркала OLTP — отличная стратегия по нескольким причинам. Прежде всего «реальная» OLTP будет использоваться только очень короткое время, во время зеркалирования. После этого она освобождается и становится доступной для конечных пользователей или для обслуживания. Ключи, отношения и представления зеркала можно изменять, что приводит к упрощению ETL-процессов, которые будет дешевле создавать и поддерживать. Можно также создавать определенные индексы в зеркале OLTP, тогда выполнять обслуживание индексов в исходном OLTP не придется. Это может повысить производительность на других этапах ETL. Наконец, благодаря зеркалированию подмножества столбцов не нужно загружать все таблицы OLTP. Это делает зеркало намного меньше и быстрее, чем полный OLTP.

Помните, что не существует единой модели, которая будет одинаково хорошо работать для любого сценария использования. Причин, по которым может понадобиться изменить модель, множество, в том числе безопасность, объем данных и производительность. Так, между CIF и архитектурой шины хранилища данных можно добавить еще один уровень, например EDW, содержащий только схемы типа «звезда». Это повысит производительность, поскольку такие схемы хорошо оптимизированы под запросы.

Мифы о методологиях Инмона и Кимбалла

Горячие многолетние дискуссии вокруг методологий Инмона и Кимбалла породили несколько устойчивых мифов. Этот раздел завершает главу, развенчивая некоторые из них.

Миф: методология Кимбалла — это подход «снизу вверх» без ориентации на предприятие.

На самом деле методология Кимбалла включает в себя как нисходящий, так и восходящий подход. Аспект «сверху вниз» проявляется в стратегическом планировании и проектировании в масштабах всего предприятия. Кимбалл подчеркивает, что необходимо предварительно потратить достаточно много времени на проектирование решения, используя инструмент, известный как матрица шины корпоративного хранилища данных (EDW bus matrix). Этот инструмент служит архитектурным наброском основных бизнес-процессов организации и связанных с ними измерений, включая согласованные измерения. Матрица шины EDW обеспечивает стратегическую перспективу «сверху вниз», чтобы гарантировать, что данные в среде хранилища данных / BI могут быть интегрированы в рамках всего предприятия.

Аспект «снизу вверх» вступает в игру на этапе исполнения. Здесь методология Кимбалла каждый раз фокусируется только на одном бизнес-процессе, обеспечивая гибкую доставку решений. Этот подход подчеркивает практическую реализацию и демонстрирует синергию использования стратегии планирования «сверху вниз» и подхода к исполнению «снизу вверх».

Миф: методология Инмона требует подготовки множества проектов, прежде чем можно будет приступить к созданию решения, подобно подходам «большого взрыва» или «водопада».

Инмон с самого начала заявлял, что способ создания хранилищ данных — итеративный. Фактически он утверждает, что самый важный фактор успеха при создании хранилища данных — это *не* использовать «большой взрыв».

Этот миф связан с давним заблуждением о том, что для создания архитектуры в стиле Инмона требуется много времени и ресурсов. Клаудия Имхофф (Claudia Imhoff), одна из самых известных экспертов по подходу Инмона, и ее соавторы говорят об этом так:

Мы никогда не рекомендуем сразу создавать единое корпоративное хранилище данных, содержащее все стратегические корпоративные данные, которые вам когда-либо понадобятся, прежде чем появится первая возможность анализа (витрина данных). Каждая последующая бизнес-программа, реализуемая с помощью очередной витрины данных, будет увеличивать датасет, служащий основой для вашего хранилища данных. В итоге объем данных, которые необходимо добавить в хранилище для поддержки новой витрины, станет незначительным, поскольку большая их часть уже будет присутствовать в хранилище¹.

Частично путаница возникает из-за того, что люди отождествляют подход Инмона к предварительному проектированию и планированию с самим архитектурным решением. Обе методологии отводят время на проектирование, планирование и изучение требований, прежде чем фактически что-то создавать. И в той и в другой соблюдается итеративный/гибкий подход, когда вы создаете часть решения, затем снова проводите проектирование и планирование, затем опять следует процесс разработки, и так снова и снова.

Шкала, представленная на рис. 8.10, показывает, какой объем проектирования выполняется перед созданием решения. В крайнем левом положении находится ситуационная модель (модель *ad hoc*), в которой до создания решения не проводятся никакие проектные работы; в крайнем правом — водопадная (*waterfall*) модель, в которой предварительно проектируется абсолютно *все* (часто эта модель называется «большим взрывом» (*big bang*)). Методологии Инмона

¹ Клаудия Имхофф, Николас Галеммо и Джонатан Г. Гейгер (Claudia Imhoff, Nicholas Galemмо, and Jonathan G. Geiger), *Mastering Data Warehouse Design*, Wiley, 2003, 21.

и Кимбалла находятся на отметке около 25 %, когда перед созданием решения выполняется достаточный объем проектирования.

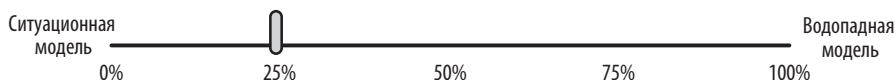


Рис. 8.10. Объем проектирования до построения решения

В качестве аналогии можно привести строительство нового города. Вы разрабатываете план города со всеми его кварталами, затем строите квартал за кварталом, и люди заселяются в них сразу по завершении строительства. В ходе этого процесса общая архитектура города может меняться, если это необходимо. Вы же не будете строить сразу все кварталы, прежде чем в них заселятся люди, и не начнете строить, не имея четкого представления о том, как должен выглядеть город в целом.

Миф: модель Инмона не позволяет создавать витрины данных со схемой «звезда».

В более поздних изданиях книг *Building the Data Warehouse* и *A Tale of Two Architectures* Инмон утверждает, что витрины данных со схемой «звезда» хороши для предоставления конечным пользователям прямого доступа к данным и что схема «звезда» подходит для витрин данных. Он совсем не против них.

Миф: очень немногие компании используют метод Инмона.

Недавние опросы показывают, что методологию Инмона применяют больше компаний, чем методологию Кимбалла. И гораздо больше компаний используют EDW без витрин данных.

Миф: архитектуры Кимбалла и Инмона несовместимы.

Эти два метода вполне могут работать вместе, предлагая более эффективное решение, как было показано в разделе «Гибридные модели» на стр. 150.

Кимбалл в своих книгах также определяет весь жизненный цикл разработки, затрагивая такие темы, как планирование проекта, определение бизнес-требований и обслуживание (см. рис. 8.11). Инмон рассказывает только о хранилище данных, но не о том, как его использовать.

Когда речь заходит о методологии Кимбалла, на самом деле часто имеется в виду размерное моделирование — они рассматриваются как синонимы. Кимбалл не избрал базовые концепции фактов и измерений; однако он создал обширный набор методов и понятий для размерного моделирования, включая согласованные измерения, медленно меняющиеся измерения, мусорные измерения (<https://oreil>).

ly/bHoUf), мини-измерения (https://oreil.ly/ZBp_W), таблицы мостов (<https://oreil.ly/5TF86>), периодические и аккумулируемые снимок-таблицы фактов (<https://oreil.ly/LyhQl>).

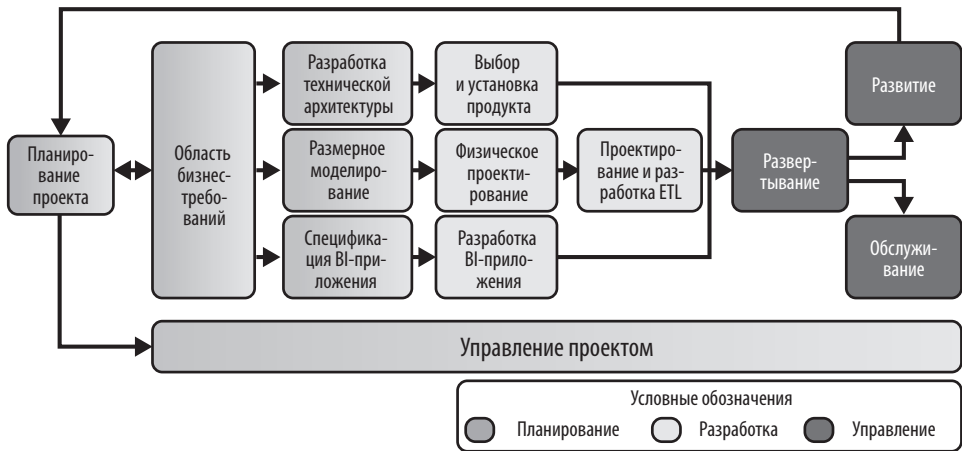


Рис. 8.11. Методология Кимбалла включает жизненный цикл разработки, который выходит за рамки создания хранилища данных (источник: Kimball, The Microsoft Data Warehouse Toolkit)

Итоги

В этой главе мы познакомились с практиками и методологиями, лежащими в основе организации данных и определения их структуры, отношений и ограничений для эффективной обработки, хранения и извлечения данных. Мы изучили процесс создания модели данных, которая служит абстрактным представлением данных предприятия. Это важнейший первый шаг в управлении данными, поскольку он дает представление о том, как данные будут храниться, использоваться и интегрироваться в системы.

Далее мы рассмотрели реляционное моделирование — классический подход, основанный на концепции нормализации данных в таблицы (или отношения), где каждая строка представляет собой запись, а столбцы являются атрибутами. Эта модель лежит в основе большинства традиционных реляционных систем баз данных и является ключевым фактором в предотвращении избыточности данных и повышении их целостности. Затем мы обсудили размерное моделирование — методологию, широко используемую в хранилищах данных: она делит данные по категориям на факты (измеримые величины) и измерения (контекстная информация); общую модель данных (CDM, common data model) — общий язык данных, обеспечивающий совместимость данных между приложениями и бизнес-процессами; и модель Data Vault — устойчивый гибридный подход, сочетающий лучшие черты

3NF и схемы «звезда». В оставшейся части главы было проведено сравнение двух конкурирующих философий построения хранилищ данных. Подход Кимбалла «снизу вверх» предполагает создание сначала отдельных витрин данных, а затем их объединение в полноценное хранилище данных, тогда как в подходе Инмона «сверху вниз» сначала создается комплексное хранилище данных, а затем из него извлекаются витрины данных.

Понимание этих методологий может существенно помочь в разработке эффективных архитектур данных, отвечающих конкретным требованиям организации и обеспечивающих баланс между операционной эффективностью и стратегической аналитикой.

Способы приема данных

В постоянно развивающемся цифровом мире обработка и осмысление всевозможных данных стали невероятно важны для любого бизнеса. Эта глава посвящена первому этапу работы с данными — их поступлению в систему.

Мы рассмотрим некоторые из основных способов доставки данных в системы. Начнем с описания двух популярных методов, известных как ETL и ELT. Кроме того, познакомимся с новой идеей, называемой *обратным ETL*, и поясним, как она перевернула все традиционные методы с ног на голову.

Поскольку у всех разные потребности в данных, мы рассмотрим два метода приема данных: пакетную обработку и обработку в реальном времени. Это поможет понять, какой из них лучше всего подходит в зависимости от того, какой объем данных и как быстро необходимо получить. Наконец, мы поговорим о важности управления данными, то есть об обеспечении точности, согласованности и доступности данных.

Цель главы — изложить простым языком сложные идеи и показать, как все они связаны с общей картиной работы с данными. Независимо от того, специалист вы или новичок в этой сфере, я с радостью проведу вас по увлекательному процессу приема данных. Добро пожаловать!

ETL и ELT

В течение многих лет самым популярным методом передачи данных из системы-источника в реляционное хранилище данных был метод «*извлечение — преобразование — загрузка*» (ETL, extract-transform-load). Но в последнее время, особенно в случае с озерами данных, стал использоваться вариант «*извлечение — загрузка — преобразование*» (ELT, extract-load-transform).

Процесс ETL включает в себя извлечение данных из внешних источников, их преобразование и очистку «на лету», чтобы они соответствовали формату и структуре данных в целевой системе, а затем загрузку в эту систему (обычно в озеро данных или реляционное хранилище).

Как избежать путаницы между ETL и ELT

Термины ETL и ELT из-за своей схожести можно легко перепутать или неправильно истолковать. Чтобы легче запомнить, что есть что, вот полезная мнемоника:

ETL (Extract-Transform-Load) = Early Transformation Leads («Ранняя трансформация рулит»).

Такая трактовка подчеркивает, что преобразование происходит на ранней стадии процесса, сразу после извлечения и перед загрузкой. Это отражает суть процесса ETL, в котором данные очищаются и преобразуются *перед* загрузкой в целевую систему.

Что касается ELT, можно попробовать следующее:

ELT (Extract-Load-Transform) = Every Load Transforms («Каждая загрузка преобразуется»).

Это означает, что после загрузки каждой партии необработанных данных в систему они подвергаются преобразованию; то есть преобразование происходит уже *после* загрузки данных в систему.

У ETL есть несколько основных недостатков. Во-первых, преобразование данных требует времени и извлечение может потребовать много ресурсов. Чем дольше происходит извлечение из системы-источника, тем выше вероятность того, что конечные пользователи этой системы столкнутся с проблемами производительности. Во-вторых, если в процессе ETL произошла ошибка и его нужно повторно запустить, придется вновь обращаться к системе-источнику, чтобы снова извлечь данные, что, вероятно, опять повлияет на производительность. В-третьих, если копируется очень большой объем данных, традиционные инструменты ETL могут не справиться с ним из-за ограниченной вычислительной мощности. В-четвертых, ETL выполняет преобразование записей по очереди, что также замедляет процесс. Наконец, некоторые инструменты ETL поддерживают ограниченный набор типов данных.

В качестве положительного момента можно отметить, что ETL зачастую идеально подходит для небольших датасетов, где процесс преобразования не слишком сложен. Он обеспечивает лучший контроль за качеством данных, поскольку их очистка и преобразование перед загрузкой помогают минимизировать ошибки. ETL может иметь ключевое значение для безопасности данных, так как загружаются только необходимые и очищенные данные, что снижает потенциальные риски нарушения безопасности. Наконец, ETL предпочтительнее, когда и источником, и получателем данных являются реляционные БД, поскольку он, как правило, более эффективен.

Процесс ELT похож на ETL, за исключением того, что порядок действий другой. ELT также копирует данные из внешних источников в целевую систему, но данные не преобразуются в процессе передачи. Напротив, они попадают в целевую систему в необработанном виде и уже в ней преобразуются и очищаются в соответствии с необходимым форматом и структурой данных. Однако при извлечении данных из источника можно отфильтровать ненужные строки и столбцы, чтобы не тратить на них ресурсы.

ELT часто оказывается предпочтительным для озер данных и особенно полезен при работе с большим объемом неструктурированных или полуструктурированных данных. Его практичность заключается в возможности сначала загрузить все необработанные данные в целевую систему и только затем применить необходимые преобразования. Как только данные попадают в озеро данных, ELT использует огромную вычислительную мощь современных технологий, что делает его эффективным при обработке очень больших объемов данных. ELT также может быстрее выполнять преобразования, поскольку обрабатывает данные пакетно, что является заметным преимуществом по сравнению с последовательным подходом ETL к преобразованию, «запись за записью». Также этот метод обеспечивает возможность изменения или оптимизации преобразований без необходимости повторного извлечения данных. Еще одним преимуществом ELT является широкий спектр доступных инструментов, в том числе разработанных специально для платформ Big Data. Это означает, что процессы ELT могут поддерживать большее разнообразие типов данных и адаптироваться к меняющимся потребностям среды данных. Кроме того, подход ELT позволяет минимизировать потенциальные проблемы с производительностью системы-источника, поскольку данные извлекаются только один раз, а последующие преобразования выполняются в целевой системе.

В заключение следует отметить, что и ETL, и ELT обладают уникальными достоинствами и подходят для различных сценариев обработки данных. ETL, будучи более старым из двух методов, обеспечивает надежный контроль качества данных, безопасность и эффективность, особенно если источником и получателем являются реляционные базы данных, что делает его идеальным для небольших датасетов. ELT, более современный подход, завоевал популярность при использовании с озерами данных, предлагая большую гибкость и масштабируемость, особенно при работе с большими объемами и неструктурированными данными. Пакетная обработка преобразований в ELT и широкие функциональные возможности инструментов делают его универсальной и эффективной технологией для платформ Big Data.

Выбор между ETL и ELT во многом зависит от конкретных потребностей и структуры среды данных. Поскольку технологии обработки данных быстро развиваются, он не является однозначным — «или-или»; напротив, речь идет о поиске баланса, максимально соответствующего требованиям к обработке данных.

Обратный ETL

Обратный ETL (reverse ETL) — это процесс перемещения данных из современного хранилища данных (MDW) в стороннюю систему или системы, чтобы сделать данные операционными. На рис. 9.1 представлено сравнение методов ETL, ELT и обратного ETL. Традиционно данные из DW применяются для аналитических приложений и бизнес-аналитики — для выявления долгосрочных тенденций и влияния на долгосрочную стратегию — и не копируются за пределы хранилища. Однако некоторые компании теперь используют эти данные и для операционной аналитики.

Операционная аналитика помогает принимать повседневные решения и направлена на повышение эффективности бизнес-процессов. Проще говоря, она заставляет данные компании работать, помогая сотрудникам принимать более эффективные и обоснованные бизнес-решения. Например, предположим, что вы загружаете данные о клиентах в хранилище, а затем очищаете и обрабатываете их (ELT). Затем можете скопировать эти данные в несколько систем типа «ПО как услуга» (SaaS, software-as-a-service), например в систему Salesforce, чтобы получить согласованное представление каждого клиента во всех системах. Данные о клиентах можно также скопировать в систему поддержки клиентов и/или в систему продаж.

Можно даже выявить клиентов с высоким риском оттока (которые могут откатиться от продуктов или услуг компании), отображая в CRM-системе данные об их активности.

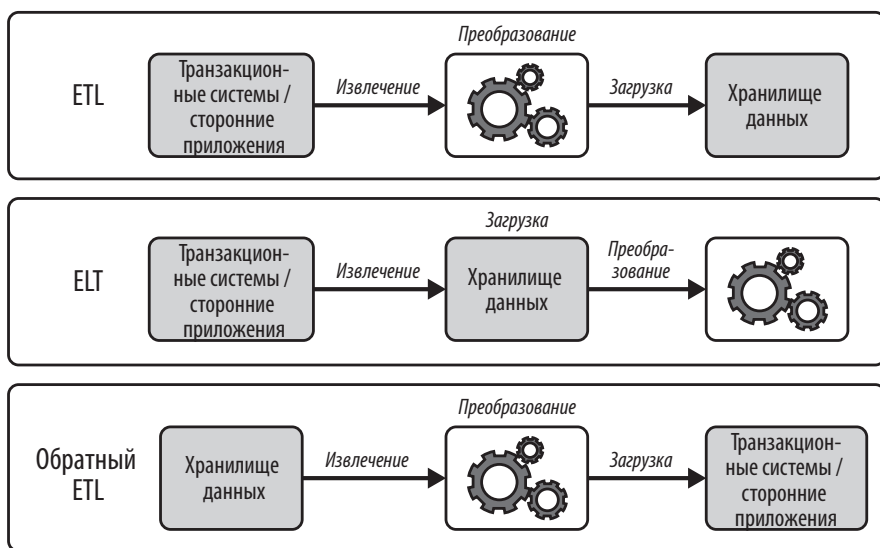


Рис. 9.1. Сравнение методов ETL, ELT и обратного ETL

Компании создают важные метрики на базе SQL поверх своих хранилищ данных, чтобы лучше понимать клиентов и взаимодействовать с ними. Эти метрики могут включать в себя пожизненную ценность клиента (прогноз общей чистой прибыли, которую компания рассчитывает получить от будущих отношений с клиентом), квалифицированный лид продукта (потенциальный клиент, который взаимодействовал с продуктом и может стать реальным покупателем), индекс склонности к покупке (вероятность того, что пользователь купит продукт) и многие другие.

Хотя отчеты и визуализации на основе этих данных легко создавать в BI-инструментах или SQL, гораздо большего эффекта можно достичь, если полученные инсайты будут напрямую влиять на повседневную работу ваших команд в тех инструментах, которые они используют постоянно. Например, можно создавать BI-отчеты из хранилища данных и обучать торговых представителей пользоваться ими — или реализовывать их анализ на практике, поручая аналитику данных вводить показатели лидов из хранилища данных в настраиваемое поле в системе Salesforce. Другой пример: команда специалистов data science рассчитывает индекс склонности к покупке (propensity score) и сохраняет его в хранилище данных или озере данных. Используя обратный ETL, можно перенести этот индекс в операционную рабочую базу данных, чтобы предоставлять клиентам персонализированные предложения в реальном времени. На практике это означает, что компания может улучшить уровень удовлетворенности клиента, предоставляя более релевантный и адаптированный контент, рекомендации или предложения на основе того, насколько вероятно, что клиент купит продукт. Приложение становится более вовлекающим и чутко реагирующим на индивидуальные предпочтения и поведение клиента.

Пакетная обработка и обработка в реальном времени

В методах ETL и ELT есть два способа, от выбора которых зависит, когда и как часто извлекать данные из системы-источника: пакетная обработка и обработка в реальном времени. Рассмотрим подробнее каждый из них.

Пакетная обработка

Пакетная обработка — это эффективный способ обработки больших объемов данных. Набор однотипных транзакций из системы-источника группируется вместе («пакетируется») на определенный период времени. Через регулярные промежутки времени система автоматически «запускает задание», и весь пакет копируется из исходной системы в целевую (в озеро или хранилище данных), как правило, в нерабочее время. Основная функция системы пакетной обработки заключается в регулярном запуске пакетов в запланированное время (например, каждую ночь) или по мере необходимости.

Ваш счет за электричество — пример такой пакетной обработки. Счетчик электроэнергии собирает данные о вашем потреблении за определенный период

(обычно за месяц), а затем электроснабжающая организация обрабатывает весь пакет данных и на основе этого выставляет вам ежемесячный счет.

Обработка в реальном времени

Обработка данных в реальном времени — это метод непрерывной обработки данных по мере их сбора, в течение нескольких секунд или миллисекунд, что позволяет получать информацию практически мгновенно. Система реального времени в архитектуре, основанной на событиях, быстро реагирует на новую информацию. Когда она обнаруживает новый файл или запись в системе-источнике, запускается событие, и она копирует файл или запись в место назначения.

IoT-устройства, сообщения в социальных сетях и финансовые торговые системы — все они используют системы обработки данных в реальном времени. Например, если происходит подозрительная транзакция по кредитной карте, банк должен немедленно предупредить владельца карты, поэтому для обнаружения мошенничества банки используют обработку в реальном времени. Вы же не хотите в таком случае дожидаться следующего дня, когда будет выполнен ночной пакет транзакций!

Еще один хороший пример — приложения для анализа дорожной обстановки. Навигационное приложение Waze автоматически в режиме реального времени обновляет информацию о загруженности дорог, используя данные, полученные с различных мобильных устройств и дорожных датчиков в заданном районе, чтобы предложить оптимальный и кратчайший путь к пункту назначения. Очевидно, что, если бы информацию о пробках пришлось ждать до ночного запуска пакета, она была бы совершенно бесполезной.

Обработка в реальном времени мгновенно обновляет целевую систему, поэтому отчеты и запросы никогда не устаревают. Это позволяет быстро обнаруживать бизнес-операции, требующие немедленного внимания, и обновлять приложения или отчеты, которые работают на основе данных реального времени.

Традиционно в реляционных хранилищах данных использовалась только пакетная обработка. Данные в реальном времени стали очень популярны в последние несколько лет. Системы, использующие этот тип обработки, в основном хранят данные в озере данных, которое гораздо лучше подходит для работы с данными в реальном времени, поскольку может поддерживать миллионы событий в секунду.

Каждый метод имеет свои преимущества и недостатки при использовании в хранилищах данных.

Плюсы и минусы пакетной обработки

Пакетная обработка эффективна при работе с большими объемами данных, поскольку все данные обрабатываются сразу, а не по отдельности. Это также

позволяет планировать выполнение задач пакетной обработки в часы наименьшей нагрузки, что сводит к минимуму перебои в обычной работе системы. Кроме того, при пакетной обработке снижается риск сбоя системы, поскольку завершившиеся ошибкой задания можно повторно выполнить без существенных осложнений.

К недостаткам пакетной обработки можно отнести возможные задержки в доступе к данным, поскольку на одновременную обработку большого объема данных требуется время. При неправильной организации управления это может привести к нерациональному использованию ресурсов. Кроме того, пакетная обработка не подходит для приложений, требующих проведения аналитики в режиме реального времени, так как обновления больше по объему и происходят реже, а любые изменения в данных или расчетах требуют полного перезапуска для всего пакета, что может повлиять на производительность.

Плюсы и минусы обработки в реальном времени

В отличие от пакетной, обработка в реальном времени обеспечивает ежеминутное получение актуальной информации, позволяя действовать немедленно на основе имеющихся данных. Она особенно полезна для систем, требующих постоянного обновления и способных эффективно обрабатывать потоковые данные. Кроме того, обработка в реальном времени более гибкая и оперативно реагирует на меняющиеся потребности бизнеса.

Обработка в реальном времени имеет и свои недостатки. Она требует больше системных ресурсов для непрерывной обработки и создает повышенный риск сбоя системы. Обработка ошибок и восстановление в системах реального времени могут оказаться достаточно сложными и нуждаются в надежных инструментах. Обеспечение согласованности данных может быть осложнено необходимостью постоянных обновлений. Более того, затраты, связанные с обработкой в реальном времени, могут быть гораздо выше, поскольку необходимо обеспечивать непрерывность процесса.

Выбор между пакетной обработкой и обработкой в реальном времени подразумевает учет таких факторов, как тип данных, потребности в обработке и допустимая задержка. Обычно он сводится к соблюдению баланса между потребностью в немедленном получении данных и способностью системы выделять необходимые ресурсы для обработки в реальном времени — это часто называют *толерантностью к задержкам* (latency tolerance). Если бизнес-процесс или система может позволить себе небольшую задержку в доступе к данным — другими словами, если они обладают высокой толерантностью к задержкам, — пакетная обработка может оказаться самым подходящим выбором. И наоборот, если потребность в немедленном получении данных критически важна, а система оснащена соответствующими инструментами для управления необходимыми ресурсами, что свидетельствует о низкой толерантности к задержкам, то лучше использовать обработку в реальном времени.

Управление данными (data governance)

Управление данными (data governance) — это стратегия общего управления данными в рамках всей организации. Она включает в себя разработку политик и процедур для сбора, хранения, защиты, преобразования и представления данных. В частности, она используется для того, чтобы обеспечить соблюдение организацией правовых и нормативных требований. Она также включает в себя мониторинг качества и точности данных — например, обеспечение надлежащей очистки и преобразования данных.

Управление данными должно включать в себя структуру, которая определяет сотрудников и роли, ответственные за управление, обслуживание и использование данных в организации. Один из способов добиться этого — создать центр совершенства (CoE, center of excellence) в области управления данными. CoE служит центром разработки политик, процедур и стандартов управления данными организации, а также для распределения ролей, обязанностей и процессов принятия решений в рамках работы с данными.

Перед созданием любого решения по хранению данных важно заранее определить фреймворк управления данными и создать CoE. Слишком часто проекты терпят неудачу, потому что этим аспектам не было уделено достаточно внимания.

На рис. 9.2 представлена модель зрелости управления данными (data governance maturity model), с помощью которой вы можете определить направления развития для вашей организации.

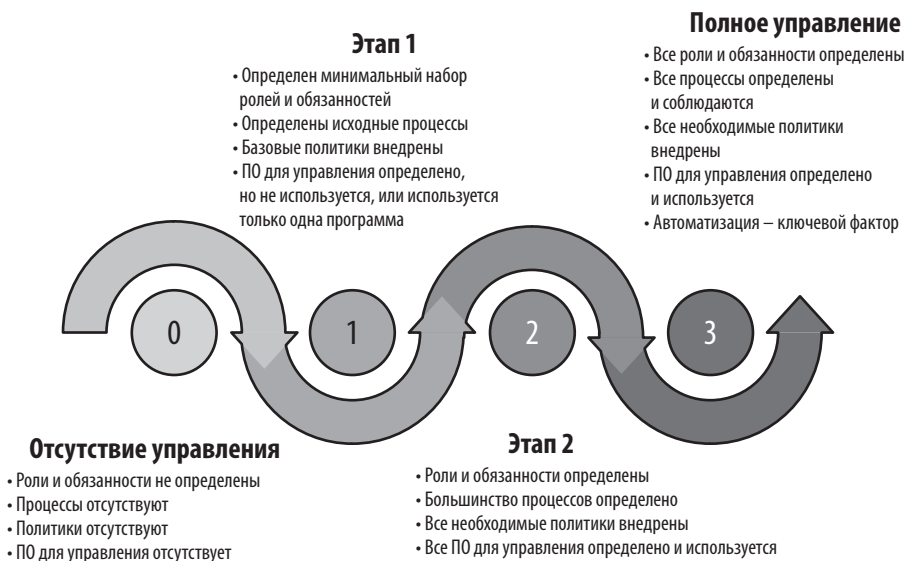


Рис. 9.2. Модель зрелости управления данными

Итоги

В этой главе мы рассмотрели методы и стратегии, используемые для передачи данных из различных источников в централизованную систему хранения, готовую к обработке и анализу.

Мы сравнили два основных метода: ETL (извлечение, преобразование, загрузка) и ELT (извлечение, загрузка, преобразование). Подход ETL подразумевает извлечение данных из источников, преобразование их в соответствии с операционными потребностями и последующую загрузку в целевую систему. В свою очередь, ELT-процесс загружает данные в систему до их преобразования. Обе методологии имеют свои преимущества и подходят для разных сценариев в зависимости от объемов данных, сложности структуры, требований к производительности и архитектурных соображений.

Далее мы ввели понятие обратного ETL — метода, позволяющего компаниям переносить данные из хранилища или озера данных обратно в работающие системы. Этот процесс необходим для того, чтобы передать инсайты, полученные в результате анализа данных, обратно в бизнес-приложения для практического применения.

Мы изучили два противоположных подхода к обработке данных: пакетную обработку и обработку в реальном времени. Затем мы перешли к такому важному аспекту, как управление данными (data governance), — системе управления качеством данных, конфиденциальностью, безопасностью и комплаенсом. Фреймворки управления данными помогают поддерживать целостность и надежность данных на протяжении всего их жизненного цикла, обеспечивая их точность, согласованность и безопасность.

Выбор правильной стратегии приема данных — это важное бизнес-решение, которое отчасти определяет, насколько эффективно организация сможет использовать свои данные для принятия операционных решений и ведения бизнеса. Ставки высоки: неверная стратегия может привести к низкому качеству данных, проблемам с производительностью, увеличению затрат и даже к нарушению нормативно-правовых требований. Поэтому тщательно продумайте подход к приему данных не только как к технической необходимости, но и как к бизнес-потребности.

Архитектуры данных

Теперь, когда вы хорошо освоили все понятия архитектуры данных, пришло время перейти к основной части книги. В пяти главах части III рассматриваются четыре архитектуры в порядке их возникновения: современное хранилище данных (modern data warehouse), ткань данных (data fabric), озеро-хранилище данных (data lakehouse) и сетка данных (data mesh).

Я привожу собственную интерпретацию архитектур данных; у других могут быть другие мнения. В конце концов, ваше решение может быть даже комбинацией этих архитектур.

Современное хранилище данных

Во второй части этой книги вы узнали о реляционных хранилищах данных (RDW, relational data warehouses) и озерах данных (data lakes) — двух ключевых составляющих экосистемы управления данными. Теперь давайте обратимся к динамичному миру современного бизнеса. Каждый день организациям приходится просеивать огромные объемы данных, чтобы получать информацию, принимать решения и способствовать росту своей компании. Представьте себе городской супермаркет, перешедший с традиционных баз данных на современное хранилище данных (MDW, modern data warehouse). Теперь менеджеры могут получать доступ к данным о товарных запасах в режиме реального времени, прогнозировать тенденции продаж и оптимизировать процесс совершения покупок для своих клиентов. В этом и заключается сила MDW. Оно сочетает в себе лучшее из двух технологий: структуру RDW и гибкость озер данных.

Почему стоит обратить внимание на MDW? Потому что они находятся в самом сердце быстро развивающейся экосистемы данных, позволяя организациям использовать информацию, необходимую для внедрения инноваций и повышения конкурентоспособности. В этой главе мы разберемся, что такое MDW и чего можно достичь с их помощью, а также расскажем о некоторых важных особенностях, которые следует иметь в виду. Мы рассмотрим архитектуру, функциональность и общие этапы создания MDW, а в завершение приведем показательный пример. Время погрузиться в мир современных хранилищ данных, где данные — это не просто цифры, а двигатель успеха.

Архитектура MDW

Рис. 10.1 иллюстрирует гибридную природу MDW, в которой RDW и озеро данных объединяются, создавая среду, позволяющую гибко манипулировать данными и получать надежную аналитику.

RDW работает по нисходящему принципу («сверху вниз»), подробно описанному в главе 4. Структура такого хранилища предполагает значительную предварительную подготовку, прежде чем можно будет начать загрузку в него данных («схема при записи», schema-on-write). Такой подход эффективен при создании ретроспективных отчетов, позволяя аналитикам углубляться в описательную (*что* произошло) и диагностическую аналитику (*почему* это произошло).

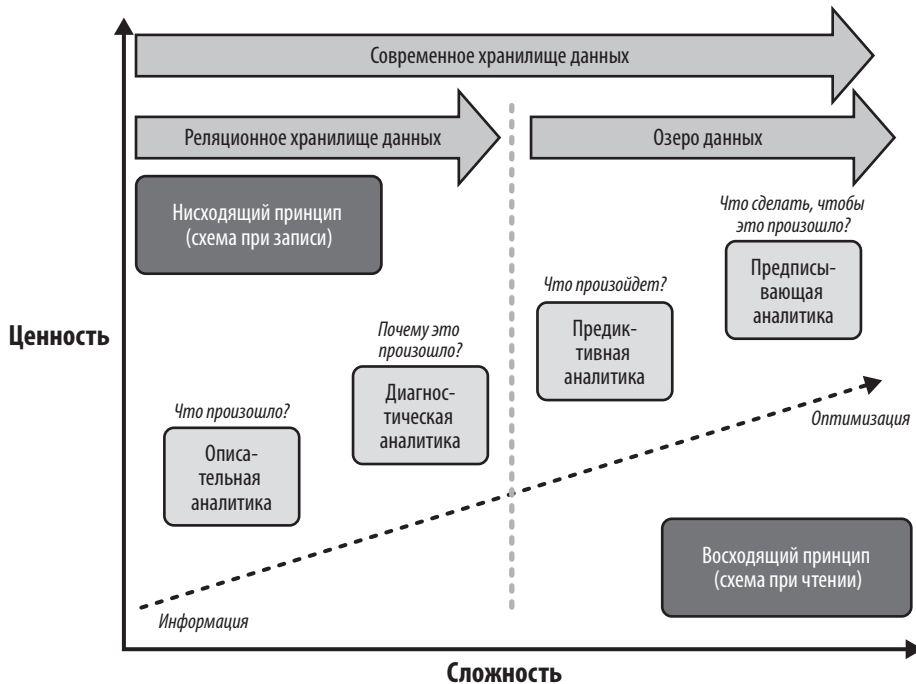


Рис. 10.1. Полный спектр аналитики, поддерживаемой современным хранилищем данных

Озеро данных, напротив, определяется философией «снизу вверх» (восходящий принцип), описанной в главе 5. Для того чтобы начать использовать данные, требуется минимум предварительной работы (schema-on-read, «схема при чтении»), что позволяет быстро развернуть модели машинного обучения (ML) для предиктивной (прогнозирование того, что произойдет) и предписывающей аналитики (предложения решений для достижения желаемых результатов).

В архитектуре MDW озеро данных — это не просто репозиторий для хранения огромных объемов информации и обучения ML-моделей; это динамический компонент, предназначенный для преобразования данных и прочих целей, рассмотренных в главе 5. Например, оно может выступать в качестве хаба для приема потоковых данных, шлюза для пользователей, позволяющего быстро исследовать данные и создавать отчеты, и централизованного источника для поддержки единой версии истины.

Важной особенностью архитектуры MDW является то, что по крайней мере часть данных в ней должна быть реплицирована в RDW. Без такого дублирования это была бы просто архитектура озера-хранилища данных, описываемого в главе 12.

В середине 2010-х годов многие компании пытались применять озеро данных во всех сценариях использования и полностью отказаться от RDW. После того как такой подход потерпел неудачу, самой популярной архитектурой стало MDW, что было обусловлено экспоненциальным ростом объемов данных, их разнообразия и скорости поступления. Компании в различных предметных областях, включая здравоохранение, финансы, розничную торговлю и технологии, осознали необходимость более гибкого и масштабируемого подхода к хранению и анализу данных. Их RDW уже не годились для решения задач в области Big Data, а MDW предлагали гибкую интеграцию RDW и озер данных. В качестве ведущих поставщиков выступили такие ключевые IT-игроки, как Microsoft с сервисом Azure Synapse Analytics, Amazon со службой Redshift, Google с хранилищами BigQuery и Snowflake. Они произвели революцию в способах доступа предприятий к своим данным и в их использовании.

Когда Big Data, потоковые и полуструктурированные данные стали еще более популярными, крупные предприятия начали использовать архитектуры типа ткань данных (data fabric) и озеро-хранилище данных (data lakehouse), которые обсуждаются в главах 11 и 12. Однако у клиентов с небольшим объемом данных (скажем, менее 10 терабайт) MDW по-прежнему востребованы и, скорее всего, останутся таковыми, по крайней мере до тех пор, пока озера данных не станут работать так же хорошо, как RDW.

На самом деле, до сих пор существуют компании, создающие облачные решения, которые вообще не задействуют озера данных. В основном это происходит в таких сценариях использования, когда компания имеет небольшой объем данных и осуществляет переход с локального решения, где не было озера данных. Я по-прежнему считаю такой подход приемлемым, но с той оговоркой, что компания должна быть *абсолютно уверена*, что хранилище данных не будет слишком сильно расти (это очень сложно предсказать заранее). Кроме того, если у вас небольшой объем данных, возможно, целесообразнее использовать для RDW более дешевое решение с симметричной многопроцессорной обработкой (SMP), чем решение с массово-параллельной обработкой (MPP) (см. главу 7).

Теперь разберемся в деталях MDW. На рис. 10.2 показан типичный поток данных через озеро данных и RDW в MDW.

По мере циркулирования по архитектуре MDW данные проходят через пять этапов, которые пронумерованы соответствующим образом на рис. 10.2. Проследим за прохождением каждого из них, чтобы составить общее представление об архитектуре MDW.

Этап 1. Прием

MDW может обрабатывать практически любые типы данных, и они могут поступать из разных источников, как локальных, так и облачных. Данные

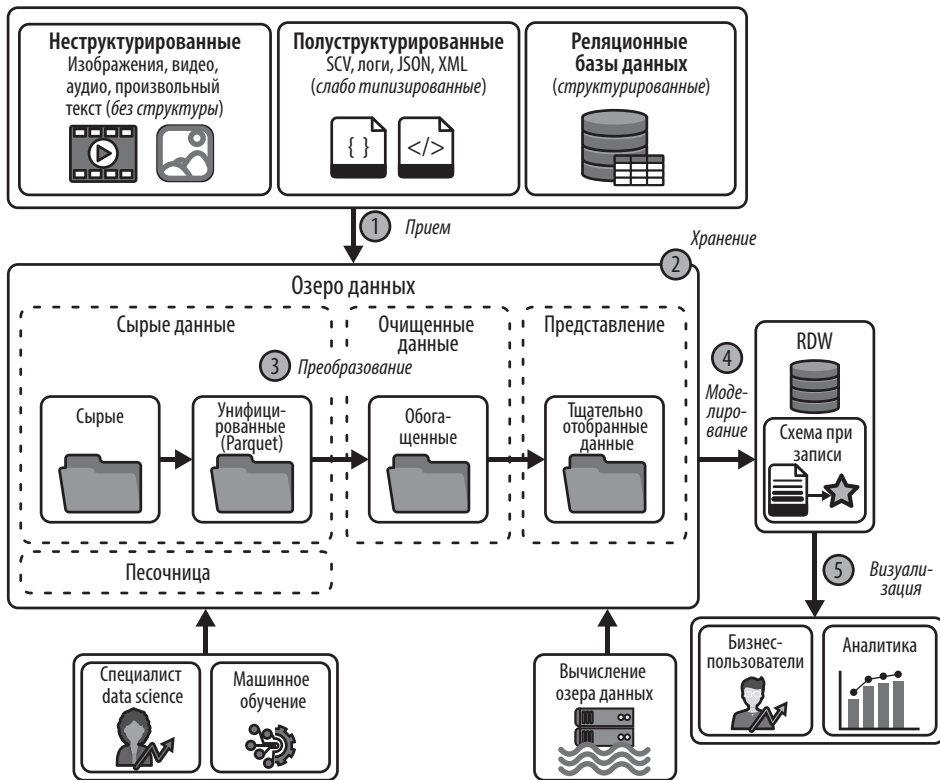


Рис. 10.2. Путь, который проходят данные в архитектуре современного хранилища данных

могут различаться по объему, скорости поступления и типу; они могут быть неструктурированными, полуструктурированными или реляционными; они могут поступать пакетами или в потоковом режиме реального времени; в виде небольших или огромных файлов. Такое разнообразие может стать проблемой при приеме данных.

Этап 2. Хранение

Принятые данные попадают в озеро данных, которое содержит все уровни, описанные в главе 5, что позволяет конечным пользователям получать доступ к ним независимо от своего местонахождения (при условии, что у них есть доступ к облаку). Каждый провайдер облачных услуг предлагает неограниченное хранилище в озере данных, и его стоимость относительно невысока. В озере данных заложены высокая доступность и надежное аварийное восстановление, а также множество опций безопасности и шифрования.

Этап 3. Преобразование

Озеро данных является хабом хранения данных. Однако, чтобы по-настоящему придать смысл данным и работать с ними, необходимо приложить некоторые усилия, и здесь на помощь приходят вычислительные мощности. Существенным преимуществом MDW является четкое разграничение между хранением данных (в озере данных) и их обработкой (с использованием вычислительных мощностей). Разделение этих двух функций позволяет выбирать из множества вычислительных инструментов, предоставляемых облачными провайдерами или другими источниками. Эти инструменты, предназначенные для извлечения и анализа данных из озера, могут считывать данные в разных форматах, а затем упорядочивать их в более стандартизированный формат, например в Parquet. Такое разделение обеспечивает гибкость и повышает эффективность.

Затем облачное средство вычисления получает файлы с уровня унифицированных данных, преобразует их (обогащает и очищает) и сохраняет на уровне очищенных данных озера. Наконец, инструмент вычисления берет файлы с уровня обогащенных данных и выполняет дополнительные преобразования для повышения производительности или удобства использования (например, объединяет данные из нескольких файлов и агрегирует их), а затем записывает их на уровень представления данных озера.

Этап 4. Моделирование

Создание отчетов непосредственно на основе данных в озере может быть медленной, недостаточно защищенной и сложной для конечных пользователей процедурой. Вместо этого можно скопировать все или часть данных из озера в реляционное хранилище. Это означает, что вы создадите реляционную модель для данных в RDW, где модель обычно находится в третьей нормальной форме (см. главу 8). Возможно, вы также захотите скопировать ее в схему «звезда» в RDW для повышения производительности и упрощения использования.

Этап 5. Визуализация

Как только данные окажутся в RDW в удобном для восприятия формате, бизнес-пользователи смогут проводить их анализ с помощью привычных инструментов, создавая отчеты и дашборды.

На этапе приема данных нужно определить, как часто вы будете извлекать данные и использовать ли инкрементное или полное извлечение (см. главу 4). Не забывайте о размере и пропускной способности конвейера от источников данных до облака, где расположено озеро данных. Если вам нужно передавать большие файлы, а пропускная способность невелика, возможно, придется загружать данные несколько раз в день небольшими фрагментами, а не одной большой партией раз в день. Возможно, у вашего облачного провайдера есть варианты покупки дополнительной пропускной способности.

На разных этапах этого пути специалисты data science могут использовать машинное обучение на основе данных из MDW для обучения и построения моделей данных. В зависимости от поставленных задач они могут брать данные озера с уровней необработанных (сырых), очищенных данных или уровня представления, а также с *уровня «песочницы»* (это выделенное пространство для проведения экспериментов с данными, их изучения и предварительного анализа, которые не влияют на данные других уровней), или из RDW.

Существуют некоторые исключения из только что описанной последовательности этапов пути данных по архитектуре MDW в зависимости от характеристик исходных данных. Например, не все данные с уровня представления в озере нужно копировать в RDW, особенно с учетом новых инструментов, упрощающих запросы и создание отчетов на основе данных в озере. Это означает, что на этапе визуализации бизнес-пользователю не нужно обращаться к RDW за всеми данными — он может просто получить прямой доступ к тем данным в озере, которые не были скопированы в RDW.

Другое исключение: не все исходные данные нужно копировать в озеро. В некоторых случаях, особенно для новых проектов, создаваемых в облаке, быстрее скопировать данные из источника прямо в RDW, минуя озеро данных, особенно для структурированных (реляционных) исходных данных. Например, справочные таблицы, используемые в качестве таблиц измерений, не нуждаются в очистке, являются полными выборками и могут быть легко извлечены из источника при необходимости повторного запуска пакета ETL. В конце концов, извлечение данных из реляционной БД, копирование их в озеро (с потерей ценных метаданных, таких как типы данных, ограничения и внешние ключи), а затем импорт их в другую реляционную БД (хранилище данных) могут оказаться достаточно трудоемкими.

Это особенно справедливо для типичной миграции с локального решения, которое не использует озеро данных и имеет множество пакетов ETL, копирующих данные из реляционной БД в RDW. Чтобы сократить трудозатраты на миграцию, необходимо немного изменить существующие пакеты ETL — поменять только целевое назначение. Со временем вы сможете модифицировать все пакеты так, чтобы они использовали озеро данных. Однако для быстрого запуска в работу новой базы можно заменить только несколько медленно работающих пакетов ETL, чтобы они использовали озеро данных, а все остальное сделать позже.

Исходные данные, идущие в обход озера данных, не получают некоторых его преимуществ — в частности, для них будет невозможно резервное копирование на случай повторного запуска пакета ETL. А еще такой обход озера данных может привести к излишней нагрузке на RDW, которое в этом случае будет отвечать за очистку данных. Кроме того, те, кто использует озеро, обнаружат, что эти данные в нем отсутствуют, что не позволит озеру служить единым источником истины.

Плюсы и минусы архитектуры MDW

У MDW множество преимуществ и несколько недостатков. К преимуществам относятся:

Интеграция нескольких источников данных

MDW могут работать как со структурированными, так и с неструктурированными данными из различных источников, включая RDW и озера данных, обеспечивая всестороннее представление информации.

Масштабируемость

MDW разработаны таким образом, чтобы расти вместе с бизнесом, легко масштабироваться для обработки возросшего объема данных.

Анализ данных в реальном времени

MDW способствуют проведению аналитики в режиме реального времени, позволяя предприятиям принимать своевременные решения на основе текущих данных.

Повышение производительности

Благодаря использованию передовых технологий и оптимизированной обработке данных MDW обеспечивают более высокую производительность запросов и извлечения полезной информации.

Гибкость

MDW обеспечивают гибкость в моделировании данных, позволяя использовать как традиционные структурированные запросы, так и методы обработки Big Data.

Повышенная безопасность

Ведущие провайдеры MDW обеспечивают надежные меры безопасности для защиты конфиденциальных данных.

Некоторые недостатки архитектуры MDW.

Сложность

Внедрение и управление MDW могут быть сложными, особенно в гибридных архитектурах, сочетающих разные типы хранения и обработки данных.

Стоимость

Первоначальная настройка, текущее обслуживание и масштабирование могут стоить слишком дорого, особенно для малого и среднего бизнеса. Кроме того, потребуются расходы на хранение и дополнительные конвейеры данных для создания нескольких копий данных (как для RDW, так и для озера данных).

Требования к квалификации сотрудников

Использование MDW в полном объеме требует специальных знаний и навыков, что может привести к сложностям с набором сотрудников или дополнительным расходам на их обучение.

Возможное возникновение обособленных хранилищ данных

Без надлежащей интеграции и управления MDW могут привести к появлению обособленных (локальных) хранилищ данных, или мест, где информация становится изолированной и труднодоступной для части сотрудников организации.

Проблемы с комплаенсом

Обеспечить комплаенс в среде, где используются разные типы и источники данных, может быть непросто.

Зависимость от поставщика

Если вы используете облачный сервис MDW, это может означать зависимость от конкретного поставщика услуг и привести к потенциальной привязке к нему и ограничению гибкости в будущем.

По мере того как данные копируются в MDW и перемещаются в озеро данных, а затем в RDW, они меняют формат и их становится удобнее использовать. Это облегчает проведение ориентированной на пользователя бизнес-аналитики (BI) самообслуживания, когда конечные пользователи могут создавать отчеты, просто перетаскивая поля из списка в рабочую область без необходимости объединять какие-либо таблицы. Хотя IT-отдел должен проделать дополнительную предварительную работу, чтобы данные действительно стало удобно использовать, эта работа в большинстве случаев стоит того, поскольку освобождает IT-отдел от дальнейшего участия (и, скорее всего, от затруднений) в создании отчетов и дашбордов.

Комбинирование RDW и озера данных

В MDW озеро данных используется для *предварительного хранения и подготовки* данных, а RDW — для *обслуживания, обеспечения безопасности и комплаенса*. Рассмотрим подробнее, в чем заключается функциональность RDW и озера данных при их одновременном использовании.

Озеро данных

В озере данных специалисты data science и опытные пользователи с более высокими техническими навыками имеют эксклюзивный доступ, что обусловлено сложной структурой папок и файлов и отдельным хранением метаданных.

Непрофессионалам, скорее всего, будет трудно ориентироваться в озере, а для доступа могут потребоваться более сложные инструменты. Функции озера данных включают как пакетную обработку, когда данные преобразуются в пакеты, так и обработку в реальном времени, которая служит пунктом назначения для потоковых данных (см. главу 9). Озеро также применяется для уточнения и очистки данных, предоставляя платформу с несколькими вариантами вычислений на выбор, позволяющую использовать столько вычислительных мощностей, сколько необходимо. Кроме того, оно подходит для задач ELT.

С учетом вышесказанного озеро данных служит местом для хранения более старых данных или резервных копий исходных данных, чтобы не хранить их в RDW. Это также место для резервного копирования данных из самого RDW. Пользователи могут легко создавать копии данных в озере на уровень песочницы, позволяя другим пользоваться этими данными и «играть» с ними. Озеро также предоставляет возможность просматривать и исследовать данные, если вы пока не знаете, что хотите из них извлечь; вы можете определить их ценность до того, как скопируете их в хранилище данных. Озеро облегчает быструю подготовку отчетов и доступ к данным, особенно учитывая то, что в нем используется «схема при чтении», поэтому данные быстро попадают в хранилище.

Реляционное хранилище данных

Бизнес-пользователи часто обращаются к RDW как к источнику данных, доступному нетехническим специалистам, в частности если они привыкли к реляционным базам данных. Эти базы данных отличаются низкой задержкой доступа, что позволяет выполнять запросы гораздо быстрее, особенно если используется технология MPP (см. главу 7). Они могут обрабатывать большое количество объединений таблиц и сложных запросов, а также отлично подходят для выполнения интерактивных ad hoc запросов благодаря высокой производительности, которую обеспечивает технология MPP. Это позволяет постоянно выполнять тонкую настройку запросов без длительного ожидания. RDW также могут поддерживать одновременную работу многих пользователей, параллельно выполняющих запросы и создающих отчеты.

Дополнительные функции безопасности RDW в контексте MDW включают такие опции, как безопасность на уровне строк и столбцов. RDW также обеспечивают широкую поддержку различных инструментов, и поскольку они существуют гораздо дольше, чем озера данных, то и инструментов для них доступно гораздо больше. Озера данных не обладают достаточной производительностью для поддержки дашбордов, а в RDW есть возможность запускать дашборды благодаря его превосходной производительности вплоть до миллисекунд. Обязательный уровень метаданных над данными требует больше предварительной работы, но значительно упрощает BI-самообслуживание. Как правило, когда вы создаете RDW, вы уже знаете, ответы на какие вопросы хотите получить с помощью данных, и можете их предусмотреть, что делает RDW мощным и универсальным инструментом.

Промежуточные ступени на пути к MDW

Создание MDW — важный, но долгий и трудный процесс, требующий значительных инвестиций в технологии и человеческие ресурсы. Он представляет собой эволюцию в управлении данными, где интеграция, доступность, безопасность и масштабируемость имеют первостепенное значение. Пока этот процесс только начинается, большинству организаций требуются промежуточные временные решения для удовлетворения текущих потребностей в хранении данных. Эти решения, которые служат ступеньками на пути к полноценному MDW, позволяют организации продолжать извлекать ценность из своих данных, в то же время оставаясь гибкой и оперативно реагирующей на потребности бизнеса. Это не просто временные решения, а жизненно важные компоненты стратегической миграции.

Существуют три распространенных типа переходных архитектур:

- дополненное корпоративное хранилище данных (EDW);
- временное озеро данных плюс EDW;
- все в одном.

Каждый из этих подходов имеет свои преимущества и недостатки. Их пригодность в качестве путей к созданию MDW может варьироваться в зависимости от потребностей организации, существующей инфраструктуры, бюджета и стратегических целей. Рассмотрим каждый из них более подробно.

Расширенное EDW (augmented EDW)

Эта архитектура обычно используется в тех случаях, когда компания, у которой долгое время было большое локальное EDW, хочет извлечь пользу из своих данных, но ее EDW не может работать с Big Data из-за нехватки пространства хранения, вычислительных мощностей, доступного времени в окне обслуживания для загрузки данных или отсутствия поддержки полуструктурированных данных.

Принцип работы

Вы создаете озеро данных в облаке и копируете в него большие данные. Пользователи могут их запрашивать и создавать отчеты из данных в озере, но основные данные по-прежнему хранятся в EDW, как показано на рис. 10.3.

Преимущества

Архитектура расширенного EDW повышает информационную емкость и гибкость данных за счет использования облачного озера данных наряду с существующим EDW. Эта стратегия обеспечивает масштабируемость и экономическую эффективность, создавая возможности для инновационной аналитики при сохранении текущих структур. Она обеспечивает сбалансированный подход, который согласуется и с ростом бизнеса, и с его непрерывностью.

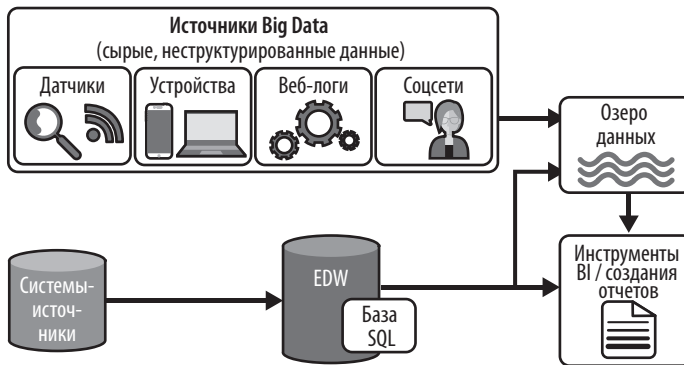


Рис. 10.3. Архитектура расширенного EDW

Сложности

Эта архитектура сопряжена с некоторыми потенциальными сложностями.

- Если необходимо объединить данные из EDW с данными в озере, придется скопировать данные из EDW в озеро.
- Существующие инструменты для создания запросов могут не работать с озером данных.
- Для очистки данных в озере данных понадобится новая форма вычислений, которая может оказаться дорогостоящей и потребовать новых профессиональных навыков.
- Такая архитектура не поможет разгрузить EDW.

Миграция

Эта архитектура может стать началом поэтапного подхода к миграции локального EDW в облако. После того как архитектура будет запущена и начнет работать с большими данными, можно начать миграцию данных из локального EDW в озеро данных. Данные из систем-источников сначала поступают в озеро данных, а затем, при необходимости, в новое реляционное хранилище в облаке, которое является частью настоящего MDW.

Временное озеро данных плюс EDW

Компании обычно используют временное озеро данных вместе с EDW, когда у них уже есть EDW и необходимо задействовать Big Data, но их преобразование займет слишком много времени. Такие компании хотят сократить окно обслуживания EDW, избавив EDW от нагрузки преобразования данных.

Принцип работы

В этой архитектуре озеро данных используется исключительно в качестве временной промежуточной области хранения и уточнения данных. Оно не применяется

для запросов или создания отчетов; все это делается из EDW, как показано на рис. 10.4. Такое ограничение его функциональности позволяет интегрировать озеро данных гораздо быстрее. Иногда можно даже скопировать данные из EDW в озеро, уточнить их и переместить обратно в EDW.

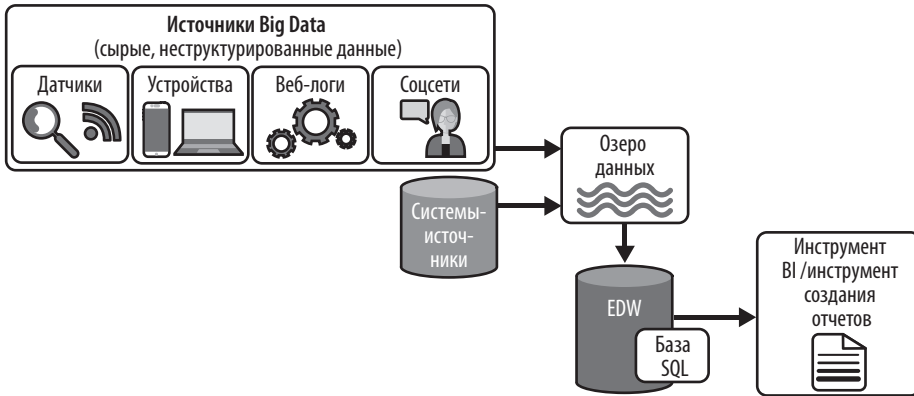


Рис. 10.4. Использование EDW с временным озером данных

Преимущества

Такая архитектура позволяет переложить обработку данных на озеро, снизив нагрузку на EDW и повысив общую производительность. Использование других типов вычислений в озере может обеспечить большую скорость и расширенную функциональность, позволяя гибко работать с большими данными. Такая стратегия предлагает экономически эффективное решение для включения больших массивов данных без нарушения существующей работы EDW, что обеспечивает его гибкость и масштабируемость.

Сложности

Несмотря на использование озера данных, такая архитектура не обеспечивает всех его преимуществ (рассмотренных в главе 5).

Миграция

Превратить такую архитектуру в полноценное MDW можно с помощью всего нескольких модификаций, поэтому это хорошая ступень на пути к MDW.

«Все в одном»

Архитектура «все в одном» обычно используется в организациях, которые стремятся к упрощенному подходу к работе с данными, например стартапах или малом бизнесе. Она предпочтительна для быстрого создания прототипов или для достижения конкретных краткосрочных целей. Это также хороший вариант, когда

основными пользователями являются технические специалисты, предпочитающие интегрированную платформу.

Принцип работы

Как показано на рис. 10.5, все отчеты и запросы выполняются с помощью озера данных. Никаких RDW не задействовано. (Архитектура «все в одном» тесно связана с архитектурой «озеро-хранилище данных», о которой говорится в главе 12.)

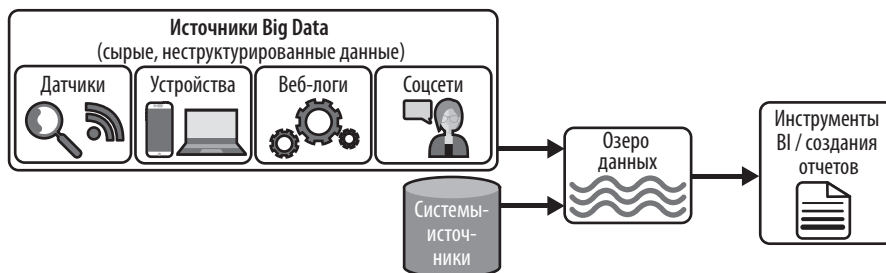


Рис. 10.5. Архитектура «все в одном»

Преимущества

Этот подход позволяет быстро внедрить решение и получить мгновенный результат, что особенно привлекательно для технических команд, нацеленных на быстрый прогресс. Консолидация отчетов и запросов в рамках озера данных упрощает архитектуру, потенциально снижая сложность обслуживания и интеграции. Такая архитектура может быть более гибкой и адаптивной, позволяя использовать разные типы данных и упрощая разработку.

Сложности

Отсутствие RDW влечет за собой снижение производительности, безопасности и удобства использования, а также нарушение целостности ссылок.

Миграция

Для некоторых компаний и сценариев использования, например, когда данные в озере будут использоваться только специалистами data science, может подойти подход с использованием только озера данных. Но чтобы он стал ступенькой к MDW, необходимо будет добавить RDW.

Сценарий использования: стратегический переход компании Wilson & Gunkerk на MDW

Wilson & Gunkerk, вымышленная фармацевтическая компания среднего размера, расположенная на Среднем Западе США, использовала для управления своими

данными локальное решение. Поскольку объем данных никогда не превышал 1 ТБ, компания не чувствовала необходимости переходить на более современные решения, такие как озеро данных или MDW.

Проблема

По мере усиления конкуренции компания все острее ощущала потребность в аналитической информации о тенденциях рынка и поведении клиентов. В существующей системе данных начали проявляться определенные ограничения, и перед организацией встал выбор: либо переходить на более комплексное решение с озером данных, либо внедрять MDW. Еще одним важным моментом была неопределенность относительно будущего роста объема данных, особенно с учетом готовящихся новых проектов исследований и разработок.

Решение

После тщательного анализа компания Wilson & Gunkerk решила внедрить облачное MDW. Это решение было обусловлено следующими факторами.

- Существующий объем данных был значительно меньше 1 ТБ, а рост прогнозировался в пределах до 10 ТБ.
- Миграция на облачное MDW обеспечивала более плавный переход без необходимости адаптации от существующего локального решения к совершенно новой технологии, такой как озеро данных.
- Внедрение MDW было бы экономически эффективным. Компания Wilson & Gunkerk выбрала решение SMP, а не более дорогой вариант MPP, поскольку ее потребности в данных были достаточно скромными.
- Выбранное MDW обеспечивало надежное функционирование и неплохую масштабируемость в соответствии с ожидаемым ростом объемов данных в допустимых пределах.

Результат

Переход на MDW был завершен в течение нескольких месяцев с минимальными нарушениями текущих бизнес-процессов. Компания Wilson & Gunkerk сразу увидела положительные результаты.

Улучшенная аналитика

Новая система позволила улучшить аналитику данных, что дало возможность принимать более обоснованные решения. Компания вышла за рамки простого ретроспективного анализа и перешла к предиктивной аналитике. До появления MDW анализ ограничивался минувшими трендами. Новая система помогла интегрировать в систему различные источники данных и использовать сложные алгоритмы обработки данных. Это позволило компании создавать

прогностические модели, например прогнозировать эффективность лекарств на основе демографических данных и генетических факторов пациента. Переход на новую систему дал возможность принимать более взвешенные решения, обеспечивая более строгий подход к разработке лекарственных средств и персонализированных медицинских решений.

Экономия средств

Решение SMP обеспечило отличную производительность при меньших затратах по сравнению с возможными MPP-решениями или полноценным озером данных.

Перспективность

Оставляя возможность перехода на новые технологии в будущем, MDW позволяет развиваться постепенно и адаптироваться к меняющимся условиям, не делая больших вложений в технологии, в которых компания пока не нуждается.

Пример компании Wilson & Gunkerk иллюстрирует баланс при выборе решений для управления данными. MDW по-прежнему отлично подходит компаниям, работающим с небольшими объемами данных и ищущим масштабируемый и экономически выгодный инструмент. Потребности и траектория роста каждой организации уникальны, поэтому тщательный анализ настоящих и будущих запросов крайне важен для определения наиболее подходящей архитектуры данных.

Итоги

Мы начали главу с подробного описания современного хранилища данных (MDW), а затем — пяти ключевых этапов, которые данные проходят по мере продвижения по MDW: прием, хранение, преобразование, моделирование и визуализация. Эти этапы представляют собой жизненный цикл данных, поскольку данные изначально собираются, надежно хранятся, дорабатываются для удобства использования, затем моделируются для получения аналитических результатов и, наконец, визуализируются для упрощения интерпретации и принятия решений.

Были подробно описаны преимущества MDW, включая интеграцию, масштабируемость, анализ в реальном времени и безопасность, а также рассмотрены такие недостатки, как сложность, высокая стоимость и зависимость от поставщиков услуг. Затем мы остановились на сочетании традиционных реляционных хранилищ с озерами данных, подчеркнув уникальные возможности такой комбинации и повышенную гибкость при работе с разными типами и структурами данных.

Далее мы подробно рассмотрели три переходные архитектуры, которые часто используются в качестве временных решений хранения данных: дополненное

корпоративное хранилище данных, временное озеро данных плюс корпоративное хранилище и решение «все в одном». Каждая модель служит временной мерой, пока организация работает над внедрением полноценного MDW.

В заключение мы привели пример фармацевтической компании Wilson & Gunkerk и ее стратегического перехода на MDW.

В следующих главах мы более подробно рассмотрим еще две архитектуры: ткань данных (data fabric) и озеро-хранилище данных (data lakehouse), их преимущества и возможные сценарии использования с учетом быстро меняющегося ландшафта управления данными.

Ткань данных

Архитектура «*ткань данных*» (data fabric) — это эволюция архитектуры современного хранилища данных (MDW): более продвинутый уровень, построенный на базе MDW для повышения доступности (accessibility), безопасности (security), обнаруживаемости (discoverability) и готовности (availability) данных. Представьте себе, что ткань данных пронизывает всю вашу компанию, накапливая данные и предоставляя их всем, кому они необходимы, внутри компании или даже за ее пределами. Это архитектура, которая может потреблять данные независимо от их объема, скорости поступления или типа. Самая важная часть философии «ткань данных» заключается в том, что ткань может использовать абсолютно любые данные организации.

Это *мое* определение ткани данных; другие специалисты определяют ее по-другому. Некоторые даже используют этот термин как взаимозаменяемый с *современным хранилищем данных*! Например, консалтинговая фирма Gartner начинает с аналогичного определения (https://oreil.ly/fan_P), утверждая, что ткань данных — это

концепция дизайна, которая служит интегрированным элементом (тканью) из данных и связующих их процессов. Ткань данных использует непрерывную аналитику существующих, обнаруживаемых и извлеченных посредством анализа метаданных для поддержки проектирования, развертывания и применения интегрированных и пригодных для повторного использования данных во всех средах, включая гибридные и мультиоблачные платформы.

Точка зрения Gartner расходится с моей в том, что виртуализация данных (о которой вы узнали в главе 6) является основным элементом технологии «ткань данных», которая снижает необходимость перемещения или копирования данных из разрозненных систем. Фирма Gartner представляет себе «умную ткань данных», которая использует графы знаний, искусственный интеллект и машинное обучение для автоматизации поиска и каталогизации данных, нахождения взаимосвязей между данными, оркестрации и интеграции данных, а также автоматического обнаружения метаданных. Однако многие из необходимых технологий еще не существуют в природе, и пока нет еще такой ткани данных, которая бы соответствовала определению Gartner. Я надеюсь, что когда-нибудь такая технология появится, но пока рекомендую сосредоточиться на архитектуре ткани данных в том виде, в котором она существует сегодня.

Ткань данных — это наиболее распространенная архитектура для новых решений, создаваемых в облаке, особенно для больших объемов данных (более 10 терабайт). Многие компании «проводят апгрейд» своей архитектуры MDW до уровня ткани данных, добавляя дополнительные технологии, в особенности обработку в режиме реального времени, политики доступа к данным и каталог метаданных. В этой главе описаны восемь функциональных элементов, которые превращают MDW в ткань данных. Здесь есть некоторая двусмысленность: если добавить три из этих функций, получится MDW или ткань данных? Между ними нет четкой границы. Лично я называю соответствующую архитектуру тканью данных, если в ней применяются как минимум три из этих восьми технологий. Многие вообще не используют ни одну из них, но тем не менее называют свое решение «тканью данных», поскольку придерживаются философии ткани данных (я считаю это вполне нормальным).

В этой главе приводится описание архитектуры «ткань данных» и ее восьми ключевых компонентов: политик доступа к данным, каталога метаданных, управления мастер-данными (MDM), виртуализации данных, обработки в реальном времени, API, сервисов и продуктов.

Архитектура «ткань данных»

Архитектура «ткань данных» содержит все компоненты MDW, но имеет несколько дополнительных функций. На рис. 11.1 показана общая схема этой архитектуры, где этапы прохождения данных обозначены цифрами, как на рис. 10.2. Данные по ткани данных проходят тот же путь, что и по MDW; как вы должны помнить из главы 10, это следующие этапы: (1) прием, (2) хранение, (3) преобразование, (4) моделирование и (5) визуализация.

Итак, что еще предлагает ткань данных, помимо MDW? В этом разделе мы рассмотрим ее дополнительные возможности.

Политики доступа к данным

Политики доступа к данным — это ключ к управлению данными. Они представляют собой набор руководящих принципов, правил и процедур, которые определяют, кто имеет доступ к какой информации, как эта информация может быть использована, когда доступ может быть предоставлен или запрещен в пределах организации. Они помогают обеспечить безопасность, конфиденциальность и целостность важных данных, а также соответствие законам и нормативным актам, таким как Общий регламент по защите данных Великобритании (GDPR, General Data Protection Regulation) (<https://oreil.ly/eYnqO>) и Закон об ответственности и переносе данных о страховании здоровья граждан США (HIPAA, Health Insurance Portability and Accountability Act) (<https://oreil.ly/qI9YI>). Эти политики обычно касаются таких аспектов, как классификация данных, аутентификация и авторизация пользователей, шифрование данных, хранение данных, резервное копирование и восстановление данных, а также удаление данных.

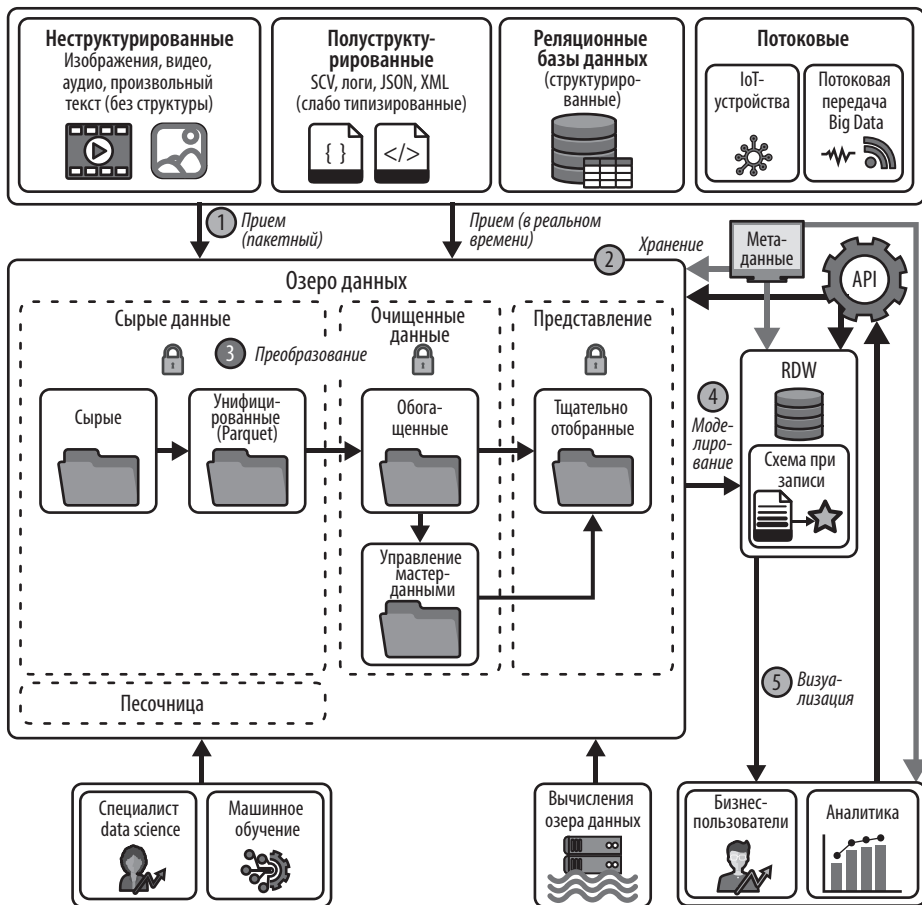


Рис. 11.1. Путь, который проходят данные в архитектуре «ткань данных»

Все запросы данных в организации должны соответствовать установленным политикам доступа и обрабатываться с помощью специальных механизмов, таких как API, драйверы или виртуализация данных, что обеспечивает соблюдение нормативных требований. Например, политики доступа к данным в организации здравоохранения могут потребовать, чтобы доступ к медицинским картам пациентов имел только уполномоченный медицинский персонал. Эти политики должны определять процедуры проверки учетных данных сотрудников и допустимые способы использования информации. Чтобы обеспечить такой контролируемый доступ, организация может внедрить API-интерфейсы для взаимодействия с защищенными системами аутентификации. При наличии такой защиты, когда поставщик медицинских услуг запрашивает запись пациента, API проверяет его учетные данные на соответствие политикам доступа, прежде чем этот доступ предоставить.

Каталог метаданных

Ткань данных включает в себя *каталог метаданных*: репозиторий, в котором хранится информация об активах данных, включая их структуру, взаимосвязи и характеристики. Он обеспечивает централизованный, организованный способ управления и обнаружения данных, облегчая пользователям поиск и понимание нужных им данных. Предположим, пользователь ищет в каталоге метаданных слово «клиент». В результате будут найдены любые файлы, таблицы базы данных, отчеты или дашборды, содержащие данные о клиентах. Возможность увидеть то, какие данные уже загружены и какие отчеты уже созданы, помогает избежать ненужной двойной работы.

Важной частью каталога является информация о *происхождении данных* (data lineage), то есть запись истории любого фрагмента данных, включая то, откуда они взялись, как были преобразованы и где хранятся. Происхождение данных используется для соблюдения таких нормативных требований, как законы о персональных данных. Здесь также можно отследить источник данных и все преобразования, которые они претерпели, что важно для понимания того, как они были созданы и насколько они надежны. Вы наверняка не хотите принимать важные решения на основе данных, которые нельзя считать полностью достоверными! Например, если пользователь спрашивает о конкретном значении в отчете, вы всегда можете проследить его происхождение по каталогу метаданных, чтобы ответить, откуда появилось это значение.

Управление мастер-данными (MDM)

Согласно главе 6, MDM (master data management) — это сбор, консолидация и поддержка согласованных и достоверных данных из разных источников в рамках компании для создания единого доверительного источника мастер-данных. Эти данные, как правило, не связаны с транзакциями и описывают ключевые сущности компании, такие как клиенты, продукты, поставщики и сотрудники.

MDM помогает компаниям принимать обоснованные решения и избегать проблем, связанных с данными, таких как дублирование записей, рассогласование и неверная информация.

Виртуализация данных

В главе 6 вы уже сталкивались с виртуализацией данных. Напоминаю, виртуализация данных — это программная архитектура, позволяющая приложениям и конечным пользователям получать доступ к данным из нескольких источников, как если бы они хранились в одном месте. Виртуализированные данные хранятся на логическом уровне, который выступает в качестве посредника между конечными пользователями и источниками данных. Этот виртуальный уровень представляет собой единую точку доступа, которая абстрагирует сложность базовых источников данных, что позволяет конечным пользователям легко получать доступ к данным из

нескольких источников и проводить их объединение в режиме реального времени без необходимости их копирования или физической интеграции.

Использование виртуализации не означает, что нужно подключать *все* источники данных. По большей части она используется лишь в небольшом количестве сценариев, при этом основная часть данных по-прежнему остается централизованной. Некоторые IT-специалисты рассматривают виртуализацию данных как определяющую функцию ткани данных, утверждая, что архитектура без виртуализации не является тканью данных; я с этим не согласен. В моем определении ткани данных виртуализация не является обязательным условием.

Обработка в реальном времени

В главе 9 мы определили, что под *обработкой в реальном времени* будем понимать обработку данных и немедленное получение результатов, как только данные становятся доступны, без какой-либо заметной задержки. Это позволяет принимать решения на основе актуальной информации (например, информации о дорожной обстановке во время движения).

API

Вместо того чтобы полагаться на строки подключения (connection strings), API-интерфейсы предоставляют данные в стандартизированном виде из разных источников, таких как озеро данных или реляционное хранилище, не раскрывая подробностей местоположения этих данных. Если данные были перемещены, достаточно только обновить внутренний код API; ни одно из приложений, вызывающих API, при этом не затрагивается. Это упрощает использование преимуществ новых технологий.

API также могут включать в себя несколько типов мер безопасности. Кроме того, они могут обеспечить гибкость с помощью фильтрации безопасности, предоставляя детальный контроль над тем, какие пользователи к каким данным могут получить доступ.

Сервисы

Ткань данных может быть построена в виде «блоков», что облегчает повторное использование кода. Например, в компании могут быть сотрудники, которым не нужен доступ к полной функциональности ткани данных, а нужен только код для приема данных или их очистки. Этот код можно сделать обобщенным и инкапсулировать его в общедоступный сервис.

Продукты

Всю ткань данных можно объединить в продукт и продавать его. Он будет особенно привлекателен, если настроить его для определенной сферы деятельности, например для здравоохранения.

Зачем переходить с MDW на архитектуру «ткань данных»?

Компании решают перейти с MDW на ткань данных по нескольким причинам. Хотя MDW традиционно служили основой инфраструктуры данных многих компаний, они не отличаются гибкостью и иногда трудно масштабируемы, в то время как типы данных продолжают быстро развиваться. Ткань данных, напротив, изначально разрабатывалась с учетом требований масштабируемости. Присущая ей гибкость позволяет легко адаптироваться к разным типам и источникам данных, что делает ее устойчивой к текущим требованиям к данным и, в определенной степени, защищенной от устаревания (future-proof).

Кроме того, в какой-то момент разнообразие источников данных может оказаться слишком большим. Ткань данных позволяет органично объединять разные источники, обеспечивая единое, унифицированное представление данных, что значительно упрощает и повышает эффективность работы с многогранными потоками данных.

В быстро меняющейся бизнес-среде способность обрабатывать данные в реальном времени и сразу получать аналитические результаты имеет решающее значение. Ткань данных удовлетворяет эту потребность, поддерживая обработку данных в реальном времени и предоставляя компаниям мгновенный доступ к самым свежим данным.

Кроме того, по мере роста обеспокоенности утечками данных и усиления акцента на соблюдении правил защиты данных безопасность данных и управление ими становятся еще более важной задачей. Ткань данных решает ее с помощью продвинутых политик доступа, обеспечивая более безопасную среду данных. Она также улучшает управление данными (data governance) путем ограничения доступа к данным прошедших авторизацию пользователей, защищая их конфиденциальную информацию от потенциальных угроз. А для многонациональных компаний, работающих в разных юрисдикциях, каждая из которых имеет свои правила защиты данных, расширенные возможности управления, предоставляемые тканью данных, помогают соблюдать комплаенс независимо от того, где находятся данные.

Наконец, некоторые сферы, такие как биржевая торговля и электронная коммерция, отличаются быстро меняющейся ситуацией на рынке. В условиях постоянных изменений оставаться в курсе событий в режиме реального времени — не просто преимущество, а жизненная необходимость. Именно в таких случаях бесценным помощником оказывается ткань данных. Поддержка обработки данных в реальном времени позволяет таким компаниям быть гибкими, информированными и готовыми адаптироваться к любой ситуации.

Потенциальные недостатки

Несмотря на многочисленные преимущества архитектуры «ткань данных», она не лишена недостатков. Переход от MDW к ткани данных может быть ресурсоемким

и на первых этапах сопровождаться трудностями, связанными с высокими затратами, обучением персонала и интеграцией. Кроме того, не все компании нуждаются в расширенных возможностях ткани данных — для малого бизнеса, использующего ограниченное количество источников данных и имеющего простые потребности в обработке данных, может вполне хватить и MDW.

Более того, присущая ткани данных сложность может затруднить поиск и устранение неполадок. Крайне важно убедиться, что в организации или у ее партнеров есть знания и опыт, необходимые для перехода на новую систему.

Несмотря на то что ткань данных предлагает передовое решение для современных задач управления данными, прежде чем приступить к переходу на нее, каждая компания должна оценить свои индивидуальные потребности, потенциальную окупаемость инвестиций и долгосрочные стратегические цели.

Итоги

В этой главе мы подробно рассмотрели концепцию архитектуры «ткань данных» — продвинутой модификации MDW, призванной повысить доступность, безопасность, обнаруживаемость и готовность данных. Мы дали определение ткани данных и провели его сравнение с принятым в IT-индустрии, при этом подчеркнув такие различия, как роль виртуализации данных. Мы рассмотрели восемь ключевых функциональных элементов, которые превращают MDW в ткань данных, но не проводят при этом строгого разграничения между этими архитектурами.

Далее мы рассмотрели основные компоненты архитектуры «ткань данных», включая политики доступа к данным, каталог метаданных, управление мастер-данными, обработку в реальном времени, API и дополнительные элементы, такие как виртуализация данных. В главе подчеркивается высокая адаптивность ткани данных, особенно при работе с большими объемами данных в реальном времени.

Мы также рассмотрели причины, по которым следует переходить с MDW на ткань данных, а также потенциальные недостатки, такие как сложность и ресурсоемкость перехода. Коротко говоря, каждая организация должна тщательно оценить свои индивидуальные потребности и возможности, прежде чем воспользоваться этим передовым решением.

Озеро-хранилище данных

Я кратко упомянул об озере-хранилище данных как о модели, сочетающей концепции озера данных и хранилища данных. Идея *озера-хранилища данных* (data lakehouse) заключается в том, чтобы упростить работу, используя для хранения всех имеющихся данных только одно озеро данных вместо нескольких отдельных реляционных хранилищ данных. В этом случае озеру данных потребуется больше функциональности для замены функций RDW. И здесь на помощь приходит продукт Delta Lake от компании Databricks.

Delta Lake — это программный слой транзакционного хранения, который запускается поверх существующего озера данных и добавляет функции, подобные функциям RDW, которые повышают надежность, безопасность и производительность озера. *Сам по себе Delta Lake не является системой хранения.* В большинстве случаев превратить озеро данных в Delta Lake очень просто: достаточно при сохранении данных в озеро указать, что вы хотите использовать формат Delta Lake (в отличие от других форматов типа CSV или JSON).

Под капотом, когда вы сохраняете файл в формате Delta Lake, он хранится определенным образом: в виде файлов Parquet в папках и журнала транзакций для отслеживания всех изменений, внесенных в данные. Хотя фактические данные хранятся в озере данных в привычном формате, добавленный журнал транзакций превращает озеро в Delta Lake, расширяя его возможности. Но это означает, что все, что взаимодействует с Delta Lake, должно поддерживать формат Delta Lake; для большинства продуктов так и есть, поскольку формат стал очень популярным.

Delta Lake — не единственный вариант расширения функциональных возможностей озера данных; два других популярных решения — Apache Iceberg и Apache Hudi, обладающие схожими возможностями. Однако в этой главе мы сосредоточимся на Delta Lake.

Вы уже познакомились с реляционным хранилищем данных (глава 4), озером данных (глава 5), современным хранилищем данных (глава 10) и тканью данных (глава 11). В этой главе мы добавим к этому списку озеро-хранилище данных. На рис. 12.1 показаны все эти архитектуры в хронологической последовательности, по мере их появления.

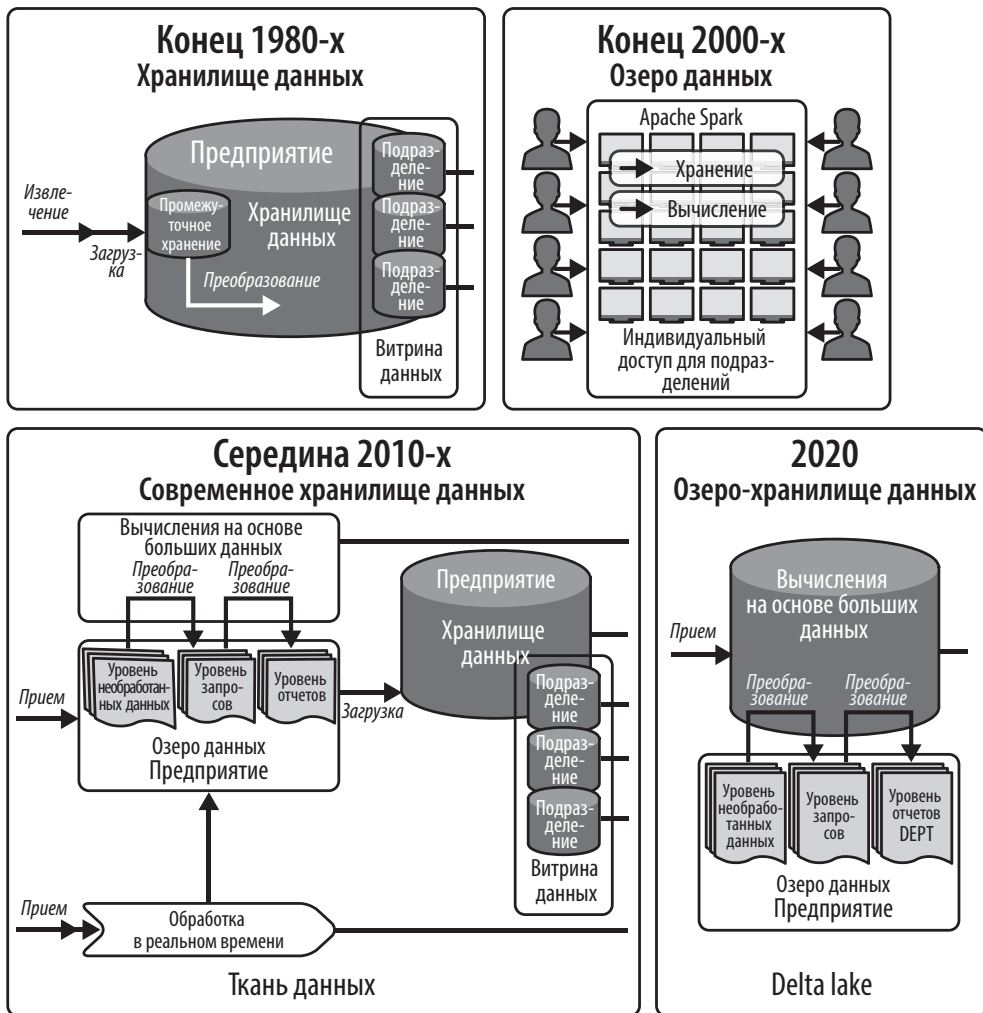


Рис. 12.1. Хронология появления архитектур данных

Функции Delta Lake

Delta Lake добавляет в озеро данных несколько функций, подобных имеющимся у реляционного хранилища. В этом разделе мы рассмотрим некоторые из них.

Возможно, главная причина, по которой компании используют Delta Lake, заключается в том, что оно поддерживает команды *языка обработки данными* (DML, data manipulation language), такие как INSERT, DELETE, UPDATE и MERGE. Эти команды упрощают сложные задачи управления данными, делая обработку данных в Delta Lake более гибкой и надежной. Озера данных не предоставляют встроенную

поддержку этих операций, поскольку они оптимизированы под пакетную обработку и хранение больших объемов данных, а не для быстрых обновлений в режиме реального времени.

Обновление данных в озере обычно предполагает чтение всего файла, внесение необходимых обновлений и запись обновленного файла обратно в озеро данных, что может занять много времени, особенно для больших файлов. В отличие от этого, когда Delta Lake изначально работает с таблицей — по сути, файлом, представляющим собой организованную коллекцию данных в строках и столбцах, — оно разбивает эту таблицу на несколько небольших цифровых файлов для простоты управления. В результате получается *дельта-таблица* (Delta Table). Затем Delta Lake использует журнал транзакций для отслеживания изменений, что позволяет командам DML работать гораздо быстрее благодаря оптимизированному хранилищу (с колоночным форматом хранения), обработке в оперативной памяти и оптимизации под пакетную обработку. Например, посредством оператора `UPDATE` Delta Lake находит и выбирает все файлы, содержащие данные, которые соответствуют предикату и поэтому должны быть обновлены. Затем он считывает каждый такой файл в память, обновляет соответствующие строки и записывает результат в новые файлы данных. Это позволяет эффективно обновлять данные, не переписывая всю дельта-таблицу. Журнал транзакций хранит историю изменений данных и гарантирует, что данные всегда находятся в согласованном состоянии, даже в случае сбоя или отказов системы.

Часто при работе с базами данных вы будете слышать аббревиатуру ACID (atomicity, consistency, isolation, durability), которая расшифровывается как *атомарность, согласованность, изолированность и долговечность*. Это четыре свойства, которые обеспечивают надежность и целостность транзакции в базе данных. Они гарантируют, что транзакция может быть завершена как единая надежная операция, при этом изменения данных будут либо полностью применены, либо полностью отменены. Хотя можно сказать, что Delta Lake поддерживает транзакции ACID, это утверждение требует уточнения. В отличие от обработки ACID-транзакций в реляционной базе данных, такой как SQL Server, поддержка ACID в Delta Lake ограничена одной дельта-таблицей. Выполнение DML-транзакций для более чем одной дельта-таблицы в Delta Lake не гарантирует целостности ACID. В реляционной базе данных целостность ACID работает с DML-транзакциями, которые охватывают несколько таблиц.

Delta Lake также предлагает функцию «путешествие во времени», которая позволяет запрашивать данные, хранящиеся в дельта-таблицах, в том виде, в котором они существовали в определенный момент. История изменений данных хранится в журнале транзакций Delta Lake, вместе с метаданными, например время внесения изменения и идентификатор пользователя, который внес это изменение. Пользователи могут просматривать и получать доступ к предыдущим версиям данных, а при необходимости даже возвращать данные к более раннему состоянию. Это может быть полезно для аудита, отладки или восстановления данных в случае непреднамеренного изменения данных или других проблем (например, для отката транзакций).

Озера данных часто сталкиваются с проблемой «маленьких файлов», когда большое количество файлов малого размера может замедлять операции чтения и записи и увеличивать стоимость хранения. Delta Lake решает ее с помощью оптимизированных алгоритмов уплотнения (сжатия) файлов, позволяющих эффективно объединять маленькие файлы в большие. Уплотнение выполняется автоматически в фоновом режиме, и его можно настроить для выполнения по расписанию или на запуск вручную.

С помощью Delta Lake пользователи могут выполнять как пакетную обработку, так и потоковую передачу данных в режиме реального времени для одних и тех же данных в дельта-таблице, что избавляет от необходимости поддерживать отдельные конвейеры и системы данных для пакетной и потоковой обработки. Это унифицированное решение облегчает управление и поддержку конвейеров и упрощает архитектуру обработки данных, что, в свою очередь, означает, что Delta Lake поддерживает лямбда-архитектуру (см. главу 7).

Принудительное применение схемы (schema enforcement) — это функция дельта-таблицы, которая позволяет указать ожидаемую схему для данных в дельта-таблице и применить такие правила, как ограничения на нулевые значения полей, ограничения на тип данных и ограничения на уникальность данных. Это гарантирует, что данные, записанные в дельта-таблицу, будут соответствовать указанной схеме, это помогает предотвратить повреждение данных. Без принудительного применения схемы в папку может быть добавлен файл с недопустимой схемой, что может привести к ошибке при выполнении заданий ELT. Если входящие данные не соответствуют схеме, Delta Lake отклонит операцию записи и выдаст ошибку.

Повышение производительности

Использование Delta Lake позволяет повысить производительность озера данных несколькими способами.

Пропуск данных

Delta Lake может пропускать нерелевантные данные при чтении из дельта-таблицы, что может значительно повысить производительность запросов.

Кэширование

Delta Lake поддерживает кэширование данных в Spark, что может значительно повысить производительность повторяющихся запросов.

Быстрое индексирование

Delta Lake использует оптимизированную структуру индексирования для быстрого поиска данных, что сокращает время выполнения запросов.

Оптимизация запросов

Delta Lake интегрируется со Spark SQL и может использовать возможности Spark по оптимизации запросов, что повышает скорость выполнения и эффективность запросов.

Передача предикатов

Delta Lake поддерживает *перенос предикатов* (predicate pushdown), это означает, что условия фильтрации переносятся на уровень хранения, при этом сокращается объем данных, которые необходимо обработать.

Обрезка столбцов

При *обрезке столбцов* (column pruning) считываются только столбцы, необходимые для конкретного запроса, что сокращает объем данных, которые надо обработать.

Векторизованное выполнение

При *векторизованном выполнении* (vectorized execution) несколько точек ввода данных обрабатываются за одну инструкцию процессора, что повышает производительность.

Параллельная обработка

Delta Lake поддерживает параллельную обработку, это означает, что несколько задач могут выполняться параллельно, приводя к повышению производительности.

Z-упорядочивание

Z-упорядочивание, также известное как упорядочивание Мортонa, — это метод индексации данных, используемый в архитектурах Delta Lake для организации данных таким образом, чтобы ускорить и повысить эффективность доступа к данным и выполнения запросов.

Архитектура озера-хранилища данных

Как показано на рис. 12.2, данные в озере-хранилище проходят те же пять этапов, которые мы рассматривали в архитектурах MDW и «ткань данных». Это (1) прием, (2) хранение, (3) преобразование, (4) моделирование и (5) визуализация данных.

Как следует из рис. 12.2, в архитектуре озера-хранилища данных есть только одно хранилище (озерo данных, использующее Delta Lake), а не два (озерo данных и реляционное хранилище). Реляционное хранилище заменяется дополнительным реляционным сервисным уровнем, описанным далее в этой главе. Это решают шесть задач, часто встречающихся в архитектурах MDW и «ткань данных».

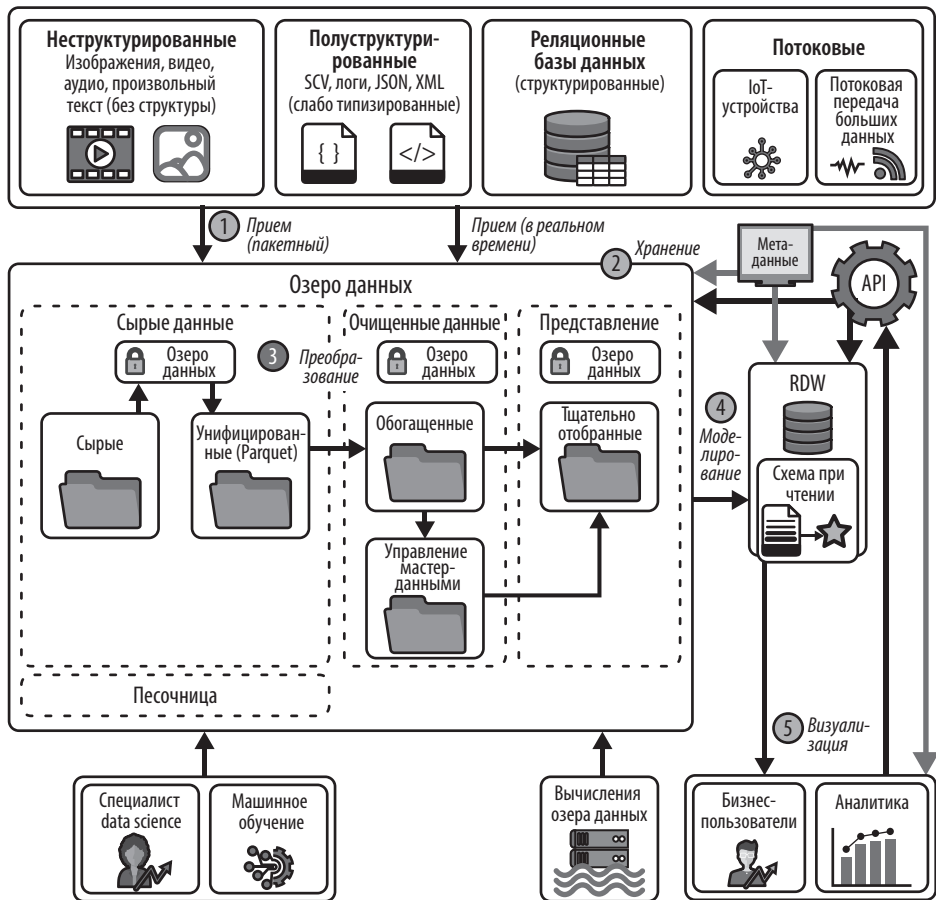


Рис. 12.2. Архитектура озера-хранилища данных

Надежность

Поддержание согласованности озера данных и реляционного хранилища может стать проблемой, особенно если большие объемы данных часто приходится копировать из озера в хранилище. Если задания копирования завершаются сбоем, данные копируются неточно или размещаются не в том месте в озере данных, это может вызвать проблемы с надежностью. Например, запрос отчетов на основе RDW может возвращать результаты, отличные от результатов отчетов на основе тех же данных из озера данных. В случае с озером-хранилищем, поскольку в нем нет RDW, в которое копировались бы данные, такой проблемы не возникает.

Устаревание данных

Данные в RDW всегда будут старше, чем эквивалентные данные в озере. Насколько — будет зависеть от частоты копирования данных из озера в хранилище;

а чтобы не повлиять на производительность выполнения запросов и отчетов, вы не будете запускать эти задания слишком часто. Как и в случае с проблемами надежности, описанными выше, это может привести к тому, что отчеты на основе данных из RDW и озера будут содержать разные результаты. В случае с озером-хранилищем данных, поскольку в нем нет реляционного хранилища, в которое копировались бы данные, такой проблемы не возникает.

Ограниченная поддержка расширенной аналитики

Немногие системы и инструменты искусственного интеллекта/машинного обучения хорошо работают с RDW, поскольку специалисты data science обычно предпочитают работать с файлами в озере данных. В архитектуре озера-хранилища данных они получают такую возможность.

Совокупная стоимость владения данными

Несмотря на относительно дешевизну хранения, вычисления, необходимые для копирования данных в RDW, требуют дополнительных затрат. Кроме того, в результате получаются две копии данных, что увеличивает расходы на хранение. В озеро-хранилище данных, напротив, существует только одна копия данных. Вдобавок вычисления, используемые для выполнения запросов и создания отчетов в RDW, обычно стоят гораздо дороже, чем вычисления, используемые в озере данных. К тому же управление озером данных и RDW требует разных компетенций, и подбор разных сотрудников для работы с этими сущностями также может привести к дополнительным расходам.

Управление данными

Наличие двух копий данных в двух разных средах хранения, возможно, с двумя разными типами безопасности, повышает риск того, что кто-то увидит данные, которые ему видеть не следует. Также сложно обеспечить, чтобы обе системы следовали одинаковым правилам качества данных и преобразования данных. В случае с озером-хранилищем данных, где копия данных единственная, таких проблем не существует.

Сложность

Управление как озером данных, так и RDW может быть сложной задачей, требующей специальных навыков и ресурсов. Поскольку в озеро-хранилище данных нет RDW, для работы с ним требуется меньший объем специальных знаний.

Что, если отказаться от реляционного хранилища данных?

Этим мы открываем ящик Пандоры. Еще недавно я бы сказал вам, что отказ от RDW — очень плохая идея. Теперь, когда благодаря Delta Lake новые функции, аналогичные имеющимся в реляционном хранилище, продолжают добавляться

к озеру данных, я начинаю видеть все больше и больше сценариев использования, для которых оптимальной архитектурой будет озеро-хранилище данных. Аргумент в пользу использования озера-хранилища данных особенно убедителен при работе с небольшими датасетами. Поскольку у большинства облачных провайдеров есть бессерверные вычисления, которые можно использовать в работе с Delta Lake, вы можете экономить, оплачивая отдельные запросы. Вы также экономите за счет того, что больше не нужно копировать данные в реляционное хранилище, которое стоит дороже и использует более дорогие реляционные вычисления. А если вы применяете специализированные вычисления для реляционного хранилища, а не переходите на бессерверные, вы платите, даже если не используете их. Как и с любой архитектурой, если вы решили отказаться от RDW, очень важно учесть возможные недостатки и компромиссы такого решения.

Первый недостаток заключается в том, что запросы к реляционной базе данных выполняются быстрее, чем запросы к Delta Lake, особенно если в реляционном хранилище используется технология MPP (см. главу 7). Реляционные хранилища имеют функции для повышения производительности запросов, которые недоступны в Delta Lake, например:

- продвинутая индексация (в том числе кластеризованные поколоночные индексы и полнотекстовые индексы);
- продвинутая статистика;
- кэширование (если вы не используете Spark);
- продвинутая оптимизация плана выполнения запроса;
- материализованные представления;
- продвинутая оптимизация объединения.

Отдельные функции производительности Delta Lake, такие как Z-упорядочивание, могут частично компенсировать некоторые из этих недостающих функций.

В Delta Lake также отсутствуют некоторые основные функции безопасности реляционного хранилища, такие как безопасность на уровне строк, безопасность на уровне столбцов, шифрование данных в состоянии покоя, шифрование на уровне столбцов, прозрачное шифрование данных (TDE, transparent data encryption) и динамическое маскирование данных (которое автоматически заменяет или скрывает часть данных, так что неавторизованные пользователи видят замаскированную версию данных вместо фактической конфиденциальной информации). Оно также не поддерживает представление в SQL, ссылочную целостность, управление рабочей нагрузкой или расширенные функции аудита и комплаенса, такие как аудиторские следы, политики хранения данных и сертификаты соответствия. Реляционные хранилища также поддерживают более высокий уровень конкурентности, чем Delta Lake, поскольку они предоставляют расширенные функции, такие как продвинутая блокировка, уровни изоляции и управление транзакциями.

Сложность также является проблемой. При использовании реляционного хранилища у вас есть *обязательный уровень метаданных* — вы должны создать базу данных, схему и таблицу с полями, описывающими тип данных, а затем загрузить данные («схема при записи»). Это означает, что метаданные всегда находятся поверх фактических данных. Это требует предварительной работы, но большое преимущество заключается в том, что метаданные и данные всегда связаны друг с другом, поэтому метаданные легко использовать для поиска данных. Вы никогда не «потеряете» метаданные, и они всегда будут точно описывать данные. Это сильно отличается от Delta Lake, где все основано на папках и файлах. Это основная причина, по которой конечные пользователи, привыкшие к реляционному хранилищу, часто испытывают трудности с Delta Lake.

В Delta Lake метаданные не обязательно должны находиться рядом с данными. Они могут быть в одном или нескольких отдельных файлах, в самом файле, содержащем данные, в одной папке с данными, в отдельной папке или вообще отсутствовать. Ко всему прочему, метаданные могут быть очень неточными, поскольку они не имеют той однозначной связи с данными, которая есть в реляционном хранилище. Можно представить, как это путает конечных пользователей, которым сложно найти метаданные и поэтому они ошибочно решают, что метаданных нет, или даже используют не те метаданные. Наконец, может произойти рассинхронизация метаданных с данными, если изменения вносятся за пределами Delta Lake.

Использование определенных функций архитектур Delta Lake может привести к тому, что придется прибегнуть к Spark. Кроме того, если вы переходите с продукта, использующего версию SQL, отличную от Spark SQL, приготовьтесь переписать хранимые процедуры, представления, отчеты, дашборды и так далее.

Если вам все же придется использовать Spark SQL, это может означать переобучение конечных пользователей, которые уже привыкли взаимодействовать с реляционным хранилищем и знакомы с необходимыми инструментами. Они могут использовать ANSI-совместимый SQL или RDW-продукт, например T-SQL (который применяется в продуктах Microsoft). Они также привыкли к реляционной модели и знают, как быстро создавать запросы и отчеты на ее основе. Переход к миру папок и файлов, скорее всего, вынудит их осваивать новые инструменты, привыкать к «схеме при чтении» и решать проблемы, связанные со скоростью передачи, безопасностью, отсутствием функций, сложностью и изменениями SQL, описанными ранее. Придется учиться слишком многому.

Существующие технологии могут частично снять некоторые из этих проблем, а со временем и вовсе сделают их неактуальными. Но пока этого не произошло, учитывайте их применительно к конкретным потребностям, когда будете решать, использовать ли архитектуру озера-хранилища данных. Например, если в архитектуре «озеро-хранилище данных» запросы к озеру данных занимают в среднем пять секунд, является ли это проблемой? Если конечных пользователей это устраивает — хорошо, но если они применяют дашборды и нуждаются в миллисекундном

времени отклика на запросы, придется скопировать данные для дашбордов, в реляционное хранилище (то есть в итоге использовать архитектуру MDW). Вместо этого можно применить продукт для создания отчетов, который позволяет импортировать данные в свою память и в итоге получить время отклика на запрос порядка миллисекунд, но данные при этом будут не такими свежими, как в Delta Lake, и их придется обновлять в памяти этого инструмента для создания отчетов через определенные промежутки времени.

И снова мы говорим о компромиссах. Если вы архитектор, ваша основная задача — определение продуктов, которые можно использовать в рамках каждой архитектуры, а также выявление и анализ всех их плюсов и минусов, чтобы выбрать наиболее подходящую архитектуру и соответствующие продукты под конкретный сценарий использования.

Ни одна из вышеперечисленных проблем сама по себе не является непреодолимой, но в совокупности они могут послужить достаточным основанием для использования реляционного хранилища. Многие компании начинают с проверки концепции, чтобы определить, не вызовут ли компромиссы, связанные с отказом от реляционного хранилища, каких-либо проблем. Если нет, то они переходят к озеру-хранилищу данных.

Реляционный сервисный уровень

Поскольку Delta Lake работает по принципу «схема при чтении» (см. главу 2), схема применяется к данным в момент их чтения, не раньше. Delta Lake — это система файлов и папок, поэтому она не предусматривает контекста, описывающего, что из себя представляют данные. (В отличие от уровня представления метаданных в реляционном хранилище, который находится поверх данных и связан с ними напрямую.) Определенных отношений в Delta Lake не существует. Каждый файл находится на своем изолированном острове, поэтому, чтобы напрямую связать метаданные с данными, необходимо создать «сервисный уровень» поверх данных в Delta Lake. Чтобы помочь конечным пользователям быстро разобраться в данных, вы, скорее всего, захотите представить данные в реляционной модели, поэтому создадите «реляционный сервисный уровень» (relational serving layer). С помощью этого уровня, расположенного поверх данных, можно определить отношения между файлами, если требуется объединить несколько файлов. Реляционный сервисный уровень может принимать разные формы: SQL-представление, датасет в инструменте для создания отчетов, таблица Apache Hive или специальный SQL-запрос. Если уровень организован грамотно, конечный пользователь даже не догадается, что он получает данные из Delta Lake, — он будет думать, что они получены из реляционного хранилища.

Многие компании создают SQL-представления поверх файлов в Delta Lake, а затем используют инструмент создания отчетов для вызова этих представлений. Это облегчает конечным пользователям создание отчетов и дашбордов.

Даже при наличии реляционного сервисного уровня с решением Delta Lake все равно возникают некоторые проблемы. Например, реляционный сервисный уровень может неверно отображать данные, или в итоге могут получиться два уровня, которые указывают на одни и те же данные, но имеют разные метаданные. Метаданные, не привязанные к данным, — это имманентная проблема Delta Lake. В реляционном хранилище подобные проблемы отсутствуют, поскольку оно использует универсальную модель данных, которой могут пользоваться все.

Итоги

В этой главе была рассмотрена концепция озера-хранилища данных, причем особое внимание было уделено роли Delta Lake в качестве транзакционного уровня хранения, который значительно повышает надежность, безопасность и производительность существующих озер данных.

Вы узнали о потенциальных недостатках, связанных с отказом от традиционного RDW, с акцентом на проблемы, касающиеся скорости, безопасности и конкурентности. Просто помните об этих недостатках и, если для вас они не представляют проблем, используйте озеро-хранилище данных. Если один из этих недостатков окажется слишком серьезным для какого-то конкретного датасета, вы можете скопировать в реляционное хранилище только эти данные (не обязательно копировать все). По мере совершенствования технологий Delta Lake и аналогичных, таких как Apache Iceberg и Apache Hudi, а также технологий хранения и вычислений, причин для поддержки реляционных хранилищ будет становиться все меньше и меньше. Большинство новых архитектур данных будут представлять собой озера-хранилища данных.

Мы ознакомились с оригинальным подходом «схема при чтении» для Delta Lake, когда интерпретация данных происходит в момент чтения, но не раньше. Вы также узнали о структуре файлов и папок, которая лишена контекстного описания и тем самым значительно отличается от структурированного слоя представления метаданных в реляционном хранилище. Это обуславливает необходимость создания «реляционного сервисного уровня» для установления прямой контекстной связи с данными в Delta Lake.

Стремительный технологический прогресс продолжает влиять на стратегии архитектуры данных. Еще несколько лет назад отказ от реляционного хранилища в архитектуре считался бы серьезной ошибкой, но современные тенденции указывают на растущее число сценариев использования, когда озеро-хранилище данных становится оптимальным выбором.

Все архитектуры, которые мы рассматривали до сих пор, представляли собой централизованные решения, то есть исходные данные копировались в одно центральное место, которым владел IT-отдел. В следующей главе мы разберем кардинально отличающийся сценарий, поскольку будем обсуждать децентрализованное решение — сетку данных.

Основы сетки данных

Сетка данных (data mesh) представляет собой децентрализованную архитектуру данных с четырьмя ключевыми особенностями. Во-первых, она требует, чтобы независимые команды в определенных доменах владели своими аналитическими данными. Во-вторых, в сетке данных данные обрабатываются и предоставляются как продукт, чтобы потребителю данных было легко находить их, доверять им и использовать их в любых целях. В-третьих, она опирается на автоматизированное выделение инфраструктуры. И в-четвертых, она использует управление данными (data governance), чтобы гарантировать, что все независимые продукты данных являются безопасными и следуют глобальным правилам.

Хотя понятия, лежащие в основе сетки данных, не новы, заслуга Жамак Дегани (Zhamak Dehghani), генерального директора и основателя компании Nextdata (<https://nextdata.com>), состоит в том, что она ввела термин *сетка данных* и объединила все эти понятия¹. Хотя звучат разные мнения о том, что такое сетка данных, в этой книге мы придерживаемся определения, данного в работе Дегани, в частности основанного на четырех принципах сетки данных, которые будут рассмотрены ниже. Очень важно понимать, что сетка данных — это концепция, а не технология. Речь идет об организационном и культурном сдвиге в рамках всей компании. Технология, используемая для создания сетки данных, может либо следовать архитектуре современного хранилища данных, ткани данных или озера-хранилища данных, либо разные домены могут иметь разную архитектуру.

В этой и следующей главах мы будем углубляться в тонкости архитектуры сетки данных, и я предлагаю дорожную карту для навигации по ее сложному ландшафту, основанному на принципе децентрализации. В этой главе мы разберем децентрализованную архитектуру данных, прояснив причины ажиотажа вокруг сетки данных и описав четыре ее основополагающих принципа. Мы изучим, что собой, собственно, представляет сетка данных, и все тонкости доменов данных, познакомимся с логической архитектурой и разнообразными топологиями, поддерживающими инфраструктуру, и проведем различия между сеткой данных и тканью данных. И наконец, обсудим сценарии использования сетки данных.

¹ См.: Дегани Ж., How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh (<https://oreil.ly/cv2sX>) MartinFowler.com, 20 мая 2019 г.; Data Mesh Principles and Logical Architecture), (<https://oreil.ly/NTOLG>) MartinFowler.com, 3 декабря 2020 г. и «Data Mesh: Delivering Data-Driven Value at Scale» (O'Reilly, 2022)

В главе 14 мы немного сместим фокус и рассмотрим потенциальные проблемы при внедрении сетки данных, развенчивая распространенные мифы, чтобы помочь организациям в принятии обоснованных решений относительно ее внедрения. Далее я расскажу о проведении организационной оценки с целью определения, соответствует ли внедрение сетки данных потребностям бизнеса. После этого мы перейдем к рекомендациям по успешному внедрению сетки данных, обеспечивающему бесшовный переход и максимальные преимущества. Затем мы рассмотрим перспективы развития сетки данных в постоянно меняющемся ландшафте data science и в завершение обсудим, когда следует использовать каждую из четырех архитектур: современное хранилище данных, ткань данных, озеро-хранилище данных и сетку данных.

Присоединяйтесь, чтобы раскрыть все тонкости архитектуры сетки данных в ходе краткого, но глубокого обзора ее децентрализованной структуры.

Децентрализованная архитектура данных

Архитектуры данных, о которых ранее шла речь в этой книге, — современное хранилище данных, ткань данных и озеро-хранилище данных — представляют собой централизованные архитектуры, где все аналитические данные создаются ИТ-командой и принадлежат ей. Кардинальное отличие сетки данных от прочих архитектур данных состоит в том, что сетка является децентрализованной. Это означает, что данные, а также все, что используется для управления ими, не имеют отношения к центральному управляющему органу или единой ИТ-команде, а принадлежат отдельным командам, или «доменам», внутри организации, управляются ими и сгруппированы по бизнес-доменам. Эти команды доменов отвечают за сбор и обработку своих данных и управление ими и обладают автономией в принятии решений об использовании и распространении своих данных. Данные хранятся в пределах домена; конечные пользователи могут получать доступ к ним и запрашивать их там, где они находятся, не копируя их в некое центральное местоположение. Это приводит к повышению ответственности и гарантирует, что управление и обслуживание данных осуществляются теми, кто обладает наибольшими знаниями в соответствующей области.

Такая схема решает главную проблему централизованных архитектур — организационное и техническое масштабирование. По мере роста компании и необходимости быстро централизовывать все больше данных ИТ-отдел часто испытывает излишнюю нагрузку и становится узким местом.

Когда я говорю о том, что все остальные архитектуры данных в этой книге являются централизованными, я имею в виду самые разные параметры этих архитектур. Центральная ИТ-команда не только владеет данными и контролирует их, она также отвечает за:

- интеграцию данных из различных источников;
- хранение данных в централизованном месте, которым она владеет;

- создание моделей данных и их размещение в центральном репозитории;
- управление данными и комплаенс, включая определение и соблюдение стандартов качества данных, безопасности и конфиденциальности;
- создание и поддержку конвейеров данных и логики их преобразования;
- обеспечение качества данных с помощью профилирования, проверки, очистки и других процессов;
- оптимизацию производительности;
- управление всеми вычислительными технологиями;
- управление инструментами аналитики и создания отчетов;
- хранение метаданных и управление ими;
- аварийное восстановление и резервное копирование.

Кроме того, в централизованной архитектуре аппаратные средства и вычислительные мощности рассчитаны только на вертикальное масштабирование.

Ажиотаж вокруг сетки данных

Сетка данных привлекает к себе много внимания с момента своего появления в 2019 году. У консалтинговой компании Gartner есть хорошо известный «цикл хайпа» (hype cycle) для управления данными, который измеряет зрелость технологий на основе текущего уровня внедрения и количества лет до массового внедрения. В своем исследовании «Hype Cycle for Data Management 2023»¹ (<https://oreil.ly/xgsrW>) компания поместила сетку данных вблизи вершины цикла (на подходе к «пику завышенных ожиданий») и с «умеренным» рейтингом преимуществ. Проникновение сетки данных на рынок составляет от 5 до 20 %, и, что интересно, Gartner прогнозирует, что «сетка данных устареет еще до выхода на плато. Практика и поддерживающие технологии будут развиваться в сторону ткани данных, поскольку организации начнут собирать пассивные метаданные».

Многие, кто создает ажиотаж вокруг сетки данных, утверждают, что проекты по созданию хранилищ данных достаточно часто терпят неудачу, являются неустойчивыми и больше не могут обрабатывать большие данные, потому что не могут масштабироваться. Учитывая всю эту шумиху, можно подумать, что создание сетки данных — это решение подобных «проблем» с хранилищами данных. Правда заключается в том, что проекты по созданию хранилищ данных редко проваливаются, и происходит это не потому, что они не могут достаточно масштабироваться для работы с большими данными или потому, что архитектура или технология не способны это делать. Неудача почти всегда связана с человеческим фактором и/или с процессом либо с тем, что организация выбрала совершенно неверную

¹ В технической литературе обычно называется «Кривая зрелости технологий для управления данными», хотя буквальный перевод — «цикл хайпа». — *Примеч. ред.*

технологию. Иногда мне кажется, что сетка данных — это решение в поисках задачи, а люди, расхваливающие сетку данных, слишком упрощают все проблемы управления и интеграции данных.

Я видел, как решения для Big Data отлично работали в течение многих лет без сетки данных начиная с конца 1980-х годов (когда терабайт считался «Big Data») и заканчивая сегодняшними петабайтами данных. Технология не стоит на месте. Централизованные архитектуры данных и облачные продукты вполне способны обрабатывать практически любые объемы пакетных и потоковых данных.

Эти решения работают с большими данными уже несколько десятилетий. Почему мы должны предположить, что они перестанут работать в будущем?

Конечно, это все теории. Сетка данных все еще в новинку, но очевидно, что не все поддаются ажиотажу вокруг нее. Я отношусь к лагерю скептиков. Я работаю с базами данных уже более 35 лет, а с хранилищами данных — 15 и видел множество надуманных технологий, которые приходят и уходят. Хотя я не верю, что сетка данных устареет, но также не верю, что какие-то компании создадут «настоящую» (чистую) сетку данных. Возможно, часть компаний построит модифицированную сетку данных, а остальные будут использовать другую архитектуру.

Я видел очень мало компаний, внедривших модифицированную сетку данных в продакшен-среде. Многие заявляют об этом, желая казаться продвинутыми, но в большинстве случаев это на самом деле ткань данных или озеро-хранилище данных. Но даже при очень вольной трактовке понятия «сетка данных» процент компаний, создающих такую сетку, очень мал. Когда я говорю об архитектуре данных, всегда спрашиваю аудиторию: «Многие ли слышали о сетке данных?» Обычно не менее 75 % поднимают руки. Затем я спрашиваю: «Сколько из вас внедряют сетку данных?» Возможно, из ста человек поднимется одна или две руки. Затем я спрашиваю: «Сколько из вас используют сетку данных на практике?» Я еще не видел ни одной поднятой руки. Другие коллеги сообщают об аналогичном опыте. Хотя это и анекдот, он, по-видимому, указывает на то, что сетка данных не оправдывает ажиотажа вокруг нее. Тем не менее у архитектуры сетки данных есть много плюсов, которые могут улучшить любую из остальных архитектур. В этой и следующей главах мы поговорим об этом более подробно.

Четыре принципа сетки данных по Дегани

Сетка данных пытается решить четыре самые большие проблемы централизованных архитектур данных: отсутствие владельца данных, низкое качество данных, техническое масштабирование и организационное масштабирование. Четыре принципа, которые излагает Дегани, направлены на решение этих проблем¹. Рассмотрим по очереди каждый из них.

¹ См.: Дегани Ж. «Data Mesh: Delivering Data-Driven Value at Scale» (издание на русском языке: «Data Mesh. Новая парадигма работы с данными»).

Принцип № 1. Владение доменом

Принцип № 1 рекомендует децентрализовать ответственность и распределить ее между людьми, которые находятся ближе всего к данным, чтобы поддерживать непрерывные изменения и масштабируемость (например, производство, продажи, поставщики).

В централизованной архитектуре обычно возникает путаница (а на некоторых совещаниях и громкие споры) по поводу того, кто «владеет» аналитическими данными.

Большинство исходных данных генерируется собственными действующими системами, инструментами CRM, такими как Salesforce и Microsoft Dynamics, и инструментами планирования ресурсов предприятия (ERP), такими как SAP и Microsoft Dynamics. Каждая из этих доменно-ориентированных систем и приложений управляется специальной группой эксплуатации, члены которой обычно практически не общаются с центральной IT-командой платформы данных и зачастую даже не знают, что «их» данные передаются на платформу данных.

Когда данные находятся в операционной базе данных, они принадлежат операционному домену, который владеет базой данных. Когда данные копируются в централизованное хранилище данных (обычно в озеро данных), ими, как правило, владеет IT-отдел, но некоторые специалисты считают, что они все равно должны принадлежать домену.

Со своей стороны, центральная IT-команда платформы данных редко имеет четкое представление о том, как работают действующие системы или бизнес-приложения. Они просто плохо разбираются в данных и не имеют представления о том, как эти данные были сгенерированы и что они на самом деле означают. Они просто видят имена таблиц и полей или файлы. В результате им сложно получить качественные данные и полезную аналитику.

Эта проблема разрешается с помощью первого принципа сетки данных — владение доменом. Сотрудники, отвечающие за операционные базы данных, всегда являются владельцами аналитических данных и отвечают за управление данными и их поддержку. Ответственность и владение децентрализованы и распределены между теми, кто находится ближе всего к данным, для поддержки непрерывных изменений и масштабируемости. Эти сотрудники группируются по бизнес-доменам, таким как производство, продажи или поставщики; определение доменов в компании может быть непростой задачей. Каждый домен владеет своими операционными и аналитическими данными.

Принцип № 2. Данные как продукт

Принцип № 2 предлагает рассматривать аналитические данные, предоставляемые доменами, как продукт, а потребителей этих данных — как клиентов. У каждого домена есть команда домена, API-код, данные и метаданные, а также инфраструктура.

В централизованной архитектуре команда IT-специалистов владеет аналитическими данными и, следовательно, отвечает за их качество, но ее члены обычно не слишком хорошо разбираются в этих данных и могут допускать ошибки при их преобразовании. Команды доменов, напротив, обладают глубоким пониманием данных и их контекста в рамках своих бизнес-доменов. Эти знания позволяют им выявлять и решать проблемы качества данных, которые могут быть упущены централизованными IT-командами, поскольку у них меньше информации для понимания потребностей бизнеса.

Второй принцип сетки данных заключается в том, что владельцы данных относятся к ним как к *полноценному продукту*, за который они несут ответственность, а не рассматривают аналитические данные как некие входные данные, актив или побочный продукт процесса, которым управляют другие. Они относятся к потребителям этих данных (пользователям данных, аналитикам, специалистам data science) как к клиентам. Это означает, что команды доменов применяют продуктивное мышление к своим данным, чтобы убедиться, что они легко находимы, надежны, доступны, безопасны и понятны.

Первый шаг к тому, чтобы относиться к данным как к продукту, — определить, что именно в организации является продуктом данных. *Продукт данных* — это самостоятельная, независимо разворачиваемая единица данных, обладающая бизнес-ценностью. Он содержит аналитические данные, полученные из операционных или транзакционных приложений. Он проектируется и создается одной командой продукта. Каждый продукт данных автономен и управляется совершенно независимо от других продуктов данных. Он решает конкретную бизнес-задачу и должен быть разработан с четким пониманием потребностей целевых пользователей. Продукт данных связан с определенным доменом данных, поэтому он обслуживает конкретную область бизнеса и должен соответствовать бизнес-целям этой области. Обычно в каждом домене существует несколько продуктов.

Имейте в виду, что *продукт данных* — это не то же самое, что *данные как продукт*. *Данные как продукт* — это идея, согласно которой владельцы данных рассматривают данные как полноценный продукт, за который они несут ответственность, а не как побочный продукт процесса, которым управляют другие, и должны сделать данные доступными для других доменов и потребителей. Понятие *продукта данных* относится к архитектуре воплощения данных в виде продукта. Некоторые IT-специалисты рассматривают датасеты, аналитические модели и дашборды с отчетами как продукты данных, фокусируясь на физическом представлении данных, и не считают метаданные, код или инфраструктуру продуктами данных.

Обратите внимание, что продукт данных может состоять из подпродуктов, каждый из которых отвечает за предоставление определенного подмножества данных или функциональности для конкретной предметной области. Например, продукт данных для данных о клиентах может иметь подпродукты для транзакционных, демографических и поведенческих данных, каждый из которых управляется отдельной командой.

Продукты данных потребляются через *контракты данных*, которые представляют собой соглашения между доменами данных или продуктами данных, определяющие формат и структуру обмена данными между производителем и потребителем. Контракт данных — это очень обширный документ, который обычно содержит метаданные, схемы данных, информацию о преобразовании данных и правила доступа к данным. Он также содержит информацию о том, как другие домены, продукты или потребители будут получать доступ к продукту данных, как правило, через API, базу данных, стандартный формат файла, поток событий или граф.

Поскольку команды доменов непосредственно отвечают за данные, они могут быстро выявлять и устранять любые проблемы с качеством. Это приводит к ускорению процесса решения проблем и снижает риск использования неточных данных при принятии бизнес-решений. Наконец, доменные команды интегрированы в бизнес, поэтому они лучше понимают специфические потребности в данных для своих доменов. Это гарантирует, что данные управляются и поддерживаются таким образом, чтобы соответствовать потребностям бизнеса, что приводит к получению более качественных и актуальных аналитических данных.

Еще одно замечание по поводу принципа № 2: каждая команда домена отвечает за приобретение и управление собственной инфраструктурой и распределение ресурсов, а также за набор сотрудников для создания архитектуры и управления данными. Это серьезное изменение: вместо очистки и обработки данных для потребления (копирования оперативных данных в аналитические), чем занимался ранее IT-отдел, эта ответственность теперь ложится на команду домена. Доменные команды также должны определять бизнес-потребности в своих данных. Это означает, что каждая команда может определять свои потребности в ресурсах, исходя из требований домена и объема данных. Каждая команда также может самостоятельно предоставлять и масштабировать ресурсы по мере необходимости, чтобы реагировать на колебания спроса, поэтому доменные команды теперь не конкурируют за инфраструктурные ресурсы от центральной IT-команды.

Принцип № 3. Самообслуживаемая инфраструктура данных как платформа

Принцип № 3 рекомендует упростить создание продуктов данных и управление ими за счет автоматизации выделения инфраструктуры (например, хранилищ, вычислений, конвейеров данных и контроля доступа).

Как уже отмечалось ранее в этой главе, в централизованных архитектурах центральная IT-команда может стать узким местом, затрудняя эффективное масштабирование. Эта команда поддерживает и контролирует всю архитектуру, включая хранилища, вычисления, конвейеры данных и политики доступа к данным. Часто им просто не хватает людей, чтобы своевременно предоставлять данные.

В сетке данных ситуация иная. Поскольку каждая команда домена владеет данными своего домена и рассматривает их как продукт, ей нужны собственные команды по

работе с данными и инфраструктура для создания, развертывания, мониторинга и предоставления доступа к продукту данных. Это расширяет ответственность доменных команд — от разработки операционных приложений до построения аналитики и создания решения для совместного использования данных. Говоря коротко, первые два принципа сетки данных возлагают дополнительную нагрузку на команды инженеров данных домена. Третий принцип направлен на ее снижение.

Команды доменов могут прийти в ужас при мысли о том, что им придется заниматься менеджментом, набирать штат сотрудников, контролировать и оплачивать разработку и управление своим продуктом данных. Чтобы избежать разочарования и отторжения, вам нужно дать им «подсказку», как справиться с этой дополнительной работой. Совсем не обязательно, чтобы каждый домен создавал собственную инфраструктуру с нуля; дублирование усилий на то, чтобы «изобрести велосипед» в каждом домене, существенно затян Timer в ввод в действие сетки данных, увеличит затраты и приведет к риску крупномасштабных несоответствий и несовместимости между доменами.

Чтобы предотвратить эти проблемы, платформа сетки данных должна создаваться специальной командой центральной платформы. Не все в сетке данных децентрализовано. Платформа должна содержать все инструменты и интерфейсы, необходимые команде инженеров данных домена для упрощения жизненного цикла создания, тестирования, развертывания, защиты, поддержки и обмена продуктами данных с потребителем или с другими доменами данных. Командам домена не нужно беспокоиться о выделении ресурсов базовой инфраструктуры. Это можно сделать, предоставив набор стандартизированных API-интерфейсов или скриптов платформы, которые разработчик продукта данных может использовать для указания своих потребностей в инфраструктуре. Затем вы можете предоставить платформе возможность создавать и управлять хранилищем, вычислениями, конвейерами данных и контролем доступа. В результате вы получите стандартизированный способ создания продуктов данных, их защиты, поиска, подключения к ним и чтения данных из них. В сетке данных команды инженеров данных домена состоят из технических специалистов широкого профиля, а не узкой направленности, поэтому снижение сложности с помощью платформы ячейки данных может освободить их ресурсы для внедрения новых технологий работы с данными.

Задача этого принципа — экономия затрат, уменьшение сложности, облегчение работы команд доменов, снижение потребности в технической специализации сотрудников и автоматизация политик управления.

Принцип № 4. Федеративное вычислительное управление

Принцип № 4 (federated computational governance) гласит, что коллективное управление данными между доменами и центральной командой по работе с данными существует для определения, внедрения и мониторинга глобальных правил (например, интероперабельности, качества данных, безопасности данных, установленных правил и моделирования данных).

Управление данными в централизованной архитектуре намного проще, чем в сетке данных, поскольку при централизованном подходе к архитектуре одна центральная команда по работе с данными владеет всеми данными и определяет, внедряет и контролирует все управляющие процессы. Однако такой централизованный подход часто приводит к возникновению «узких мест», замедляя процесс принятия решений и ограничивая способность отдельных доменов быстро реагировать на меняющиеся условия и особые ситуации, с которыми они сталкиваются. Это также может привести к нехватке специальных знаний и понимания контекста, каковые имеются у команд отдельных доменов, поскольку центральной команде приходится управлять огромным разнообразием данных без глубокого понимания нюансов каждого домена.

В сетке данных центральная команда по работе с данными определяет и контролирует все стандарты и политики управления данными, включая такие вопросы, как качество данных, безопасность данных, нормативные требования и моделирование данных. Центральная команда по работе с данными состоит из представителей доменов и профильных экспертов (по вопросам комплаенса, юридическим вопросам, безопасности и т. д.). Однако внедрение и мониторинг управления данными возлагаются на домены, поскольку они владеют данными и знают их лучше остальных.

Проблема распределения ответственности между командами доменов заключается в вероятности возникновения несоответствий и конфликтующих практик, что приводит к неэффективности работы и повышению сложности. Домены могут использовать разные стандарты качества данных (например, полные названия штатов США вместо двухбуквенных почтовых аббревиатур); либо разные системы безопасности или методы моделирования данных. Но не все правила управления могут быть установлены глобально; некоторые из них зависят от контекста и должны задаваться в рамках домена. Например, центральная команда по работе с данными может указать, что пользователи обязательно должны проходить аутентификацию, а доменные команды определяют, к каким продуктам данных будет иметь доступ каждый пользователь.

Такая несогласованность может привести к появлению некачественных данных, к тому, что пользователи будут видеть данные, которые им не следует видеть, и к другим проблемам. Централизованные системы зачастую страдают от этого, поскольку пытаются навязать универсальную политику, что может привести к несогласованию и неспособности удовлетворить конкретные потребности и запросы разных отделов и подразделений.

Именно поэтому концепция федеративного управления данными, сочетающая централизованный надзор с автономией отдельных доменов, имеет решающее значение для поддержания согласованности и безопасности в рамках всей организации. Необходимо найти баланс между независимостью доменов и надзором с стороны центральной команды по работе с данными. По возможности следует автоматизировать внедрение и мониторинг политик, а также маркировать данные для облегчения их идентификации.

«Настоящая» сетка данных

Теперь, когда вы разобрались с этими четырьмя принципами, взгляните на рис. 13.1. На нем показана высокоуровневая архитектура сетки данных, которая использует все эти четыре принципа. Первые два заключаются в децентрализации: каждый домен создает аналитические данные с помощью собственной ИТ-команды и инфраструктуры. Но для реализации двух других принципов требуются централизованные команды: одна — для создания платформы инфраструктуры данных, другая — для реализации общего управления для всех используемых доменов.

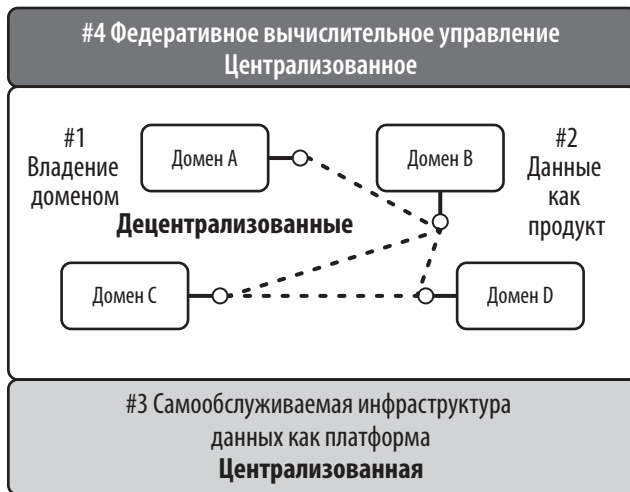


Рис. 13.1. Архитектура сетки данных

Так что же делает сетку данных *настоящей*? Вот тут-то и начинается путаница. Не существует универсальных правил, по которым решение можно назвать сеткой данных.

Если компания создает сетку данных, полностью соответствующую всем четырем принципам, это решение с уверенностью можно назвать «настоящей» сеткой данных. Но что если в решении используются только три принципа? Можно ли по-прежнему считать его сеткой данных? А если два?

А что, если решение использует все четыре принципа, но при этом не соответствует полностью всем требованиям принципа? Например, можно следовать принципу № 2, но при этом домены будут совместно использовать одну инфраструктуру, имея одно физическое озеро данных, логически разделенное на доменно-ориентированные папки, доступ к которым имеют только соответствующие домены; или, возможно, несколько доменов будут назначены одному из нескольких вычислительных кластеров. Другой пример: центральный ИТ-отдел, используя принцип № 3, может предложить сценарии, которые помогут организовать хранилище и управление

доменами, но при этом оставить доменам возможность самостоятельно создавать остальную инфраструктуру. И еще один пример: в соответствии с принципом № 4 центральный IT-отдел может установить стандарты в области безопасности данных и регулирования, но при этом обязать домены самостоятельно отслеживать соответствие нормативным требованиям и предоставить им свободу заниматься качеством данных и моделированием по своему усмотрению.

Некоторые компании создают решение с двумя или тремя частично реализованными принципами и называют его сеткой данных. Другие создают отдельные озера данных для каждого домена и говорят, что построили сетку данных. Однако простая группировка данных по отделам или организация данных в домены не является сеткой данных. Даже если предположить, что при этом домены спроектированы верно (это тема следующего раздела), то в таком случае применяется только принцип № 1, а другие нет.

Некоторые поставщики будут уверять вас, что если вы используете их продукт, то у вас будет сетка данных, хотя сетка данных — это организационный и культурный сдвиг, а не какая-то технология. Я даже слышал, как некоторые говорили, что все, что нужно сделать для создания сетки данных, — это соединить все озера и хранилища данных с помощью программы для виртуализации! Неважно, что при этом не соблюдается *ни один* из четырех принципов.

Как видите, в рамках каждого принципа может быть множество исключений из концепции чистой сетки данных. Сколько нужно сделать исключений, чтобы решение уже не считалось сеткой данных (пусть минимально жизнеспособной)? Если не принять четыре принципа в целом, не стоит даже пытаться ответить на этот вопрос. И должны ли мы полностью принять определения Дегани или нужно скорректировать их на основе мнений других отраслевых экспертов? Нужен ли нам отраслевой комитет, чтобы уточнить определение сетки данных и утвердить его? И как избежать мнений, основанных исключительно на корыстных интересах?

Боюсь, у меня нет ответов на все эти вопросы. Я надеюсь, что эта книга положит начало здоровой дискуссии, внеся некоторую ясность в общие концепции архитектуры данных и архитектур, которые предшествовали сетке данных.

Домены данных

Процесс проектирования архитектуры, ориентированной на домен, называемый *доменно-ориентированным проектированием* (DDD, domain-driven design), очень сложен и требует много времени. DDD возникло в контексте проектирования программного обеспечения, но применительно к данным это понятие усложняется. Первый шаг — определить, что такое *домен* в контексте организации.

Согласно первому принципу Дегани, аналитическими данными должны владеть те бизнес-домены, которые находятся ближе всего к ним, — либо источники данных,

либо их основные потребители. Она определяет три типа доменно-ориентированных аналитических данных.

Доменные данные, ориентированные на источник

Это аналитические данные, тесно связанные с операционными источниками, из которых они поступают. Данные копируются из операционных баз данных приложения-источника в домен и преобразуются в совместно используемые аналитические данные. Эти данные не подгоняются и не моделируются для конкретного потребителя. Домены, ориентированные на источник, могут иметь один или несколько источников: данные обычно поступают из операционных систем данного домена, но могут быть также операционными или аналитическими данными из других доменов. Например, компания, продающая книги онлайн, может разделить свои данные на домены на основе пути клиента к покупке: поиск товара, просмотр, оформление заказа и оплата. Проблема создания доменов заключается в вопросе владения данными; например, если данные предприятия разделены на домены приложений (домены, основанные на поддержке одного конкретного приложения), а вместо этого их необходимо разделить на бизнес-домены (основанные на бизнес-возможностях), то каждый бизнес-домен может владеть более чем одним приложением.

Агрегированные доменные данные

Это аналитические данные, которые объединяют и/или агрегируют данные из других доменов, как правило, для повышения производительности запросов. Например, можно объединить данные из домена производства и домена продаж, чтобы легче и быстрее создать отчет о прибылях и убытках или создать модель данных, которую гораздо проще использовать потребителям.

Доменные данные, ориентированные на потребителя

Это аналитические данные, преобразованные в соответствии с потребностями одного или нескольких отделов или сценариев использования. Домены, ориентированные на потребителя, почти всегда получают данные из доменов, ориентированных на источник. Например, данные для домена производства могут быть смоделированы и описаны двумя способами: один — для тех, кто знаком с производством (ориентированные на источник), другой — для тех, кто не связан с производством (ориентированные на потребителя). Другой пример — преобразование данных, ориентированных на источник, в данные, ориентированные на потребителя, для упрощения обучения ML-моделей.

Если вы хотите узнать больше о доменах данных и доменно-ориентированном проектировании, я настоятельно рекомендую книгу Питхейна Стренгхольта (Piethein Strengholt) «Data Management at Scale»¹ (O'Reilly, 2023).

¹ Стренгхольт П. Масштабируемые данные. Лучшие шаблоны высоконагруженных архитектур. СПб, издательство «Питер».

Логическая архитектура сетки данных

На рис. 13.2 показано, как может выглядеть логическая архитектура сетки данных для компании с несколькими доменами. Каждый из этих доменов будет иметь один или несколько продуктов.

На рис. 13.2 производство, продажи и поставщики — это домены, ориентированные на источник. Каждый домен имеет свои собственные операционные и аналитические данные. Например, домен продаж может использовать Salesforce для отслеживания всей информации о своих клиентах (операционные данные). Затем отдел продаж может поместить свои данные в архитектуру озера-хранилища данных (см. главу 12) и объединить их с другими источниками данных, содержащими операционные данные, а также с аналитическими данными из домена поставщика. Таким образом создаются аналитические данные отдела продаж, которые затем могут попасть в озеро-хранилище данных. Отдел продаж также создает API для обмена этими аналитическими данными и связанными с ними метаданными с другими доменами и потребителями.

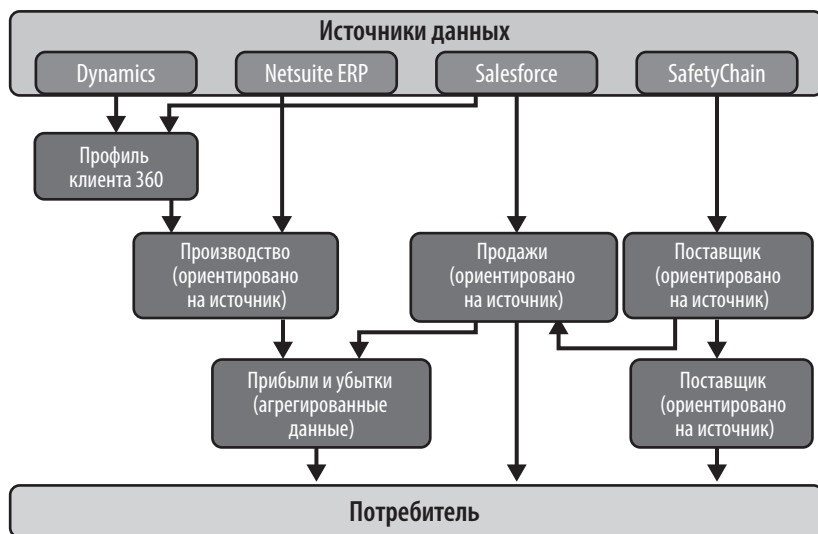


Рис. 13.2. Логическая архитектура сетки данных

Обратите внимание на домен поставщика, ориентированный на потребителя. Он был создан потому, что аналитические данные в домене поставщика, ориентированном на источник, были слишком сложны для понимания людьми, не входящими в этот домен. В домене поставщика, ориентированном на потребителя, модель данных упрощена, а жаргон поставщика более понятен.

Существует также домен агрегированных данных прибылей и убытков, который строится на основе данных из домена производства и домена продаж,

ориентированных на источник. В значительной степени он создан из соображений производительности. Выполнение запроса или отчета, требующего объединения данных из нескольких доменов, может оказаться слишком медленным, если эти домены имеют отдельные озера данных, расположенные географически очень далеко друг от друга.

Наконец, есть домен «Профиль клиента 360» (Customer 360). Он получает данные о клиентах (такие как демографические, поведенческие, транзакционные данные и данные об отзывах клиента) из разных источников, очищает и обрабатывает их (см. главу 6), а затем объединяет, чтобы сформировать полную картину о клиенте. Все домены, которым нужны данные о клиенте, берут их из домена «Профиль клиента 360». Это гораздо лучше, чем если бы каждый домен самостоятельно собирал, очищал и обрабатывал данные о клиенте.

Как вы уже убедились, большое количество доменов может привести к неразберихе. Представьте, как будет выглядеть диаграмма на рис. 13.2, если на нее добавить десятки или даже сотни доменов. Вот почему доменное проектирование так важно. В этой главе мы кратко затронули эту тему, однако я рекомендую вам обратиться к другим ресурсам, в которых она рассматривается более подробно. Прежде чем создавать свою сетку данных, стоит уделить достаточно времени проектированию доменов и продуктов, чтобы через несколько месяцев не пришлось возвращаться назад и все переделывать.

Различные топологии

Вариаций или топологий архитектуры сетки данных существует множество. На рис. 13.3 показаны три типа, о которых я расскажу.

Тип сетки 1

В этой архитектуре все домены используют одну технологию. Например, все они могут работать с одним и тем же облачным провайдером и быть ограничены продуктами этого провайдера, поэтому каждый домен будет применять одинаковые продукты для хранения, создания конвейеров данных, безопасности, реляционных хранилищ данных, формирования отчетов, каталога метаданных, MDM, виртуализации данных, API-интерфейсов, инструментов машинного обучения и т. д. Для некоторых продуктов могут быть опции, но минимальные. Каждый домен будет иметь свою инфраструктуру, поэтому все будет децентрализовано — за исключением хранения данных. Вместо собственного озера данных для каждого домена будет создано единое корпоративное озеро данных, причем каждый домен получит свой собственный контейнер или папку в этом озере, доступ к которым будет иметь только он.

Логически домены имеют собственные озера данных, но физически данные для всех доменов находятся в одном озере. Многие клиенты используют такую топологию из-за проблем с производительностью, которые могут возникнуть при

объединении данных из нескольких физически разных озер данных, находящихся на расстоянии многих километров друг от друга. Кроме того, наличие одного физического озера данных значительно упрощает защиту и мониторинг данных, а также выполнение резервного копирования для аварийного восстановления.

Тип сетки 2

В этой архитектуре домены используют ту же технологию, что и в первом типе. Однако вместо одного корпоративного озера данных для всех доменных данных каждый домен имеет собственное озеро. Все озера данных используют одну технологию. Это делает инфраструктуру по-настоящему децентрализованной, но за это приходится платить: технически сложно связать все озера данных вместе и добиться приемлемой производительности при объединении данных из нескольких доменов. Вот почему большинство компаний используют первый тип сетки.

Тип сетки 3

В этой архитектуре каждый домен может использовать любые технологии и любых облачных провайдеров по своему усмотрению, а также может иметь собственное озеро данных (которое, в свою очередь, тоже может применять любую технологию). Например, у вас могут быть домены A и D, использующие Azure, домен B — AWS и домен C — GCP, при этом домен A выбирает Azure Synapse для своего реляционного хранилища данных, а домен D — SQL Server в виртуальной машине для своего хранилища. В этом случае сложности будут связаны с разными типами безопасности для каждого домена, поиском и содержанием специалистов для поддержки десятков продуктов, разработкой стандартов управления для многих продуктов в разных облаках (принцип № 4), созданием автоматизированной инфраструктуры, позволяющей учитывать все это разнообразие (принцип № 3), и при этом с попытками объединить данные из нескольких продуктов в разных облаках. Все это вписывается в категорию «настоящей сетки данных», о которой мы говорили выше. Однако внушительный список проблем — это причина, по которой я считаю, что сетка третьего типа никогда не будет реализована.

В левой части рис. 13.3 представлена архитектура с наибольшей централизацией, а по мере продвижения вправо они становятся все более распределенными.

Сетка данных в сравнении с тканью данных

Ткань данных и сетка данных — концепции, занимающие важное место в современном ландшафте данных, но они выполняют разные роли, и их не следует путать. По своей сути ткань данных — это *архитектурный фреймворк*, предназначенный для использования в одном или нескольких доменах внутри сетки данных. А сетка данных — это *целостная (холистическая) концепция*, включающая в себя технологии,

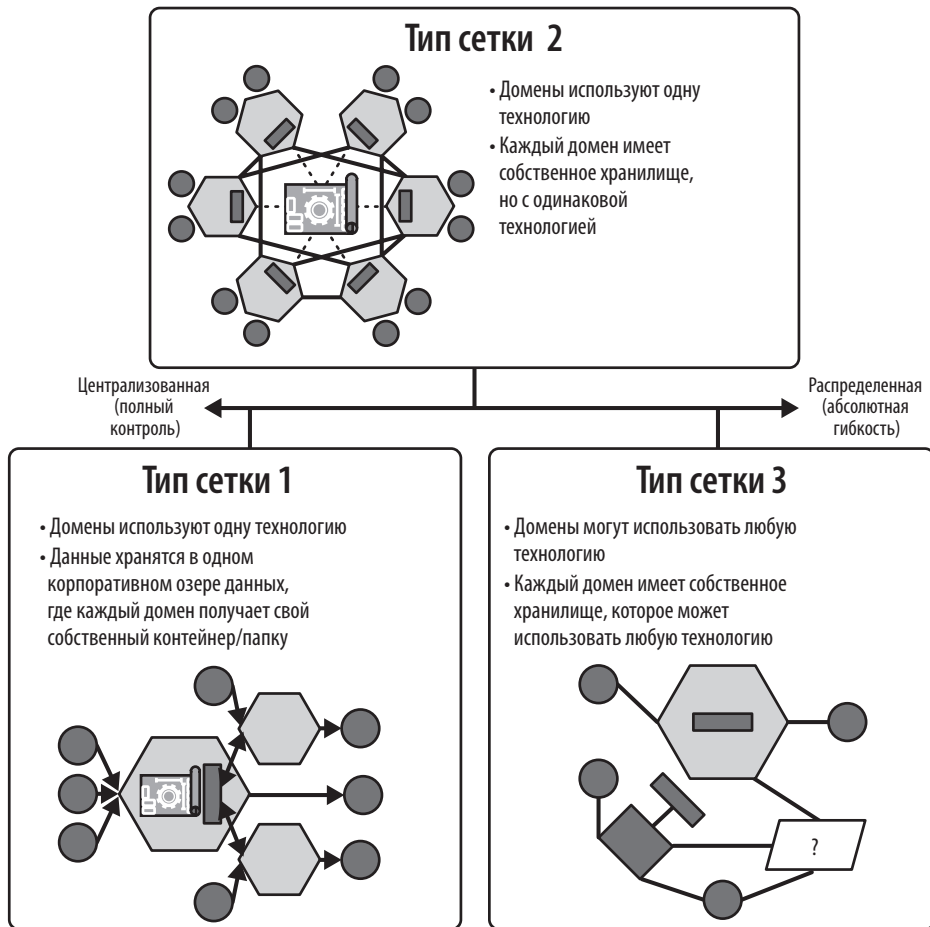


Рис. 13.3. Три топологии сетки данных в диапазоне от полного контроля до абсолютной гибкости

стратегии и методологии. Фактически ее определение настолько всеобъемлюще, что я даже не решаюсь назвать ее архитектурой.

С другой стороны, технологические и архитектурные основы ткани данных могут быть адаптированы для поддержки различных функций и компонентов внутри сетки данных. Соотношение между ними — не конкуренция и не взаимозаменяемость, а скорее синергия. Одна сетка данных может включать в себя несколько тканей данных, каждая из которых адаптирована к уникальным потребностям соответствующего домена.

В этом контексте каждый домен в сетке данных можно рассматривать как собственную экосистему. В рамках этой экосистемы домен может создавать, совершенствовать и оперировать собственной тканью данных, обеспечивая ее

соответствие конкретным требованиям и целям. Другими словами, если сетка данных представляет собой набросок децентрализованного и доменно-ориентированного подхода к данным, то ткань данных предоставляет архитектурную и технологическую инфраструктуру, обеспечивающую работу этих доменов. Их использование в тандеме может привести к созданию более интегрированной и эффективной инфраструктуры данных. Это относится и к сочетанию сетки данных с архитектурами современного хранилища данных или озера-хранилища данных.

Сценарии использования

Надеюсь, к этому моменту вы уже хорошо понимаете, что такое сетка данных в определении Дегани, и именно ее я буду называть «настоящей». Хотя «настоящая» сетка данных в теории кажется отличным решением, на самом деле она слишком гипотетична и для ее поддержки существует ограниченное количество технологий. Поэтому практические решения для сетки данных будут включать в себя массу исключений из концепции Дегани. Большинство таких решений содержат так много отклонений от настоящей сетки, что эти реализации даже не стоит называть сеткой данных.

Главный вопрос, который необходимо задать, заключается в следующем: пытаетесь ли вы осуществить только федерализацию данных, то есть интегрировать данные из нескольких разрозненных источников без физической консолидации либо хранения данных в едином репозитории, принадлежащем и управляемом IT-отделом, или же вы действительно хотите создать сетку данных? Вы можете реализовать федерализацию данных без построения сетки данных; базовую федерализацию данных можно рассматривать как удовлетворение принципа № 1 сетки данных, но не трех других принципов. Если вы пойдете по этому пути, может ли ваше решение по-прежнему называться сеткой данных? Я думаю, оно может называться как угодно, но, возможно, точнее будет назвать его корпоративным озером-хранилищем данных или архитектурой ткани данных, особенно если каждый домен имеет свое собственное рабочее пространство. Наиболее точное описание того, что вы построили, — это корпоративное озеро-хранилище данных или архитектура «ткань данных» с несколькими рабочими пространствами (по одному рабочему пространству на каждый домен). Помните, что вы можете соблюсти принцип № 1 сетки данных, если каждый домен владеет аналитическими данными, полученными из своих операционных данных, даже если эти аналитические данные создает IT-отдел. Однако вы не сможете соблюсти принцип № 2, пока каждый домен не создаст собственные аналитические данные и интерфейс для обмена этими данными.

Не забывайте также, что для того, чтобы компания соответствовала принципу № 1, она должна пройти через процесс реорганизации и разделения на бизнес-домены. Я сомневаюсь, стоит ли заниматься созданием доменов, если при этом не соблюсти

принцип № 2 (данные как продукт). Если очисткой данных будет заниматься IT-отдел, а не домены, то пропадет весь смысл реализации сетки данных.

В качестве примера можно привести ситуацию, когда компания приобретает другие компании. Каждая из приобретенных компаний владеет своими данными, и их IT-отделы используют ПО виртуализации для доступа к ним. В этом сценарии вы автоматически выполнили федерализацию данных, но у вас определенно отсутствует сетка данных; вы не разделяете свою организацию на домены данных (если только вы не определите домены по бизнес-функциям и не решите, что приобретенные компании сами собой являются доменами, — а это плохое решение, потому что тогда у вас будет, например, несколько HR-доменов с дублирующимися HR-продуктами). Но даже в этом случае организация доменов на основе приобретенных компаний удовлетворяет только принципу № 1. Я бы сказал, что для того, чтобы назвать архитектуру сеткой данных, должен быть соблюден принцип № 2, поскольку он реализует два самых больших преимущества сетки данных: организационное и техническое масштабирование.

И последнее: в стремлении найти баланс между централизацией и децентрализацией в сетке данных решающее значение имеет распределение ответственности. На рис. 13.4 представлена шкала, показывающая, как должны распределяться зоны ответственности в нашем вымышленном сценарии использования.

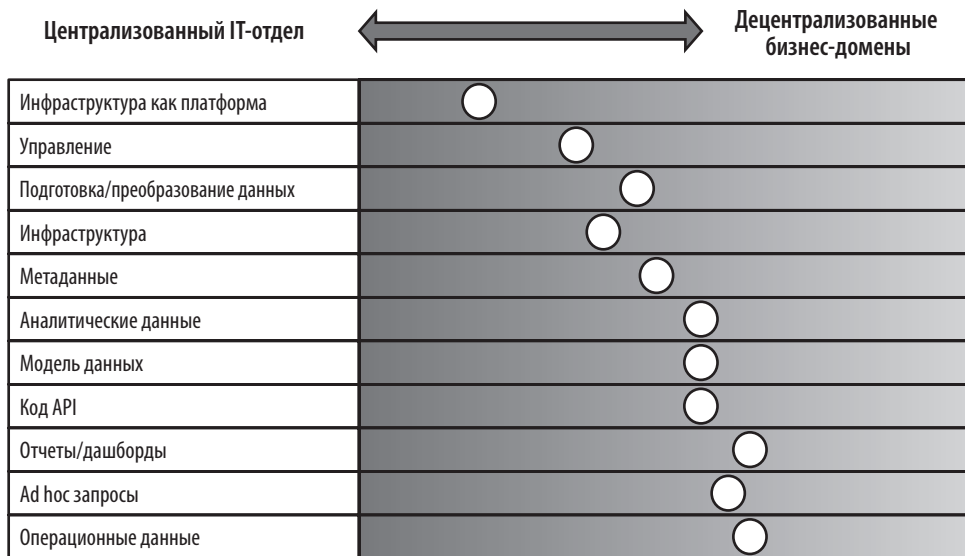


Рис. 13.4. Разграничение ответственности и владения данными между централизованным IT-отделом и децентрализованными бизнес-доменами (вымышленный пример)

На рис. 13.4 слева показан централизованный IT-отдел, а справа — децентрализованные бизнес-домены. Кругик в крайнем левом углу означает, что централизованная

IT-служба выполняет все работы по данному пункту, а кружок в крайнем правом углу — что все работы по данному пункту выполняет каждый децентрализованный бизнес-домен. Кружки между ними показывают границы зоны ответственности в этом спектре.

Если большинство кружков находится в крайнем левом углу, значит, решение настолько централизовано, что называть его сеткой данных, возможно, не совсем уместно. Его лучше классифицировать как корпоративную ткань данных или озеро-хранилище данных.

Итоги

В этой главе был представлен комплексный обзор концепции сетки данных и ее основополагающих принципов. Мы начали со сравнительного анализа централизации и децентрализации, подчеркивая переход от монолитной архитектуры к децентрализованному ландшафту данных, где данные рассматриваются как продукт, сам по себе имеющий ценность, и владеют им кросс-функциональные команды. Затем сформулировали четыре принципа сетки данных Жамак Дегани: доменно-ориентированное децентрализованное владение данными, данные как продукт, самообслуживаемая инфраструктура данных как платформа и федеративное вычислительное управление. Эти концепции подчеркивают важность понимания данных как целостной сущности, которая принадлежит определенным командам домена, а также поддерживается и используется этими командами. Идея состоит в том, чтобы рассматривать каждый блок данных как отдельный продукт с собственным жизненным циклом, управляемый командой, которая лучше всего понимает его ценность и сценарии использования.

После подробного описания основополагающих принципов мы остановились на «настоящей» сетке данных, подчеркнув ее абстрактность. Затем перешли к изучению доменов данных, обратив особое внимание на их роль в структурировании децентрализованной среды. Также были рассмотрены логическая архитектура сетки данных и различные топологии сетки данных и показано их влияние на поток и взаимодействие данных. Сравнив сетку данных с тканью данных, мы выяснили их отличительные особенности и области применения. Наконец, мы рассмотрели сценарии использования сетки данных.

Несмотря на то что концепция сетки данных может кардинально изменить наши взгляды на обработку данных, на пути ее внедрения немало препятствий и трудностей, о которых я подробнее расскажу в следующей главе. В ней мы рассмотрим частые заблуждения, касающиеся сетки данных, а также приведем некоторые соображения, которые помогут вам понять, стоит ли внедрять сетку в своей организации.

Стоит ли внедрять сетку данных?

Мифы, проблемы и перспективы

Сразу скажу, что эта глава о проблемах сетки данных — одна из самых длинных в книге. Не потому, что я считаю сетку данных плохой идеей или что другие архитектуры, которые мы обсуждали, лучше; дело в том, что вокруг нее существует множество мифов, сомнений и проблем, о которых следует знать. Если вы решите заняться созданием сетки данных, я хочу, чтобы вы сделали осознанный выбор, а не просто поддались ажиотажу вокруг нее.

Ниже мы разберем распространенные заблуждения, связанные с сеткой данных, рассмотрим вопросы, которые действительно вызывают беспокойство и часто остаются невысказанными, а также оценим, соответствует ли она вашей организационной структуре, и дадим практические рекомендации по ее успешному внедрению. Наконец, мы рассмотрим потенциальные горизонты развития сетки данных и закончим обсуждением наилучшего сценария использования для каждой из четырех архитектур данных.

Мифы

По мере того как концепция сетки данных становится все более обсуждаемой в технологическом сообществе, возникает целый ряд заблуждений, которые зачастую затуманивают понимание ее реальных масштабов, преимуществ и проблем. В этом разделе я постараюсь развеять подобные мифы, детально разобрав, что на самом деле подразумевает сетка данных и как она вписывается в более широкий ландшафт архитектуры данных.

Миф: использование сетки данных — это универсальное средство, которое поможет решить все проблемы с данными

В реальности все совсем не так. Построение сетки данных занимает гораздо больше времени, чем создание других архитектур, о которых шла речь в этой книге. Дегани совсем не утверждает, что сетка данных — это универсальное решение на все случаи жизни, но из-за раздутого ажиотажа ее идеи часто воспринимаются именно так. Поэтому важно обсуждать сложности и проблемы, связанные с использованием сетки данных.

Миф: сетка данных заменит озеро данных и хранилище данных

Как отмечалось в главе 13, каждый домен в сетке данных по-прежнему нуждается в озере данных и, возможно, в реляционном хранилище данных. Отличие в том, что это больше не «централизованное» озеро данных и реляционное хранилище данных, которыми владеет центральная ИТ-команда. Вместо хранения всех данных для всех доменов в одном озере данных и одном реляционном хранилище каждый домен хранит свои данные отдельно. Таким образом, при использовании сетки данных требуется на самом деле больше озер данных и хранилищ данных, с которыми домены должны работать совместно.

Миф: все проекты хранилищ данных провальные, но сетка данных решит эту проблему

Я часто слышу утверждения, что существующие решения для работы с Big Data не масштабируются, и в качестве решения предлагается использовать сетку данных. С подобным я не согласен. Несмотря на то что сотни тысяч организаций успешно внедрились решения для работы с большими данными, реально используемых сеток данных среди них очень мало (и ни одна из них не является «настоящей» сеткой данных).

За 15 лет работы в сфере хранения данных я видел множество «монолитных» архитектур, которые отлично масштабировали свои технологии и ИТ-команды, поддерживая даже петабайты данных.

Конечно, многие проекты с большими данными оказываются неудачными. Однако чаще всего это происходит по причинам (обычно связанным с человеческим фактором и процессами), которые привели бы к провалу и сетку данных. Доступные технологии централизации данных значительно усовершенствовались и будут продолжать это делать. Современные бессерверные варианты отвечают потребностям большинства организаций в Big Data, позволяя масштабировать решения и экономить средства.

Миф: создание сетки данных означает децентрализацию абсолютно всего

Это, пожалуй, самый распространенный миф. На самом деле даже в случае «настоящей» сетки данных центральная ИТ-команда все равно отвечает за создание инфраструктуры как сервиса (принцип № 3), чтобы ускорить запуск инфраструктуры каждого домена. Центральная ИТ-команда также отвечает за создание и мониторинг политик управления данными (data governance), которым следует каждый домен (принцип № 4). Нет смысла создавать собственную инфраструктуру с нуля для каждого домена, как и собственные политики управления данными, — в итоге вы получите столько типов безопасности и интерпретаций стандартов и правил, сколько существует доменов.

Миф: с помощью виртуализации данных можно создать сетку данных

Я слышал, как многие сторонники виртуализации данных говорят, что если с ее помощью соединить в единую систему все озера и хранилища данных, то получится сетка данных! Но такой подход полностью игнорирует четыре принципа сетки данных (см. главу 13).

Это заблуждение, что можно взять ткань данных одного предприятия (глава 11), применить к ней виртуализацию данных и превратить ее в сетку данных. Вы можете попытаться это сделать, но результат будет далек от сетки данных — это *федерализация данных*, то есть интеграция данных из нескольких разрозненных источников без их физической консолидации или хранения в едином центральном репозитории. Децентрализация — это лишь малая часть того, что представляет собой сетка данных; простое добавление виртуализации не обеспечит соблюдения ни одного из четырех принципов сетки данных!

Например, предположим, что у компании есть корпоративная ткань данных, которая использует виртуализацию данных. Каждый домен генерирует собственные аналитические данные, а ткань данных подключается к ним с помощью ПО для виртуализации данных. Является ли это сеткой данных? Рассмотрим четыре принципа, чтобы выяснить это.

- Принцип № 1 касается владения доменами. Это действительно домены данных или просто отдельные образования внутри компании?
- Принцип № 2 заключается в отношении к данным как к продукту. Есть ли в каждом домене своя команда, которая создает аналитическое решение, используя собственную инфраструктуру этого домена? Следуют ли эти команды контракту данных, который регулирует, каким образом они предоставляют доступ к данным и метаданным для всех остальных? И использует ли эта организация виртуализацию данных для подключения ко всем своим доменам или только к некоторым?

Важно отметить, что если у каждого домена нет собственной команды и инфраструктуры, то специализированная команда инженеров данных остается узким местом в ИТ-инфраструктуре. Аналогично, если какие-то из доменов не используют виртуализацию данных, их данные необходимо собирать и централизовать. Помощь в масштабировании инфраструктуры и организации — два основных преимущества сетки данных, и, если домены не следуют принципу № 2, они не реализуют эти преимущества.

- По поводу принципа № 3 мы можем задать следующий вопрос: создает ли отдельная команда самообслуживаемую инфраструктуру данных как платформу?
- А для принципа № 4 — такой: существует ли отдельная команда, которая занимается федеративным вычислительным управлением?

Как видите, решения для настоящей сетки данных и ткани данных с использованием виртуализации данных заметно различаются.

Проблемы

Вокруг сетки данных поднимают много шума, но при этом очень мало говорят о ее недостатках и проблемах. В этом разделе мы попытаемся всесторонне осветить эти проблемы, сгруппировав их по темам и предложив решения, если таковые существуют. Если, прочитав его, вы будете уверены, что преодолеете все сложности, — отлично! Тогда сетка данных, возможно, для вас. И ни одна из этих проблем не станет для вас сюрпризом — вы будете уже готовы. Я надеюсь, что это знание поможет решить их еще до того, как вы начнете работать с сеткой данных, и значительно повысит ваши шансы на успех.

Хотя этот список ориентирован в основном на сетку данных, многие из указанных в нем проблем встречаются и в других архитектурах, поэтому полезно изучить его целиком. Я на собственном опыте убедился, что знакомство с сеткой данных стимулирует оживленные дискуссии, которые помогают улучшить любую используемую архитектуру.

Философские и концептуальные вопросы

Оригинальные блоги и книга Жамак Дегани о сетке данных задали стандарт, но разные люди интерпретируют ее труды по-разному. Некоторые придумали «исключения» из настоящей сетки данных и другие идеи о том, какой должна быть сетка данных. Итак, если вы хотите построить сетку, какое определение вы будете использовать? На то, чтобы все, кто участвует в создании сетки данных, пришли к консенсусу, могут уйти месяцы. Это особенно сложно, когда разработчики множества продуктов, используемых для создания сетки данных, имеют разные представления о том, что это такое.

Чтобы привести команду к общему пониманию сетки данных, возможно, стоит организовать кросс-функциональный управляющий комитет для выработки основополагающих определений и стандартов. Проведите семинары для заинтересованных лиц и используйте документированные определения, чтобы убедиться, что все вовлеченные сотрудники понимают сетку данных одинаково. Практическую проверку этих определений можно провести на пилотных проектах, что позволит адаптировать их по мере необходимости. Открытая коммуникация и периодические проверки помогут поддерживать достигнутое согласие по мере внедрения сетки данных.

Наличие единого источника истины помогает гарантировать, что организация будет принимать решения на основе достоверных данных, без двусмысленности или разногласий относительно того, какие данные являются верными. Когда все используют один набор информации, работа становится более эффективной и результативной.

Данные в сетке могут считываться из одного домена, а преобразовываться и храниться в другом. Такая динамичная топология может сильно затруднить поддержание

единого источника истины. Фактически Дегани назвала идею единого источника истины «мифом». Я с этим категорически не согласен; я лично видел десятки компаний, внедрявших единый источник истины в разных архитектурах данных, и читал о сотнях других случаев. Его можно внедрить и в рамках сетки данных, если организация установит строгий процесс управления данными и будет использовать централизованный каталог данных (см. главу 6). В сочетании с метриками качества кросс-доменных данных и тесным сотрудничеством между доменами данных единый источник истины не только реален, но и сможет повысить гибкость и масштабируемость сетки.

Внедрение сетки данных сопряжено с огромными рисками, особенно по сравнению с проверенным успехом хранилища данных. Это требует радикальных организационных изменений и совершенно новой архитектуры. Кроме того, сетка данных предполагает, что каждая система-источник может динамически масштабироваться для удовлетворения спроса потребителей. Это может оказаться особенно сложной задачей, когда определенные активы данных становятся «горячими точками» в экосистеме, где наблюдается всплеск запросов или использования.

Объединение данных в децентрализованной среде

Для выполнения запросов и создания отчетов всегда придется объединять данные из нескольких доменов. Иногда доменов могут быть десятки, и они могут находиться во множестве отдельных озер данных, географически удаленных друг от друга. Вам нужно будет тщательно продумать, как преодолеть проблемы производительности, поскольку такие решения, как виртуализация данных (см. главу 6) и домены агрегированных данных, имеют свои недостатки.

Еще одним недостатком наличия автономных доменов является то, что каждый домен концентрируется только на продуктах данных для своих аналитических нужд. Часто владельцы доменов не задумываются над тем, как объединять их данные с данными из других доменов, и мало что делают для того, чтобы упростить объединение своей модели данных с другими. Когда кому-то понадобится объединить данные из нескольких доменов, он может столкнуться с большими трудностями.

Принцип № 4 может помочь определить набор правил моделирования данных для всех доменов, но для этого необходимо, чтобы хоть кто-то разбирался в данных *всех* доменов — это именно та особенность централизованных архитектур, от которой сетка данных пытается уйти. При ориентации доменов в сетке данных на свои внутренние потребности в компании может не оказаться ни одного человека, кто исследовал бы способы извлечения ценности из объединенных данных. Поэтому важно, чтобы такой человек был.

Если вы применяете общую модель данных (CDM, common data model; см. главу 8), то ее необходимо использовать в каждом домене. Вам придется координировать работу, чтобы убедиться, что каждый домен имеет свой набор уникальных

идентификаторов (ID) для записей с одними и теми же типами данных, что и в других доменах (например, клиенты или продукты). В противном случае при объединении данных из нескольких доменов может возникнуть дублирование идентификаторов. Это приведет к ошибкам подсчета и агрегирования и несоответствию данных в запросах и отчетах. Чтобы предотвратить дублирование идентификаторов в рамках CDM в нескольких доменах, можно использовать глобально уникальный идентификатор (GUID) или создать централизованную систему управления идентификаторами для присвоения уникальных ID. В качестве альтернативы можно задать пространство имен, где каждый идентификатор имеет префикс или суффикс с кодом, специфичным для домена.

Вдобавок, по мере того как домены создают свои продукты и модели данных, другие домены и потребители будут использовать их данные и объединять данные из нескольких доменов на основе этих моделей данных. Если домен меняет свою модель данных или интерфейс, зависимости могут привести к проблемам у других доменов и потребителей, использующих его основные модели данных или запросы, объединяющие модели данных домена. Необходимо координировать работу всех доменов при изменении модели данных любого отдельного домена.

Наконец, команды доменов могут иметь разные представления о том, что считать «чистыми» или «стандартизированными» данными. Если домены расходятся во мнениях по таким, например, вопросам, как определять штаты — с помощью аббревиатур или полного названия, объединение данных из нескольких доменов может привести к путанице. А что, если у доменов нет времени на написание кода для тщательной очистки данных? Чтобы решить проблему несоответствия чистоты данных в разных доменах, создайте централизованную структуру управления, которая установит стандартные определения для чистых или стандартизированных данных. Предложите общие шаблоны очистки данных или совместно используемые библиотеки, чтобы сэкономить время и обеспечить единообразие. Управляющий комитет или сотрудник, ответственный за управление данными, помогут обеспечить соблюдение этих стандартов, что позволит без проблем объединять данные из нескольких доменов.

Другие вопросы децентрализации

Домены агрегированных данных и домены, ориентированные на потребителя (см. главу 13), не являются децентрализованными. Чтобы создать домен агрегированных данных, берут данные из нескольких доменов и централизуют их. В случае с доменами агрегированных данных и доменами, ориентированными на потребителя, необходимо решить, кто будет владеть ими, создавать и поддерживать их — проблема, которую призван предотвратить принцип № 1. Создание таких доменов также приводит к дублированию данных. Все это, казалось бы, противоречит духу сетки данных.

Чтобы совместить домен агрегированных данных и домены, ориентированные на потребителя, с принципами сетки данных, рассмотрите возможность совместного

управления и назначьте ответственных за данные для управления этими доменами. Используйте виртуализацию данных, чтобы свести к минимуму дублирование, и создайте уровень абстракции (API). Поддерживайте прозрачность с помощью надежных метаданных и аудиторских следов.

Безопасность также представляет проблему. В централизованной архитектуре за защиту всех данных отвечает одна команда. В сетке данных эта ответственность передается десяткам или даже сотням доменов, что создает гораздо больше «дверей», через которые может проникнуть злоумышленник. Это значительно повышает риск нарушения безопасности.

Необходимо выделить человека для поиска персонально идентифицируемой информации (personally identifiable information, ПИ) и проверять, кто получает к ней доступ, а также специалиста, который будет устранять нарушения, если кто-то видит ПИ, к которой он не должен иметь доступа. Выделенный сотрудник должен отвечать за гарантию достаточного уровня безопасности для всех доменов.

Кроме того, сложность представляет обеспечение последовательности выбора и внедрения технологии в разных доменах. Хотя принцип № 3 (инфраструктура как услуга) и призван защищать от таких ситуаций, но, если у вас десятки доменов, сотрудники в которых имеют разный уровень навыков и опыта, они, скорее всего, выберут разные технологии и настроят свою инфраструктуру и параметры облачного продукта по-разному. Это может серьезно затруднить потребление данных.

Можно ограничить количество технологий, автоматически предоставляемых каждому домену, но это нарушает принцип № 1, поскольку вы ограничиваете свободу каждого домена выбирать технологию для своего сценария использования. Поддерживать много разных технологий может быть сложно, как и создать общий интерфейс для всех доменов. В качестве компромисса можно создать большой каталог одобренных, проверенных технологий, предоставляя доменам возможность выбора, но в рамках единого фреймворка, что обеспечит более простую интеграцию.

Кроме того, хотя принцип № 4 (федеративное управление) предусматривает общие политики сетки данных, каждый домен должен реализовывать их самостоятельно. Это может привести к несогласованным реализациям, что потенциально снижает производительность и повышает затраты.

Сложность

Идеи, лежащие в основе сетки данных, очень сложны. Признаюсь честно, мне пришлось десятки раз перечитать блоги Дегани, чтобы полностью понять эту архитектуру. Даже спустя годы я иногда осознаю, что не совсем правильно понял ту или иную часть концепции. И я такой не один; почти всех, с кем я общаюсь, сетка данных сбивает с толку. Эта сложность в сочетании с отсутствием стандартного определения концепции означает, что большинству команд, которые планируют построить сетку данных, придется предварительно потратить много времени на

то, чтобы убедиться, что все сотрудники ее понимают (и понимают правильно). Поскольку для того, чтобы начать работу, потребуется много времени, удостоверьтесь, что сетка данных будет достаточно полезна, чтобы это стоило затраченных времени и усилий.

Сложность природы сетки данных проявляется и в организационной сложности, хотя сторонники сетки и утверждают обратное. В распределенных командах работает гораздо больше людей, и они не всегда могут эффективно коммуницировать.

Дублирование

Дегани утверждает, что дублирование исходных данных не является проблемой в сетке данных, и пишет, что для каждого домена «весьма вероятно, не понадобится озеро данных, поскольку распределенные журналы и хранилища, в которых содержатся исходные данные, доступны для исследования из различных адресуемых неизменяемых датасетов как продукты»¹. Однако не всегда можно напрямую получить доступ к исходным данным из этих неизменяемых датасетов, поскольку часто возникают проблемы с безопасностью или немедленным извлечением. (У меня до сих пор много клиентов, использующих мейнфреймы, к которым невозможно подключиться.) Поэтому, чтобы что-то сделать с этими исходными данными, вам все равно придется их копировать и сохранить копию в озере данных. На самом деле, в сетке данных может быть *больше* копий данных, чем в других архитектурах, поскольку доменам может потребоваться копировать данные из других доменов для полноты своих продуктов данных либо для создания доменов агрегированных данных или доменов, ориентированных на клиента. Синхронизация всех копий может потребовать дополнительных усилий и затрат.

Точно так же *согласованное измерение*, используемое в схеме «звезда» (см. главу 8), имеет одинаковое значение для каждого факта, с которым оно связано. Согласованные измерения позволяют категоризировать и описывать факты и измерения (например, измерение даты или клиента) одним и тем же способом, обеспечивая согласованную отчетность в рамках всего предприятия. Согласованные измерения очень популярны в решениях со схемой «звезда». В сетке данных эти согласованные измерения должны быть дублированы в каждом домене, имеющем связанные таблицы фактов. Это большое количество дублированных данных, которые необходимо синхронизировать.

Скорее всего, ваша компания приобретает сторонние данные, такие как данные о погоде, акциях или конкурентах. В централизованной архитектуре этим занимается один человек или группа, поэтому вероятность того, что две команды в одной компании приобретут одни и те же данные, очень мала. Однако в сетке данных каждый домен покупает собственные данные. Как не допустить, чтобы два домена

¹ Дегани Ж., How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh (<https://oreil.ly/cv2sX>), сайт MartinFowler.com, 20 мая 2019 г.

тратили деньги впустую, покупая одни и те же датасеты? Ни один из принципов сетки данных, похоже, не охватывает эту область, но можно расширить принцип № 4, создав централизованную команду, которая будет обрабатывать все потребности в сторонних данных.

Невзирая на принцип № 3, некоторые домены, скорее всего, будут создавать похожие платформы для приема данных или аналитики. При централизованном подходе платформу создает только одна команда, а в сетке данных это могут делать десятки доменов. Поэтому, чтобы домены не затрачивали лишние усилия, дублируя работу друг друга, и не упускали полезные решения, созданные другими доменами, имеет смысл способствовать обмену этими решениями.

Возможность реализации

Переход на сетку данных — это масштабное мероприятие, требующее значительных организационных и технических изменений, гораздо больших, чем при использовании других архитектур данных. Он подразумевает намного более высокие затраты и более длительные сроки реализации — многие месяцы, возможно, даже годы, до момента запуска в продакшен. Нужно приложить немало усилий, чтобы домены начали использовать созданные ими самими отчеты и дашборды настолько хорошо, чтобы поддержка центрального IT-отдела больше не требовалась. В то же время, если ваша компания не является совершенно новой, вам придется поддерживать и работу существующей платформы. Это может быть довольно сложно, особенно по мере того как новые домены переходят от предоставления операционных данных в центральный IT-отдел к хранению их в пределах своего домена.

Отметим еще раз: сетка данных — это концепция, а не технология. Речь идет об организационных и культурных изменениях в компании. В книге Дегани не говорится о том, как использовать существующие технологии для создания сетки данных. Вероятно, это связано с тем, что в доступных технологиях имеются серьезные пробелы в таких областях, как квант (блок) данных, автоматизация, совместное использование продуктов, создание доменной инфраструктуры и совместное управление данными.

Принцип № 2 требует наличия технологии для публикации и совместного использования доменов, однако современные технологии с этой точки зрения находятся в зачаточном состоянии. Принцип № 3 требует очень сложных решений для автоматизации выделения инфраструктуры, но существующие решения в лучшем случае выполняют лишь небольшую часть этой задачи. А создание и управление глобальными правилами из принципа № 4 — притом что домены создают свои правила и управляют ими — означает, что для каждого продукта данных необходимо во многом полагаться на автоматизацию и решения по написанию кода для управления жизненным циклом продуктов данных, на стандарты как код, политики как код, автоматизированные тесты, автоматизированный мониторинг и на встроенное выполнение политик. Для этого требуются очень сложные программные

инструменты, которые пока очень ограничены, если вообще существуют. Компаниям приходится создавать их самостоятельно или просто пропускать этот шаг, тем самым нарушая принципы сетки данных.

Дегани описывает продукты данных как имеющие порты для вывода данных (интерфейсы) с четко определенными контрактами и API. Эти API будут обслуживать многомодельный доступ, битемпоральные данные и неизменяемый доступ только для чтения, а также смогут передавать огромные объемы данных и масштабироваться на огромное количество запросов. Кроме того, эти API будут охватывать одну или несколько физических инфраструктур, несколько облаков и локальных сред размещения. Пока что технологии для API, поддерживающей все это, не существует. И, скорее всего, никогда не будет существовать.

Кроме того, API редко используются для работы с данными; взаимодействие с данными обычно оставляют за SQL-запросами. Создание API для всех взаимодействий с данными — это большой сдвиг в культуре, который займет гораздо больше времени, чем просто использование SQL-запросов.

В своей книге Дегани представляет новую единицу логической архитектуры, называемую *квантом данных*. Квант данных контролирует и инкапсулирует все структурные компоненты, необходимые для автономного обмена данными в виде продукта данных, который могут использовать другие пользователи. Сюда входят данные, метаданные, код (вычисления), политика и инфраструктурные зависимости. Данные и вычисления логически объединены в единое целое, но базовые физические инфраструктуры, где размещаются код и данные, остаются отдельными, так как в настоящее время для этого не существует технологии. Хотя Дегани и основала компанию по созданию кванта данных с использованием контейнеров продуктов данных, могут пройти годы, прежде чем ее компания или любая другая создаст компоненты для решения большинства этих задач.

Я считаю, что квант данных — очень рискованная концепция. Вы, конечно, можете построить сетку данных и без этого. Подумайте, как проследить происхождение данных при использовании концепции кванта данных. Если метаданные каждого домена доступны только в самом продукте данных, чтобы увидеть его метаданные, происхождение или информацию о качестве данных, вам придется выполнять сложные федеративные запросы ко *всем* продуктам данных. Можно реплицировать все метаданные в центральное местоположение, но это потребует постоянной синхронизации и значительно усложнит общую архитектуру. Кроме того, возникает вопрос безопасности: придется обновлять метаданные, такие как метки чувствительности, классификации конфиденциальности и информация о владельце, для данных, которые физически находятся в нескольких местах.

Проблемы с копированием данных распространяются и на продукты данных. Если они копируются, то копируются и их метаданные, что приводит к появлению нескольких копий одних и тех же метаданных, которые необходимо синхронизировать. Любые обновления метаданных должны выполняться одновременно

в нескольких контейнерах продуктов данных, что требует наличия постоянного доступа к этим контейнерам. Вы рискуете получить разных владельцев для одних и тех же метаданных.

Наконец, одновременный запуск сотен или даже тысяч небольших продуктов данных значительно повышает сложность, увеличивает использование ресурсов инфраструктуры и компьютерных сетей, усложняет управление конфигурацией и создает проблемы с производительностью и согласованностью ссылочных данных.

Дегани считает, что, поскольку еще не созданы технологии, изначально ориентированные на сетку данных, «можно использовать технологии хранилищ данных в качестве базового хранилища и режима доступа SQL, но при этом выделять отдельную схему для каждого продукта данных», тем самым разделяя вычисления. Но это приводит к централизации инфраструктуры и, как следствие, нарушению принципа № 1 и утрате главного преимущества сетки данных: возможности технического масштабирования.

Люди

При внедрении сетки данных понадобится найти и нанять квалифицированных инженеров и других сотрудников для каждого домена. В целом достаточно сложно подыскать хороших специалистов даже для центральной IT-команды, а в сетке данных каждый домен должен не только научиться создавать инфраструктуру и аналитическую платформу, но и сформировать свою собственную мини IT-команду. Если у вас много доменов, то вам понадобится намного больше людей, чем при централизованной архитектуре, — это огромный недостаток горизонтального масштабирования организации.

Принцип № 3 призван устранить потребность в глубоких технических навыках и вместо них использовать более продвинутые кросс-функциональные навыки. Это часто побуждает организации привлекать к внедрению в каждой продуктовой группе сотрудников из бизнес-доменов, а не специалистов с большим техническим опытом, тем более что найти достаточное количество высококвалифицированных специалистов с соответствующими техническими знаниями достаточно сложно. А неопытные сотрудники, как правило, создают дорогие, сложные и плохо работающие продукты данных.

Сетка данных предполагает, что инженеры данных центральной IT-команды не обладают знаниями в области бизнеса и конкретного домена, поэтому лучше использовать тех, кто знаком с доменами, которым принадлежат данные. Кроме того, предполагается, что каждая команда домена обладает необходимыми навыками для создания надежных продуктов данных или может эти навыки приобрести.

Это не всегда так. Удивительно, но я видел специалистов центральной IT-команды, знающих домен лучше, чем эксперты домена! Кроме того, IT-специалисты понимают, как объединять данные из разных доменов. Иногда люди, управляющие

действующими системами, очень слабо разбираются в своих данных, особенно с точки зрения аналитики; они просто стараются поддерживать работу системы. Если инженеры данных центральной IT-команды не обладают необходимыми знаниями о домене, почему бы не дать им возможность получить эти знания и не улучшить взаимодействие между IT-отделом и доменами? Это может оказаться более эффективным, чем внедрять совершенно новые методы работы.

Сторонники сетки данных хвастливо утверждают, что вместо специалистов по работе с данными, обладающих глубокими знаниями только в одной области, сетка данных воспитывает *универсалов*. Никто не работает над одним изолированным шагом. Сетка данных предполагает создание кросс-функциональных команд, состоящих из специалистов во многих областях: например, потоковой обработке данных, пакетной обработке ETL, проектировании хранилищ данных, визуализации данных, а также в таких практиках программной разработки, как кодирование, тестирование и непрерывная доставка (DataOps). Эдакий «мастер во всем, но не специалист ни в чем», умеет все, но по чуть-чуть. Опять же, если у вас много доменов, это означает много позиций, которые нужно заполнить, и зарплат, которые нужно платить. Нелегко найти людей с таким широким спектром навыков, не говоря уже о том, что их требуется много. Вопрос в том, будут ли эти универсальные специалисты лучше, чем люди с многолетним глубоким опытом в одной области.

Препятствия на уровне доменов

Представим, что ваш домен наконец получил разрешение на то, чтобы центральный IT-отдел добавил крайне необходимые вам функции в аналитику домена и вы могли извлечь реальную пользу из своих данных. Вы с центральным IT-отделом разработали план создания аналитики на следующие несколько месяцев. Затем вы узнаете, что ваша компания будет создавать сетку данных и вам предстоит набрать совершенно новую команду для создания этих необходимых функций и API для них. А после этого вас еще просят отложить проект на несколько месяцев, пока компания будет реализовывать принципы № 3 и № 4, проектировать домены данных и составлять контракт, согласно которому будет осуществляться модернизация каждого домена.

Как бы вы себя чувствовали? А теперь представьте, что люди еще в десятках доменов думают так же. Компания не может иметь домен-изгой; потребители, а также другие домены зависят от доступности данных из каждого домена, поэтому очень важно, чтобы каждый домен участвовал в проекте сетки данных. Для этого вам потребуется следующее:

- предоставить убедительные стимулирующие механизмы, чтобы компенсировать дополнительную работу, неудобства и отсрочки. Недостаточно сказать людям, что они делают это «ради всеобщего блага»;
- заручиться поддержкой идеи внедрения сетки данных для всех доменов;

- добиться одобрения со стороны руководителей высшего звена;
- донести четкую концепцию, демонстрирующую конкретные выгоды для каждого домена;
- запустить пилотные программы, чтобы продемонстрировать эффективность нового подхода;
- предложить материальные и нематериальные стимулы для поощрения участия;
- наладить прозрачные каналы для постоянного общения;
- предложить поддержку через центры компетенций или центры передового опыта.

Заранее продумайте, что вы будете делать, если какой-либо домен не захочет следовать правилам. Это может быть особенно проблематичным для безопасности данных и стандартов качества данных. Если вы решите, что домены, которые не следуют правилам, не могут присоединиться к сетке данных, у вас появится обособленное (локальное) хранилище данных.

Также может оказаться, что какие-то домены *не могут* присоединиться к сетке данных, даже если очень захотят. Возможно, у них нет бюджета на формирование команды для создания аналитической платформы, или обстоятельства вынуждают их продолжать использовать старые технологии, которые не могут работать с аналитической платформой. Чтобы включить их данные в сетку, может понадобиться централизованная архитектура, помимо сетки данных.

Эффективный обмен продуктами имеет решающее значение для успеха сетки данных. Обмен продуктами внутри и вне каждого домена в сетке сильно отличается от обмена данными. Вам понадобится способ, с помощью которого другие домены или потребители смогут отслеживать все домены вашей компании и их продукты данных. Для этого потребуется, чтобы все метаданные домена были где-то каталогизированы с организацией механизма поиска и способа запроса доступа: своего рода маркетплейс данных, как описано в главе 6.

Основные провайдеры облачных услуг предлагают множество программных решений для обмена данными, но очень мало — для обмена продуктами данных (и все выпущены недавно, непосредственно перед написанием этой книги). Вам придется решить: покупать ли одно из таких решений или создать свое собственное?

Результатом внедрения сетки данных является переход к модели самообслуживания в запросах данных, что создает определенные трудности. Традиционно домены полагались на центральные IT-отделы для удовлетворения всех своих потребностей в аналитических данных, от управления данными до специализированных запросов и отчетов. При использовании сетки данных эта ответственность децентрализуется и возлагается на отдельные домены. Потенциальный риск в том, что некоторым доменам может не хватить опыта сотрудников для эффективного

управления этими задачами. Кроме того, они не могут просто копировать решения, реализованные центральным IT-отделом, а должны модифицировать их, чтобы стала заметна польза сетки данных. И существует вероятность того, что эти вновь созданные уникальные решения могут оказаться не столь эффективными и надежными, как централизованные системы, которые они заменяют.

Организационная оценка: стоит ли внедрять сетку данных?

Теперь у вас есть понимание, что такое сетка данных и какие потенциальные проблемы с ней связаны. Если вы все еще считаете, что сетка данных вам подходит, следующий шаг – тщательная оценка *готовности* организации к ее успешному внедрению. Для оценки можно воспользоваться критериями, приведенными в таблице 14.1, которые взяты из книги Дегани. Дегани рекомендует компаниям рассматривать создание сетки данных, только если они частично или в значительной степени соответствуют всем этим критериям.

Таблица 14.1. Оценка готовности компании к внедрению сетки данных

Критерий	Оценка (выберите одно)		
Организационная сложность слишком велика для текущей системы, а существующая архитектура данных стала тормозом; она не способна масштабироваться достаточно быстро, чтобы извлекать ценность из данных в большинстве секторов бизнеса	Не применимо	Частично применимо	В значительной степени применимо
В компании есть стратегия, ориентированная на данные, а также стратегическое видение использования данных и извлечения из них ценности	Не применимо	Частично применимо	В значительной степени применимо
Руководители поддерживают изменения в организационной культуре и методах работы, необходимые для создания сетки данных, и стремятся мотивировать всех сотрудников на внедрение этих изменений	Не применимо	Частично применимо	В значительной степени применимо
Данные рассматриваются как ценный актив, регулярно собираются, анализируются и используются для принятия бизнес-решений. Работа, процессы принятия решений, организационная культура и стратегии строятся на основе данных. Значительные средства вкладываются в инфраструктуру технологий обработки данных, аналитические платформы и машинное обучение. Сотрудники достаточно грамотны в вопросах работы с данными	Не применимо	Частично применимо	В значительной степени применимо
Компания любит быть в авангарде отрасли и всегда одной из первых принимает и внедряет новые технологии и инновации, даже если это сопряжено с определенным риском	Не применимо	Частично применимо	В значительной степени применимо

Критерий	Оценка (выберите одно)		
Компания использует современные практики и технологии инженерии данных, включая принятие решений на основе данных и проектирование с ориентацией на пользователя, что позволяет создать множество небольших команд вместо одной большой централизованной команды	Не применимо	Частично применимо	В значительной степени применимо
Организация структурирована по конкретным бизнес-областям или доменам данных, в каждом из которых есть специальные технические команды, которые создают и поддерживают продукты данных, обслуживающие данный домен. У каждого домена есть (или скоро появится) бюджет на создание аналитических данных на основе своих оперативных данных и на создание собственной инфраструктуры сетки данных	Не применимо	Частично применимо	В значительной степени применимо
Даже если это занимает много времени, компания стремится использовать машинное обучение и аналитику для получения конкурентных преимуществ над конкурентами. Компания привержена процессу изменения корпоративной культуры и стремится к разработке и созданию технологий, необходимых для внедрения сетки данных, а также к формированию сети домен-ориентированных команд	Не применимо	Частично применимо	В значительной степени применимо

Основываясь на собственном опыте, скажу, что в реальности лишь крошечный процент компаний, возможно только 1 %, находит, что *абсолютно все* эти критерии частично или в значительной степени применимы к ним. Особенно редко встречаются высокие оценки по критериям «один из первых, кто внедряет», «ориентация на домены» и «современная инженерия данных».

Рекомендации по внедрению успешно работающей сетки данных

Если у вас есть решение для работы с данными и вы решили внедрить сетку данных, я рекомендую использовать модель типа «ступица и спицы» (hub-and-spoke), в которой сетка данных создается в качестве расширения централизованного решения для работы с данными (рис. 14.1). Можно начать с использования новых данных для создания новых доменов сетки данных, но дополнить эти домены текущим централизованным решением для данных в соответствии с потребностями бизнеса. Переносите централизованные данные в домены сетки данных постепенно, по мере добавления новых доменов из новых данных — не пытайтесь преобразовать все централизованные данные в домены сетки данных одним махом.

Постепенное внедрение сетки данных поможет управлять рисками, связанными с полной перестройкой существующей архитектуры данных. Если в одной части

сетки произойдет сбой, он будет локализован и не повлияет на всю систему. Вы получаете больший контроль над соблюдением нормативных требований и мерами безопасности, а также время на формирование надлежащего управления и протоколов безопасности. Такой постепенный подход также позволяет равномерно распределить затраты и нагрузку на персонал. Существующие бизнес-процессы, которые зависят от централизованных данных, могут продолжать бесперебойно функционировать во время перехода, сводя к минимуму перебои в работе бизнеса.

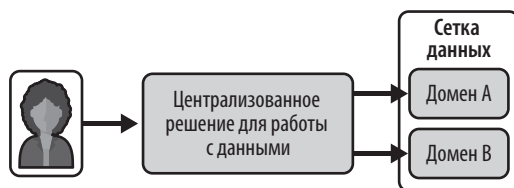


Рис. 14.1. Модель типа «ступица и спицы» (hub-and-spoke), в которой сетка данных дополняет и расширяет централизованное решение для работы с данными

Я предлагаю итеративный подход. Постепенное внедрение новых доменов позволит извлекать уроки на каждом этапе, вносить необходимые коррективы и применять эти знания для последующих доменов, создавая более надежную систему в долгосрочной перспективе. Кроме того, такой подход позволяет использовать существующие инвестиции в технологии при переходе к более современной архитектуре.

Вдобавок он дает возможность гибко устанавливать приоритеты для определенных доменов, которые приносят большую немедленную ценность или лучше соответствуют стратегическим бизнес-инициативам. Это позволяет согласовывать переход с бизнес-потребностями, а не форсировать все сразу. Вы также получаете возможность оптимизировать производительность инфраструктуры каждого домена и можете адаптировать ее к конкретным потребностям и ограничениям этого домена.

И наконец, с помощью подхода, основанного на модели типа «ступица и спицы», можно со временем произвести культурный сдвиг, способствуя кросс-функциональному и доменно-ориентированному сотрудничеству между владельцами продуктов данных, инженерами данных и бизнес-стейкхолдерами.

Поэтапное внедрение сетки данных — это стратегия, которая позволяет тщательно планировать, управлять рисками и ориентироваться на бизнес-цели. Она поддерживает постепенный переход от централизованной архитектуры данных к децентрализованной с использованием как старых, так и новых систем в течение переходного периода. Это более адаптивный и устойчивый подход, чем стремление сразу осуществить полную миграцию, что может быть сопряжено с рисками и сложностями.

Перспективы сетки данных

Я общался со многими людьми, которые считают, что идея централизованной команды, создающей всю архитектуру данных, уходит в прошлое и что со временем все компании перейдут на децентрализованную модель. В конце концов, централизованные модели плохо масштабируются.

Это обсуждалось еще во времена дебатов между Кимбаллом и Инмоном (см. главу 8). В лагере Инмона IT-специалисты централизовали все, а в лагере Кимбалла придерживались методологии создания слабо централизованных витрин данных по предметным областям и объединения их с помощью согласованных измерений в архитектуре шины. Однако на практике это вело к появлению большого количества доменов, создающих собственные витрины данных изолированно, иногда с использованием разных технологий и, как следствие, к децентрализованной архитектуре. Когда кому-то требовалось объединить и агрегировать данные из нескольких доменов, они создавали дополнительные витрины данных и/или OLAP-кубы.

Иногда эта стратегия показывала хорошие результаты, но зачастую она приводила к двойной работе с использованием всевозможных технологий и продуктов. Она также была сопряжена с многочисленными сложностями и обычно с бесконтрольными затратами. Чтобы избежать этих проблем, компания в конце концов возвращалась к централизованной модели.

Приведет ли сетка данных к тому же круговороту проблем? Что может измениться на сей раз? Хотя со времен витрин данных технологии и шагнули далеко вперед, дело не в них — а в людях и процессах, и мы не так хорошо продвинулись в работе с ними. Вы можете придумывать новые крутые словечки и методы, которые вам нравятся, но очень трудно заставить людей измениться, а сетка данных требует массы изменений.

Я предполагаю, что очень немногие компании перейдут на архитектуру сетки данных, и не верю, что хотя бы одно из этих решений будет «настоящей» сеткой данных. Считаю, что из тех, кто назовет свое решение сеткой данных, большинство — возможно, 90 % — внедрят доменное владение (принцип № 1), и меньшее количество — примерно 70 % — введут данные как продукт (принцип № 2). Возможно, очень небольшой процент из них дойдет до уровня кванта данных. Около трети компаний будут использовать ту или иную форму автоматизированной инфраструктуры для каждого домена (принцип № 3: самообслуживаемая инфраструктура данных как платформа), и, возможно, половина примет федеративное вычислительное управление (принцип № 4). Я думаю, что очень большой процент решений будет накладывать ограничения на технологии, которые может использовать каждый домен.

Вместо того чтобы пытаться внедрить каждый элемент сетки данных, я рекомендую сосредоточиться на том, как помочь своим командам по работе с данными

быстрее и чаще предоставлять бизнес-ценность клиентам. Исходя из этого, вы можете пойти в обратном направлении, определить и внедрить конкретные элементы сетки данных, которые помогут достичь этой цели.

В конечном счете сетка данных — поле битвы между централизацией и децентрализацией. Любое решение сетки данных всегда будет подразумевать и то и другое. Главное — найти правильный баланс для вашего сценария использования.

Общая картина: представление об архитектурах данных и их применении

В современном мире стремительного технологического прогресса компании находятся на перепутье, окруженные обилием данных. Эти данные, если их правильно использовать, могут открыть беспрецедентные возможности, оптимизировать работу и обеспечить конкурентное преимущество. Ключ к раскрытию этого потенциала лежит в архитектуре — структуре, определяющей, как данные будут храниться, обрабатываться и как к ним будет осуществляться доступ.

На протяжении всей книги мы подробно разбирались в тонкостях разных архитектур данных, их эволюции и практическом применении. Теперь, когда эта глава подходит к концу, очень важно сделать шаг назад и взглянуть на открывающийся ландшафт с высоты птичьего полета.

Перед вами множество вариантов. Как понять, какой из них подходит именно вашей организации? Чтобы ответить на этот вопрос, рассмотрим некоторые высокоуровневые сценарии использования для каждой архитектуры, сгруппированные по стоимости и сложности.

Современное хранилище данных

Хранилища MDW больше всего подходят для предприятий, которые работают с небольшим объемом данных и уже привыкли к реляционным хранилищам данных, что делает переход на новое решение плавным.

Ткань данных

По мере роста и диверсификации компании часто сталкиваются с проблемой работы с данными из множества источников, каждый из которых отличается по объему данных, скорости доступа к ним и их типу. Ткань данных — это подходящая архитектура для тех, кому нужен бесшовный способ интеграции этих разнообразных потоков данных.

Озеро-хранилище данных

Это решение можно представить как промежуточное. Используйте озеро-хранилище данных в качестве основного решения до тех пор, пока не столкнетесь с его

ограничениями. Когда это произойдет, вы сможете перенести определенные наборы данных в реляционное хранилище, чтобы оптимально удовлетворить потребности организации.

Сетка данных

Разработанная специально для очень крупных доменно-ориентированных компаний, сетка данных предназначена для организаций, которые сталкиваются с серьезными проблемами масштабируемости и имеют ресурсы и время для инвестиций в надежное решение.

Эти архитектуры не исключают друг друга. В реальных условиях очень редко встречаются архитектуры той или иной модели в чистом виде. Большинство организаций приходят к выводу, что их уникальные потребности требуют функций нескольких архитектур. Такое индивидуальное решение может сочетать в себе признаки одной или сразу всех этих архитектур, адаптированные к конкретным сценариям использования данных и развивающимся возможностям бизнеса.

Итоги

В этой главе мы рассмотрели распространенные мифы о сетке данных, прояснили связанные с ней реальные проблемы и оценили ее применимость в разных организационных структурах. Мы дали ключевые рекомендации по успешному внедрению сетки данных, основанные на реальных практиках и конкретных примерах. Изучили перспективы сетки данных в контексте развивающихся технологий и тенденций бизнеса. Наконец, обобщили все архитектуры данных, чтобы вам было легче выбрать наиболее подходящую архитектуру данных для своих конкретных бизнес-потребностей. Надеюсь, теперь у вас есть как теоретическое понимание, так и практические знания, которые позволят вам принимать обоснованные решения о внедрении сетки данных.

Перспективы бизнеса неразрывно связаны с будущим данных. С течением времени выбранные архитектуры будут определять не только способы хранения и доступа к данным, но и то, как будет развиваться, внедрять инновации и конкурировать весь бизнес. Очень важно подходить к этому выбору с глубоким пониманием, готовностью к изменениям и видением будущего. Помните, что архитектура, которую вы выберете сегодня, проложит путь вашему бизнесу завтра. Выбирайте с умом, и пусть данные ведут вас вперед.

В предыдущих главах рассматривалась технологическая сторона архитектур данных. В главах 15 и 16 мы затронем такие важные темы, как люди и процессы, изучим внутреннюю структуру команды, причины неудач проектов и факторы, способствующие их успеху.

Люди, процессы и технологии

Поздравляю! Вы изучили основные разделы, в которых речь шла об общих концепциях архитектуры данных и четырех архитектурах данных. Но мы еще не закончили. Глава 15 посвящена самым важным факторам, определяющим успех любого создаваемого решения: людям и процессам. Глава 16 охватывает вопросы технологий архитектур данных. Чтобы сохранить актуальность на долгие годы, обсуждение затрагивает не зависящие от поставщиков услуг исторические и высокоуровневые аспекты.

Люди и процессы

В сложном мире архитектур данных, когда кажется, что все внимание уделено технологиям и инструментам, люди и процессы остаются его сердцем и душой. По мере того как организации переходят от традиционных моделей данных к инновационным структурам, таким как современное хранилище данных, ткань данных или озеро-хранилище, и даже, более того, к децентрализованным парадигмам, таким как сетка данных, человеческий фактор остается ключевым. Почему?

За многие годы работы над созданием хранилищ данных, приложений и инструментов для составления отчетов я видел, что помогает проектам добиваться успеха, а что приводит к их провалу. Представьте, что вы строите чудо архитектуры — современнейший небоскреб. Чертеж безупречен, материалы первоклассные. Однако, если строители, инженеры и менеджеры не совпадают в своем видении результата или если отсутствуют процессы, обеспечивающие слаженность действий, проект может рассыпаться, даже не начавшись.

Точно так же трансформация архитектуры данных требует не только технологических сдвигов, но и глубоких перемен в организационной культуре. Изменение методов управления данными, их обработки и оценки означает перестройку мышления команд и их взаимодействия с данными. Надо бросить вызов устоявшимся нормам и принять новые методологии. Такие переходы никогда не бывают просто техническими — они неразрывно связаны с человеческим фактором.

Центральное место в работе с людьми отводится распределению ролей. Кто чем занимается? Кто несет ответственность за обеспечение бесперебойности потока данных, их правильную обработку, этичность и эффективность использования? Неправильное понимание или недооценка этих ролей — и вы окажетесь на волоске от неэффективности или, что еще хуже, на грани провала проекта. И наоборот, грамотное распределение ролей — залог того, что каждый член команды понимает свои обязанности и имеет ресурсы для их выполнения — сродни закладке прочных камней в фундамент архитектурного шедевра.

Как и любое другое крупное начинание, проекты по работе с данными имеют свои истории успеха и поучительные примеры. Результат — триумфальный переход на новую архитектуру данных или провальное решение — часто можно определить по тому, как осуществлялось управление людьми и как были организованы процессы. Изучение как успешных, так и неудачных примеров дает бесценные знания

о том, как справиться с проблемами и сложностями трансформации архитектуры данных.

В этой главе мы рассмотрим тонкости внутренней организации команды, особенности ролей в разных структурах данных, а также критические факторы, которые определяют успех и неудачу проектов работы с данными. В конце нашего пути вы поймете, что, хотя инструменты и технологии активно развиваются, именно люди и процессы определяют, будут ли эти инструменты использованы для достижения успеха или останутся в списке упущенных возможностей.

Организация команды: роли и обязанности

Для создания архитектуры данных требуется команда (или несколько команд) с разнообразным набором знаний и ролей. В этом разделе описаны некоторые ключевые роли, соответствующие им обязанности и способы их взаимодействия при создании современного хранилища данных, ткани данных или озера-хранилища данных. (Для создания сетки данных понадобятся дополнительные роли, о которых я расскажу позже.) В небольших компаниях один человек может выполнять несколько ролей; например, быть одновременно и менеджером по управлению данными, и менеджером по качеству данных. В крупных компаниях одну роль могут выполнять несколько человек — например, инженеров по работе с данными может быть целый десяток.

Роли для работы с современным хранилищем данных, тканью данных или озером-хранилищем данных

Сначала рассмотрим основные роли.

Архитектор данных

Разрабатывает высокоуровневую структуру архитектуры данных (современное хранилище, ткань данных или озеро-хранилище данных) и решает, какие технологии и политики управления данными следует использовать в проекте.

Инженер данных

Отвечает за создание, тестирование и поддержку архитектуры данных. Пишет сценарии для извлечения, загрузки и преобразования данных (ELT) из разных источников в выбранное решение для работы с данными и тесно сотрудничает с архитектором данных для реализации разработанной архитектуры.

Управляющий данными (data steward)

Отвечает за качество данных и управление ими. Определяет и внедряет правила использования данных и стандарты, контролирует очистку данных, управляет правами доступа к данным и работает с пользователями для решения любых проблем, связанных с качеством данных или доступом к ним.

Администратор базы данных (DBA)

Отвечает за работу, обслуживание и производительность системы данных. Реализует планы резервного копирования и восстановления, оптимизирует запросы для повышения производительности и обеспечивает бесперебойную работу системы.

Аналитик данных

Собирает и обрабатывает большие массивы данных, а также проводит их статистический описательный и диагностический анализ, выявляя закономерности и тенденции, преобразуя сложные результаты в понятные выводы и доводя их до сведения заинтересованных лиц. Создает подробные отчеты и дашборды для отслеживания бизнес-метрик.

Специалист data science

Использует передовые статистические методы, прогнозное моделирование и машинное обучение для извлечения ценной информации из данных и решения сложных задач. Разрабатывает и внедряет ML-модели, выполняет предиктивный и предписывающий анализ, проводит A/B-тесты и занимается майнингом данных. Кроме того, вносит основанные на данных предложения и рекомендации, влияющие на принятие стратегических и оперативных решений.

Бизнес-аналитик

Осуществляет коммуникацию между командой по работе с данными и бизнес-пользователями и помогает определить бизнес-требования к архитектуре данных, а затем преобразует их в технические требования. Он также проверяет выводы аналитиков данных и специалистов data science и доводит их до сведения бизнес-пользователей.

Менеджер проекта / скрам-мастер (scrum master)

Координирует работу команды, следит за выполнением поставленных задач и устраняет любые затруднения. Кроме того, в его обязанности входит информирование стейкхолдеров о статусе проекта, а также управление любыми рисками.

Специалист по защите данных

Обеспечивает соблюдение законов о персональных данных, совместно с управляющим данными разрабатывает политики использования данных и реагирует на любые инциденты, связанные с конфиденциальностью данных. В некоторых отраслях или регионах эта роль является обязательной в соответствии с нормативными требованиями.

Менеджер по управлению данными (data governance manager)

Обеспечивает доступность, удобство использования, целостность и безопасность данных организации, а также создает и внедряет политики и процедуры

управления данными для передачи передового опыта во все бизнес-подразделения.

Менеджер по качеству данных

Контролирует полноту, согласованность и точность данных, включая внедрение правил обеспечения качества данных, мониторинг метрик качества данных и координацию действий с другими командами для устранения любых проблем с качеством данных.

Роли при реализации сетки данных

Как было отмечено в главах 13 и 14, подход, основанный на сетке данных, рассматривает данные как общий актив, распределенный по доменам организации. В рамках модели с сеткой центральная команда по работе с данными не занимается всеми вопросами работы с данными — обязанности распределяются между кросс-функциональными командами доменов. Каждая команда отвечает за данные, которые она производит, включая предоставление данных как продукта, обеспечение их качества и предоставление доступа к ним другим сторонам. Это довольно сильно меняет состав требуемых команд, роли и обязанности. Разберем эти команды.

Команды доменов

В каждом домене есть одна команда домена. Ее члены отвечают за создание аналитического решения для своего домена и реализацию политик управления и принципов сетки данных (описанных в главе 13). Большинство ролей в команде домена, например инженеры данных домена, аналитики данных и управляющие данными домена, аналогичны ролям, перечисленным выше для современного хранилища данных, ткани данных или озера-хранилища данных, за исключением того, что их сферой ответственности является конкретный домен, а не вся организация. Поскольку каждый домен имеет свое собственное решение, большинство, если не все, из этих ролей присутствуют в каждом домене. Еще одна роль, характерная только для сетки данных, — это владелец продукта данных.

Владелец продукта данных берет на себя ответственность за данные, которые производит его домен. Эти обязанности включают в себя управление жизненным циклом данных, обеспечение их качества, доступности и соответствия потребностям бизнеса. Владелец продукта должен понимать все параметры данных — как технические, так и относящиеся к бизнесу.

Команда самообслуживаемой платформы инфраструктуры данных

Команда платформы, ориентированная на самообслуживаемую инфраструктуру данных, в соответствии с третьим принципом сетки данных предоставляет доменным командам технологическую инфраструктуру, инструменты и рекомендации для создания и управления их продуктами данных. Роли в этой команде:

Архитектор сетки данных

Разрабатывает общую архитектуру и руководящие принципы, координирует усилия доменных команд, а также способствует их коммуникации и обучению. Хотя каждая команда домена управляет своими данными, роль архитектора заключается в обеспечении общей согласованности всех элементов сетки данных.

Архитектор платформы

Разрабатывает общую архитектуру платформы данных для поддержки потребностей доменов и обеспечения возможности работы с разными типами данных, приложениями и инструментами аналитики.

Инженеры/разработчики платформы

Создают и поддерживают инфраструктуру платформы, включая базы данных, API, оркестрацию контейнеров и др.

Инженер данных

Работает над конвейерами, которые перемещают и преобразуют данные в рамках платформы, обеспечивая чистоту, согласованность и доступность данных для всех, кому они нужны.

Инженер DevOps

Автоматизирует развертывание, масштабирование и управление платформой, обеспечивая стабильность и надежность ее работы, а также способность удовлетворять потребности команд, работающих с данными.

Инженер DataOps

Управляет жизненным циклом и моделями data science и оптимизирует их целиком и полностью, в том числе контролируя качество данных и обеспечивая соблюдение требований к защите персональных данных и комплаенса.

Специалист по защите данных и комплаенсу

Обеспечивает соблюдение платформой всех необходимых норм и передовых практик в области безопасности и конфиденциальности данных.

Владелец/менеджер продукта

Выступает в качестве связующего звена между технической командой и бизнес-стейкхолдерами, расставляя приоритеты и управляя функциями и дополнительными возможностями платформы в соответствии с потребностями пользователей и целями бизнеса.

UX-дизайнер

Разрабатывает максимально простые в использовании и эффективные интерфейсы, которые команды доменов используют для взаимодействия с платформой.

Сотрудники службы поддержки и обучения персонала

Помогают командам доменов в использовании платформы, устраняют неполадки и обучают новых пользователей.

Команда платформы федеративного вычислительного управления

В соответствии с четвертым принципом сетки данных еще одна команда платформы должна заниматься федеративным вычислительным управлением, предоставляя каждому домену шаблоны, которые помогают им соблюдать правила безопасности и защиты персональных данных. Большинство команд доменов реализуют эти шаблоны самостоятельно, но команда платформы может иногда внедрять глобальные политики. Она предоставляет необходимые рекомендации, чтобы помочь каждой доменной команде управлять своими данными в соответствии с политикой и стандартами организации. Эта команда должна хорошо понимать специфические потребности каждого домена в данных и находить баланс этих потребностей и общих требований к управлению данными в организации. Роли в этой команде:

Руководитель службы управления данными (data governance lead)

Руководит общей стратегией управления и следит за тем, чтобы инициативы команды соответствовали стратегическим целям организации. Координирует свои действия с представителями доменов и отраслевыми экспертами для определения стандартов и политик управления.

Специалист по управлению данными (data governance specialist)

Разрабатывает и контролирует выполнение стандартов и политик, связанных с данными, тесно сотрудничая с командами доменов, чтобы убедиться в правильности их внедрения.

Специалист по защите данных и комплаенсу

Обеспечивает соблюдение командами доменов правил защиты персональных данных и внутренних политик комплаенса, консультируя их по правовым и нормативным вопросам совместно с внешними экспертами в этой области.

Аналитик по безопасности данных

Определяет стандарты и процедуры безопасности, которым должны следовать команды доменов, дает им указания и следит за соблюдением всех параметров безопасности, предусмотренных политиками управления.

Специалист по качеству данных

Определяет стандарты качества и процедуры для обеспечения качества данных, создаваемых в рамках организации. Работает с командами доменов над созданием метрик качества данных и систем мониторинга.

Представитель домена (domain representative)

Выступает в качестве представителя своей команды домена, предоставляя команде платформы управления информацию о данных, специфичных для данного домена, и следит за тем, чтобы общие политики управления данными соответствовали потребностям его домена.

Архитектор данных

Определяет стандарты моделирования данных, включая разработку и управление логическими и физическими моделями данных.

Инженер DataOps

Автоматизирует внедрение и мониторинг политик управления данными. Может также поддерживать системы и инструменты, обеспечивающие маркировку данных и другие механизмы идентификации данных и управление ими.

Конкретные роли и обязанности членов каждой команды будут различаться в зависимости от специфики организации и ее сетки данных. Самое главное, что каждая команда домена берет на себя ответственность за создание данных как продукта в рамках своего домена.

Члены команды должны тесно сотрудничать друг с другом, чтобы архитектура данных отвечала потребностям бизнеса и пользователей. Архитектура должна быть гибкой, чтобы реагировать на изменения в бизнесе и технологическом ландшафте, и достаточно надежной, чтобы гарантировать качество и доступность данных.

По мере того как мы углубляемся в тонкости динамики внутри команды, необходимо признать, что даже самые хорошо организованные команды могут столкнуться с непредвиденными проблемами. Переключим внимание с надлежащего распределения ролей на распространенные подводные камни, которые могут помешать проекту, и на то, как принимать определенные превентивные меры.

Почему проекты проваливаются: распространенные ошибки и способы их избежать

Теперь поговорим о распространенных причинах неудач проектов с большими данными. Некоторые исследования (<https://oreil.ly/ZwGQY>) показывают высокий процент неудач в этой области; например, в 2019 году сайт *VentureBeat* сообщил (<https://oreil.ly/JmAw5>), что «87 % проектов в области data science никогда не доходят до потребителя». В этом разделе описаны некоторые из наиболее распространенных причин, по которым проекты оказываются провальными, а также способы этого избежать.

Ошибка: позволить руководителю думать, что BI — «это просто»

Я встречал топ-менеджеров с оторванными от жизни представлениями о том, сколько времени занимает создание архитектуры данных. Они часто оказывают давление на руководителей проектов, заставляя их устанавливать нереальные сроки и контрольные точки, тем самым обрекая проект на провал.

Чтобы не попасть в эту ловушку, необходимо обучать руководителей. Помогите им лучше понять процесс и время, необходимое для проведения надлежащей разработки и тестирования решения. Я даже предлагал топ-менеджерам поучаствовать в однодневной коддинг-сессии, чтобы они своими глазами увидели, насколько сложно создавать решения для работы с данными.

Ошибка: использование неподходящих технологий

Когда компании не информированы обо всех доступных продуктах для работы с данными и сценариях их использования, они могут выбрать не тот продукт, который нужен в данном конкретном случае. (Например, я видел, как компания использовала SQL Server в качестве озера данных!) Люди не могут знать то, чего не знают, поэтому они используют только те продукты, о которых слышали. Как говорится, когда у вас есть молоток, все становится гвоздями.

Лица, принимающие решения, должны выделить время на изучение всех возможных инструментов, а затем выбрать тот, который подходит для работы, чтобы избежать ошибок. Я рекомендую сформировать несколько комитетов, каждый из которых будет заниматься изучением определенной группы продуктов, например ETL, инструментов аналитики или инструментов для создания отчетов.

Ошибка: собирать слишком много бизнес-требований

В начале карьеры я был разработчиком в компании, которая больше года собирала бизнес-требования для решения по работе с данными, а наша команда сидела и ждала, когда они закончат. В итоге мы получили несколько сотен страниц требований и большое отставание от графика. Никто так и не успел прочитать весь документ; этап сбора требований занял так много времени, что перед началом разработки мы успели изучить лишь несколько разделов.

Чтобы избежать подобной ошибки, уделите сбору требований несколько недель, собрав достаточно, чтобы разработчики могли начинать писать код, и продолжайте сбор параллельно с работой программистов.

Ошибка: собирать слишком мало бизнес-требований

Некоторые проектные команды практически не выделяют время на сбор требований от конечных пользователей; другие вообще оставляют разработчикам право самим догадываться, чего же хотят конечные пользователи. В итоге частенько

реализуются не те функции, которые хотели бы иметь пользователи, и совсем не те, что им действительно нужны.

Если вы примените итеративный подход к разработке проекта, сможете избежать этой проблемы. Такой подход предполагает создание отдельных частей решения, а затем повторное возвращение на этапы проектирования и планирования, что позволяет непрерывно вносить коррективы на основе обратной связи и меняющихся требований.

Ошибка: представление отчетов без их предварительной проверки

Если вы представите конечному пользователю отчет, сформированный с помощью вашего нового решения, а в нем окажутся неверные данные, его первое впечатление будет негативным. Если пользователи теряют доверие к отчетам, его очень трудно потом вернуть. Компании не всегда выделяют достаточно времени на то, чтобы убедиться, что в отчете все правильно, и это может привести к серьезным проблемам.

Устройте строгий процесс проверки качества, включающий сравнение итоговых показателей отчета, сформированного в новом решении, с показателями отчета из рабочего решения, который заведомо является достоверным. По возможности обратитесь за помощью к конечным пользователям: они не только смогут подтвердить правильность отчетов, но и почувствуют свою причастность к созданию нового решения.

Ошибка: привлечение неопытной консалтинговой IT-компании

У большинства компаний не хватает либо времени, либо опыта, чтобы самостоятельно создать хранилище данных, поэтому они нанимают для этого консультантов. Это отличная идея, нужно только выбрать хорошую консалтинговую компанию. Она может быть экспертом — но в совершенно другом типе проектов. Консалтинговые компании иногда привлекают экспертов с соответствующим опытом, когда пытаются получить заказ, но заменяют их менее опытными исполнителями, как только начинается работа над проектом.

Чтобы не попасться на эту удочку, задавайте вопросы. Есть ли у консультантов опыт создания конкретного решения, которое вам нужно? Если да, то будут ли именно эти люди работать над вашим проектом?

Ошибка: привлекать консалтинговую компанию, которая передает разработку на аутсорсинг офшорным разработчикам

Сторонние компании, преимущественно выполняющие разработку на заказ, часто используют офшорных разработчиков для экономии средств, но это может привести к проблемам. Разница во времени может затруднить проведение совещаний, а языковой барьер — правильно изложить требования; кроме того, качество их работы может оказаться намного ниже.

Чтобы избежать подобных проблем, задавайте вопросы, чтобы убедиться, что консалтинговая компания не передаст ваш проект офшорной команде.

Ошибка: передача прав на владение проектом консалтинговой компании

Может возникнуть соблазн, так сказать, «передать ключи» от проекта консалтинговой компании, поручив ей не только разработку проекта, но и полное управление им. Это может привести к отсутствию прозрачности; проект может превратиться в «черный ящик», когда вы не будете знать, над чем работают сотрудники консалтинговой компании и насколько хорошо они справляются со своей работой.

Убедитесь, что вы получаете полную и правдивую информацию о ходе проекта. Не стоит ограничиваться отчетами о состоянии дел; назначьте менеджеров проекта, которые будут внимательно следить за ходом работ.

Ошибка: пренебрежение передачей информации обратно в организацию

Если вы пользуетесь услугами консалтинговой компании, не допускайте ситуации, когда она выполняет проект и уходит, не обучив вас пользоваться тем, что она создала. Вы должны иметь возможность исправлять ошибки и добавлять функции без необходимости снова привлекать подрядчика.

Пусть часть ваших сотрудников поработает вместе с консультантами хотя бы несколько недель, чтобы ознакомиться с создаваемым решением. Выделите время для передачи информации ближе к концу проекта и попросите консалтинговую компанию провести тренинги и рабочие совещания, чтобы ознакомить сотрудников вашей организации с новым решением.

Ошибка: сокращение бюджета в середине проекта

Плохо спланированные проекты иногда быстро исчерпывают свой бюджет, и деньги заканчиваются как раз в тот момент, когда финиш уже близок. Если вам нужен гарантированный провал, начните сокращать бюджет, в том числе увольнять разработчиков, тестировщиков, менеджеров программ или администраторов баз данных; урезать аппаратное обеспечение или сокращать обучение конечных пользователей работе с новой системой. Не делайте этого.

Чтобы избежать подобной ошибки, либо увеличьте бюджет, либо сократите расходы в менее важных областях. Например, изначально уменьшите количество систем-источников, из которых нужно принимать данные, оставив это на последующие этапы проекта.

Ошибка: исходить из даты окончания проекта и строго следовать графику

Ориентация на жесткие сроки может негативно сказаться на качестве проекта. Во-первых, спешка на сложных этапах поиска данных, интеграции, определения

стандартов качества, построения ETL-процессов, проектирования моделей данных и создания пользовательских интерфейсов может привести к ошибкам и неэффективности системы. Во-вторых, жесткие временные рамки могут также ограничить важные фазы проекта, такие как дискавери и исследование, и в итоге созданная система не будет отвечать фактическим потребностям организации. В-третьих, нереалистично предполагать, что масштаб проекта будет полностью понятен изначально. Жесткая дата окончания проекта затрудняет учет неожиданных изменений или новых требований, возникающих по ходу проекта, что может привести к задержкам или неполному результату. Наконец, фиксированная, не подлежащая коррекции дата окончания проекта может ограничить сотрудничество с основными стейкхолдерами, и, как следствие, продукт не сможет должным образом удовлетворить потребности всех пользователей.

Не делайте сроки и график работы фиксированными. Примиритесь с тем, что при возникновении новых обстоятельств может потребоваться изменить исходный график.

Ошибка: создание структуры хранилища данных в соответствии с исходными данными, а не с потребностями бизнеса

Исходные данные извлекаются из информационных систем, которые обычно не предназначены для комплексного анализа данных или задач бизнес-аналитики, что может ограничить удобство использования современного хранилища данных. Эти системы также не учитывают стратегические цели и ключевые показатели эффективности бизнеса (KPI), из-за чего можно упустить ценные аналитические сведения (а это, в свою очередь, негативно сказывается на принятии решений). Моделирование системы на основе исходных данных также означает, что изменения информационных систем, которые обычно происходят по мере развития бизнеса, потребуют серьезных модификаций хранилища, а это приведет к сбоям в работе и увеличению объемов технического обслуживания.

Создавайте структуру современного хранилища данных (или озера-хранилища данных) в соответствии с потребностями бизнеса. Не стоит просто бездумно повторять структуру исходных данных.

Ошибка: представлять конечным пользователям решение с большим временем отклика или другими проблемами производительности

Верный способ получить массу жалоб от конечных пользователей — это представить им решение, которое медленно создает отчеты, дашборды или запросы, особенно если предыдущее решение делало это быстрее. Как и в случае с неверными данными, если у пользователей создается плохое первое впечатление из-за длительного времени отклика, они теряют уверенность в вашей компетентности, доверие к вам и создаваемому решению, и это доверие будет очень сложно вернуть.

Не ждите жалоб, чтобы начать повышать производительность. Измерьте время отклика для перемещаемых на новую платформу отчетов и настройте новые так, чтобы время было как минимум таким же. Попросите пользователей определить приемлемый базовый уровень производительности для отчетов, которых в старой системе не было.

Ошибка: чрезмерное (или недостаточное) проектирование архитектуры данных (overdesigning/underdesigning)

Если вы уделите слишком мало времени проектированию архитектуры данных для своего решения, рискуете создать нечто, что не сможет работать с некоторыми источниками данных (определенного размера, типа или скоростью передачи); либо будет немасштабируемым; либо будет иметь низкую производительность или создавать риски для безопасности данных. С другой стороны, слишком тщательное проектирование архитектуры может привести к чрезмерной сложности, отсутствию гибкости, невозможности проводить анализ и увеличению расходов на реализацию.

При создании архитектуры данных важно соблюдать баланс. Проект должен быть достаточно всеобъемлющим, чтобы обеспечить надежную, масштабируемую и безопасную структуру, но при этом достаточно гибким и простым, чтобы его было легко понять, реализовать и в дальнейшем адаптировать в соответствии с меняющимися потребностями.

Ошибка: плохая коммуникация между IT-специалистами и бизнес-доменами

Эти две важнейшие группы специалистов часто используют разный язык, имеют разные приоритеты и смотрят на вещи с разных точек зрения, поэтому разногласия между ними не редкость. IT-специалисты обычно думают о технических вопросах и внедрении системы, в то время как бизнес концентрируется на стратегических целях и прибыльности. Недостаток коммуникации может привести к таким серьезным проблемам, как несогласованность целей, неэффективное использование ресурсов, упущенные возможности и снижение морального духа.

Успех проекта по работе с данными во многом зависит от сотрудничества этих двух групп. IT-команда должна понимать потребности бизнеса, а бизнес должен правильно оценивать потенциал и ограничения IT-инфраструктуры. Очень важно формировать культуру открытого диалога и взаимного уважения. Такие роли, как бизнес-аналитик, могут помочь преодолеть этот разрыв, переводя бизнес-требования на языке технических спецификаций и наоборот. Обе группы должны регулярно встречаться, проводить прозрачные обсуждения и четко документировать требования. Эффективное сотрудничество может существенно повлиять на то, будет ли проект успешным.

Рекомендации по достижению успеха

После прогулки по минному полю потенциальных проблем и подводных камней, способных сорвать даже самые многообещающие проекты, следует скорректировать свои представления. Насколько важно знать про то, что может пойти не так, настолько же важно понимать, что двигает проект к цели. Приведенные далее советы помогут вам использовать проверенные стратегии, лучшие практики и житейскую мудрость, чтобы избежать возможных ошибок и приблизиться к успеху.

Не экономьте на инвестициях

Иногда лучше потратить вдвое больше времени и денег, чтобы создать решение, которое будет в 50 раз лучше старого. Обращайте внимание на долгосрочную ценность вложений времени и ресурсов в создание высококачественных, надежных и масштабируемых решений для работы с данными, которые поддерживают возможности расширенной аналитики. Это очень важно для формирования культуры, ориентированной на данные; она способствует инновациям и повышению эффективности работы.

Спешка при реализации проектов или экономия ресурсов могут привести к системным ошибкам, неточным результатам и проблемам с комплаенсом. В конечном счете постоянное обслуживание или даже полное перепроектирование неработающей системы может обойтись дороже, чем изначальные высокие инвестиции в передовые инструменты. Выделение большего количества времени и ресурсов на предварительном этапе позволит лучше спланировать работу, правильно подобрать сотрудников и применить более совершенные инструменты — все это поможет создать надежную, точную и масштабируемую систему.

Специалисты с большим опытом и обширными техническими знаниями стоят дорого, но создание решения для работы с данными требует сочетания бизнес-знаний, технических навыков и управленческих способностей, что бесценно. Кроме того, в проекте должен быть *хотя бы один* человек, который хорошо разбирается в архитектурах данных. Он должен возглавить процесс принятия решения о том, какую архитектуру данных выбрать. Это очень важное решение. Если вы оставите его непрофессионалам, то результат, скорее всего, не будет соответствовать базовым требованиям, что приведет к проблемам с производительностью, невозможности получить данные из некоторых источников, ошибкам и другим проблемам. Привлекайте людей, которые знают свое дело. То же касается и инструментов для создания отчетов и любых других инструментов, которые вы используете; убедитесь, что у вас есть люди, которые понимают их возможности и функционал, чтобы вы получали максимальную отдачу от своих инвестиций.

В целом начальные затраты и вложения времени будут с лихвой компенсированы высокой долгосрочной окупаемостью инвестиций, в том числе снижением долгосрочных эксплуатационных расходов, а также стратегическими преимуществами и ценной аналитикой.

Привлекайте пользователей, покажите им результаты и заинтересуйте их

Как можно раньше вовлекайте конечных пользователей, чтобы они работали вместе с вами, а не против вас. В самом начале проекта пригласите их помочь решить такие вопросы, как, например, выбор источников данных или способов очистки, агрегирования и проверки данных. Если они будут чувствовать себя причастными к проекту, они будут поддерживать его и захотят помочь решить любые возникающие проблемы, а не жаловаться на них. Если вы не вовлечете их в процесс, им будет казаться, что изменения им навязывают. Они быстро начнут жаловаться на качество решения и откажутся его использовать.

Если ваше решение потребует от пользователей работы с незнакомыми новыми технологиями (например, инструментами для создания отчетов), обучите их! Участие в тренингах — отличная возможность для пользователей подготовиться к работе с новым решением и даже научиться быстрее и лучше выполнять свою работу. Помогите им понять все новые полезные функции и возможности, которые предлагает новый инструмент, чтобы они с нетерпением ждали его успешного выпуска.

Для этого сразу демонстрируйте результаты и делайте это как можно чаще. Не заставляйте пользователей и заинтересованных лиц ждать результатов месяцами. Используйте интерактивный, гибкий подход с частыми релизами. Разбейте проект на этапы с быстрыми результатами, демонстрирующими выгодные преимущества. Например, при добавлении источников начните с одного, который имеет высокий приоритет, но при этом небольшой и простой. Затем повторите этот процесс с другим источником. Это заложит прочный фундамент и поможет разработчикам выработать лучшие практики для создания решения.

Повысьте ценность новых отчетов и дашбордов

Если вы осуществляете миграцию — например, из локальной среды в облако, — не просто повторяйте существующие отчеты, а повысьте их ценность, предоставив пользователям новые функции и возможности. Создайте дашборды с KPI, которых они раньше не видели; покажите им искусство возможного и новые способы получения информации из данных. Найдите способы помочь им выполнять свою работу быстрее и легче. Если новое решение не будет лучше старого, они спросят: «Зачем мы потратили все эти деньги на создание того, что у нас уже есть?» Убедите их, что деньги на новое решение были потрачены не зря.

Попросите конечных пользователей создать прототип

Единственная часть решения для работы с данными, которую увидит и будет использовать большинство конечных пользователей, — это инструмент для создания отчетов. Вы должны убедиться, что все пользователи знакомы с большинством, если не со всеми его функциями. Многие даже не догадываются о функциях, которые могли бы сэкономить их время и помочь получить нужную информацию. Это

все равно что ехать на Lamborghini на первой передаче, потому что вы не знаете, что у него есть еще и другие! Обучите пользователей работе с инструментом для создания отчетов и попросите одного из экспертов продемонстрировать его работу на собственных данных.

Чтобы конечные пользователи освоили необходимый функционал инструмента для создания отчетов, важно привлечь их к созданию прототипа. Когда вы определите бизнес-требования проекта и обучите людей работе с этим инструментом, попросите конечных пользователей создать так называемый прототип. Он должен состоять из образцов отчетов, содержащих всю необходимую информацию. Создав такое практическое представление, пользователи смогут лучше понять расширенные возможности новых отчетов и дашбордов. Современные продвинутые инструменты для создания отчетов стали настолько простыми в использовании, что для создания прототипа потребуются минимальные технические навыки. Говорят, что лучше один раз увидеть, чем сто раз услышать. Рабочий прототип — отличный способ убедить конечных пользователей в преимуществах и возможностях нового решения.

Найдите спонсора проекта

Спонсор проекта (project champion/project sponsor) — это руководитель высшего звена или влиятельная персона, который поддерживает и отстаивает проект. Его роль важна и многогранна. Во-первых, он выступает проводником ваших интересов внутри компании, рассказывая о долгосрочных преимуществах хорошо построенной архитектуры данных, включая повышение эффективности принятия решений, операционной эффективности и стратегического планирования. Во-вторых, он играет важную роль в обеспечении необходимыми ресурсами и финансированием. Учитывая, что проекты по работе с данными требуют значительных инвестиций в технологии, кадровые ресурсы и время, роль спонсора проекта, способного мобилизовать эти ресурсы, может стать решающей для его успешного завершения.

В-третьих, спонсор проекта играет важную роль в налаживании кросс-функционального взаимодействия между отделами с разными интересами, согласовывая приоритеты каждого для достижения общей цели проекта. Наконец, он борется с сопротивлением изменениям — значительной проблемой большинства проектов по работе с данными, — помогая вовлечь сотрудников и устраняя их опасения по поводу новых систем и процессов. Это помогает обеспечить более плавный переход на новое решение и повышает вероятность успеха проекта.

Составьте план проекта с целевым показателем эффективности работы сотрудников 80 %

Сотрудник, ответственный за ежедневное управление и контроль выполнения проекта, часто это *менеджер проекта* или *руководитель проекта*, должен обеспечить оптимальную вовлеченность всех участников. Эта роль отличается от роли

спонсора проекта, который оказывает стратегическую поддержку и отстаивает интересы проекта на более высоком уровне. Менеджер проекта находится «на передовой», координируя задачи, сроки и ресурсы. В идеале каждый член команды должен уделять проекту около 80 % своего рабочего времени. (Остальные 20 % предположительно уйдут на отпуск или больничный, встречи, не связанные с проектом, обучение и т. д.) Вам понадобится хорошо продуманный план проекта, направленный на минимизацию времени ожидания сотрудниками, пока другой закончит ту или иную зависимую задачу. План также должен обеспечивать, что все, кому следует, выполняют то, что нужно, и что все пройдут обучение до начала проекта. Любая эффективность менее 80 % может привести к срыву сроков и перерасходу средств.

Итоги

В этой главе мы рассмотрели ключевую роль человеческого фактора в архитектуре данных. Хотя инструменты и технологии могут поражать и внушать благоговение, они лишь орудия в руках своих владельцев. То, как организации используют эти инструменты и как они справляются с культурными сдвигами в процессе, может поддержать или разрушить их архитектурные начинания. Как вы уже поняли, создание правильной структуры команды, четкое распределение ролей и обязанностей, а также глубокое понимание потенциальных ошибок и маркеров успеха — все это необходимо для удачной реализации проекта.

Нужно понимать, что самая сложная и передовая технология будет настолько мощной, насколько сильны люди, ее использующие, и процессы, регулирующие ее использование. Идеально спроектированное озеро-хранилище данных или надежная сетка данных могут дать результат только в том случае, если их дополняет хорошо организованная, информированная и сильная команда. Истинный успех преобразования данных заключается в этом тонком балансе между людьми и технологиями.

Однако, несмотря на то что эта человекоцентричная точка зрения является основополагающей, мы не можем игнорировать технологические прорывы, которые за последние годы изменили ландшафт архитектур данных. В следующей главе мы погрузимся в технологическую сферу, исследуя инновации, движение за открытый исходный код, революционные возможности Hadoop, преимущества преобразования данных, которые доступны благодаря облачным технологиям, и предложения основных облачных провайдеров. Кроме того, я открою вам мир мультиоблачных стратегий и познакомлю с такими прорывными технологиями, как Databricks и Snowflake.

Люди и технологии — это не выбор «или-или». Они должны гармонично сочетаться и способствовать развитию друг друга. Продолжим наше познавательное путешествие с пониманием, что технологии по-настоящему реализуются, только когда их поддерживают правильные люди и процессы.

Технологии

Первая часть главы будет посвящена важному выбору между решениями с открытым исходным кодом и предложениями облачных провайдеров. Делая его, необходимо учитывать ряд соображений. Я помогу вам оценить масштабы ваших потребностей в данных и требования к гибкости организации.

Помимо этого, мы рассмотрим весь спектр моделей облачных сервисов, от предоставляемых основными провайдерами до сложных мультиоблачных решений. В конце главы мы познакомимся с тремя важнейшими программными фреймворками: Hadoop, Databricks и Snowflake.

От того, как вы ответите на вопросы, поставленные в этой главе, в значительной степени будет зависеть ваша будущая архитектура данных. Речь идет не столько об отличии решений с открытым исходным кодом от коммерческих, сколько о поиске идеального сочетания технологий под ваши индивидуальные потребности в мире архитектур данных.

Выбор платформы

Решение об использовании программ с открытым исходным кодом или предоставляемых провайдерам облачных услуг зависит от разных факторов, включая конкретные требования пользователя, бюджет, квалификацию персонала, чувствительность данных и предпочтения в управлении ими. Тщательно оцените потребности своей организации и рассмотрите альтернативу между преимуществами облачных программных продуктов и гибкостью и широкими возможностями кастомизации решений с открытым исходным кодом.

Решения с открытым исходным кодом (OSS)

ПО с открытым исходным кодом (open source software, OSS) — это программное обеспечение, которое распространяется бесплатно, а его исходный код находится в открытом доступе с возможностью просмотра, изменения и распространения всеми желающими. Как правило, оно разрабатывается совместно сообществом программистов, которые вкладывают свои знания и опыт в создание и совершенствование этого продукта.

У OSS-решений богатая история, которая насчитывает несколько десятилетий. Она ведет свое начало от Движения за свободное программное обеспечение (Free Software Movement) (<https://www.fsf.org/history>), возникшего в 1980-х годах и возглавляемого Ричардом Столлманом (Richard Stallman), которое выступало за свободу программного обеспечения. Инициатива открытого исходного кода (OSI, Open Source Initiative) (<https://opensource.org>), основанная в 1998 году, сформулировала определение открытого исходного кода и установила стандарты лицензирования OSS, которые гарантируют, что ПО остается бесплатным и открытым для использования всеми желающими. Значительную роль в популяризации OSS сыграл Linux. Использование решений с открытым исходным кодом стало популярным в 2010-х годах благодаря их гибкости и низкой стоимости. Такие платформы для совместной работы, как GitHub, способствовали разработке OSS, и это движение вышло за рамки ПО и распространилось на инициативы в области открытых данных (Open Data) и открытого аппаратного обеспечения. К числу ведущих OSS-проектов относятся MySQL, PostgreSQL, MongoDB, Apache Hadoop, Apache Spark, Apache Kafka, Presto и Apache Airflow.

Многие архитектуры данных используют OSS, но лишь некоторые полностью построены на его основе. Большинство архитектур используют OSS в сочетании с продуктами облачных провайдеров. Включение OSS в архитектуру данных — идея, заслуживающая рассмотрения.

Вот лишь несколько основных причин, по которым стоит обратиться к использованию OSS вместо продуктов облачных провайдеров.

Экономия средств

Решения OSS обычно бесплатны. Это делает их привлекательными для организаций, стремящихся оптимизировать бюджет.

Гибкость и возможность кастомизации

OSS предоставляет свободу модификации и настройки исходного кода в соответствии с конкретными потребностями. Это позволяет адаптировать программы к требованиям организации, добавляя или удаляя функции по мере необходимости.

Прозрачность и доверие

Исходный код OSS находится в открытом доступе, и каждый может проверить его на наличие уязвимостей безопасности, ошибок и потенциальных баггеров. Такая прозрачность способствует укреплению доверия и созданию атмосферы сотрудничества для код-ревью и улучшения кода.

Быстрые инновации и сотрудничество

Преимуществом OSS является наличие глобального сообщества разработчиков, которые делятся своим опытом и вносят улучшения. Такая атмосфера сотрудничества способствует быстрому внедрению инноваций, укорачивает циклы

программной разработки и приводит к появлению более надежных решений с богатым функционалом.

Независимость от провайдера

Используя OSS, организации не привязаны к конкретному провайдеру и не ограничены лицензионными программами. Они могут свободно менять провайдеров услуг или адаптировать ПО под конкретные нужды по мере необходимости.

Однако существуют и потенциальные недостатки, которые необходимо учитывать.

Ограниченная поддержка

В отличие от облачных продуктов, OSS может не иметь специализированной поддержки клиентов или соглашений об уровне обслуживания. Если полагаться на поддержку сообщества, то ожидание ответа может затянуться и решение критических проблем займет больше времени. Чтобы этого избежать, компании часто приобретают контракт на поддержку у провайдера. Кроме того, некоторые поставщики ПО и оборудования отдают предпочтение совместимости и поддержке облачных решений по сравнению с OSS. Это может привести к ограниченной доступности драйверов для оборудования или отсутствию поддержки некоторых программных приложений.

Скорость обучения и удобство работы пользователей

Чтобы эффективно внедрить, настроить и поддерживать OSS-решения, часто требуются квалифицированные разработчики или технические специалисты. Это может привести к дополнительным расходам на обучение или подбор сотрудников. Проекты с OSS также различаются по качеству документации, удобству интерфейсов и простоте установки. В некоторых проектах может не быть полной документации или интуитивно понятных пользовательских интерфейсов, что усложняет начальную установку и настройку.

Фрагментация, стандартизация и совместимость

Из-за распределенного характера разработки OSS и огромного количества проектов и версий в OSS практически отсутствует стандартизация стиля написания кода, соглашений или подходов. Это может привести к фрагментации, что затрудняет выбор правильного решения или обеспечение совместимости между компонентами OSS. Интеграция с существующими системами может потребовать дополнительных усилий (больше информации см. в разделе «Nadoor» на стр. 272).

Безопасность и ответственность

Даже при наличии тщательной проверки, осуществляемой глобальным сообществом, в OSS по-прежнему присутствуют уязвимости и баги. Если вы используете решения с открытым кодом, то должны быть бдительны при установке

исправлений и обновлений системы безопасности. Кроме того, если организация меняет OSS-код, она берет на себя ответственность за поддержание безопасности и целостности модифицированной версии.

Неопределенность в обслуживании проекта

Не все проекты OSS активно развиваются или предлагают долгосрочное обслуживание. Иногда оно прекращается после выпуска небольшого количества обновлений или патчей безопасности. Оцените уровень активности и поддержки сообщества, прежде чем принимать решение об использовании OSS.

Вопросы интеллектуальной собственности

При использовании или участии в создании OSS организации должны учитывать права интеллектуальной собственности и лицензионные обязательства. Очень важно понимать положения и условия лицензий OSS, чтобы обеспечить комплаенс и избежать юридических проблем.

Локальные решения

Если у вашей компании есть локальное решение для хранения данных, вы, возможно, рассматриваете вариант его переноса в облако или, по крайней мере, создания новых проектов уже в облаке. Однако в некоторых случаях вам все же необходимо локальное решение.

- Если у вас медленный интернет или вообще отсутствует подключение к нему, например в глубокой шахте или на подводной лодке, морской нефтяной вышке или круизном лайнере.
- Если требуется миллисекундная производительность, как, например, для почтовых серверов, которые обрабатывают большие объемы почты.
- Если перенос приложений в облако нарушает контракт на поддержку с третьей стороной.
- Если у вас заключен договор на долгосрочную фиксированную аренду дата-центра или вы только что совершили очень крупную покупку оборудования.
- Если у вас очень большой объем локальных данных, которые необходимо переносить в облако, а ваш облачный конвейер данных слишком ограничен, чтобы удовлетворить потребности пользователей в создании отчетов. В этом случае ваши отчеты не будут отражать самые последние данные, что может привести к задержкам в принятии решений.
- Если вы планируете заменить свои приложения и базы данных или вывести их из эксплуатации в ближайшем будущем.
- Если ваши данные чрезвычайно чувствительны (хотя это уже не мешает многим организациям размещать данные в облаке — у большинства облачных провайдеров теперь существуют облака с высокой степенью защиты).

Если что-либо из перечисленного справедливо для баз данных вашей компании, вам придется оставить их в локальном хранилище, но остальные всегда можно перенести в облако. За последние несколько лет я говорил с сотнями компаний, и почти каждая из них хотя бы частично хранит свои данные в облаке. А большинство уже закрыли все свои локальные дата-центры. Для любой новой компании переход в облако не является чем-то из ряда вон выходящим.

Если ваше хранилище данных локальное, вам следует знать о связанных с этим ограничениях.

Ограниченная масштабируемость

Возможность масштабирования инфраструктуры ограничена тем, какое оборудование вы можете приобрести. Аппаратное обеспечение может стать ограничением, если вы вынуждены закупать его у определенных поставщиков, если имеются проблемы совместимости с существующим оборудованием или если поставщики оборудования испытывают сложности с производством. Аналогичная ситуация с дата-центрами — они предоставляют только определенное физическое пространство. Когда у вас заканчивается место, для установки дополнительных серверов существуют только два очень дорогих варианта: расширение имеющегося дата-центра или строительство еще одного. Все это может занять много месяцев.

Первоначальные затраты

Приобретение оборудования для локальных решений требует очень больших первоначальных капитальных затрат. Аппаратное обеспечение включает в себя серверное оборудование, помещение под центр обработки данных для размещения оборудования, дополнительные устройства хранения, брандмауэры, сетевые коммутаторы, высокоскоростную сеть (с резервированием) для доступа к данным, электропитание и резервные источники питания, необходимые для поддержания работоспособности системы, а также расходы на защиту и резервное копирование данных.

Если ваше хранилище данных имеет критически важное значение, вам также потребуется настроить узел аварийного восстановления, что фактически удваивает расходы.

Я часто говорю компаниям, что если у них есть собственный локальный дата-центр, то они занимаются кондиционированием воздуха, поскольку им придется охлаждать все это оборудование. Не лучше ли сосредоточить все усилия на анализе данных? Большинство компаний предпочитают ежегодную оплату услуг облачного провайдера.

Затраты на персонал

Вам придется платить сотрудникам или консультантам, обладающим специальными знаниями и опытом, за настройку, управление, администрирование

и поддержку аппаратного и программного обеспечения, настройку продуктов с целью увеличения производительности и развертывание продуктов и решений в рабочей среде. Это создает потенциальное узкое место при возникновении проблем, при этом ответственность за систему остается на заказчике, а не на поставщике.

Затраты на программное обеспечение

Организации часто платят сотни тысяч долларов за лицензионное ПО для хранилищ данных и дополнительных пакетов к нему, а также за дополнительные лицензии для конечных пользователей, таких как клиенты и поставщики. Годовые контракты на поддержку такого ПО могут достигать 20 % от первоначальной стоимости лицензии.

Теперь посмотрим на облачные сервисы.

Облачные решения

Облако — это своего рода огромный невидимый склад, парящий где-то в небесах. Оно позволяет хранить данные, получать к ним доступ и обмениваться ими из любой точки мира, обладая при этом неисчерпаемым пространством и исключая потерю ценных данных. Оно работает со всеми архитектурами, описанными в этой книге, в корне меняя представления о работе с данными. Облачная вычислительная среда предоставляет по мере необходимости доступ через интернет к общим вычислительным ресурсам, таким как серверы, хранилища и приложения, поэтому вам больше не нужно покупать физические дата-центры и серверы, владеть ими и обслуживать их. Это делает вычисления масштабируемыми, гибкими и экономически выгодными.

Облачные решения для хранения данных имеют множество преимуществ.

Масштабируемость и эластичность

Облачные продукты способны масштабироваться бесшовно. При их использовании можно динамически выделять хранилища и вычислительные ресурсы, практически «на лету», чтобы удовлетворить потребности при меняющихся рабочих нагрузках в периоды пиковой и постоянной загрузки. Вы получаете необходимые вычислительные мощности в нужный момент.

Стоимость

В облачной системе сложности и затраты на планирование и администрирование вычислительных мощностей (например, определение объема, балансировка и настройка системы) учтены, автоматизированы и покрываются стоимостью подписки на облачные услуги. Облачные провайдеры также предлагают гибкие модели ценообразования, что позволяет оптимизировать расходы, оплачивая только те ресурсы, которые вы фактически используете. Вы можете сокращать

аппаратное обеспечение по мере снижения спроса или использовать бессерверные варианты.

Это означает, что вы можете создать решение быстро, а если оно не сработает — просто удалить все облачные ресурсы, которые использовались в проекте. Цена неудачи значительно снижается, а значит, можно рисковать и пробовать новое.

Важно также учитывать косвенную экономию: вы не платите за оборудование, электроэнергию, персонал, который создает и обслуживает дата-центр, и не тратите деньги на разработку собственных обновлений.

Простота развертывания и управления

Облачные продукты часто поставляются с уже интегрированными инструментами развертывания и управления, которые упрощают процессы настройки и конфигурации. Эти инструменты оптимизируют выделение ресурсов и задачи масштабирования, мониторинга и обслуживания, снижая операционные издержки. Вы можете создать все необходимые облачные ресурсы (например, вычисления, хранилища и услуги сети) за несколько часов, а то и минут, в то время как запуск локального решения может занять недели или даже месяцы.

Поддержка от поставщика и соглашения об уровне обслуживания (SLA)

Облачные провайдеры предлагают специализированную поддержку и SLA (service-level agreements), обеспечивая помощь, устранение неполадок и гарантированную доступность услуг.

Интегрированные экосистемы

Как правило, облачные программные продукты легко интегрируются с другими сервисами провайдера. Это облегчает интеграцию и взаимодействие между сервисами и помогает использовать более широкие возможности, предоставляемые провайдером.

Управляемые сервисы

Облачные провайдеры часто занимаются отдельными частями программной инфраструктуры, например управляемыми сервисами баз данных, машинного обучения или бессерверными вычислениями. Эти управляемые сервисы упрощают эксплуатацию, позволяя организациям сосредоточиться на своей основной деятельности, а не на управлении базовой инфраструктурой. Это также избавляет от необходимости искать и оплачивать услуги сторонних поставщиков.

Постоянные инновации и обновления функций

Облачные провайдеры регулярно внедряют новые функции, услуги и обновления в свои программные предложения. Вы получаете выгоду от их инноваций,

не вкладывая средства в модернизацию или интеграцию новых функций самостоятельно.

Упрощение поиска сотрудников

Облачные продукты используются огромным количеством людей, в то время как OSS-продукты гораздо менее популярны. Поэтому вам будет намного проще найти специалистов, знакомых с облачными технологиями, которые использует ваша организация.

Глобальная высокая доступность

Дата-центры облачных провайдеров разбросаны по всему миру, что позволяет развертывать программы ближе к целевой аудитории или использовать резервирование в нескольких регионах для обеспечения высокой доступности. Облачные провайдеры применяют балансировку нагрузки, чтобы равномерно распределять трафик между несколькими серверами, гарантируя, что ни один сервер не будет перегружен и не приведет к простоям в работе.

Такая глобальная инфраструктура обеспечивает низкую задержку служб и является отказоустойчивой. В случае аппаратного или программного сбоя служба автоматически переключается на резервный экземпляр, поддерживая бесперебойный и надежный доступ. Кроме того, можно настроить аварийное восстановление хранилища, чтобы это происходило в кратчайшие сроки.

Безопасность и комплаенс

Облачные провайдеры вкладывают значительные средства в такие меры безопасности, как шифрование, контроль доступа и регулярный аудит, чтобы защитить данные и обеспечить соответствие отраслевым стандартам и нормативам.



Совокупная стоимость владения

Все облачные провайдеры предлагают калькулятор совокупной стоимости владения (TCO, total cost of ownership), который можно использовать для оценки экономии средств, получаемой за счет переноса работы приложений и баз данных в облако. Просто добавьте краткое описание своей локальной среды и получите мгновенный отчет от Microsoft Azure (<https://oreil.ly/UlvTP>), Amazon Web Services (<https://calculator.aws>) или Google Cloud (https://oreil.ly/e7_uF).

Модели облачных сервисов

Существуют три основные модели сервисов для предоставления и управления ИТ-ресурсами в облаке.

Инфраструктура как сервис (IaaS, Infrastructure as a Service)

Облачный провайдер размещает у себя аппаратную инфраструктуру и управляет ею. Он предлагает виртуализированные вычислительные ресурсы, такие как

серверы, хранилища и вычислительные сети, которые легко масштабировать в сторону как расширения, так и сокращения по мере необходимости; клиент управляет вычислительной системой, приложениями и данными. Большинство сервисов размещается на виртуальной машине (VM), к которой конечный пользователь получает удаленный доступ. Эта модель обеспечивает большую гибкость и масштабируемость по сравнению с локальной, но управление вычислительной системой, приложениями и данными требует определенного уровня ИТ-знаний.

Платформа как сервис (PaaS, Platform as a Service)

Облачный провайдер предлагает разработчикам платформу для создания, тестирования и развертывания приложений и баз данных, а также базовую инфраструктуру (аналогично IaaS). Она также включает в себя инструменты разработки, системы управления базами данных и сервисы бизнес-аналитики. Облачный провайдер контролирует среду выполнения, промежуточное ПО и операционные среды — никаких виртуальных машин нет. Это позволяет разработчикам сосредоточиться на написании кода и управлении приложениями, не заботясь об управлении платформой и базовой инфраструктурой. Эта модель еще более абстрактна и проста в использовании, чем IaaS, но ее гибкость и возможности кастомизации ограничены.

Программное обеспечение как сервис (SaaS, Software as a Service)

Облачный провайдер размещает на своем хостинге и полностью управляет ПО и инфраструктурой; клиенты получают доступ к программам через интернет и не нуждаются в самостоятельном управлении аппаратным и программным обеспечением. Это самая простая и удобная в использовании модель, но она может иметь ограниченную гибкость в плане кастомизации и управления. Некоторые облачные провайдеры предлагают инструменты архитектуры данных, которые являются «SaaS-подобными»; то есть они не настолько полностью управляемы, как настоящие SaaS-инструменты, такие как Salesforce, а требуют лишь незначительного управления приложениями.

С каждой из моделей облачных сервисов вы получаете последнюю версию любого используемого продукта базы данных. Например, облачная база данных Azure SQL Database представляет собой более новую версию локальной базы данных SQL Server 2022. Считайте, что SQL Database — это SQL Server 2022+. Новые функции добавляются каждые несколько недель — вы можете выбирать, использовать их или нет. Каждые пару лет эти новые функции собираются вместе и добавляются в коробочную версию SQL Server, но с SQL Database вы получаете их сразу. И ваш код не пострадает при установке обновлений.

Все облачные провайдеры предлагают встроенную функцию расширенного обнаружения угроз, используя мониторинг для выявления аномальной активности, которая может указывать на необычные и потенциально опасные попытки доступа

к базам данных или эксплуатации уязвимостей; оценки, которые помогают обнаруживать, отслеживать и устранять потенциальные уязвимости баз данных; и расширенные возможности для обнаружения, классификации, маркировки и составления отчетов о чувствительных данных в ваших БД.

В целом четыре модели, показанные на рис. 16.1, представляют собой различные уровни абстракции и контроля над ИТ-инфраструктурой и программным стеком. Локальное решение обеспечивает полный контроль, но требует значительных инвестиций и опыта, в то время как IaaS, PaaS и SaaS предлагают все более высокий уровень абстракции и простоты использования в ущерб возможностям контроля и кастомизации.

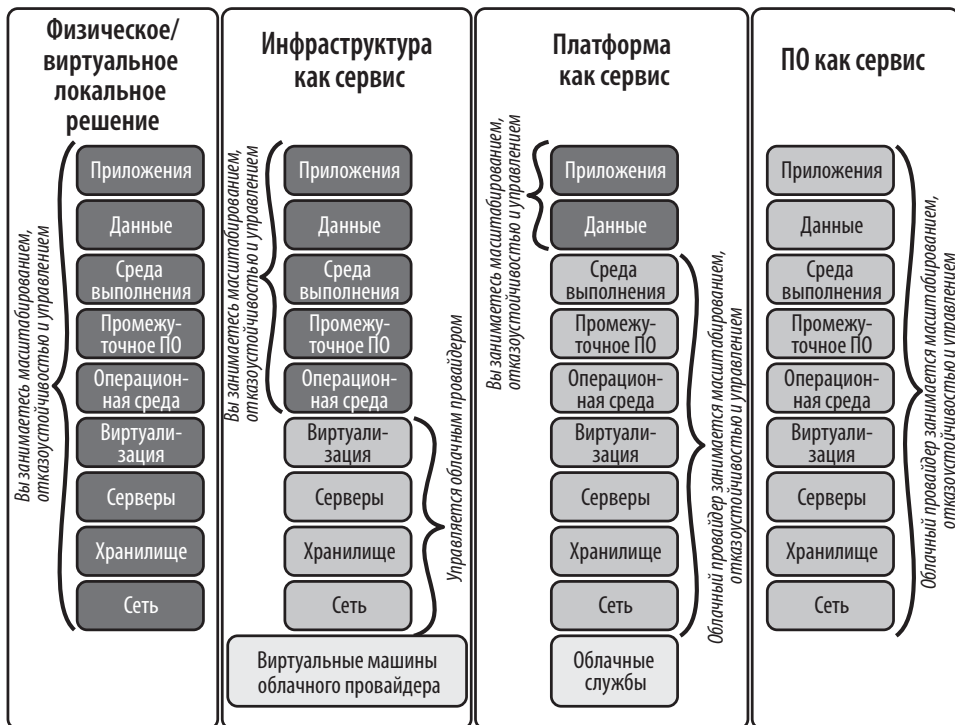


Рис. 16.1. Сравнение моделей сервисов

Выбор модели сервиса зависит от потребностей и целей организации. Однако лично я предпочитаю для хранилищ данных по возможности выбирать модель PaaS по нескольким причинам. Прежде всего вам не нужно иметь дело с виртуальной машиной. Она уже где-то находится, и на ней размещаются базы данных, но вам никогда не придется удаленно входить на сервер или каким-либо образом управлять ею — все делается через портал облачного провайдера или с помощью сторонних инструментов. Вы также не отвечаете за установку патчей и обновлений:

большинство баз данных от облачных провайдеров патчатся в рабочем режиме, без отключений и простоев. Вы получаете резервное копирование баз данных сразу в стандартной комплектации — никаких дополнительных настроек и мониторинга.

Наконец, PaaS значительно упрощает процесс аварийного восстановления. Большинство решений IaaS для аварийного восстановления — это сплошная головная боль с настройкой и мониторингом, но в случае PaaS достаточно указать, в какой части страны разместить базу данных для аварийного восстановления. А о ее настройке и поддержании работоспособности позаботится облачный провайдер.

Ведущие облачные провайдеры

Три основных облачных провайдера — Microsoft Azure (<https://azure.microsoft.com>), Amazon Web Services (AWS) (<https://aws.amazon.com>) и Google Cloud Platform (GCP) (<https://cloud.google.com>) — занимают большую часть рынка (по оценкам, на II квартал 2023 года их доля составляла около 65 % (<https://oreil.ly/oOIx->)). Чтобы узнать более подробную и актуальную информацию о предложениях каждого провайдера, рекомендую вам зайти на их сайты. В таблице 16.1 приведено сравнение основных функций, предоставляемых провайдерами, чтобы дать вам исчерпывающее представление о каждом из них.

Таблица 16.1. Сравнение трех крупнейших облачных провайдеров

Критерий сравнения	Azure	Amazon Web Services	Google Cloud Platform
Год начала работы	2008	2006	2008
Количество регионов с размещенными дата-центрами	60	31	24
Доля мирового рынка	23 %	32 %	10 %
Предлагаемые сервисы	Более 200+ сервисов в разных категориях	Около 200+ сервисов в разных категориях	100+ сервисов, самые популярные — в области аналитики данных и машинного обучения
Ценообразование	Преимущества для пользователей Microsoft	Больше вариантов для типов экземпляров	Простое и понятное клиенту
Интеграция / открытый исходный код	Сильная интеграция с продуктами Microsoft	Широкий спектр интеграций с приложениями с открытым исходным кодом	Хорошая интеграция с различными платформами с открытым исходным кодом

Критерий сравнения	Azure	Amazon Web Services	Google Cloud Platform
Особенности обучения	Проще осваивать тем, кто знаком с другими продуктами Microsoft	Широкий спектр услуг, способствующий быстрому обучению	Удобный дружественный интерфейс, простой процесс обучения
Продукты для анализа данных	Azure Synapse Analytics, Microsoft Fabric	Amazon Redshift	Google BigQuery

У каждого из этих трех облачных провайдеров есть уникальные преимущества, и выбор между ними часто сводится к конкретным потребностям и существующей инфраструктуре. Бизнес, глубоко встроенный в экосистему Microsoft, может тяготеть к Azure; предприятие, ориентированное на расширенную обработку данных, может выбрать Google Cloud Platform; а компания, нуждающаяся в широком спектре сервисов, может остановиться на Amazon Web Services. В конечном счете при принятии решения необходимо учитывать как технические требования и стоимость, так и стратегическое направление бизнеса.

Мультиоблачные решения

Многие организации раздумывают над тем, выбрать ли одного из трех крупных облачных провайдеров или воспользоваться услугами нескольких, что часто называется *мультиоблачным подходом*. Некоторые компании считают, что полагаться только на одного облачного провайдера — значит «класть все яйца в одну корзину», или опасаются, что облачный провайдер может стать их конкурентом в бизнесе. (Так произошло, например, когда Amazon купила Whole Foods, став конкурентом других супермаркетов.)

Мультиоблачный подход действительно имеет ряд серьезных преимуществ, главным образом потому, что он позволяет использовать сильные и слабые стороны каждого из облачных провайдеров. Если у одного провайдера меньше время ожидания, а у другого — лучшая доступность или аварийное восстановление данных, использование обоих может повысить способность организации соблюдать свои SLA. Подобные сравнения можно провести по стоимости, вычислительной мощности, функциям и продуктам; если у одного облачного провайдера нет всего, что вам нужно, вы можете прибегнуть к услугам другого. Однако теперь это уже не так актуально, как раньше; сегодня все три облачных провайдера предлагают очень похожие услуги, с небольшим количеством уникальных функций и продуктов.

Цифровой суверенитет данных — это также немаловажный фактор. Не все провайдеры имеют одинаковые облачные регионы, что может быть критично для обеспечения нормативно-правового соответствия. Например, если вы используете MongoDB в Azure, но у Azure нет региона на Тайване, ваши тайваньские подразделения могут использовать MongoDB в Amazon Web Services.

Представьте себе вымышленного мирового гиганта электронной коммерции, которого мы назовем TechTreasures Inc. Допустим, компания TechTreasures базируется в США, а именно в Понте-Ведра, штат Флорида, но работает по всему миру, включая Северную Америку, Европу и Азию.

Для обеспечения оптимальной производительности, надежности и соответствия региональным нормативным требованиям к обработке данных компания TechTreasures применяет мультиоблачный подход. Для своих операций в Северной Америке она использует Microsoft Azure, поскольку сеть Azure достаточно надежна, имеет малое время задержки и дата-центры в этом регионе. В Европе компания пользуется услугами Google Cloud Platform из-за его возможности в области расширенной аналитики данных и обширного присутствия в европейских облачных регионах. Для своих азиатских операций TechTreasures использует Amazon Web Services, который предлагает облачные регионы на ключевых азиатских рынках, включая Тайвань.

Такая мультиоблачная стратегия позволяет TechTreasures предоставлять быстрые и надежные услуги своей глобальной клиентской базе, обеспечивая при этом соответствие нормам цифрового суверенитета данных. Это стратегический выбор, который позволяет компании использовать уникальные преимущества каждого из крупных облачных провайдеров для оптимизации бизнес-операций.

Но у мультиоблачного подхода много недостатков. Поскольку эти три облака не предназначены для совместной работы, перемещение данных и вычислений туда и обратно может оказаться слишком медленным и неудобным. Также существует проблема совместимости; продукты одного облачного провайдера не всегда хорошо работают с продуктами других провайдеров. Перемещение данных и приложений от одного облачного провайдера к другому и подключение другого продукта могут быть очень дорогостоящим мероприятием, особенно если при этом придется перепроектировать приложения для поддержки нескольких облаков. Кроме того, провайдеры взимают плату за отказ от услуг, если вы переносите данные из их облака в облако конкурента, и совсем не поощряют миграции данных к другому провайдеру. А если вы вложили средства в более крупный интернет-пайплайн, такой как ExpressRoute от Azure, вам придется сделать аналогичные инвестиции в любого другого облачного провайдера, с которым вы работаете. Более того, хранение данных в нескольких облаках повышает их подверженность угрозам безопасности.

Если вы хотите упростить переход из одного облака в другое, вам придется использовать модель сервиса IaaS вместо PaaS, что лишит вас многих более ценных услуг, предлагаемых каждым облачным провайдером, а это упущенные возможности, которые стоят дорого.

Что касается персонала, то вам придется нанимать и/или обучать сотрудников, чтобы они разбирались в двух или более облаках, что значительно увеличивает расходы. Администрирование дополнительного облачного провайдера, обеспечение

безопасности и разработка политик, стандартов и процедур для него также требуют большего количества сотрудников и дополнительных знаний. Вам также придется сделать так, чтобы разные типы безопасности работали вместе, и у вас будет два места для добавления, удаления или обновления пользователей, два места для мониторинга и два набора счетов, что значительно усложняет администрирование (и увеличивает расходы).

Наконец, наличие двух поставщиков облачных услуг вряд ли спасет вас от сбоев. Облачные провайдеры специально проектируют свои сети таким образом, чтобы перебои в одном регионе не влияли на другие регионы. Решение для аварийного восстановления должно заключаться в переходе на другой регион в рамках одного облачного провайдера, а не в переходе на другого провайдера.

Многие считают, что использование мультиоблачного подхода заставляет облачных провайдеров вступать в конкурентную борьбу, позволяя вам заключить более выгодную сделку. Однако это по большей части миф. При заключении корпоративных лицензионных соглашений (ELA, enterprise license agreement) облачные провайдеры предоставляют более высокие скидки в зависимости от того, какой объем ресурсов вы можете использовать в их облаке, — чем больше вы используете, тем больше ваши скидки¹. Если добавить сюда многоуровневое ценообразование и скидки за длительное использование, вы, скорее всего, сэкономите больше, если будете работать только с одним провайдером. Однако некоторые компании могут угрожать перейти к другому облачному провайдеру, чтобы контролировать свои расценки. Если ваша компания является крупным предприятием, сотрудничество с одним облачным провайдером также может принести выгоду. Например, компания Microsoft сотрудничает с Ernst & Young, чтобы обеспечить рост на 15 миллиардов долларов и внедрять технологические инновации в разных отраслях.

Я не ожидаю, что мультиоблачный подход получит широкое распространение. Со временем, думаю, большинство компаний выберут одного облачного провайдера и будут стараться глубже и полнее использовать его услуги, PaaS и SaaS, а также экосистему. Некоторые очень крупные компании останутся на паре облаков, если облачные провайдеры будут предлагать разные услуги, подходящие для конкретной рабочей нагрузки. Это имеет смысл, если компания достаточно велика, чтобы выделять время и человеческие ресурсы на глубокое изучение обоих облаков и использование преимуществ PaaS-сервисов каждого из поставщиков.

Перейдем от обсуждения моделей облачных сервисов к программным фреймворкам — от создания фундамента архитектуры данных к изучению инструментов,

¹ Облачные провайдеры используют разные названия для этих скидок, основанных на степени лояльности клиента; вы можете встретить, например, «цена зарезервированного экземпляра», «скидка за предзаказ», «коммерческая скидка» или «скидка за объем».

использующих этот фундамент. В следующем разделе мы рассмотрим ключевые программные фреймворки, которые используют облачную инфраструктуру для раскрытия всего потенциала данных.

Программные фреймворки

В мире архитектуры данных выбор программных фреймворков может иметь решающее значение в том, насколько эффективно вы обрабатываете, анализируете и извлекаете информацию из своих данных. Эти фреймворки служат механизмом для работы с данными, позволяя извлекать ценную информацию и принимать основанные на данных решения. В этом разделе мы совершим путешествие в мир программных фреймворков и рассмотрим три самых выдающихся в этой сфере: Hadoop, Databricks и Snowflake. Каждый из этих фреймворков обладает уникальными преимуществами, и понимание их возможностей необходимо для построения архитектуры данных, отвечающей конкретным целям и потребностям.

Hadoop

Hadoop — это фреймворк с открытым исходным кодом, созданный в 2005 году компанией Apache Software Foundation. Он предназначен для распределенных вычислений. Разнообразные инструменты и библиотеки расширяют возможности Hadoop, делая его популярным в организациях, которые работают с огромными объемами данных и нуждаются в экономически эффективно масштабируемых решениях.

По своей сути, Hadoop состоит из двух основных компонентов. Первый, распределенная файловая система Hadoop Distributed File System (HDFS), позволяет эффективно хранить и обрабатывать большие данные, разделяя их на более мелкие блоки и распределяя по нескольким узлам и кластерам компьютеров, как показано на рис. 16.2. Это обеспечивает высокую отказоустойчивость и выполнение таких задач, как пакетная обработка, интеграция данных и аналитика. HDFS обеспечивает высокопроизводительный доступ к данным и надежность даже при отказе оборудования.

Второй, фреймворк MapReduce, — это программная модель, используемая для параллельной обработки и анализа больших массивов данных в кластере опять же путем разделения рабочей нагрузки на более мелкие задачи и распределения их по нескольким узлам кластера. Он разбивает большие сложные вычисления на множество задач и распределяет их по отдельным рабочим узлам, которые координируют между собой и консолидируют результаты. Фреймворк MapReduce автоматически справляется с распараллеливанием задач, отказоустойчивостью и распределением данных. Программа MapReduce состоит из процедуры `Map()`, которая фильтрует и сортирует данные (например, сортирует студентов по именам и выстраивает их в очереди, по одной очереди на каждое имя), и процедуры `Reduce()`, которая суммирует данные (например, подсчитывает количество студентов в каждой очереди,

получая частоту появления данного имени). Библиотеки MapReduce существуют на многих языках программирования (обычно на Java) с разными уровнями оптимизации.

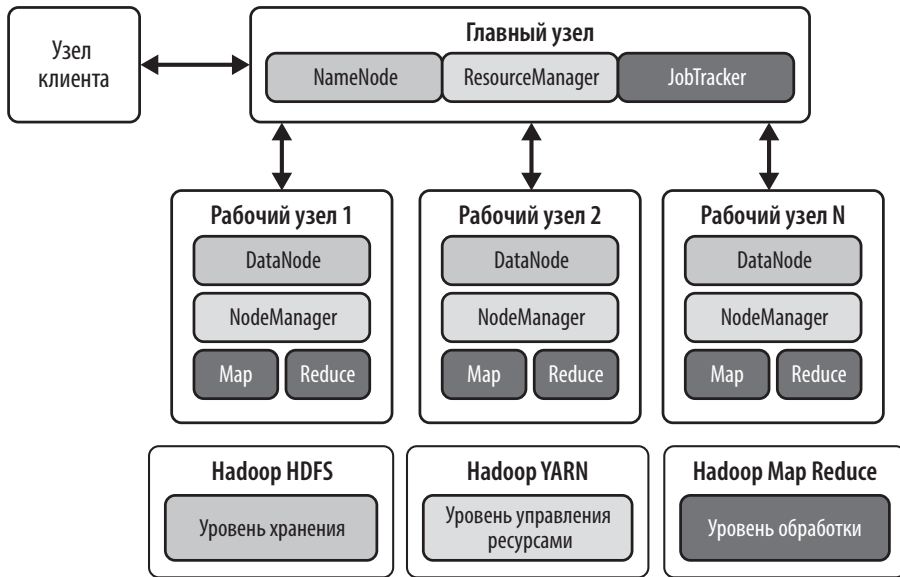


Рис. 16.2. Архитектура кластера Hadoop

Hadoop обладает рядом преимуществ, которые делают его популярным инструментом для работы с большими данными.

Масштабируемость

Hadoop предоставляет масштабируемую архитектуру, способную работать с большими объемами данных. Он позволяет легко масштабировать возможности хранения и обработки данных путем добавления в кластер большего количества оборудования, что делает его экономически эффективным и гибким.

Распределенные вычисления

Hadoop распределяет данные и задачи их обработки по кластеру машин, обеспечивая параллельную обработку. Распределение работы между несколькими узлами позволяет быстрее обрабатывать большие массивы данных.

Отказоустойчивость

Hadoop реплицирует данные на нескольких узлах кластера, обеспечивая их сохранность и доступность даже при отказе оборудования. Если один из узлов выходит из строя, Hadoop автоматически перенаправляет задачи на другие доступные узлы, обеспечивая бесперебойную обработку.

Экономически эффективное хранилище

HDFS позволяет хранить большие объемы данных на стандартном и доступном оборудовании, которое можно купить в готовом виде, а не в дорогих специализированных системах хранения.

Локальность данных

Надоор оптимизирует обработку данных, перемещая вычисления ближе к месту хранения данных, а не передавая их по сети, что снижает нагрузку на сеть и повышает общую эффективность.

Гибкость и совместимость

Надоор совместим с разными форматами данных и может обрабатывать структурированные, полуструктурированные и неструктурированные данные. Он поддерживает широкий спектр языков программирования и инструментов, позволяет разработчикам пользоваться привычными языками и интегрировать различные компоненты в свои конвейеры обработки данных.

Экосистема инструментов

Надоор имеет богатую экосистему инструментов и фреймворков, которые расширяют его возможности и предоставляют абстракции более высокого уровня, языки обработки данных и аналитику. К ним относятся:

- YARN (<https://oreil.ly/xGJCB>) — система управления ресурсами и планирования заданий.
- Hive (<https://hive.apache.org>) — система хранения данных.
- Pig (<https://pig.apache.org>) — высокоуровневый процедурный язык для анализа данных.
- HBase (<https://hbase.apache.org>) — база данных NoSQL.
- Spark (<https://spark.apache.org>) — фреймворк для обработки данных в оперативной памяти.
- Presto (<https://prestodb.io>) — распределенная система SQL-запросов.

Поставщики Надоор продают платформы, которые объединяют, тестируют и предварительно интегрируют вместе несколько продуктов с открытым исходным кодом, что позволяет экономить время и силы на самостоятельной интеграции и тестировании компонентов.

Поддержка сообщества

Надоор имеет большое и активное сообщество разработчиков ПО с открытым исходным кодом, которое вносит свой вклад в его развитие, совершенствование и поддержку, а также предоставляет пользователям множество ресурсов, документации и полезных знаний.

К потенциальным недостаткам Hadoop относятся:

Сложность

Hadoop требует длительного обучения и может оказаться слишком сложным в настройке и управлении. Необходимы специальные знания и опыт работы с распределенными системами, что может представлять проблему для организаций, не имеющих соответствующего опыта или специальных ресурсов.

Время ожидания

MapReduce, основная платформа обработки данных в Hadoop, базируется на пакетной обработке, что приводит к задержкам, поэтому она может не подходить для сценариев обработки данных в реальном времени или интерактивном режиме. Hadoop наиболее эффективен для решения крупномасштабных задач обработки данных, таких как пакетная аналитика и обработка огромных массивов данных. Если вам нужны мгновенные результаты или данные «почти в реальном времени», вам больше подойдут альтернативные решения или фреймворки, такие как Apache Kafka или Apache Flink.

Требования к аппаратному обеспечению

Распределенная архитектура Hadoop требует кластера машин для эффективной работы. Настройка и обслуживание такого кластера, включая аппаратное обеспечение и сетевую инфраструктуру, могут быть сопряжены со значительными затратами и сложностями.

Накладные расходы на управление данными

Хотя HDFS Hadoop обеспечивает отказоустойчивость и репликацию, репликация данных между узлами требует дискового пространства для хранения. Это приводит к дополнительным накладным расходам на хранение данных.

Сложная экосистема

Несмотря на то что экосистема Hadoop богата и обширна, она также и сложна, поэтому управление и интеграция ее компонентов могут быть непростой задачей. Это требует понимания возможностей и совместимости каждого инструмента, а также поддержки конфигураций и зависимостей всех инструментов.

Безопасность данных и управление

Распределенный характер Hadoop и репликация данных могут создавать специфические проблемы с точки зрения безопасности данных, конфиденциальности и управления. В отличие от традиционных систем хранения данных, распределенный характер Hadoop требует от организаций выходить за рамки стандартных методов обеспечения безопасности и принимать специальные меры безопасности и контроля доступа, учитывающие все тонкости этой системы.

Эти меры необходимы для обеспечения защиты данных и соответствия требованиям и лучшим практикам, характерным для Hadoop.

Hadoop получил широкое распространение в начале 2010-х годов, но по мере развития сферы больших данных и облачных вычислений, внедрения таких технологий, как бессерверные вычисления, облачные хранилища данных и платформы потоковой обработки данных, использование Hadoop пошло на убыль. Сегодня облачная инфраструктура может заменить кластеры Hadoop, а облачное хранилище объектов — HDFS. Основные принципы Hadoop, такие как распределенное хранение и обработка данных, остаются актуальными, но его отдельные компоненты и инструменты не получили одинакового признания. Например, такие технологии, как Apache Spark, завоевали популярность в качестве альтернативы или дополнения к MapReduce.

Databricks

Databricks — это облачная платформа для выполнения задач в Apache Spark, движке с открытым исходным кодом для аналитической обработки данных. Изначально Spark был разработан в лаборатории AMPLab Калифорнийского университета в Беркли в 2009 году как способ преодоления ограничений MapReduce¹ — более быстрая и простая в использовании альтернатива, применяемая как для пакетной обработки данных, так и для обработки потоковых данных в реальном времени.

Некоторые из создателей Spark были в числе основателей Databricks в 2013 году. Databricks продолжила работу над коммерциализацией Spark, представив лицензионный программный продукт Unified Data Analytics Platform (унифицированная платформа аналитики данных) — полностью управляемый сервис, который предлагает комплексную упрощенную платформу для анализа больших данных, объединяющую data science, инжиниринг и бизнес. Она призвана упростить для предприятий использование Spark, без необходимости заботиться о создании и поддержке собственной инфраструктуры, делая расширенную аналитику и машинное обучение доступными для более широкого круга пользователей.

С момента своего создания компания Databricks внесла значительный вклад в развитие Apache Spark, работая над повышением скорости обработки данных в рамках проекта Tungsten — серии оптимизаций, направленных на использование современных архитектур процессоров с целью повышения эффективности. Компания также разработала оптимизированную систему автоматического масштабирования, которая динамически регулирует количество машин, выделяемых для выполнения задания, в зависимости от требуемой вычислительной мощности, повышая как экономическую эффективность, так и производительность.

¹ Spark стал программным продуктом с открытым исходным кодом в 2010 году под лицензией Berkeley Software Distribution, а в 2013 году — проектом Apache Software Foundation.

Компания Databricks представила различные библиотеки для расширения возможностей Spark, такие как MLlib (для машинного обучения), GraphX (для вычисления графов) и Spark Streaming (для данных в реальном времени). Она также сыграла решающую роль в обучении и поддержке сообщества Spark, что ускорило распространение Spark и позволило пользователям успешнее применять его возможности.

Кроме того, Databricks сосредоточилась на упрощении использования Spark, представив высокоуровневые API на языках Python, Java и Scala, чтобы сделать написание приложений Spark более доступным. Компания также разработала блокноты Databricks — интерактивные среды, в которых специалисты data science и инженеры данных могут писать код для Spark, визуализировать результаты и создавать модели данных. Эта платформа позволила повысить эффективность совместной работы специалистов data science и инженеров данных, объединив data science, инженерию данных и бизнес-аналитику.

Чтобы устранить ограничения озер данных и повысить надежность и качество данных, компания Databricks также представила Delta Lake (см. главу 12), слой хранения данных с открытым исходным кодом, который работает поверх существующих озер данных. Он расширяет возможности Spark по работе с большими данными за счет ACID-транзакций, масштабируемой обработки метаданных и унифицированной потоковой и пакетной обработки данных. Delta Lake решает проблемы традиционных озер данных, связанные с качеством данных, надежностью и производительностью. В 2019 году Databricks сделала Delta Lake программой с открытым исходным кодом, передав этот проект в Linux Foundation, чтобы дать возможность более широкому кругу разработчиков внести свой вклад. Сегодня Delta Lake получила всеобщее признание.

В 2020 году компания Databricks представила новую архитектуру данных — озеро-хранилище данных (которая также рассматривается в главе 12). Эта инновация ознаменовала собой значительный сдвиг в парадигме управления данными. Вместо того чтобы выбирать между озерами данных и хранилищами данных, организации могут выбрать гибридную архитектуру, сочетающую в себе как масштабируемость и экономичность озер данных, так и надежную структуру, а также безотказность и производительность хранилищ данных.

По мнению Databricks, современные архитектуры данных должны быть открытыми, поддерживать совместную работу и работать со всеми типами данных и сценариями использования расширенной аналитики. Таким образом, компания поддерживает архитектуры, позволяющие предприятиям легко проводить масштабирование по мере их роста и изменения с течением времени.

Благодаря своей платформе Unified Data Analytics Platform и таким инструментам, как Delta Lake, компания Databricks может предоставить базовую инфраструктуру для каждого продукта данных или домена в сетке данных (см. главы 13 и 14). Платформа может обрабатывать разные типы данных и поддерживать обработку данных как в пакетном режиме, так и в режиме реального времени, что делает ее

достаточно универсальной для удовлетворения потребностей разных доменов данных. Основные функции Delta Lake, такие как транзакции ACID, принудительное применение схемы и версионирование, позволяют обеспечить качество и согласованность данных в каждом домене данных. Более того, она предоставляет возможность своего рода «путешествий во времени» (то есть доступа к предыдущим версиям данных). Это может быть особенно полезно при использовании сетки данных, где командам может потребоваться провести аудит или отменить изменения в соответствующих доменах. Среда для совместной работы от Databricks также позволяет кросс-функциональным командам в сетке данных работать вместе более эффективно, что отвечает принципу создания доменно-ориентированных децентрализованных команд.

Snowflake

Snowflake — это полностью управляемое облачное решение для хранения данных, позволяющее хранить, упорядочивать и анализировать большие объемы данных, которое, помимо традиционной функциональности хранения данных, может служить еще и озером данных.

Компания была основана в 2012 году для решения проблем, связанных с ограничениями традиционных хранилищ данных. Ее основатели разработали уникальную архитектуру, которая разделяет вычисления и хранение, что позволяет масштабировать эффективную аналитику данных и рабочие нагрузки независимо и автоматически в зависимости от потребностей. Платформа, запущенная в 2014 году, получила признание благодаря своей способности обрабатывать разные типы данных, а также хранить и обрабатывать широкий спектр форматов данных, включая CSV, JSON, Parquet и Avro. Ее облачный подход и архитектура привлекли широкий круг клиентов из разных отраслей. Она предлагает оплату по мере использования, надежную защиту данных, гибкость и безотказность.

Данные можно загружать в разных форматах непосредственно в Snowflake, как и в случае с озером данных, что избавляет от необходимости преобразовывать или предварительно обрабатывать их перед загрузкой. Snowflake еще больше упрощает этот процесс, автоматически определяя схему загружаемых данных. После загрузки данных Snowflake предоставляет структурированную и масштабируемую среду для их организации и каталогизации, позволяя пользователям создавать таблицы, определять схемы и применять теги метаданных. Это обеспечивает эффективную организацию и легкий доступ к данным.

Использование Snowflake в качестве озера данных дает преимущества унифицированной платформы данных, устраняя необходимость в отдельной инфраструктуре для озера данных и понижая сложность. Она предлагает масштабируемую и производительную среду для хранения, управления и анализа разных типов данных, что делает ее привлекательным выбором для организаций, желающих использовать возможности озера данных в рамках единой экосистемы.

Сильными сторонами Snowflake являются высокооптимизированные возможности хранения данных, а также эффективное выполнение запросов и анализ данных в рамках самой платформы. Однако если вам требуются специфические вычисления или специализированная обработка данных, которую лучше выполнять с помощью других вычислительных систем, понадобится экспортировать данные из Snowflake и использовать эти системы. Хотя Snowflake и обеспечивает бесшовный экспорт данных, она напрямую не поддерживает запуск внешних вычислительных систем или инструментов для данных, хранящихся на платформе.

Snowflake хорошо согласуется с принципами сетки данных (рассмотренных в главе 13). Компания признает важность предоставления экспертам и командам доменов прав владения своими данными. С помощью Snowflake различные бизнес-подразделения или домены могут иметь отдельные учетные записи или виртуальные хранилища, что дает им возможность управлять своими активами данных. Такой децентрализованный подход способствует повышению гибкости и автономности, позволяя командам доменов принимать основанные на данных решения и брать на себя ответственность за качество и удобство использования своих данных.

Благодаря широким возможностям Snowflake в управлении данными, включая управление схемами, тегирование метаданных и каталогизацию, данные становятся ценным активом, к которому могут получать доступ и который могут использовать другие команды внутри организации, что способствует развитию продуктового мышления в отношении данных. Snowflake также обеспечивает масштабируемость и эластичность, которые характерны для концепции сетки данных: ее облачная архитектура позволяет независимо масштабировать вычислительные ресурсы, помогая командам адаптироваться к меняющимся требованиям и эффективно справляться с обработкой и аналитикой данных. Подводя итог, можно сказать, что Snowflake упрощает применение сетки данных в организациях, что способствует сотрудничеству, автономности и гибкости их экосистем данных.

Итоги

Выбор технологий архитектур данных будет иметь определяющее значение для успеха ваших инициатив в области работы с данными. Эта глава поможет вам разобраться в основах существующих решений и сделать этот выбор, что заложит фундамент вашей инфраструктуры данных.

Мы рассмотрели три ключевых программных фреймворка: Hadoop, Databricks и Snowflake, каждый из которых предлагает уникальный подход к обработке и анализу данных. Независимо от того, выбираете ли вы решения с открытым исходным кодом, пользуетесь ли предложениями облачных провайдеров или комбинируете оба варианта, учитывайте масштабы своих потребностей в данных и требования вашей организации к гибкости.

Кроме того, помните, что мир технологий обработки данных не является бинарным. Он включает в себя целый ландшафт моделей облачных сервисов, где основные провайдеры предлагают разнообразные сервисы, а мультиоблачные решения добавляют дополнительный уровень сложности к процессу принятия решений.

Путешествуя по архитектурам данных, помните, что выбранные в этой главе решения не работают изолированно; они взаимосвязаны и будут определять структуру вашей среды данных. Речь идет не просто о выборе решений с открытым исходным кодом или коммерческих, а о создании гармоничного сочетания технологий, которые соответствуют целям и задачам вашей организации. Руководствуясь этими соображениями, вы сможете легко ориентироваться в постоянно меняющемся мире технологий архитектур данных.

Счастливого пути!

Об авторе

Джеймс Серра (James Serra) является архитектором решений для Big Data и хранилищ данных в компании Microsoft, где он проработал большую часть последних девяти лет. Он является одним из лидеров мнений в области использования и применения Big Data и расширенной аналитики, включая такие архитектуры данных, как современное хранилище данных, озеро-хранилище данных, ткань данных и сетка данных. Ранее в роли независимого консультанта он работал архитектором и разработчиком хранилищ данных/бизнес-аналитики. Джеймс — главный создатель минимальной жизнеспособной версии (MVP) SQL Server с более чем 35-летним опытом работы в сфере IT, а также популярный блогер (<https://www.jamesserra.com>) и лектор, выступавший на десятках крупных мероприятий, включая SQLBits, PASS Summit, Data Summit и конференцию Enterprise Data World.

Иллюстрация на обложке

На обложке книги «*Архитектуры данных: современные решения для любых задач*» изображен голубой дискус (*Symphysodon aequifasciata*) — пресноводная рыба, обитающая в притоках реки Амазонки. Волнистые полосы и оригинальная окраска делают голубых дискусов популярными среди аквариумных рыбок, но их естественную среду обитания трудно воспроизвести и поддерживать.

Голубые дискусы питаются червями и мелкими ракообразными и вырастают до 25 см в длину. Мальки питаются выделениями желез своих родителей.

Многие из животных, представленных на обложках O'Reilly, находятся под угрозой исчезновения, все они важны для нашего мира.

Иллюстрация на обложке выполнена Карен Монтгомери (Karen Montgomery) на основе черно-белой гравюры из книги *Akademie der Wissenschaften*. Дизайн этой серии книг разработан Эди Фридманом (Edie Freedman), Элли Фолькхаузен (Ellie Volckhausen) и Карен Монтгомери.

Джеймс Серра
**Архитектуры данных: современные решения
для любых задач**

Перевела с английского Н. Григорьева
Научный редактор А. Петраки

Изготовлено в России. Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес:
010000, Казахстан, город Астана, район Алматы, проспект Рақымжан Қошқарбаев, д. 10/1, н. п. 18.

Дата изготовления: 02.2026. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 10.12.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 23,220. Тираж 700. Заказ 0000.



Харрисон Ферроне

ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ UNITY. ПОПУЛЯРНЫЕ ШАБЛОНЫ И ЛУЧШИЕ ПРАКТИКИ СОЗДАНИЯ ИГР НА UNITY И C#

Хотите научиться писать в Unity понятный и простой в сопровождении код для игр? Тогда вы пришли по адресу! Изучение популярных паттернов проектирования Unity позволит использовать весь их потенциал при создании захватывающих проектов. Изучая практические примеры, вы освоите такие порождающие паттерны, как Прототип, помогающий эффективно создавать врагов, и поведенческие паттерны, например Наблюдатель, для реализации реактивной игровой механики. По мере чтения вы также научитесь замечать негативные последствия плохих архитектурных решений и справляться с ними с помощью простых, но эффективных методов.

Когда вы прочтете эту книгу, ваши приемы разработки игр в Unity изменятся. Вы не только станете более опытным разработчиком, но и научитесь использовать в работе передовые паттерны проектирования.

*Саурабх Шривастава,
Ниланджали Шривастав*

**SOLUTIONS ARCHITECT:
АРХИТЕКТУРА
И ПРОЕКТИРОВАНИЕ
ИТ-РЕШЕНИЙ
3-е издание**



Овладейте искусством дизайна архитектур и станьте успешным архитектором решений. Книга, написанная опытными техлидами AWS Саурабхом Шриваставой и Ниланджали Шривастав, выходит за рамки традиционных руководств для подготовки к сертификации. В ней вы найдете подробную аналитику и передовые методы, предназначенные для удовлетворения конкретных потребностей клиентов и решения проблем, с которыми сталкиваются современные архитекторы решений.

Издание описывает новые интересные возможности, которые позволят вам оставаться в авангарде этой стремительно развивающейся сферы. Большие языковые модели, генеративный ИИ, инновации в области глубокого обучения – все эти революционные достижения формируют будущее технологий. Такие темы, как облачно-ориентированные архитектуры, инженерия данных, облачная оптимизация, модернизация мейнфреймов и создание экономичных и безопасных архитектур, продолжают оставаться актуальными в современной технологической сфере. В книге рассматриваются эти перспективные и ключевые технологии, а также приводится пошаговое описание процесса проектирования архитектур решений, начиная с ключевых принципов. Она снабдит вас знаниями, необходимыми, чтобы стать успешным архитектором решений, а также поможет усовершенствовать гибкие навыки и тем самым получить возможность для развития карьеры.

Раскройте потенциал передовых технологий, изучайте практические выводы на реальных сценариях и повышайте мастерство.

SPRINT
book



Марк Прайс

.NET 8: ИНСТРУМЕНТЫ И НАВЫКИ. ЛУЧШИЕ ПРАКТИКИ И ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ, ОТЛАДКИ И ТЕСТИРОВАНИЯ

Раскройте богатство возможностей разработки .NET-приложений с новой книгой Марка Прайса! Здесь вы узнаете, как управлять исходным кодом и проектами с помощью Git, обеспечивая при этом контроль версий. Откроете для себя передовые методы отладки и стратегии выявления и устранения проблем, а также увидите, как правильно документировать код, API и сервисы, чтобы повысить удобство обслуживания проекта. Погрузитесь в мир криптографии и освоите приемы обеспечения конфиденциальности и целостности на протяжении всего цикла разработки.

Вы сможете расширить свои навыки, познакомившись с такими продвинутыми темами, как создание интеллектуальных приложений с поддержкой пользовательских чат-сервисов на основе LLM, внедрение зависимостей, оптимизация производительности с помощью тестирования и запуск сервисов в контейнерах Docker. В книге описаны возможности разработки облачных сервисов с применением .NET Aspire и преимущества современных облачных платформ. А знакомство с особенностями передовых архитектур программного обеспечения позволит вам создавать надежные и масштабируемые приложения, удобные в обслуживании.

*Саурабх Шривастава,
Ниланджали Шривастав*

**.NET 8: ПРИЛОЖЕНИЯ И СЕРВИСЫ.
ПРАКТИКА СОЗДАНИЯ ПРОЕКТОВ
С ИСПОЛЬЗОВАНИЕМ BLAZOR,
.NET MAUI, GRPC, GRAPHQL
2-е издание**



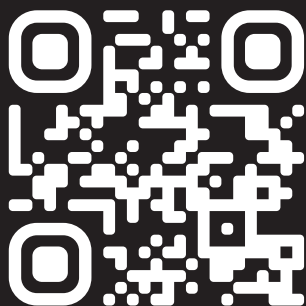
С новым изданием «.NET 8: приложения и сервисы» вы сможете поднять свои навыки владения C# и .NET на новый уровень. В книге описаны практические приемы использования различных технологий, включая Web API, gRPC, GraphQL и SignalR, что выделяет ее среди других источников. Она познакомит вас с новой унифицированной моделью Blazor Full Stack и расскажет, как использовать .NET MAUI для разработки мобильных и десктопных приложений. В новом издании представлены последние достижения, включая бесшовную реализацию веб-сервисов со встроенной поддержкой Ahead-of-Time (AOT) ADO.NET SqlClient. Помимо других популярных библиотек, в книге рассматриваются Humanizer и Noda Time. А совершенно новая глава расскажет об архитектуре, кэшировании, организации очередей и приемах построения надежных фоновых сервисов. Обретите глубокие знания и создайте многофункциональные и эффективные приложения.

SPRINT
book

Read IT Club

**Комьюнити рецензентов
и переводчиков ИТ-литературы**

Миссия участников клуба – обеспечить высокое качество профессиональной переводной литературы на русском языке. «Книжные дебагеры» проверяют корректность терминологии и подписей на схемах и иллюстрациях, чтобы сделать книги более понятными русскоязычному читателю. Стать участником Read IT Club может любой ИТ-специалист, готовый поделиться опытом с сообществом.



присоединиться к нам